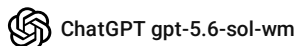


 transolver(10).pdf
9.5 MB linearNO(20260911-065214).pdf
9.9 MB LRSA(7).pdf
6.9 MB LNO(1).pdf
2.8 MB FLARE(1).pdf
3.8 MB IPOT(1).pdf
7.9 MB SVD-NO(1).pdf
371.1 KB

我要开始一个新的研究方向，在本聊天中忽略以前我的研究方向，我会给你提供一些参考的论文阅读，在阅读完我提供的论文后你也可以补充阅读一些与他们相关的论文，在我提供的论文中先系统的比较下transolver,linearno,lrsa,lno,flare的模型架构，要体现出它们的每一处区别，细节也要把控好



我会先把这次研究视为全新的独立方向，不沿用此前方案。接下来我会逐篇核对五篇论文的正文、公式、架构图和附录；IPOT 与 SVD-NO 暂作为后续关联材料，不混入第一轮五模型主比较。我会使用 PDF 阅读流程，重点统一到同一套比较坐标：输入表示、几何/坐标处理、token 或 latent 构造、核心算子、低秩机制、局部-全局交互、编码/解码、复杂度与适用域。

收到。我会把同一条记录里的 7 篇附件全部纳入本轮精读：前五篇是主比较对象，IPOT 与 SVD-NO 用来校准“低秩究竟发生在哪里、是否显式构造基/核、是否保留非线性 latent mixing”等判断。最终仍以 Transolver、LinearNO、LRSA、LNO、FLARE 的逐层架构差异为主，不把另外两篇喧宾夺主。

目前已经确认一个容易被论文措辞掩盖、但对比较非常关键的分界：这五个模型并不只是“都把 N 个点压到 M 个 latent”。LNO 是整网只编码一次、在 latent 中连续处理、最后只解码一次；而 Transolver、LinearNO、LRSA、FLARE 都是在每个 block 内把信息送入低秩通道再回到点域。后四者之间又分别对应“共享对称路由 + latent attention”“非对称线性路由”“独立上下行 cross-attention + 非线性 latent mixing”“对称双 SDPA 路由但无 latent mixing”。这会成为最终对照的主轴。

我已把这次讨论视为全新的独立研究方向。下面的比较以你上传的五篇主论文为核心，同时用 IPOT 与 SVD-NO 校准“latent bottleneck”和“低秩核分解”的概念，不沿用此前研究方案。

阅读依据：[Transolver](#)、[LinearNO](#)、[LRSA](#)、[LNO](#)、[FLARE](#)，以及辅助参照 [IPOT](#)、[SVD-NO](#)。

一、最核心的架构结论

这五个模型不能简单概括成“把 N 个网格点压缩为 M 个 latent token”。它们实际分成两类。

1. 每层返回物理点域

Transolver、LinearNO、LRSA、FLARE 都维持点特征

$$H^{(\ell)} \in \mathbb{R}^{N \times d}$$

作为网络主状态。每个 block 内部暂时经过 M 个低秩通道，再回到 N 个点：

$$N \longrightarrow M \longrightarrow N.$$

因此，下一层仍然面对全部物理点，并重新计算一次低秩路由。

2. 整个网络只进入和离开 latent 空间一次

LNO 的结构是：

$$N_{\text{in}} \xrightarrow{\text{PhCA encoder}} M \xrightarrow{L \text{ 个 latent Transformer}} M \xrightarrow{\text{PhCA decoder}} N_{\text{out}}.$$

它在 encoder 后就不再维护 N 个物理点的隐藏状态，而是让所有深层计算都发生在 M 个 latent token 上。这是 LNO 与另外四个模型最根本的区别。

二、统一的低秩视角

可以把一次点到点全局交互抽象成

$$\Delta H = U(H) \mathcal{G}(V(H)^\top \mathcal{V}(H)),$$

其中：

- $V^\top$ ：把 N 个点压缩到 M 个低秩通道；
- $\mathcal{G}$ ：在 M 个通道内部进一步处理；
- U ：将 M 个通道重建回 N 个点；
- $\mathcal{V}(H)$ ：需要传递的 value 特征。

五种模型在这个模板中的位置如下。

模型	压缩 $V^\top$	latent 处理 $\mathcal{G}$	重建 U	上下行是否绑定
Transolver	feature-dependent slice aggregation	latent self-attention	使用原 slice 权重 deslice	强绑定，同一 slice assignment
LinearNO	独立 Key 投影 $\psi(K)^\top$	恒等映射 I	独立 Query 投影 $\phi(Q)$	解耦
LRSA	learnable queries 到物理点的 cross-attention	FFN + self-attention + FFN	物理点到处理后 latent 的另一套 cross-attention	完全解耦
LNO	坐标驱动 PhCA encoder	整个 latent Transformer backbone	坐标驱动 PhCA decoder	attention-projector 参数共享
FLARE	$\text{softmax}(QK^\top)$	恒等映射 I	$\text{softmax}(KQ^\top)$	强绑定、近似互为伴随

因此最精炼的架构关系是：

Transolver :	$U = V, \quad \mathcal{G} = \text{latent attention}$
LinearNO :	$U \neq V, \quad \mathcal{G} = I$
LRSA :	$U \neq V, \quad \mathcal{G} = \text{nonlinear Transformer}$
FLARE :	$U \approx V, \quad \mathcal{G} = I$
LNO :	全网络级 $U\mathcal{G}^L V^\top$

这里的 $U = V$ 是指“上下行由同一组归属分数产生”，并不意味着归一化后的两个矩阵在数值上严格相等。

三、五个模型的完整计算结构

3.1 Transolver：共享 slice 路由 + slice 间注意力

输入提升

输入坐标 g_i 与输入物理量 u_i 拼接：

$$H^{(0)} = \text{Linear}([g, u]) \in \mathbb{R}^{N \times C}.$$

坐标与物理量从一开始就被混合，不区分“位置决定路由”和“物理量作为 value”。

Slice assignment

每个点通过单个投影产生对 M 个 slice 的归属权重：

$$A = \text{softmax}_M(HW_s) \in \mathbb{R}^{N \times M}.$$

其中 A_{ij} 表示第 i 个点属于第 j 个 slice 的程度。

Softmax 沿 M 维进行，因此每个点对全部 slice 的权重和为 1。

聚合为 physics-aware tokens

$$z_j = \frac{\sum_{i=1}^N A_{ij} v_i}{\sum_{i=1}^N A_{ij}}, \quad Z \in \mathbb{R}^{M \times d_h}.$$

关键点是：

- 点到 slice 的归属先沿 M 归一化；
- 聚合时又除以每个 slice 收到的总权重；
- 因此需要额外的显式 sum-normalization；
- 通常需要构造 $N \times M$ slice-weight matrix。

Slice 间注意力

$$Z' = \text{softmax} \left(\frac{ZW_Q(ZW_K)^\top}{\sqrt{d_h}} \right) ZW_V.$$

这是 Transolver 独有的“先分组，再让物理状态之间交互”的核心解释。

Deslice

$$\Delta H_i = \sum_{j=1}^M A_{ij} Z'_j.$$

重建直接复用压缩时的 A 。也就是说，一个点以多大权重向某个 slice 写入，也以相同归属权重从该 slice 读取。

Block

$$\begin{aligned} \widehat{H} &= H + \text{PhysicsAttn}(\text{LN}(H)), \\ H' &= \widehat{H} + \text{FFN}(\text{LN}(\widehat{H})). \end{aligned}$$

因此每层完整经历：

$$N \rightarrow M \rightarrow M \rightarrow N.$$

几何与局部处理

原论文中 slice projector 可以是：

- 不规则网格：point-wise linear；
- 规则网格或结构网格： 3×3 convolution。

所以 Transolver 的核心 slice 机制本身不要求网格连接，但其实验版本在规则域上引入了局部卷积先验。卷积并不是 Physics-Attention 数学定义中不可缺少的部分。

3.2 LinearNO：把 Transolver 化为非对称线性注意力

LinearNO 的输入编码、输出投影、pre-norm residual block 基本沿用 Transolver，真正替换的是 Physics-Attention。

第一步：解耦 Query 与 Key 路由

LinearNO 使用两套独立投影：

$$\begin{aligned} U &= \phi(Q) = \text{softmax}_M(HW_Q) \in \mathbb{R}^{N \times M}, \\ V^\top &= \psi(K)^\top = \text{softmax}_N((HW_K)^\top) \in \mathbb{R}^{M \times N}. \end{aligned}$$

这里存在两个重要区别：

1. $W_Q \neq W_K$ ，压缩与重建不再共享参数；
2. 两个 Softmax 的方向不同：
 - Query map 沿 latent 维 M 归一化；
 - Key map 沿物理点维 N 归一化。

第二步：删除 **slice self-attention**

先聚合：

$$S = V^\top (HW_V) \in \mathbb{R}^{M \times d_h},$$

再直接重建：

$$\Delta H = US.$$

合并即

$$\text{LinearNO}(H) = \phi(Q) [\psi(K)^\top \text{Linear}_V(H)].$$

这里没有：

- slice-to-slice self-attention；
- latent FFN；
- M^2 的 token mixing。

因此它对应

$$\mathcal{G} = I.$$

与 **Transolver** 的本质差异

Transolver 是：

$$A \mathcal{G}_{\text{attn}} A^\top,$$

LinearNO 是：

$$UV^\top, \quad U \neq V.$$

LinearNO 认为 Transolver 的效果主要来自 $N \rightarrow M \rightarrow N$ 的低秩投影，而不是 slice token 之间的注意力。

输出位置限制

基础 LinearNO 与 Transolver 一样，默认输入点和输出点对应同一组 N 个位置。论文附录另外提出了 Cross-LinearNO，用于 Burgers 稀疏观测和超分辨率，但它不是主干 LinearNO block 的默认结构。

3.3 LRSA：完全标准的 cross-attention + 非线性 latent operator

LRSA 同样每层执行 $N \rightarrow M \rightarrow N$ ，但不再使用手工 slice/deslice 公式，而是全部使用标准 scaled dot-product attention。

输入

$$h_i^{(0)} = \text{Lift}([f_i, \text{PE}(x_i)]).$$

与 Transolver 不同，LRSA 明确允许先对坐标使用 Fourier features 等位置编码，再与物理特征拼接。

压缩

每层拥有自己的可学习 latent queries

$$P^{(\ell)} \in \mathbb{R}^{M \times d}.$$

压缩为：

$$Z = \text{Attn}(PW_Q^\downarrow, HW_K^\downarrow, HW_V^\downarrow).$$

这里：

- latent queries 是固定的可学习参数；
- Key、Value 来自当前点特征；
- Softmax 沿 N 个物理点执行；

- 完全是标准 cross-attention。

Latent mixing

LRSA 在 latent 内部不是只做一次 self-attention，而是：

$$\begin{aligned}\tilde{Z} &= Z + \text{FFN}_{\text{in}}(\text{Norm}(Z)), \\ \hat{Z} &= \tilde{Z} + \text{SelfAttn}(\text{Norm}(\tilde{Z})), \\ Z' &= \hat{Z} + \text{FFN}_{\text{out}}(\text{Norm}(\hat{Z})).\end{aligned}$$

这使它的 latent operator $\mathcal{G}$ 比 Transolver 更完整，也明显比 LinearNO 和 FLARE 更强。

重建

$$\Delta H = \text{Attn}\left(HW_Q^\uparrow, Z'W_K^\uparrow, Z'W_V^\uparrow\right).$$

然后

$$H' = H + \Delta H.$$

压缩和重建：

- 使用不同的参数；
- 使用不同的 Q/K/V；
- 重建权重还依赖已经过 nonlinear latent mixing 的 Z' 。

因此 LRSA 的解耦程度比 LinearNO 更强。LinearNO 的 U, V 都直接由同一个输入 H 生成，而 LRSA 的重建矩阵还取决于处理后的全局 latent 状态。

外层 pointwise block

LRSA 的 global mixing 后仍有点域 FFN residual。因此一个 LRSA block 同时包含：

- 点域 channel mixing；
- $N \rightarrow M$ cross-attention；
- latent FFN；
- latent self-attention；
- $M \rightarrow N$ cross-attention；
- 再一次点域 FFN。

硬件含义

每层使用：

- 两次标准 cross-attention；
- 一次标准 latent self-attention。

这些都可以直接映射到 PyTorch SDPA/FlashAttention，而不需要显式保存 slice assignment 并在算子外手工归一化。

3.4 LNO：位置—物理量解耦的一次性 latent backbone

LNO 最特别的并不是“也有 latent”，而是：

1. 全网络只编码、解码一次；
2. attention 权重只由位置决定；
3. 物理量只进入 value；
4. 原生允许 $N_{\text{in}} \neq N_{\text{out}}$ 。

两套输入投影

对观测点位置 x_i ：

$$\hat{x}_i = \text{TrunkProjector}(x_i).$$

对位置-数值对：

$$\hat{y}_i = \text{BranchProjector}([x_i, f(x_i)]).$$

于是：

- $\hat{X}$ ：只包含位置；
- $\hat{Y}$ ：包含位置和物理量。

这与 Transolver、LinearNO、LRSA、FLARE 都不同。后四者通常让当前完整隐藏特征共同决定路由。

PhCA encoder

LNO 定义 M 个可学习的“假想 latent 位置”：

$$H_{\text{latent}} \in \mathbb{R}^{M \times D}.$$

编码：

$$Z^{(0)} = \text{PhCA}(H_{\text{latent}}, \hat{X}, \hat{Y}).$$

展开为

$$Z^{(0)} = \text{softmax}(W_1 \hat{X}^\top) \hat{Y} W_V.$$

注意权重只依赖坐标嵌入 $\hat{X}$ ，不依赖物理量；物理量只通过 value 被聚合。

Latent backbone

$$Z^{(\ell+1)} = \text{TransformerBlock}_\ell(Z^{(\ell)}), \quad \ell = 0, \dots, L-1.$$

这 L 层全部在 M 个 latent token 上运行，中间不返回点域。

PhCA decoder

输出查询位置 y_j 先经同一个 trunk projector：

$$P_j = \text{TrunkProjector}(y_j),$$

再解码：

$$U = \text{softmax}(P W_2^\top) Z^{(L)} W_V'.$$

最后用 MLP 投影到输出物理量。

参数共享

LNO 设置

$$W_1 = W_2,$$

即 encoder 和 decoder 的 attention-projector 共享参数。论文实验认为共享优于分别学习。

这与：

- LinearNO 的上下行解耦；
- LRSA 的完全独立上下行 attention

形成直接对照。

原生优势

因为 decoder 的 query 是任意输出坐标 y_j ，LNO 可以原生处理：

- 稀疏观测到完整场；
- 输入、输出分辨率不同；
- forward PDE；

- inverse PDE;
- 任意输出查询点。

另外四个模型的基本 block 则更接近“同一组点上的特征更新”。

3.5 FLARE: 对称双 SDPA 路由 + 无 latent mixing

FLARE 和 LinearNO 都删除了 latent self-attention，但两者选择了几乎相反的路由设计。

固定 learnable queries

每个 head 拥有独立 latent query:

$$Q_h \in \mathbb{R}^{M \times d_h}.$$

点特征通过深层 residual MLP 得到:

$$K_h = \text{ResMLP}_K(H), \quad V_h = \text{ResMLP}_V(H).$$

与 LinearNO 的浅层线性投影不同，FLARE 特意使用较深的 ResMLP 补偿固定 query 的表达限制。

Encoding

$$Z_h = \text{SDPA}(Q_h, K_h, V_h; \text{scale} = 1).$$

即

$$W_{\text{enc},h} = \text{softmax}_N(Q_h K_h^\top),$$

$$Z_h = W_{\text{enc},h} V_h.$$

Decoding

FLARE 交换 Query 和 Key 的角色:

$$Y_h = \text{SDPA}(K_h, Q_h, Z_h; \text{scale} = 1),$$

即

$$W_{\text{dec},h} = \text{softmax}_M(K_h Q_h^\top),$$

$$Y_h = W_{\text{dec},h} Z_h.$$

合并为:

$$Y_h = W_{\text{dec},h} W_{\text{enc},h} V_h.$$

因此点到点通信矩阵

$$W_h = W_{\text{dec},h} W_{\text{enc},h}$$

严格满足

$$\text{rank}(W_h) \leq M.$$

这是五个模型中最容易直接写出和分析的显式低秩 attention matrix。

不进行 latent mixing

FLARE 的 latent Z_h 只作为信息路由中介:

$$N \rightarrow M \rightarrow N,$$

中间没有:

- self-attention;
- latent FFN;
- latent Transformer block。

因此 latent 是 routing hub，而不是 computational workspace。

Head-wise independent routing

每个 head 使用独立的 latent query slice，学习一个独立的低秩矩阵 W_h 。不同 head 相当于并行学习多套 rank- M 路径。

按 FLARE 论文对原实现的分析：

- Transolver 的 slice projection 在 heads 之间共享；
- LNO 使用单个全局投影；
- FLARE 则显式强调 per-head independent latents。

最终各 head 输出拼接后，再由线性层进行 channel mixing。

Block 内非线性放在哪里

FLARE 虽然删除 latent 非线性，但不是整个 block 都是线性的：

$$H \leftarrow H + \text{FLARE}(\text{LN}(H)),$$
$$H \leftarrow H + \text{ResMLP}(\text{LN}(H)).$$

同时 K, V 也由三层 residual MLP 生成。因此它把表达能力放在：

- 点级深层 K/V 表示；
- 多个 head 的并行低秩路径；
- block 间反复 $N \rightarrow M \rightarrow N$ ；
- pointwise ResMLP。

而不是放在 latent self-attention 中。

四、逐项总表

4.1 主干拓扑

比较项	Transolver	LinearNO	LRSA	LNO	FLARE
主状态所在空间	点域	点域	点域	latent 域	点域
压缩次数	每层一次	每层一次	每层一次	全网一次	每层一次
重建次数	每层一次	每层一次	每层一次	全网一次	每层一次
深度方向的信息载体	N 个点	N 个点	N 个点	M 个 latent	N 个点
每层是否重新路由	是	是	是	否	是
输入输出点数能否天然不同	否	基础版否	基础版否	是	否

4.2 Latent 的含义

模型	M 个 latent 的实际含义
Transolver	根据当前点特征形成的 physics-aware slices
LinearNO	线性注意力中的低秩 feature-map/basis channels
LRSA	由 learnable queries 聚合得到的全局 latent modes
LNO	贯穿整个网络的 hypothetical latent sampling positions
FLARE	每个 head 独立的信息 gather-scatter routing hubs

Transolver 的“物理状态切片”是模型的解释，而不是数学保证；从 LinearNO、LRSA 的低秩视角看，它同时也可以被理解作为一种特征依赖的低秩软分组。

4.3 路由生成和上下行关系

比较项	Transolver	LinearNO	LRSA	LNO	FLARE
下行路由来源	当前完整点特征	当前完整点特征	latent query 与当前点特征	latent 位置与输入坐标	固定 latent query 与当前点特征
上行路由来源	复用原 assignment	独立 Query map	当前点特征与处理后 latent	输出坐标与 latent 位置	复用同一 score 的转置方向
上下行参数	绑定	独立	完全独立	默认共享 projector	绑定
路由是否依赖物理量	是	是	是	否，权重只看位置	是
路由是否依赖处理后 latent	否	否	是	decoder 使用最终 latent value，但权重主要由位置决定	否

4.4 Latent 内部操作

模型	Latent 内操作
Transolver	一次多头 self-attention
LinearNO	无， $G = I$
LRSA	FFN-in $\rightarrow$ self-attention $\rightarrow$ FFN-out
LNO	全部 L 个 Transformer blocks
FLARE	无， $G = I$

这表明“有没有 latent self-attention”并不是唯一问题，还必须看它放在什么位置：

- Transolver：每层短暂进入 latent；
- LRSA：每层短暂进入 latent，但处理更强；
- LNO：整个 backbone 都在 latent；
- LinearNO、FLARE：latent 只负责低秩传输。

4.5 坐标与物理量

模型	坐标和物理量的关系
Transolver	拼接后共同形成隐藏特征和 slice assignment
LinearNO	基础版沿用 Transolver，路由由完整隐藏特征产生
LRSA	$[f_i, \text{PE}(x_i)]$ 经 Lift 后共同参与 cross-attention
LNO	严格区分：坐标生成 attention weights，坐标+物理量生成 values
FLARE	没有独立 coordinate branch；坐标若存在，作为普通输入特征进入 ResMLP

只有 LNO 把“在哪里”和“该位置有什么物理量”在架构上明确解耦。

4.6 局部模块与几何先验

模型	显式局部模块	网格连接要求
Transolver	规则/结构域可使用 3×3 conv projector	不规则域不要求连接
LinearNO	实验中基本沿用 Transolver 的卷积预处理策略	不要求连接
LRSA	无专用卷积或图消息传递	不要求连接
LNO	无	不要求连接
FLARE	无，明确依靠 attention routing 与 ResMLP	不要求连接

需要注意，pointwise FFN/ResMLP 只进行通道混合，不是空间局部邻域聚合。

五、复杂度、秩与硬件实现

忽略通道投影的常数后：

模型	Attention/路由复杂度	是否有 M^2	显式秩
Transolver	每层 $O(NM + M^2)$	有	点间路径至多经 M 个 slice
LinearNO	每层 $O(NM)$	无	$\leq M / \text{head}$
LRSA	每层 $O(NM + M^2)$	有	有效 kernel 经 M 个 latent modes
LNO	全网 $O((N_{\text{in}} + N_{\text{out}})M + LM^2)$	有	M -维 bottleneck，不是单一显式矩阵
FLARE	每层 $O(NM)$	无	严格 $\text{rank}(W_h) \leq M / \text{head}$

硬件路径

Transolver

需要显式处理：

- $N \times M$ slice weights；
- 列方向求和归一化；
- weighted aggregation；
- deslice。

这些操作不完全落入单次标准 SDPA，半精度下的分母归一化也更敏感。

LinearNO

删除了 M^2 self-attention，但仍以两组 $N \times M$ feature maps 和矩阵乘法表示。论文没有像 LRSA/FLARE 那样把整个主路径写成标准 fused SDPA 调用。

LRSA

每个 block 是两次 cross-SDPA 加一次 latent self-SDPA，可以直接使用硬件优化注意力内核，并避免在 Python/PyTorch 算子层显式维护注意力矩阵。

LNO

$N \times M$ 投影只在 encoder/decoder 出现，不随深度 L 重复；但原始 PhCA 的 attention-projector 和共享投影实现并不完全等同于直接调用标准 fused cross-SDPA。

FLARE

每个 block 恰好两次 SDPA：

```
python

Z = SDPA(Q, K, V, scale=1.0)
Y = SDPA(K, Q, Z, scale=1.0)
```

这是五者中最直接、最适合 fused SDPA 的实现。它还特意采用 `scale=1`，而不是通常的 $1/\sqrt{d_h}$ ，论文将其归因于很小的 head dimension（常见 $d_h = 4$ 或 8）。

六、几个看似矛盾、实际非常重要的结论

6.1 LinearNO 与 FLARE 对“对称路由”的结论相反

LinearNO 的结论是：

Transolver 上下行共用 slice assignment 限制有效秩；应该将 U, V 解耦。

FLARE 的结论却是：

这不是严格意义上的直接矛盾，因为两者的其他条件不同。

LinearNO	FLARE
U, V 都由当前点特征的独立线性投影产生	Q 是固定 learnable latent, K 来自深 ResMLP
上下行非对称	上下行共享 score
没有特别强化 per-head latent 独立性	每个 head 是独立低秩路径
K/V 主要是浅层线性投影	K/V 使用深 residual MLP
用非对称路由提升秩利用率	用多 head 的不同子空间提升总表达能力

因此真正需要研究的问题不是简单的“对称一定好还是不好”，而是：

$$\text{路由对称性} \times \text{head 独立性} \times \text{K/V 非线性深度} \times \text{latent mixing}$$

之间的联合关系。

6.2 LinearNO/FLARE 与 LRSA 对 latent mixing 的判断不同

LinearNO、FLARE认为 latent self-attention 可以删除；LRSA 则发现：

- 简单静态问题可能不需要很强的 latent mixing；
- 复杂、动态、长程耦合问题更依赖非线性 $\mathcal{G}$ ；
- 删除 latent self-attention 会显著限制某些任务。

关键原因是 LRSA 的 $\mathcal{G}$ 不只是额外计算，它处于独立压缩基 V 与独立重建基 U 之间：

$$V^\top H \rightarrow \mathcal{G}_{\text{nonlinear}} \rightarrow U.$$

它可以改变 latent 的内容及其重建方式。LinearNO/FLARE 则将深度放在多个点域 block 或 pointwise MLP 中。

6.3 LNO 与 LRSA 对参数共享的判断也不同

LNO 发现 encoder/decoder projector 共享更好；LRSA 发现压缩和重建解耦更好。

原因仍然是它们共享的对象不同：

- LNO 共享的是坐标驱动的全网络 encoder/decoder 基；
- LRSA 解耦的是每层 feature-dependent 的写入和读取注意力；
- LNO 希望建立稳定的统一 latent coordinate system；
- LRSA 希望每层的分析基和合成基可以承担不同功能。

所以不能直接用 LNO 的共享实验否定 LRSA 的非对称设计。

七、五个模型的真实“基因关系”

可以把它们概括成以下演化逻辑：

1. LNO/IPOT 路线

将整个 PDE operator 放在固定 latent 空间中计算，主要解决输入输出离散化解耦和深层计算成本。

2. Transolver 路线

不永久离开物理点域，而是在每个 block 中形成短暂的物理 slice，让点域表示随深度不断更新。

3. LinearNO 路线

将 Transolver 重新解释为低秩线性注意力，保留逐层点域更新，解耦上下行映射并删除 latent mixing。

4. LRSA 路线

保留 Transolver 的逐层 $N \rightarrow M \rightarrow N$ ，但改成完全标准的 cross-attention，并恢复强非线性 latent operator。

5. FLARE 路线

同样逐层 $N \rightarrow M \rightarrow N$ ，但把 latent 限定为纯路由媒介；通过对称双 SDPA、深 K/V MLP 和 head-wise 独立路径补偿不做 latent

mixing 的限制。

八、IPOT 与 SVD-NO 带来的校准

IPOT

IPOT 与 LNO 属于同一大类：

一次 encoder $\rightarrow$ 深 latent processor $\rightarrow$ 一次 decoder.

但 IPOT 使用一般的 inducing-point cross-attention；LNO 进一步强调：

- 坐标与物理量解耦；
- trunk/branch projector；
- encoder/decoder attention-projector 共享；
- forward/inverse PDE 统一。

因此，LNO 不是 Transolver 的逐层 slice 变体，而更接近专门为 PDE 重新设计的 IPOT/Perceiver 型 latent operator。

SVD-NO

SVD-NO 显式学习：

$$K(z, z') = \Phi(z) \Sigma \Psi(z')^\top,$$

其中：

- Φ, Ψ 是左右奇异函数；
- Σ 是对角奇异值矩阵；
- 使用 Gram regularization 推动 Φ, Ψ 正交。

相比之下，这五个主模型虽然都可以解释为低秩结构，但都不是严格的 SVD：

- 不要求左右基正交；
- 不显式学习排序奇异值；
- 中间矩阵不一定为对角阵；
- LRSA、Transolver、LNO 的 latent processor 还是输入依赖、非线性的。

所以更准确的说法是：它们是 latent-induced low-rank operators，而不是 SVD low-rank operators。

九、最终定位

如果只看架构而不看论文宣传语：

- **Transolver**：物理语义最强，但路由约束和手工聚合最多。
- **LinearNO**：最接近纯粹的非对称 linear attention，计算最简洁。
- **LRSA**：低秩模板最完整，兼顾独立分析/合成基和非线性 latent operator。
- **LNO**：输入输出离散化解耦最彻底，最适合稀疏观测、超分辨率和 inverse problem。
- **FLARE**：工程扩展性和 fused-kernel 适配最强，显式低秩矩阵最容易分析，但单个 block 的 latent 表达最受约束。

对后续研究最重要的设计坐标不是笼统的“选择多少 latent”，而是四个相互独立的问题：

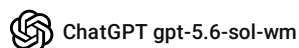
逐层路由还是一次性编码 + 对称还是非对称上下行 + 是否保留 nonlinear latent mixing + 共享还是 head-wise 独立 latent

这四个维度正好完整地地区分了上述五种架构。

- :codex-followup[绘制张量流程]{prompt="继续基于这7篇论文，为五个模型分别绘制逐步张量计算流程，给出每一步张量形状，并用同一个具体PDE样本演示。"}
- :codex-followup[扩展文献谱系]{prompt="在这五个模型之外补充检索相关低秩神经算子、latent operator和高效attention论文，构建完整的文献演化关系与分类表。"}
- :codex-followup[分析研究空白]{prompt="基于本次架构比较，系统分析五条路线各自尚未解决的gap，并提出若干独立、可实现的新研究方向。"}

这些论文的发表顺序是什么样的，你认为它们的发展体现了对低秩空间神经算子怎样的考虑，再补充阅读一些与低秩空间神经算子有关的论文，形成完整的思考链

刚才我把你对“我要开始一个新的研究方向，在本聊天中忽略以前我的研究方向，我会给你提供一些参考的论文阅读，在阅读完我提供的论文后你也可以补充阅读一些与他们相关的论文，在我提供的论文中先系统的比较下transolver,linearno,lrsa,lno,flare的模型架构，要体现出它们的每一处区别，细节也要把控好”这个问题的回答打断了，继续回答这个问题，把你刚才想回答的回答完



我会接着上一轮的断点继续，不重新换研究问题。这次重点补齐上一版尚未展开到位的部分：五个模型在每一层中的真实张量流、各自的低秩矩阵到底由什么构成、multi-head 与参数共享差异、非线性放置位置，以及这些差异会造成什么表达能力限制。我先用个人上下文检查流程确认断点，再继续按 PDF 精读流程核对原文。

上一版的总体分类没有问题，但有一处需要进一步精确到“归一化轴”：LNO 的 PhCA 虽然写成 cross-attention 形式，论文图中 encoder 与 decoder 的 Softmax 轴设置并不只是泛泛的标准注意力描述。我正在逐式核对这一点，因为它直接决定 LNO 与 Transolver 的聚合是否真正等价。

我已经把比较进一步收束成三个不能混为一谈的“低秩对象”：有些模型低秩的是点到点通信矩阵，有些低秩的是整个网络的表示瓶颈，还有些只是通过 M 个 latent 模式表达输入依赖的非线性核。接下来的补充会明确区分这些对象，并说明“rank $\leq M$ ”为什么通常不等于整个 residual block 或整个神经算子的秩不超过 M 。

继续。下面补完上一版尚未展开到位的细粒度比较，并先修正三个容易造成误解的简写。

1. Transolver、LNO、FLARE 的“上下行绑定”，准确说是共享原始 score generator，并不是归一化后的编码矩阵与解码矩阵严格相等。
2. LinearNO 中的“Linear”指线性复杂度注意力，不代表整个网络关于输入是线性的。
3. “rank $\leq M$ ”通常只指固定当前输入后、单个 head 的非局部通信路径；加上 residual、FFN、多头和输入相关路由后，整个 block 或整个神经算子并不是 rank- M 。

一、在统一记号下精确展开五个模型

设当前点域特征为

$$H \in \mathbb{R}^{N \times C},$$

attention head 数为 n_h ，每个 head 的维度为 $d_h = C/n_h$ ，低秩 latent/slice 数为 $M \ll N$ 。

1. Transolver 的精确形式

Slice assignment

对每个 head，当前点特征产生 slice logits：

$$A_h = \text{softmax}_M(\text{Project}(H_h)) \in \mathbb{R}^{N \times M}.$$

每一行沿 M 个 slice 归一化：

$$\sum_{m=1}^M A_{h,im} = 1.$$

Slice aggregation

Transolver 不是简单使用 $A_h^\top H_h$ ，而是还要对每个 slice 收到的总权重进行归一化。令

$$D_h = \text{diag}(A_h^\top \mathbf{1}_N) \in \mathbb{R}^{M \times M},$$

则

$$Z_h = D_h^{-1} A_h^\top V_h \in \mathbb{R}^{M \times d_h}.$$

其中 V_h 是该 head 的 value 特征。

这一步对应论文中的

$$z_m = \frac{\sum_i A_{im} v_i}{\sum_i A_{im}}.$$

Latent token attention

$$Z'_h = \text{SelfAttn}(Z_h) \in \mathbb{R}^{M \times d_h}.$$

Deslice

$$\Delta H_h = A_h Z'_h.$$

因此，固定当前 attention 权重后，Transolver 的空间通信路径应写成

$$K_{\text{Transolver},h} = A_h G_h D_h^{-1} A_h^\top,$$

而不是省略 D_h^{-1} 的 $A_h G_h A_h^\top$ 。

这里“对称”的准确含义

压缩和重建使用同一个 A_h ，但：

- 压缩端为 $D_h^{-1} A_h^\top$ ；
- 重建端为 A_h 。

所以它们不是严格转置或数值相等，只是共享同一组原始 slice assignment。

原论文图中每个 head 都形成自己的 slice tensor；FLARE 对其实现的分析进一步指出，Transolver 的 slice projection 映射在 heads 间共享。即使投影参数共享，由于各 head 输入子特征不同，得到的 A_h 也不必相同。

2. LinearNO 的精确形式

LinearNO 先将 Transolver 的上下行 score generator 解耦。

压缩矩阵

$$E_h = \text{softmax}_N \left((H_h W_{K,h})^\top \right) \in \mathbb{R}^{M \times N}.$$

每一行在 N 个物理点上归一化：

$$\sum_{i=1}^N E_{h,mi} = 1.$$

重建矩阵

$$U_h = \text{softmax}_M (H_h W_{Q,h}) \in \mathbb{R}^{N \times M}.$$

每个物理点在 M 个 latent 通道上归一化。

低秩聚合和重建

$$\begin{aligned} Z_h &= E_h (H_h W_{V,h}), \\ \Delta H_h &= U_h Z_h. \end{aligned}$$

合并为

$$\Delta H_h = U_h E_h (H_h W_{V,h}).$$

因此

$$K_{\text{LinearNO},h} = U_h E_h, \quad \text{rank}(K_{\text{LinearNO},h}) \leq M.$$

与 Transolver 不只是“删除 attention”

LinearNO 同时进行了两项变化：

1. 将共享 slice score 改成独立的 W_Q, W_K ；
2. 删除 $M \times M$ 的 slice self-attention，即令 $G = I$ 。

所以 LinearNO 不是单纯的 “Transolver without slice attention”，而是：

非对称分析/合成基 + 无 latent mixing

论文没有足够明确地说明 LinearNO 的 route projector 是否在不同 heads 间共享，因此不宜像 FLARE 那样断言它一定采用 fully independent per-head latent parameters。

3. LRSA 的精确形式

LRSA 使用标准 cross-attention 实现压缩和重建。

压缩

每层有自己的 learnable queries：

$$P^{(\ell)} \in \mathbb{R}^{M \times C}.$$

对每个 head：

$$E_h = \text{softmax}_N \left(\frac{(P W_Q^{\downarrow h}) (H W_K^{\downarrow h})^{\top}}{\sqrt{d_h}} \right) \in \mathbb{R}^{M \times N}, \quad Z_h = E_h (H W_V^{\downarrow h}).$$

注意：learnable queries P 只是 query，不存在从 P 到 Z 的 residual； Z 完全由点特征聚合而来。

Latent processing

LRSA 的 latent 部分为：

$$\begin{aligned} \tilde{Z} &= Z + \text{FFN}_{\text{in}}(\text{Norm}(Z)), \\ \hat{Z} &= \tilde{Z} + \text{SelfAttn}(\text{Norm}(\tilde{Z})), \\ Z' &= \hat{Z} + \text{FFN}_{\text{out}}(\text{Norm}(\hat{Z})). \end{aligned}$$

它比 Transolver 仅有一次 latent self-attention 更完整。

重建

$$U_h = \text{softmax}_M \left(\frac{(H W_Q^{\uparrow h}) (Z' W_K^{\uparrow h})^{\top}}{\sqrt{d_h}} \right) \in \mathbb{R}^{N \times M}, \quad \Delta H_h = U_h (Z' W_V^{\uparrow h}).$$

这里重建矩阵 U_h 不只是与压缩矩阵 E_h 参数独立，它还依赖处理后的 Z' 。因此其解耦程度高于 LinearNO。

外层 block

LRSA 外层准确顺序是：

$$\begin{aligned} G^{(\ell)} &= H^{(\ell)} + \text{LRSA}(\text{Norm}(H^{(\ell)})), \\ H^{(\ell+1)} &= G^{(\ell)} + \text{FFN}(\text{Norm}(G^{(\ell)})). \end{aligned}$$

不是在 LRSA 前后各放一个点域 FFN。模型整体另有：

- 所有 blocks 之前的 Lift FFN；
- 所有 blocks 之后的 Project FFN。

LRSA 明确规定所有参数，包括 $P^{(\ell)}$ ，都是 layer-specific；但论文没有规定每个 head 都拥有一套完全独立的 P_h 参数表。

4. LNO 的精确形式

LNO 需要区分两个经常被混淆的东西：

- PhCA encoder/decoder；
- latent Transformer 中的 multi-head self-attention。

论文配置中的 `Heads=8` 主要对应 latent Transformer，并不表示 PhCA 有八套独立空间路由。

输入位置与物理值解耦

$$\begin{aligned}\widehat{X} &= \text{Trunk}(x) \in \mathbb{R}^{N_{\text{in}} \times D}, \\ \widehat{Y} &= \text{Branch}([x, f(x)]) \in \mathbb{R}^{N_{\text{in}} \times D}.\end{aligned}$$

其中：

- $\widehat{X}$ 只包含位置；
- $\widehat{Y}$ 包含位置和物理量。

PhCA encoder

attention-projector 产生

$$L_{\text{in}} = \text{AttnProjector}(\widehat{X}) \in \mathbb{R}^{N_{\text{in}} \times M}.$$

Encoder 在 N_{in} 维，即论文图中的 `axis=0` 上归一化：

$$A_{\text{in}} = \text{softmax}_{N_{\text{in}}}(L_{\text{in}}).$$

然后

$$Z^{(0)} = A_{\text{in}}^{\top}(\widehat{Y}W_V) \in \mathbb{R}^{M \times D}.$$

Latent backbone

$$Z^{(L)} = \mathcal{T}_L \circ \dots \circ \mathcal{T}_1(Z^{(0)}),$$

其中每个 $\mathcal{T}_{\ell}$ 是标准 pre-norm self-attention + MLP residual block。

PhCA decoder

输出坐标经过同一个 trunk projector：

$$P = \text{Trunk}(y) \in \mathbb{R}^{N_{\text{out}} \times D}.$$

产生

$$L_{\text{out}} = \text{AttnProjector}(P) \in \mathbb{R}^{N_{\text{out}} \times M}.$$

Decoder 沿 M 维，即 `axis=1` 归一化：

$$A_{\text{out}} = \text{softmax}_M(L_{\text{out}}).$$

最终：

$$U = A_{\text{out}}(Z^{(L)}W'_V).$$

LNO 共享和不共享什么

LNO 共享：

- 输入、输出坐标使用的 trunk projector；
- PhCA encoder 与 decoder 的 attention-projector，即 $W_1 = W_2$ 。

LNO 不共享：

- encoder value projection W_V ；

- decoder value projection W'_V 。

即使 $x = y$ ，编码和解码矩阵也不是严格转置，因为：

- encoder 沿 N 归一化；
- decoder 沿 M 归一化。

共享的是原始 coordinate score generator，而不是最终路由矩阵。

5. FLARE 的精确形式

FLARE 每层、每个 head 都使用 M 个 latent routes，而不是把总共 M 个 latent 平均分给 heads。

参数形状

整体 learnable query 为

$$Q \in \mathbb{R}^{M \times C},$$

沿 channel 切分后：

$$Q_h \in \mathbb{R}^{M \times d_h}.$$

每个 head 都有 M 个 query 向量，只是每个 query 的 head 子空间维度为 d_h 。

点特征经不同的深层 ResMLP 得到：

$$K = \text{ResMLP}_K(H), \quad V = \text{ResMLP}_V(H).$$

Encoding

$$S_h = Q_h K_h^\top \in \mathbb{R}^{M \times N},$$

$$W_{\text{enc},h} = \text{softmax}_N(S_h),$$

$$Z_h = W_{\text{enc},h} V_h.$$

Decoding

$$W_{\text{dec},h} = \text{softmax}_M(S_h^\top) = \text{softmax}_M(K_h Q_h^\top),$$

$$Y_h = W_{\text{dec},h} Z_h.$$

所以

$$Y_h = W_{\text{dec},h} W_{\text{enc},h} V_h.$$

编码与解码不是严格转置

虽然使用同一个 S_h ，但两边 Softmax 的分母不同，因此：

$$W_{\text{dec},h} \neq W_{\text{enc},h}^\top.$$

更准确的说法是：二者由同一 score 转置后分别归一化，互为“差一个对角缩放的转置”或 near-adjoint。

FLARE 证明该结构的通信矩阵在适当对角相似变换下与半正定矩阵相关，这是其频谱可分析性强于 LinearNO 的重要原因。

二、五个模型正好构成一个设计矩阵

	保留 nonlinear latent mixing	删除 latent mixing
上下行 raw score 绑定	Transolver; LNO	FLARE
上下行独立	LRSA	LinearNO

还要加第三个维度：路由发生频率。

模型	路由频率
Transolver	每个 block 一次
LinearNO	每个 block 一次
LRSA	每个 block 一次
FLARE	每个 block 一次
LNO	整个 operator backbone 只编码、解码一次

这比简单说“它们都采用低秩 latent”更准确。

三、它们低秩的其实不是同一个对象

1. Transolver、LinearNO、FLARE

它们最直接限制的是条件化的点到点通信矩阵：

$$K(H) \in \mathbb{R}^{N \times N}.$$

固定当前输入和 attention weights 后，每个 head 的空间通信路径因经过 M 个通道而满足 rank 上界。

2. LRSA

LRSA 限制的是全局信息必须经 M 个 latent modes 传播：

$$\kappa(x,y;H) \approx \sum_{m,k=1}^M \beta_m(x;H) \, G_{mk}(H) \, \alpha_k(y;H).$$

但由于：

- G 是非线性的；
- reconstruction weights 依赖处理后的 Z' ；
- attention weights 本身依赖输入；

它更适合称为 conditional separable kernel 或 low-rank communication bottleneck，而不是一个固定 rank- M 线性矩阵。

3. LNO

LNO 限制的是整个网络的中间表示空间：

$$\mathbb{R}^{N_{\text{in}} \times D} \rightarrow \mathbb{R}^{M \times D} \rightarrow \mathbb{R}^{N_{\text{out}} \times D}.$$

这是 representation bottleneck。由于中间有深层非线性 Transformer，不能将整个 LNO 简化为单个 rank- M 线性核。

四、为什么整个网络并不是 rank- M

即使单个 head 的非局部更新为

$$K_\ell = U_\ell V_\ell^\top, \quad \text{rank}(K_\ell) \leq M,$$

实际 block 是

$$H_{\ell+1} = H_\ell + K_\ell(H_\ell)H_\ell + \text{FFN}(\cdot).$$

其中：

- $I + K_\ell$ 一般是 full rank；
- U_ℓ, V_ℓ 随输入改变；
- 多个 heads 具有不同的空间矩阵；
- head 输出经过 channel concat 和线性混合；
- 多层 $(I + K_L) \cdots (I + K_1)$ 可以产生远高于单层 M 的有效复杂度；
- Softmax、FFN、LayerNorm 均为非线性。

因此：

LinearNO 和 FLARE 删除了 block 内的 latent self-attention，不等于它们只能表达线性或 rank- M 的 PDE solution operator。

它们把深度移到了“多次重新路由 + 点域 FFN”中。

五、五个模型在哪里放置非线性

模型	路由前	Latent 内	返回点域后
Transolver	线性/可选卷积 projector	self-attention	pointwise FFN
LinearNO	Q/K/V 线性投影 + Softmax	无	pointwise FFN
LRSA	标准 cross-attention	FFN-in + self-attention + FFN-out	pointwise FFN
LNO	trunk/branch MLP	整个深层 Transformer	输出 MLP
FLARE	深层 K/V ResMLP	无	深层 pointwise ResMLP

所以不能只根据“有无 latent attention”判断容量。

FLARE 虽然 $G = I$ ，但它使用：

- 三层 residual K projector；
- 三层 residual V projector；
- 三层 pointwise ResMLP；
- 多个 head-specific low-rank pathways；
- 每层重新计算路由。

LinearNO 的 K/V 路径则明显更浅，主要依靠非对称 U, V 和 block 深度。

六、一次性压缩与逐层低秩校正的根本差别

LNO：一次性压缩

优点：

- $O(NM)$ 成本只在 encoder/decoder 支付；
- 深度 L 主要产生 $O(LM^2)$ 成本；
- 天然支持 $N_{\text{in}} \neq N_{\text{out}}$ ；
- 非常适合稀疏观测、inverse problem、任意坐标查询。

风险：

- 输入在第一步没有被压入 latent 的空间细节，后面无法重新从点域获取；
- 没有贯穿深度的点级 identity path；
- 对高频、小尺度和局部异常结构可能形成不可逆 information bottleneck。

另外四个模型：逐层低秩校正

每层都保持

$$H^{(\ell)} \in \mathbb{R}^{N \times C}.$$

latent 路径只提供全局 correction：

$$H^{(\ell+1)} = H^{(\ell)} + \Delta H^{(\ell)}.$$

即使 M 很小，原有点级细节仍可通过 residual 保留下来。代价是每一层都要支付 $O(NM)$ 。

这是 LNO 和其余四者在高频保持、计算复杂度和深度扩展上的核心权衡。

七、位置、物理量与路由条件

模型	路由由什么决定	同一网格不同样本的路由是否可变
Transolver	当前坐标—物理联合特征	是
LinearNO	当前隐藏特征的独立 Q/K 投影	是
LRSA	当前点特征、learnable queries、处理后 latent	是
LNO	输入/输出坐标	固定网格时基本固定
FLARE	固定 Q_h 与当前特征产生的深层 K_h	是

LNO 的压缩是一种 coordinate-conditioned basis：

- 在固定 Darcy 网格上，不同样本使用相同空间路由；
- 变化的 permeability/solution 值只影响 value；
- 当几何坐标变化时，路由才随几何变化。

而 Transolver、LinearNO、LRSA、FLARE 的路由可以随具体物理场状态变化，因此更接近 input-adaptive kernel。

八、multi-head 的差异

Transolver

- 每个 head 在自己的 channel subspace 内形成 slice 特征；
- 原实现的 slice projection 映射在 heads 间共享；
- 不同 head 的输入子特征不同，所以实际 A_h 仍可以不同。

LinearNO

- 论文明确采用 Multi-Head LinearNO；
- rank 分析也是逐 head 进行；
- 但论文没有充分说明 route projector 是否跨 head 共享，因此不应额外推断。

LRSA

- 使用标准 multi-head SDPA；
- learnable query table $P^{(\ell)}$ 是完整 $M \times C$ 表；
- 各 head 通过 Q/K/V channel projection 得到不同子空间；
- 论文没有把 independent per-head latent array 作为独立贡献。

LNO

- PhCA 是单个全局 coordinate routing matrix；
- Heads=8 主要属于 latent Transformer；
- 不存在 FLARE 式的多套并行空间低秩矩阵。

FLARE

- $Q \in \mathbb{R}^{M \times C}$ 沿通道切成 $Q_h \in \mathbb{R}^{M \times d_h}$ ；
- 每个 head 都有 M 条路线；
- 每个 head 有不同的 $W_h = W_{\text{dec},h} W_{\text{enc},h}$ ；
- concat 后的完整多通道层不能简单称为一个 rank- M 标量矩阵。

九、标准注意力与硬件实现差异

模型	每个 block 的标准 SDPA	自定义/显式路由操作
Transolver	1 次 $M \times M$ latent SDPA	slice、sum-normalization、deslice

模型	每个 block 的标准 SDPA	自定义/显式路由操作
LinearNO	0 次 latent SDPA	两组 feature map Softmax + 两次矩阵乘法
LRSA	2 次 cross-SDPA + 1 次 latent self-SDPA	无特殊聚合
LNO	latent backbone 中有 self-SDPA	全网前后各一次 PhCA
FLARE	恰好 2 次 cross-SDPA/block	无

LRSA 和 FLARE 可以让 fused SDPA kernel 在计算时不显式保存完整 $N \times M$ attention matrix；但这取决于 dtype 和后端：

- LRSA 论文的 FlashAttention-2 优势主要出现在 FP16/BF16；
- FP32 下 PyTorch 可能退回 cuDNN/math SDPA；
- FLARE 的无融合等价实现仍会显式产生 $M \times N$ 权重。

FLARE 的 K/V ResMLP 和 pointwise ResMLP 都是 $O(N)$ ，当 N 很大时，它们也可能成为实际延迟瓶颈，不能只看 attention 的 $O(NM)$ 。

十、规则网格、点云与任意输出查询

能力	Transolver	LinearNO	LRSA	LNO	FLARE
点集排列等变	pointwise 版本是	是	是	是	是
不要求 mesh edges	是	是	是	是	是
显式坐标—数值解耦	否	基础版否	否	是	否
原生 $N_{\text{in}} \neq N_{\text{out}}$	否	需 Cross-LinearNO	可在算子层扩展，但默认不充分	是	否
可选空间局部卷积	有	实验中有	无	无	无

LRSA 理论上将 reconstruction 写成任意位置 x 上的连续展开，也实验了不同分辨率；但其重建 query 是 $h(x)W_Q^\top$ ，论文没有像 LNO 那样给出完整、独立的 coordinate-only output decoder。因此：

- 不能简单说 LRSA 完全不能处理新查询点；
- 也不能说它与 LNO 一样原生支持任意输入—输出位置解耦。

十一、它们对非均匀采样都存在一个共同细节

这些模型大多使用 Softmax 将离散点视为经验测度样本，但没有显式加入：

- 单元面积；
- 网格体积；
- quadrature weights；
- 局部采样密度修正。

因此在非均匀网格上，“排列等变”不等于“离散化不变”。

理论条件也不同：

- LRSA 的连续解释明确假设 uniform sampling 和简单 quadrature；
- LinearNO 的定理建立在从测度 μ_Ω 抽样及有界性条件上；
- Transolver 的证明依赖 slice/domain 映射、平滑性和微分同胚等较强假设；
- LNO 的任意坐标查询能力并不自动保证跨任意采样测度稳定；
- FLARE 主要证明显式低秩通信和频谱性质，没有提供一般非均匀离散收敛保证。

这是比较“低秩空间神经算子”时不能忽略的一层。

十二、四组最容易混淆的关系

LinearNO 不是 FLARE 的非融合版本

共同点：

- 都删除 latent self-attention;
- 都有 rank- M 的双因子空间通信。

区别:

- LinearNO: 独立 U, E , 浅 Q/K/V 投影;
- FLARE: 共享原始 score, 深 K/V ResMLP, head-wise 路径, 双 SDPA;
- LinearNO 追求分析/合成基解耦;
- FLARE 追求 near-adjoint 稳定路由。

LRSA 不是“每层执行一次 LNO”

LNO:

- coordinate-only route;
- encoder/decoder projector 共享;
- 整个网络一次压缩;
- latent 持续存在。

LRSA:

- feature-conditioned route;
- 上下行完全独立;
- 每层重新压缩;
- latent 只在当前 block 存在;
- 点域 residual 贯穿全网。

FLARE 不是“Transolver 删除 token attention”

它还同时改变了:

- 手工 slice aggregation $\rightarrow$ 标准 SDPA;
- 浅层投影 $\rightarrow$ 深 K/V ResMLP;
- 共享式 head 结构 $\rightarrow$ 强调 head-wise latent diversity;
- 普通 attention scaling $\rightarrow$ scale=1 ;
- 物理 slice 解释 $\rightarrow$ 显式低秩 routing operator 解释。

LinearNO 不是整体线性神经算子

虽然

$$U(EV)$$

在固定 U, E 时是线性的, 但 U, E 都由输入经过 Softmax 动态产生, 外层还有 FFN、LayerNorm 和 residual。因此 LinearNO 仍然是非线性、输入条件化的神经算子。

十三、从数值方法视角理解上下行关系

这是一个有价值但属于进一步推论的解释。

- Transolver、LNO、FLARE 的 score-tied 结构类似使用相关的分析基与合成基, 接近 Galerkin 式“同源试验/检验空间”;
- LinearNO、LRSA 的独立上下行更接近 Petrov-Galerkin 或 biorthogonal analysis/synthesis;
- score-tied 通常更稳定、更容易做频谱分析;
- 独立上下行更容易表示非对称、方向性和 non-normal operator, 例如对流占优问题。

其中 FLARE 的近伴随结构尤其偏向稳定的 gather-scatter; LinearNO/LRSA 则允许一个 latent mode 以一种方式接收信息, 再以另一种方式传播信息。

不过, 这只是结构对应关系, 不能直接宣称这些模型已经满足 Galerkin 正交性或数值稳定性条件。

十四、论文实际配置也揭示了容量分配差异

模型	典型标准任务配置
Transolver	8 blocks; $C = 128, M = 64$, 高维输入常用 $C = 256, M = 32$
LinearNO	通常沿用 8 blocks、8 heads、 $C = 128/256, M = 64/32$
LRSA	标准任务多为 8 blocks; $C = 128/256, M = 32/64$
LNO	Darcy 4 latent blocks, 其余多为 8; latent size 通常 $M = 256$
FLARE	8 blocks、 $C = 64; M = 64/128/256$, 8 或 16 heads

因此原论文误差不能直接当作纯架构控制实验。例如 FLARE 在 Darcy/Airfoil 上通常使用 $M = 256$ ，明显高于 Transolver、LinearNO、LRSA 常用的 $M = 32/64$ ；同时它的 $C = 64$ 又更小。比较时需要同时控制：

M, C, n_h, d_h , block数, MLP深度和参数量.

最终完整定位

如果将五个模型放在同一设计空间中：

- **Transolver**：逐层、feature-conditioned、score-tied、带 latent attention 的物理 slice operator。
- **LinearNO**：逐层、feature-conditioned、上下行独立、无 latent mixing 的线性复杂度 operator。
- **LRSA**：逐层、feature-conditioned、上下行完全独立、带强 nonlinear latent processing 的标准 SDPA operator。
- **LNO**：一次性、coordinate-conditioned、score-tied、整个主干位于 latent 空间的 encoder-processor-decoder。
- **FLARE**：逐层、feature-conditioned、score-tied、无 latent mixing、head-wise 并行且完全由双 SDPA 实现的 routing operator。

最本质的区别可以归结为五个正交问题：

低秩发生在单层通信还是全网表示？
分析基与合成基是否共享 score？
latent 内是否执行非线性算子？
路由由坐标还是完整物理状态决定？
表达能力放在 latent、point FFN，还是多 head 路径中？

核对来源：[Transolver](#)、[LinearNO](#)、[LRSA](#)、[LNO](#)、[FLARE](#)。

- :codex-followup[比较发表顺序]{prompt="继续整理这7篇论文的首次公开时间、正式发表时间和相互引用关系，并形成低秩空间神经算子的完整发展顺序。"}
- :codex-followup[统一张量演示]{prompt="选择Airfoil或Darcy的具体输入张量，用统一的N、C、M和head设置逐步演示五个模型每一层的张量变化与计算量。"}
- :codex-followup[扩展低秩文献]{prompt="补充阅读更多低秩空间神经算子、Nyström attention、显式SVD核和latent operator论文，并构建完整的技术演化思考链。"}

分析以下问题：问题1：linearno和lrssa都将压缩和重建解耦，但是linearno中使用线性连接而非mixing attention效果更好，但lrssa就发现使用 mixing attention更好。问题2：linearno将压缩与重建解耦，才将transolver的中间的mixing attention换为线性映射，会不会之所以可以使用线性映射，是因为mixing attention换来的表达能力，如果transolver直接将mixing attention换为线性映射一样可以优于原始的transolver吗；问题3：LNO为什么相较于这里的其他模型还可以处理反问题；问题4：这些低秩空间神经算子都或多或少做了一些简化处理（比如：全部都压缩到低秩空间，有的将attention换为线性映射，有的中间不映回原来的点保持再latent空

我会把这四个问题放到同一个因果框架里分析，而不是分别复述论文结论：先区分“上下行路由表达力”“latent 内部 mixing 表达力”“跨层重复路由表达力”和“点域 residual 保真能力”，再判断各论文的消融究竟控制了哪些变量。尤其问题 1、2 不能只比较“有/无 attention”，因为 LinearNO 与 LRSA 同时改变了压缩方式、重建方式、归一化、latent 非线性和硬件实现。接下来我按 PDF 原文的消融与公式逐项核对。

这四个问题可以用一个共同结论串起来：

这些模型的“简化”不是单纯删除表达能力，而是在重新分配表达能力：有的把能力放在独立的压缩/重建基，有的放在 latent mixing，有的放在逐层重新路由，有的放在深 K/V MLP、多头和点域 residual 中。

其中最需要先澄清的是：LinearNO 并不是用“一个可学习线性层”替换 Transolver 的 mixing attention，而是直接令中间算子

$$G = I,$$

即取消 latent-to-latent mixing。LinearNO 仍然是非线性模型，因为路由矩阵由输入经过 Softmax 动态产生。

问题 1：为什么 LinearNO 删除 mixing 更好，而 LRSA 保留 mixing 更好？

1. 两篇论文的“无 mixing”并不是同一个消融

LinearNO

LinearNO 的完整计算是

$$Y = U(H)[E(H)V(H)],$$

其中

$$U(H) = \text{softmax}_M(HW_Q),$$
$$E(H) = \text{softmax}_N((HW_K)^\top).$$

中间没有任何 latent 处理：

$$G = I.$$

论文在消融中加回的是 Transolver-style slice self-attention，即比较

$$UEV \text{ 与 } UG(EV).$$

LRSA

LRSA 的完整 latent processing 是残差 Transformer：

$$\tilde{Z} = Z + \text{FFN}_{\text{in}}(\text{Norm}(Z)),$$
$$\hat{Z} = \tilde{Z} + \text{SelfAttn}(\text{Norm}(\tilde{Z})),$$
$$Z' = \hat{Z} + \text{FFN}_{\text{out}}(\text{Norm}(\hat{Z})).$$

LRSA 的 w/o Intra Attn 不是令 $G = I$ ，而是用 MLP 替换 latent processing block。因此它仍保留逐 latent 的非线性 channel mixing，只删除不同 latent token 之间的 self-attention。

所以两篇论文实际比较的是：

论文	有 mixing	无 mixing
LinearNO	非残差 Transolver-style slice attention	恒等映射 I
LRSA	残差 FFN + self-attention + FFN	仍保留 MLP，只删除跨 latent attention

这两个实验不能直接视为同一变量上的相反结论。

2. LinearNO 的独立 U, E 已经提供了跨区域通信

Transolver 在没有 latent attention 时，理想化空间矩阵为

$$K_{\text{tied}} = A D_A^{-1} A^\top.$$

其中 $A \in \mathbb{R}^{N \times M}$ 同时负责：

- 输入点写入哪个 slice；
- 输出点从哪个 slice 读取。

于是

$$K_{ij} = \sum_{m=1}^M \frac{A_{im} A_{jm}}{c_m}.$$

两个点只有在具有相似 slice membership 时，才能发生较强通信。这是一种非常受限的“同组相似性核”。

Transolver 的 latent attention

$$G \in \mathbb{R}^{M \times M}$$

主要是为了允许：

$$\text{slice } m \longrightarrow \text{slice } n.$$

因此 Transolver 的完整核近似为

$$K_{\text{Transolver}} = A G D_A^{-1} A^\top.$$

而 LinearNO 直接使用两套不同的空间因子：

$$K_{\text{LinearNO}} = U(H)E(H).$$

其中：

- E_{mj} ：输入点 j 如何写入 mode m ；
- U_{im} ：输出点 i 如何读取 mode m 。

由于 U 与 E 独立，一个输入点可以按一种方式被压缩，输出点再按完全不同的方式读取。即使 $G = I$ ，也已经能够形成全局、非对称、方向相关的点间通信。

换句话说：

Transolver 用 G 弥补“分析基和合成基被绑定”的限制；LinearNO 直接解除这个限制，因此 G 的一部分作用变得冗余。

3. LRSA 的 reconstruction 本身依赖处理后的 latent

LinearNO 的重建矩阵

$$U(H) = \text{softmax}(H W_Q)$$

在进入 latent 之前就已经由点特征决定。中间的 G 主要改变 latent values，不会重新决定输出点如何读取这些 values。

LRSA 则不同：

$$U(H, Z') = \text{softmax}\left(\frac{(H W_Q^\uparrow)(Z' W_K^\uparrow)^\top}{\sqrt{d_h}}\right).$$

这里 Z' 不仅是 decoder 的 value，也是 decoder 的 key。因此 latent self-attention 会同时改变：

1. latent 中携带的物理内容；
2. latent 之间的关系；
3. 每个点应该从哪些 latent 读取；
4. reconstruction basis 本身。

所以在 LRSA 中， G 的作用不只是“再混一次 token”，而是参与生成新的解码路由。这使它比 LinearNO 中插入的 slice attention 更有价值。

4. LRSA 的 latent attention 更容易退化成“什么也不做”

LRSA 使用 residual：

$$Z \leftarrow Z + \text{SelfAttn}(\text{LN}(Z)).$$

如果某个任务不需要 latent mixing，模型可以把 attention 输出压小，使其近似恒等映射。

Transolver 的原始形式则是：

$$Z' = \text{Attention}(Z),$$

没有显式的 latent identity residual。即使 attention 没有学到有价值的 mode interaction，所有 latent values 仍要经过该 attention。这更容易产生：

- 过度平均；
- mode contamination；
- slice 之间的无效传播；
- 有效秩下降；
- 优化噪声。

因此“同样叫 latent attention”，它们在优化上并不等价。

5. LRSA 的实验显示 mixing 的收益主要出现在复杂动态中

LRSA 的消融结果为：

数据集	Full LRSA	去掉 latent self-attention
Airfoil	0.0041	0.0043
Darcy	0.0043	0.0045
Elasticity	0.0033	0.0036
Navier-Stokes	0.0484	0.0544

可以看出：

- 静态问题上的差异较小；
- Navier-Stokes 上差异更明显。

这支持一种更细致的判断：

- 对平滑、稳态、低有效秩问题，独立 U, E 加逐层重路由可能已经够用；
- 对时间演化、模式耦合、涡结构相互作用，显式 latent mode interaction 更有价值。

但还不能把它当作普遍定律，因为两篇论文的：

- backbone；
- residual；
- latent MLP；
- rank M ；
- loss；-参数量；
- 训练随机种子统计

都没有完全对齐。LRSA 的组件消融也没有明确做严格参数量匹配。

问题 1 的最终结论

LinearNO 与 LRSA 并不矛盾：

LinearNO：独立 U, E 已承担主要空间交互

LRSA： G 还会改变 latent 内容和解码路由

此外，LRSA 使用可退化的残差式 mixing，LinearNO 对比的是非残差 Transolver-style mixing。

问题 2：LinearNO 是否因为解耦带来了表达能力，才可以删除 mixing？

答案是：是的，这基本符合 LinearNO 的结构逻辑和消融结果，但只是替代了 mixing 的一部分功能，不是数学上完全等价。

1. LinearNO 已经直接测试了这个问题

LinearNO 的两个消融开关是：

- Gen.：是否将压缩与重建解耦；
- Sim.：是否删除 slice attention，即令 $G = I$ 。

六个数据集结果如下：

路由	Latent G	Airfoil	Pipe	Plasticity	NS	Darcy	Elasticity
绑定	Attention	0.0067	0.0029	0.0016	0.1261	0.0064	0.0069
解耦	Attention	0.0054	0.0032	0.0013	0.0711	0.0061	0.0062
绑定	I	0.0071	0.0030	0.0015	0.1229	0.0052	0.0064
解耦	I	0.0049	0.0024	0.0011	0.0699	0.0050	0.0050

最后一行就是 LinearNO。

需要注意，论文中“绑定”消融的归一化写法与原始 Transolver 不一定完全 bit-exact，因此最好视为对称路由近似；但整体趋势仍有意义。

2. Transolver 直接删除 mixing 能不能优于原始 Transolver？

比较第一行与第三行：

数据集	绑定 + Attention	绑定 + 无 Attention	结果
Airfoil	0.0067	0.0071	变差
Pipe	0.0029	0.0030	变差
Plasticity	0.0016	0.0015	改善
NS	0.1261	0.1229	改善
Darcy	0.0064	0.0052	明显改善
Elasticity	0.0069	0.0064	改善

也就是说：

直接删除 mixing：六个任务中 4 个改善、2 个变差

所以答案不是“不能”，而是：

Transolver 直接删除 slice attention 确实有可能优于原始 Transolver，但不稳定，也不能在所有任务上成立。

Airfoil 很可能需要不同物理区域之间的交互，而绑定 route 在没有 G 时只能形成同源 slice similarity kernel，因此性能下降。

3. 为什么解耦后删除 mixing 会更稳定？

比较第二行与第四行：

数据集	解耦 + Attention	解耦 + I
Airfoil	0.0054	0.0049
Pipe	0.0032	0.0024
Plasticity	0.0013	0.0011
NS	0.0711	0.0699
Darcy	0.0061	0.0050
Elasticity	0.0062	0.0050

解耦后删除 mixing 在六个任务上都改善。

原因是解耦把表达能力从

$$G: \mathbb{R}^{M \times d} \rightarrow \mathbb{R}^{M \times d}$$

转移到了分析基和合成基：

$$E(H) \neq U(H)^\top.$$

绑定但无 mixing

$$K = AD^{-1}A^\top$$

近似是一个对称、半正定、同源分析/合成核。

绑定且有 mixing

$$K = AGD^{-1}A^\top.$$

G 可以在已有 latent 子空间内旋转和混合，但左右空间仍然由同一个 A 限制。

解耦但无 mixing

$$K = UE.$$

此时左右子空间独立，可以表达：

- 非对称传播；
- 上游到下游的方向性；
- 不同输入区域和输出区域的对应；
- non-normal operator；
- 不同的分析基和合成基。

这类自由度往往比在同一个受限子空间里加入 $M \times M$ mixing 更重要。

4. 解耦不能理论上完全替代 nonlinear G

若 $G = W_M$ 只是固定线性矩阵，在不考虑 Softmax 约束时：

$$UW_ME$$

可以把 W_M 吸收到 U 或 E 中，因此可能冗余。

但实际模型中：

- U, E 非负；
- U, E 经过不同方向的 Softmax；
- 每行有归一化约束；
- $G(Z)$ 可以是输入相关的 nonlinear self-attention。

所以 nonlinear G 一般不能严格被 U, E 吸收。这正是 LRSA 在复杂动态任务上仍从 G 获益的原因。

另外，如果你所说的“换成线性映射”是指插入一个可学习的

$$W_M \in \mathbb{R}^{M \times M},$$

那么这并不是 LinearNO 已经测试的结构。LinearNO 测的是

$$G = I,$$

而不是

$$G = W_M.$$

“绑定/解耦 $\times I$ /静态线性 W_M /self-attention”应当被当成新的独立消融。

问题 3：LNO 为什么更适合处理反问题？

核心原因不是“它一直在 latent 空间里计算”，而是：

LNO 显式解耦了观测位置、观测值和输出查询位置

1. LNO 的输入端

对于稀疏观测

$$\{(x_i, u(x_i))\}_{i=1}^{N_{\text{in}}},$$

LNO 分别计算：

$$\begin{aligned}\widehat{X} &= \text{Trunk}(x) \in \mathbb{R}^{N_{\text{in}} \times D}, \\ \widehat{Y} &= \text{Branch}([x, u(x)]) \in \mathbb{R}^{N_{\text{in}} \times D}.\end{aligned}$$

其中：

- $\widehat{X}$ 只用于决定在哪里采集信息；
- $\widehat{Y}$ 携带实际观测值。

编码为：

$$Z^{(0)} = A_{\text{enc}}(x)^\top \widehat{Y} W_V \in \mathbb{R}^{M \times D}.$$

无论观测点有多少、位于哪里，都被映射成固定 M 个 latent。

2. LNO 的输出端只需要查询坐标

给定任意输出位置

$$y_j, \quad j = 1, \dots, N_{\text{out}},$$

只需：

$$\begin{aligned}P &= \text{Trunk}(y), \\ \widehat{u}(y) &= \text{MLP} \left[A_{\text{dec}}(y) Z^{(L)} W'_V \right].\end{aligned}$$

它不要求：

$$N_{\text{out}} = N_{\text{in}}, \quad y = x, \quad u(y) \text{ 已知}.$$

因此可以自然地学习：

稀疏观测集合 $\longrightarrow$ 另一组位置上的未知函数.

3. Forward 和 inverse 在 LNO 中只是函数语义的交换

任务	输入函数	输出函数
Forward	系数场、初值、边界条件	PDE 解
初值反演	后续时刻的部分解	初始场
系数反演	部分解观测	系数场
场重建	稀疏或局部解	完整解场

LNO 不需要数学上反转某个已训练好的 forward network，而是针对 inverse training pairs 重新学习一个监督映射。

4. LNO 论文实际验证的“反问题”范围需要准确表述

论文真正实验的是 Burgers 方程上的：

局部时空域中的稀疏解观测 → 完整时空解场.

采用两阶段：

- Completer：稀疏观测 → 中间时间窗内的稠密解；
- Propagator：中间时间窗解 → 整个时间域解。

它没有实际验证：

- 未知黏性系数识别；
- 未知 PDE 系数场恢复；
- 方程形式发现；
- 多解后验分布反演。

所以更准确地说，它验证的是：

稀疏时空观测下的场补全和双向时间外推，属于广义 inverse/state-reconstruction task。

5. 其他模型不是理论上不能做反问题

它们只是基础架构不够方便。

- Transolver 的 deslice 使用输入点产生的 A ，默认只能回到原输入点；
- LinearNO 的 $U(H)$ 由当前输入点特征产生，默认输出仍是同一组点；
- LRSA 的 reconstruction query 和 residual 依赖已有的 $H(y)$ ；
- FLARE decoder 使用同一组输入点的 $K(H)$ 。

但经过改造都可以做：

- LinearNO 已在附录提出 Cross-LinearNO；
- LRSA 可把 up-query 改成 coordinate-only query；
- FLARE 可使用 K_{in} 编码、 $K_{\text{out}}(y)$ 解码；
- Transolver 可分别学习 A_{in} 与 A_{out} 。

因此：

LNO 独占的是原生 observation-query decoupling，而不是反问题能力本身

最后，LNO 也不会自动解决反问题的不可适定性。如果不同物理参数产生相同稀疏观测，MSE 训练的模型通常只能学习训练分布下的条件均值，而不是唯一真实解。

问题 4：这些模型的表达能力究竟来自哪里？

可以用一个统一 block 表示：

$$Z_h = E_h(H) V_h(H),$$

$$\tilde{Z}_h = G_h(Z_h),$$

$$\Delta H = W_O \text{Concat}_h \left[D_h(H, \tilde{Z}_h) \tilde{Z}_h \right],$$

$$\widehat{H} = H + \Delta H,$$

$$H' = \widehat{H} + \text{FFN}(\text{Norm}(\widehat{H})).$$

表达能力来自下面每一个环节。

1. 原始输入信息

坐标、时间、初边值、系数场、雷诺数、攻角、SDF、法向量和边界标签决定问题是否可辨识。

如果输入没有包含区分两种解所需的信息，后面的 attention 再强也无法恢复。

2. Lift/input projector

$$h_i^{(0)} = \text{Lift}([x_i, a_i, \dots]).$$

它负责：

- 将不同物理量映射到共同通道空间；
- 形成非线性局部物理特征；
- 为不同 heads 提供不同表示子空间；
- 在第一次低秩压缩前选择和组合信息。

但 pointwise Lift 只做通道变换，不会看邻居。

各模型差异：

- Transolver：较浅的 linear lift；
- LinearNO：基本继承 Transolver；
- LRSA：FFN Lift，并可加入 Fourier positional encoding；
- LNO：coordinate trunk 与 coordinate-value branch 两套 MLP；
- FLARE：两层 ResMLP input projector。

3. 坐标编码和局部几何先验

LRSA 的 positional encoding 可增强坐标高频表示。

LNO 的 trunk projector 产生连续坐标 query，是其任意位置解码的主要来源。

Transolver 在结构网格上可使用 3×3 convolution，提供：

- 局部邻域；
- 梯度和高频结构；
- 局部平移先验。

LRSA、LNO、FLARE 的核心模型没有显式空间局部模块。它们的 pointwise MLP 不能替代真正的邻域卷积或图消息传递。

4. 路由 score：决定谁与谁通信

- Transolver：当前点特征产生 slice assignment；
- LinearNO：当前点特征独立产生 U, E ；
- LRSA：learnable queries 与当前点特征匹配，重建还依赖处理后 latent；
- LNO：路由只由坐标决定；
- FLARE：固定 Q_h 与深层 $K_h(H)$ 匹配。

即使 $G = I$ ，LinearNO 和 FLARE 的核仍随样本、层和当前隐藏状态变化。

5. Softmax

Softmax 提供：

- 非线性 gating；
- 不同点或 latent 之间的竞争；
- 归一化；
- 对输入点数变化的幅值稳定性。

同时带来限制：

- 空间权重非负；
- 一次聚合通常是凸组合；
- 可能出现 uniform routing 或 route collapse；
- 非均匀网格上没有显式 quadrature weight。

带符号的物理核主要由 value projection、head output projection 和多层组合产生。

6. Value projection

路由只决定“从哪里读取”，value 决定“传递什么”。

它可以：

- 重组压力、速度、几何等物理量；
- 形成正负 channel；
- 在压缩前过滤无关信息；
- 让相同空间路由承载完全不同的物理内容。

FLARE 特别依赖深层 V -ResMLP；LinearNO 的 V 相对较浅。

7. Latent rank M

M 决定一个 head 单层能并行维持多少全局空间模式。

- M 太小：边界层、激波、多涡结构可能被合并；
- M 增大：空间模式容量提高；
- 路由塌缩时，名义 M 很大但有效秩仍很低。

LinearNO 通过独立 U, E 提高 mode 利用率；FLARE 通过 head-wise routes 获得多套低秩子空间；LRSA 同时使用独立上下行和 latent mixing。

8. Latent mixing G

G 决定已经提取的全局模式能否在返回点域前相互作用。

- Transolver：一次 slice self-attention；
- LRSA：残差 FFN + self-attention + FFN；
- LNO：整个深层 Transformer；
- LinearNO、FLARE： $G = I$ 。

Self-attention 主要增加：

- 输入相关的 $M \times M$ 非对角 mode coupling；
- 不同区域或物理状态之间的条件交互；
- 在一个 block 内表达复杂非线性全局变换的能力。

9. Reconstruction

重建不是无关紧要的“还原形状”，而是决定每个输出点如何解释 latent modes。

- Transolver：复用 slice assignment，稳定但受限；
- LinearNO：独立重建基，能够表达非对称传播；
- LRSA：重建基还依赖处理后的 Z' ，动态性最强；

- LNO：输出坐标决定重建，可查询新位置；
- FLARE：沿同一 score 的反方向 Softmax，形成 near-adjoint gather-scatter。

10. Multi-head 与输出投影

不同 heads 可以学习：

- 不同空间尺度；
- 不同边界区域；
- 不同物理变量；
- 不同传播方向；
- 不同低秩子空间。

之后的 W_O 会将各 heads 带符号地重新组合。

因此不能把完整 multi-head block 简化为一个 rank- M 标量矩阵。

11. Pointwise FFN/ResMLP

FFN 不做空间传播，但负责：

- 非线性本构关系；
- channel mixing；
- 将全局信息与当前点状态融合；
- 为下一层产生新的 Q/K/V；
- 构造高阶非线性组合。

LinearNO 删除 latent attention 后，表达能力很大一部分来自：

动态低秩传播 $\rightarrow$ point FFN $\rightarrow$ 再次动态低秩传播.

FLARE 的补偿更明显：

- K projector：深 ResMLP；
- V projector：深 ResMLP；
- block 后：深 ResMLP；
- 输入、输出端：ResMLP。

12. Residual connection

对于 Transolver、LinearNO、LRSA、FLARE：

$$H_{\ell+1} \approx (I + K_{\ell})H_{\ell}.$$

即使 K_{ℓ} 是低秩的，

$$I + K_{\ell}$$

一般仍是 full rank。Residual：

- 保留完整点级细节；
- 允许低秩支路只学习全局 correction；
- 防止所有信息永久压入 M 个 modes。

LNO 没有贯穿深层的点域 residual，因此第一次压缩没保留的信息，后面无法重新访问。

13. 深度和逐层重新路由

多层并不是重复同一个低秩矩阵：

$$H^{(L)} = (I + K_{L-1}) \cdots (I + K_0)H^{(0)}.$$

每一层：

- 使用不同参数；
- 根据新 $H^{(\ell)}$ 重新计算路由；
- 在前层全局信息基础上再次划分空间模式。

因此多次不同方向的低秩修正，可以产生远高于单层 rank M 的有效复杂度。

14. LayerNorm

LayerNorm 不增加空间通信，但决定容量是否能被训练出来：

- 控制 attention logits；
- 稳定不同物理量尺度；
- 改善深层梯度；
- pre-norm 使 residual 保留未经归一化的原始路径。

它提升的是 effective/trainable expressivity，而不是理论函数类。

15. 输出 projector/decoder

普通 pointwise output MLP/Linear：

- 将隐藏通道映射为压力、速度、应力；
- 组合不同 heads；
- 产生带符号输出；
- 不新增空间通信。

LNO decoder 不同，它同时承担：

- 从 latent 到输出坐标的空间读取；
- 任意位置插值；
- 最终物理量投影。

16. 损失函数

损失不扩大严格 hypothesis class，但决定模型使用哪部分容量。

- Relative L_2 ：更关注整体相对误差，容易由大能量区域主导；
- gradient loss：推动高频、梯度和界面恢复；
- surface/volume 加权：避免表面区域被大量体积点淹没；
- rollout loss：推动动态稳定性；
- LNO inverse MSE：在多解情况下倾向输出条件均值。

所以不能把不同论文的误差差异全部归因于 mixing attention。

五个模型的表达能力账本

模型	主要表达能力来源	主动限制/删除	补偿机制
Transo lver	动态 slices、latent attention、多头、点 FF N、逐层重路由、可选卷积	上下行 score 绑定，rank- M 路径	G 的跨 slice mixing、卷积、 点域 residual
Linear NO	独立 U, E 、动态 Softmax、多头、点 FFN、 逐层重路由	删除 G ，Q/K/V 较浅	非对称分析/合成基、更高有 效秩、深度
LRSA	独立双 cross-attention、强 latent Transforme r、动态 decoder、PE、点 FFN	仍有 rank- M 瓶颈，无显式局部模块	单 block 内最强的 mode coup ling
LNO	trunk/branch MLP、深 latent Transformer、c oordinate decoder	一次性压缩，无深层点域 residual，固定网 格时路由不随物理样本变化	较大 M 、深 latent backbon e、任意坐标 decoder
FLARE	深 K/V ResMLP、head-wise routes、重复 gat her-scatter、深 point ResMLP	$G = I$ 、固定 Q 、上下行 score 绑定	多头多样性、深 K/V、逐层重 新路由

最终不能用单一顺序排列五者的表达能力。更合理的比较量是：

$M \times \text{路由动态性} \times \text{上下行自由度} \times \text{latent mixing} \times \text{head diversity} \times \text{channel nonlinear capacity} \times \text{depth} \times \text{local prior}$

LinearNO 和 FLARE说明：删除形式上的模块，可能通过减少冗余、过度平滑和优化困难，提高有效表达能力与泛化。

LRSA说明：当任务确实存在强烈的 mode-to-mode nonlinear coupling 时，保留设计良好的残差 latent mixing 仍然有价值。

LNO则说明：表达能力还可以换成输入输出域的灵活性，即牺牲持续点域表示，换取任意 observation-to-query operator。

依据：[LinearNO](#)、[LRSA](#)、[Transolver](#)、[LNO](#)、[FLARE](#)。

- :codex-followup[设计因果消融]{prompt="基于上述分析设计一个严格控制参数量的2×4因果消融：绑定/解耦路由 × identity/静态线性/MLP/self-attention latent mixing，并给出实现和实验配置。"}
- :codex-followup[提出新型模块]{prompt="综合LinearNO、LRSA和FLARE的优点，设计一种按任务或样本自适应选择latent mixing强度的新低秩空间神经算子。"}
- :codex-followup[扩展反问题架构]{prompt="将Transolver、LinearNO、LRSA和FLARE分别改造成像LNO一样支持稀疏观测到任意输出查询位置的反问题架构，并比较改造难度。"}

1. LinearNO 已在附录提出 Cross-LinearNO；
2. LRSA 可把 up-query 改成 coordinate-only query；
3. FLARE 可使用 K_{in} 编码、 $K_{\mathrm{out}}(y)$ 解码；
4. Transolver 可分别学习 A_{in} 与 A_{out} 。这些丢失论文中已经展示出的改变方式吗，还是你设计的



我会回到四篇原文逐项核对公式、附录与实验设置，明确标成“论文已实现/论文只提供了组件基础/我提出的外推设计”。其中 Cross-LinearNO 确实来自论文；另外三项我先不凭记忆下结论。PDF 技能用于精确核页与公式。

目前可以先明确一半：第 1 条是 LinearNO 附录 E 的正式结构并有 Burgers 稀疏重建实验；第 3 条不是 FLARE 原文结构。FLARE 原文编码、解码复用同一个 $K(X)$ ，把信息广播回原输入 token； $K_{\mathrm{in}}/K_{\mathrm{out}}(y)$ 是我根据其 gather-scatter 形式提出的跨点集推广。其余两条正在做同样的逐式核对。

还有一个需要避免“一刀切”的细节：第 4 条如果只指“把 Transolver 的压缩/重建权重解耦”，LinearNO 主文的 generalization step 已经做过；如果指为不同输入/输出点集构造 $A_{\mathrm{in}}(X_{\mathrm{in}})$ 与 coordinate-only $A_{\mathrm{out}}(Y)$ ，那仍是我的跨网格设计。下面我会按这两个层次给出归属。

结论：不是四项都由论文展示。若我上次的表述让人感觉它们都已被作者实现、验证，那是不严谨的。准确归属如下。

改变方式	准确归属	论文实际做到的范围
1. Cross-LinearNO	论文正式提出并实验验证	LinearNO 附录 E
2. LRSA 使用 coordinate-only up-query	我的架构扩展建议	LRSA 只给出 feature-dependent up-query
3. FLARE 使用 K_{in} 编码、 $K_{\mathrm{out}}(y)$ 解码	我的跨点集扩展建议	FLARE 原文编码/解码复用同一个 $K(X)$
4. Transolver 分别学习 $A_{\mathrm{in}}, A_{\mathrm{out}}$	需要分两层理解	同点集解耦已在 LinearNO 中消融；跨点集版本是我的设计

1. Cross-LinearNO：论文已有

这是 [LinearNO](#) 附录 E（PDF 第 13–15 页）正式提出的：

- 输入坐标 X_K 与输出查询坐标 X_N 分开；
- 输入函数只定义在稀疏采样点 X_K ；
- 查询侧 H_N 只由坐标投影得到；
- Figure 5 给出完整网络；

- Eq. (30) 定义 Cross-LinearNO:

$$\text{CrossLinearNO}(H_N, H_K^{(L)}) = \phi_Q(H_N) \left[\psi_K(H_K^{(L)})^\top V(H_K^{(L)}) \right].$$

- Table 14 在 Burgers 的 20%, 10%, 5%, 1%, 0.5% 稀疏采样下进行了重建实验。

但这项实验严格说是

$$X_K \subseteq X_N, \quad \text{稀疏场观测} \rightarrow \text{完整场重建},$$

证明的是输入点与查询点解耦、稀疏重建和超分辨率能力，并没有验证未知 PDE 系数识别、源项反演等所有类型的反问题。

2. LRSA 的 coordinate-only up-query: 是我的设计建议

原始 LRSA 并不是 coordinate-only reconstruction。

PDF 第 3 页 Eq. (1):

$$h_i^{(0)} = \text{Lift}([f_i, \text{PE}(x_i)]),$$

因此点状态 H 同时包含坐标和输入物理量。第 5 页 Eq. (10) 的重建为:

$$\Delta H = \text{Attn}(HW_Q^\uparrow, Z'W_K^\uparrow, Z'W_V^\uparrow), \quad H' = H + \Delta H.$$

原版 up-query 是

$$Q^\uparrow = HW_Q^\uparrow,$$

并非只从输出坐标 y 产生。

论文确实留下了一个容易混淆的表述:

- 问题设置中写输出可以位于“same points (or query points)”;
- 附录 A 第 12 页 Eq. (15) 说可在任意连续位置 x 重建。

但同一公式仍定义:

$$q^\uparrow(x) = h(x)W_Q^\uparrow.$$

也就是说, 它要求查询位置已有 $h(x)$ 。对于“只知道稀疏输入点的观测、希望在新位置 y 输出”的问题, 未知位置并没有完整的 $h(y)$ 。

把它改为

$$Q_{\text{out}}(y) = \rho_{\text{out}}(\text{PE}(y))$$

是我根据 LRSA 的独立 up-cross-attention 提出的扩展。与此同时还必须取消或重新定义原来的 $H + \Delta H$ 残差, 因为新输出点上不存在原始点状态 $H(y)$ 。

所以准确说法是:

LRSA 的结构使 coordinate-only decoder 很自然, 但 LRSA 论文没有实现或验证这个版本。

3. FLARE 的 $K_{\text{in}}/K_{\text{out}}(y)$: 是我的设计建议

原始 FLARE 第 4 页 Eq. (5)-(9) 使用:

$$K = K(X), \quad V = V(X), \quad Q \in \mathbb{R}^{M \times C},$$

编码:

$$Z_h = \text{SDPA}(Q_h, K_h, V_h),$$

解码:

$$Y_h = \text{SDPA}(K_h, Q_h, Z_h).$$

同一个 $K_h(X)$ 同时决定:

$$W_{\text{enc},h} = \text{softmax}(Q_h K_h^\top),$$

以及

$$W_{\text{dec},h} = \text{softmax}(K_h Q_h^\top).$$

因此原始 FLARE 是

$$X_{1:N} \rightarrow Z_{1:M} \rightarrow Y_{1:N},$$

输出仍回到原输入 token。

我提出的 Cross-FLARE 是：

$$Z = \text{softmax}(Q K_{\text{in}}(X_{\text{in}})^\top) V_{\text{in}}(X_{\text{in}}, f_{\text{in}}),$$
$$Y(y) = \text{softmax}(K_{\text{out}}(y) Q^\top) Z.$$

该形式不在论文中。

需要补充一个重要细节：FLARE 附录 F 第 20 页确实测试过“编码与解码采用不同 query-key 参数对”，但作者报告没有精度收益。这个消融仍然是同一个输入点集上的 $N \rightarrow M \rightarrow N$ ，并不等于：

- K_{in} 依赖输入观测；
- K_{out} 只依赖新查询坐标；
- $N_{\text{in}} \neq N_{\text{out}}$ 。

Cross-FLARE 还会破坏原论文强调的 shared-score/near-adjoint routing，并需要重新设计 FLARE block 中的同点集残差 $X + \text{FLARE}(X)$ 。

4. Transolver 的 $A_{\text{in}}/A_{\text{out}}$ ：有两种不同含义

原始 [Transolver](#) 没有独立学习两套路由。其第 4-5 页 Eq. (1)-(4) 是：

$$W = \text{softmax}_M(\text{Project}(H)) \in \mathbb{R}^{N \times M},$$

压缩：

$$Z = D_W^{-1} W^\top V, \quad D_W = \text{diag}(W^\top \mathbf{1}),$$

重建：

$$H' = W Z'.$$

因此：

$$A_{\text{in}} = D_W^{-1} W^\top, \quad A_{\text{out}} = W.$$

二者并不严格互为转置，因为压缩侧有 D_W^{-1} 归一化；但它们由同一个原始 W 决定，因此是 score-tied，而不是独立学习。

不过，“解耦 A_{in} 和 A_{out} ，同时保留 Transolver 的 latent self-attention”并非完全是我的新想法。LinearNO 主文的 generalization step 已经实现了：

$$A_{\text{out}} = \phi(Q), \quad A_{\text{in}} = \psi(K)^\top,$$

其中 ϕ 和 ψ 参数独立，并在 Table 4/Table 10 的“Gen. ✓、Sim. x”一行保留了中间 slice attention。它可以理解为：

同点集、压缩与重建解耦、但仍保留 latent mixing 的 generalized Transolver。

然而，这个实验仍然是

$$H_{1:N} \rightarrow Z_{1:M} \rightarrow H'_{1:N}.$$

如果第 4 条具体指：

$$A_{\text{in}} = A_{\text{in}}(X_{\text{in}}, f_{\text{in}}), \quad A_{\text{out}} = A_{\text{out}}(Y), \quad N_{\text{in}} \neq N_{\text{out}},$$

即输出路由只由新查询坐标 Y 决定，并用于跨网格或反问题，那么这是我提出的 Cross-Transolver 推广，Transolver 原文和 LinearNO 的保留-attention 消融都没有直接验证。Cross-LinearNO 只验证了对应的 $G = I$ 版本。

所以最严格的归属是：

- Cross-LinearNO：论文已有并验证；

- LRSA coordinate-only decoder: 我的建议;
- Cross-FLARE: 我的建议;
- 同点集 untied Transolver: LinearNO 已做;
- 跨点集、coordinate-decoded Transolver: 我的建议。

因此，目前不能把 LRSA、FLARE、Transolver 都写成“论文已证明能够像 LNO 一样处理反问题”。只能说它们在结构上存在可行的改造路径，其中只有 Cross-LinearNO 已给出直接的跨查询点实验。

- :codex-followup[建立证据等级表]{prompt="把五种模型关于跨网格、稀疏重建和反问题能力整理成"论文已验证、论文公式支持、我们的推演"三级证据表"}
- :codex-followup[推导三个 Cross 版本]{prompt="严格推导 Cross-LRSA、Cross-FLARE 和 Cross-Transolver，列出所有张量维度、残差修改与复杂度"}
- :codex-followup[设计公平验证实验]{prompt="设计实验，对比 LNO、Cross-LinearNO 及三个推演版本在稀疏重建、系数反演和跨网格预测上的能力"}

2026-09-11 16:45:00 You 

根据我上面的思考，同时可以补充阅读些其他的论文（你觉得与这些论文相关的），你可以多补充月下独些计算机视觉和大语言模型架构的相关论文，也许可以带来启发（因为这些论文中的线性注意力，使用transformer原语等设计就来自其他领域），你觉得可以得到什么样的研究话题

2026-09-11 16:45:44 You 

根据我上面的思考，同时可以补充阅读些其他的论文（你觉得与这些论文相关的），你可以多补充月下独些计算机视觉和大语言模型架构的相关论文，也许可以带来启发（因为这些论文中的线性注意力，使用transformer原语等设计就来自其他领域），你觉得可以有什么设计新架构的方向（低秩神经算子），得到什么样的研究话题

 ChatGPT gpt-6-astra-wm

我会把这次问题当成“架构空间设计”，而不只是罗列相关论文：先把现有五类低秩算子的约束抽象出来，再从视觉模型的 token 压缩/局部-全局混合，以及大模型的 KV 低秩、线性递归和稀疏注意力中寻找可以真正迁移的机制。PDF 技能会用于核对你给出的论文；补充文献只采用论文原文或官方发表页。

2026-09-11 16:47:16 You 

根据我上面的思考，同时可以补充阅读些其他的论文（你觉得与这些论文相关的），你可以多补充月下独些计算机视觉和大语言模型架构的相关论文，也许可以带来启发（因为这些论文中的线性注意力，使用transformer原语等设计就来自其他领域），你觉得可以有什么设计新架构的方向（低秩神经算子），得到什么样的研究话题

 ChatGPT gpt-6-astra-wm

我会重点围绕你提出的矛盾来筛选方向：什么时候需要 latent mixing（潜空间混合）、压缩后哪些信息需要保留，以及表达能力应放在路由、潜空间还是重建端。补充阅读后，我会把每个候选题目写成“已有依据-新架构-可检验假设-风险”，明确区分论文结论与我们的设计。

补充检索发现了一个重要的“新颖性排雷”：FLARE++ 已在 2026 年 8 月提出用额外一次注意力生成输入相关的路由查询。因此，“把固定 latent query 改成动态 query”本身已经不够新。更值得追问的是：动态路由之后还剩下什么瓶颈——信息丢失、潜变量重复，还是物理交互不足？这三种问题应当对应不同的架构。

现在有两条比较有力的思路：一条是改变“怎样压缩”——让潜变量通过重建误差纠正加权汇聚中的信息混杂；另一条是改变“哪些信息必须压缩”——让全局低秩通路处理可压缩交互，同时保留少量未汇聚的细节供查询。两者都能回应你的问题，也分别有 LLM 的误差纠正记忆和压缩 + 检索式注意力作为借鉴。

另一个重要先例是 ICML 2023 的 Intention：它已经把最小二乘求解写成可堆叠的 Key-Value-Query 模块。所以，“在注意力中加入 Gram 矩阵求逆”不能算我们的新公式。这个方向若要成立，需要进一步回答 PDE 中的具体问题：基底重叠是否让 mixing attention 承担了纠错工作，以及怎样在非均匀采样下稳定地完成这种纠错。

我已把候选方向与几个很接近的先例对照了一遍。“动态 query”“局部 + 全局”“正交基”“最小二乘式注意力”都已有相关工作；更值得深入的是：什么信息需要更多压缩容量，什么交互需要额外 mixing，以及怎样用可观测误差来分配这两类计算。

我会区分“适合先验证的科学问题”和“适合发展成完整模型的方向”。前者最贴近你提出的 LinearNO/LRSA 矛盾：mixing 究竟在学习新的交互，还是补偿压缩与重建的不足？后者我更看好按误差分配低秩容量，并为难压缩的信息保留细节通路。下面也会明确哪些设计只是值得测试的模块，还不足以独立成为论文贡献。

我建议把研究重点收敛到两个问题：

- 1. 低秩表示已经建立后，到底还需要多少 latent mixing？这种需求由什么决定？
- 2. 在固定计算预算下，哪些信息值得压缩，哪些信息应当获得更多秩，哪些信息应绕过压缩？

第一个问题最直接延续你对 LinearNO、LRSA 的思考；第二个问题更适合发展成一套完整的新架构。

补充阅读视觉、语言模型和数值算子方面的相关工作后，我整理出下面六个方向。会分别说明：已有依据、可以怎样设计、真正值得研究的部分，以及哪些做法已经有先例。以下架构建议是我的设计假设，不是附件论文已经验证的结论，也不能当作已经完成全面查重的原创性保证。

一、先把你的问题提升为一个统一的架构问题

暂时省略多头、归一化和 FFN，把这类模型的一次非局部更新写成：

$$Z = E(H)V(H), \quad Z' = G(Z), \quad \Delta H = D(H, Z')Z'.$$

其中：

- E ：决定从原始点中提取什么信息，以及怎样形成 latent；
- G ：决定 latent 之间怎样交互；
- D ：决定怎样把处理后的信息送回输出位置；
- 点空间残差、局部模块、通道 MLP：提供上述低秩路径之外的表达能力。

你的问题实际上是在追问：

当 E 、 G 、 D 的表达能力可以相互补偿时，应该怎样分配计算，而不是默认每个环节都需要同样复杂？

这里有三个非常重要的区分。

第一，表示容量不等于交互复杂度。

可能需要 $M = 128$ 个 latent 才能区分输入结构，但它们之间只需要少数几种全局交互。反过来，只有少量 latent，也可能需要复杂的非线性耦合。

第二，低空间秩不等于低频。

例如：

$$K(x, y) = \sin(kx) \sin(ky)$$

始终是秩一核，但可以具有任意高的空间频率。因此，“高频区域一定需要高秩”不能直接作为理论前提，需要检验其交互结构究竟是否难以低秩近似。

第三，低秩分支不等于整个网络低秩。

每层保留点空间残差的模型，并没有把所有信息永久压入 latent。只有当后续计算不再访问原始点状态、也没有其他输入相关旁路时，压缩丢失的信息才构成真正不可恢复的瓶颈。

这三个区分，决定了下面的方向为什么值得做。

二、跨领域论文提供了哪些真正有用的启发？

下面不是简单推荐阅读，而是把机制与可迁移的研究问题对应起来。

相关工作	原论文中的关键机制	对低秩神经算子的启发与边界
TokenLearner 、 ToMe	学习少量内容相关 token；合并 token 并追踪其大小	不只是学习压缩权重，还可以研究如何分配表示数量、追踪每个 latent 代表的区域或测度。但普通 token 聚合与合并已经不是新点
Agent Attention 、 FLatten	低成本全局交互配合局部卷积分支	全局低秩路径不必承担所有细节；但“低秩+局部卷积”已有直接先例
BiFormer 、 Native Sparse Attention	先粗粒度路由，再访问选中的细粒度 token	压缩摘要可以用于决定去哪里读取未压缩的信息，而不必成为所有信息的唯一通道
Differential Transformer	两张归一化注意力图相减	可构造显式的有符号传递和公共分量抵消；但注意力相减本身已有先例
DeltaNet 、 TTT 、 MesaNet	用预测误差更新内部状态，或在前向过程中求解回归问题	压缩可以是可纠错的过程，而不只是一次加权求和；但不能把顺序相关的因果更新直接搬到无序空间点集
Intention	将最小二乘求解写成可堆叠的 Key-Value-Query 模块	是“Gram 矩阵校正编码”的直接先例；单纯在 attention 中加矩阵求逆不构成新公式
Mixture-of-Depths	在固定预算下，动态选择哪些 token 接受计算	可以研究固定总预算下的区域秩分配、交互预算分配，而不必让所有区域承担同样计算
Demystify Mamba in Vision	将注意力原语与局部偏置、门控、归一化、整体 block 分开分析	效果可能来自整个 block，而不只是 mixer；这尤其适合重新审视 LinearNO 与 LRSA 的不同消融
DeepSeek-V2 ： MLA	压缩每个 token 的联合 K/V 通道表示	提醒我们区分空间压缩与通道压缩。MLA 没有直接减少空间 token 数，其 KV-cache 收益也不能直接转化为神经算子的整场推理收益

另外有一个与选题直接相关的新先例：**2026 年 8 月的 [FLARE++](#)** 已经通过额外一次编码，把 FLARE 的 learned query 模板变成输入条件化的 query。因此，“把 FLARE 的固定 query 改成动态 query”现在不能单独作为主要创新。这里说的是 query 模板动态化，并不是原始 FLARE 的整个路由原本与输入无关。

三、方向一：把“表示秩”与“交互修正秩”分开

这是最适合从你当前思考直接启动的方向。

1. 核心问题

现有比较经常把中间模块设成两种选择：

$$G = I \quad \text{或} \quad G = \text{完整 latent self-attention.}$$

但中间还有一个很大的设计空间：

需要保留很多 latent，不代表所有 latent 都需要进行完整两两交互。

因此，建议显式引入两种预算：

- M ：表示预算，即保留多少 latent；
- r ：交互修正预算，即通过多少个方向改变这些 latent 之间的信息交换。

2. 一个最小架构

保留解耦的压缩与重建：

$$Z = E(H)V(H), \quad Z \in \mathbb{R}^{M \times d}.$$

将中间模块设计为：

$$G(Z) = Z + A_{\theta}(Z) \left[B_{\theta}(Z)^{\top} Z \right]$$

其中：

$$A_{\theta}(Z), B_{\theta}(Z) \in \mathbb{R}^{M \times r}, \quad r \ll M.$$

其含义是：

- 1. 从 M 个 latent 中提取 r 个交互摘要；
- 2. 再把这些摘要分发回 M 个 latent；
- 3. 原 latent 通过恒等路径完整保留。

通道 FFN、归一化和残差另外保留，不和“是否有跨 latent 交互”一起删除。

冻结 A, B 后，中间矩阵为：

$$G_{\text{mat}} = I + AB^{\top}.$$

必须准确理解：

- $AB^{\top}$ 的秩至多为 r ；
- $I + AB^{\top}$ 一般仍然可以是满秩 M ；
- $D(I + AB^{\top})E$ 的空间秩仍至多为 M ，不是 $M + r$ 。

所以它增加的是**低秩表示内部的交互修正能力**，不是扩大整个分支的空间秩上界。

3. 为什么能回应 LinearNO 与 LRSA？

可以检验一个更细的假设：

某些任务主要需要灵活的读写路由，少量甚至没有额外 mixing 就够；另一些任务需要在保留的 latent 之间增加少数关键耦合，而不一定需要完整 self-attention。

进一步把解码器分成：

$$D = D(H) \quad \text{与} \quad D = D(H, Z').$$

后者中，mixing 不仅改变被重建的值，还改变重建路由。这正是研究 LRSA 时不能忽略的一条路径。

4. 怎样才有研究贡献？

仅仅提出 $I + AB^{\top}$ 不够。它在代数上与低秩残差、线性注意力等已有结构接近。

真正可以形成话题的是：

表示容量与交互容量是否应该独立配置？交互预算能否由路由结构、任务和尺度预测？

建议先扫描：

$$M \in \{32, 64, 128\}, \quad r \in \{0, 4, 8, 16, 32\},$$

再与完整 residual self-attention 对照。

如果“大 M 、小 r ”能够稳定形成更好的误差—耗时折中，而且这种现象能被解释，那么这个方向就不只是换模块。

暂定研究题目：

Decoupling Representation and Interaction Budgets in Low-Rank Neural Operators

我的判断：非常适合先做机制实验；单独作为架构论文，仍需要比低秩矩阵分解本身更强的发现。

四、方向二：mixing 是否在替压缩过程“纠错”？

这是我认为最有解释力、也最需要谨慎处理已有工作的方向。

1. 一个关键假设

latent mixing 可能承担两种不同工作：

- 1. 学习任务真正需要的跨模式交互；
- 2. 补偿压缩基底之间的重叠、系数混叠或读写不匹配。

如果第二种工作占很大比例，那么不同模型中 mixing 的收益不同，就不一定说明任务需要的物理交互不同，也可能说明前后两端的表示方式不同。

这是待检验假设，不是 LinearNO 或 LRSA 已经证明的解释。

2. 一个简单推导

设输入基底为：

$$\Phi \in \mathbb{R}^{N \times M},$$

采样积分权重为：

$$W = \text{diag}(w_1, \dots, w_N).$$

假设某个特征恰好可写为：

$$V = \Phi C.$$

如果直接做加权聚合：

$$Z_{\text{raw}} = \Phi^\top W V,$$

得到的其实是：

$$Z_{\text{raw}} = \underbrace{\Phi^\top W \Phi}_{\text{Gram 矩阵}} C.$$

除非基底在该测度下正交归一，否则得到的不是原系数 C ，而是被 Gram 矩阵混合后的系数。

于是，中间的某个 mixing 模块可能在学习近似“解开”这种混合。

但也要注意：神经网络可以直接适应这种系数定义，未必需要恢复某种人为规定的“正确系数”。因此，这个推导提供的是一个诊断角度，不是证明必须求逆。

3. 可以怎样设计？

借用已有的正则化最小二乘机制：

$$Z^* = (\Phi^\top W \Phi + \lambda I)^{-1} \Phi^\top W V$$

再独立进行任务映射与输出合成：

$$\hat{H}(Y) = \Psi(Y) G(Z^*).$$

其中输入、输出基底可以分别学习；输出基底还可条件化于已知几何或观测摘要。

这里必须明确：**这个回归编码的核心公式，包括加权版本，已有 Intention 先例**。此外，ONO 已研究正交注意力，你提供的 SVD-NO 已研究分别学习并约束左右基底。因此，不能把“正交双基+Gram 校正”本身包装成首次提出。

更有价值的研究是：

编码基底的条件数、读写基底的匹配程度，能否预测 latent mixing 的边际收益？

4. 从语言模型迁移过来的另一种实现

不直接构造并求解完整 Gram 系统，而是做有限步全点集纠错：

$$Z^{t+1} = Z^t + \eta_t [\Phi^\top W (V - \Phi Z^t) - \lambda Z^t].$$

这里：

- t 是内部求解步，不是空间点顺序；
- 每一步对所有输入点并行聚合；
- 使用的是当前可见的隐藏特征 V ，不是推理时不存在的真实输出；
- 固定 Φ 时，这是正则化最小二乘的梯度迭代。

这借鉴了 DeltaNet、TTT 一类“根据当前预测误差更新表示”的思想，但避免把无序网格强行变成因果序列。

代价也需要算清楚：

- 显式 Gram 求解大致涉及 $N M^2$ 和 M^3 项；

- 因子化有限步迭代的主要乘法约为 $O(TNMd)$ ；
- 迭代稳定性需要控制步长和条件数；
- 病态基底可能放大噪声，不能默认求解更精确就预测更好。

5. 最关键的实验

做一个交叉消融：

编码方式	不做跨 latent mixing	静态混合	residual self-attention
普通聚合	测试	测试	测试
条件改善后的聚合	测试	测试	测试
有限步纠错编码	测试	测试	测试

同时记录基底重叠、条件数、重建误差与任务误差。

如果改善编码后 self-attention 的收益明显下降，支持“mixing 部分承担表示纠错”。如果不下降，说明这一解释不充分，应继续研究任务交互或解码路由。

暂定题目：

When Is Latent Mixing Necessary? Representation Conditioning in Neural Operators

我的判断：科学问题很强；但不能只把 Intention 移植过来，必须做出条件数、离散测度与 mixing 必要性之间的可信联系。

五、方向三：误差驱动的区域自适应秩

这是我更看好的完整新架构方向。

1. 从“动态权重”走向“动态容量”

附件模型普遍允许压缩权重随输入变化，但单层 latent 数量通常预先指定。

这意味着：

内容可以变化，分配给内容的表示容量却往往不变。

可以进一步问：

- 不同样本是否需要相同的 M ？
- 不同区域是否需要相同的局部表示容量？
- 同一区域在不同层中是否需要相同容量？
- 增加一个 latent，究竟能减少多少任务相关误差？

2. 一个可执行的架构

把空间分成几何区域，保留少量全局 latent；每个区域建立局部 latent 集合：

$$Z_b = E_b(H_b)V_b.$$

区域 b 使用 M_b 个 latent，在固定预算内分配：

$$\sum_b \text{Cost}_b(M_b) \leq \mathcal{B}.$$

这里应该约束实际计算成本，而不只是 $\sum_b M_b$ ：不同区域点数不同时，相同 latent 数量并不代表相同 FLOPs。

可以为每个区域设嵌套候选基底：

$$\Phi_b^{(1)} \subset \Phi_b^{(2)} \subset \cdots \subset \Phi_b^{(M_{\max})},$$

根据一个廉价指标决定启用多少个。

指标可以来自：

- 少量探测点上的表示残差；

- 新增基底带来的残差下降；
- 特征的局部差异或矩信息；
- 学到的任务相关误差预测器。

但不能直接把 attention entropy 当成“需要更多秩”的充分指标，也不能每层先计算一个昂贵的完整 SVD，再宣称节省了计算。

3. 跨领域启发具体落在哪里？

ToMe 提醒我们，合并后应追踪 token 代表的量；Mixture-of-Depths 表明，固定总预算与动态分配并不矛盾。

迁移到 PDE 后，可以进一步要求：

- 聚合使用与空间测度相适应的积分权重；
- 合并与拆分不要悄悄改变某片区域的代表权重；
- 更换采样密度后，容量分配应尽量反映物理结构，而不是只反映点密度。

例如，当离散点代表不同面积或体积时，单纯按点数平均和按积分权重聚合不是同一件事。

但“加入质量权重”也不自动等于守恒。合并规则、归一化和输出合成需要一致设计。

4. 已有工作与真正的新空间

多尺度、层级压缩、近远场分解已有充分先例：

- [Multipole Graph Neural Operator](#)；
- [HANO](#)；
- [LSM](#)。

所以值得研究的不是“再做一个多尺度网络”，而是：

在固定成本下，根据可观测表示误差与任务需求，动态决定不同区域、尺度和样本的有效秩。

还可以把方向一嵌入这里：每个区域的表示预算 M_b 与交互预算 r_b 分开配置，检验 mixing 是否主要需要发生在粗尺度或跨区域层面。

暂定题目：

Error-Aware Heterogeneous-Rank Neural Operators

我的判断：完整架构潜力较大，但关键难点是误差指标是否可靠，以及动态分配能否带来真实耗时收益。

六、方向四：让难以压缩的信息拥有独立通路

方向三仍然在问“怎样更好地压缩”。这个方向进一步问：

是否有些信息根本不值得通过同一个低秩瓶颈？

1. 先明确与普通 residual 的区别

Transolver、LinearNO、LRSA、FLARE 的点空间残差已经保留原点特征。

但“信息仍存在于本地点”不等于“其他位置能够访问这些细节”。

如果所有非局部通信仍必须经过同一个 M -latent 瓶颈，那么本地残差并不能完全解决远处细节无法传递的问题。

这就是增加细节通路的理由。

2. 一个残差驱动的双通路

先做低秩表示：

$$Z = EV, \quad \widetilde{V} = BZ,$$

定义可观测表示残差：

$$R = V - \widetilde{V}.$$

然后：

$$\Delta H = D G(Z) + \mathcal{S}_\theta(H, X, R)$$

其中：

- 第一项负责低成本全局通信；
- 第二项是稀疏细节通信；
- 稀疏通路的候选位置、区域或计算预算受到残差影响。

注意：除非 B, E 确实构成指定内积下的正交投影，否则 R 只能叫表示残差，不能直接叫正交补。

3. 与 NSA 的联系

[Native Sparse Attention](#) 的重要启发不是“压缩得更狠”，而是同时使用压缩摘要和被选中的细粒度内容。

对应到神经算子：

1. 先建立全局摘要；
2. 判断哪些区域的细节值得进一步访问；
3. 对选中的未压缩点或细粒度块进行通信；
4. 保留完整输出点集，不把稠密预测问题变成删点问题。

这样补充的是**细节的跨位置可访问性**，不仅是局部特征保留。

4. 什么才可能构成新贡献？

普通“低秩+局部”已被 Agent Attention、FLatten 和多个神经算子采用；近场稀疏、远场低秩也已有 [MGNO](#)、[FMMformer](#) 等先例。

更值得研究的是：

让未被低秩表示解释的信息，决定细节通信的预算与位置。

但这里有两个风险：

- 大残差可能是噪声，不一定是输出所需要的信息；
- 路由、搜索和 gather 的成本可能抵消稀疏计算收益。

若每个查询只访问 k 个候选点，主要通信可近似为：

$$O(NMd + Nkd),$$

但必须另计候选搜索成本。先算完整 $N \times N$ 分数再做 top- k ，并没有解决复杂度问题。

暂定题目：

Residual-Guided Sparse-Low-Rank Neural Operators

我的判断：**有机会突破统一低秩瓶颈，但它应被准确称为低秩-稀疏混合算子，而不是整个空间交互仍然严格低秩。**

七、方向五：点空间与 latent 空间的“按需交换”

这直接回应你关于“每层映回原空间”与“始终留在 latent 空间”的思考。

1. 不必只在两个端点之间选择

当前两类设计大致是：

- 每层进行点空间-latent 空间往返；
- 编码一次，在 latent 中深度处理，最后解码。

可以设计第三种：

点状态与 latent 状态同时保留，各自更新，但只在部分层交换信息。

例如：

- 点状态 H ：负责局部变化和细节保存；
- latent 状态 Z ：负责全局演化；
- 每隔几层才重新读取点信息、更新路由并向点空间写回。

2. 为什么可能有价值？

频繁往返的收益可能来自：

- 根据新特征重新分组；
- 让非局部更新与点级非线性交替；
- 让 latent 重新访问之前没提取到的信息。

但如果某几层的路由与表示几乎没变，重复交换可能收益很小。

可以研究：

什么时候 latent 表示已经过时，必须重新访问点空间？

3. 实现顺序

我建议先做固定交换间隔，例如每 $1/2/4$ 层交换一次；确认存在好的折中后，再研究数据相关的交换策略。

若 L 层中只有 J 次交换，点-latent 通信项大致由：

$$O(LNMd)$$

变为：

$$O(JNMd).$$

但点空间 FFN、局部计算以及状态存储仍存在，不能据此宣称整个模型都降低了 L/J 倍成本。

此外，不应为了判断“是否需要交换”，每层又完整计算一次 $N \times M$ 探测。可以考虑小规模探测点、低阶统计或廉价残差指标。

4. 创新边界

重复读取输入并非新概念，Perceiver 已有相关设计；持久 latent 表示和 latent rollout 也有 UPT 等先例。

因此，研究重点应是：

有明确成本与误差依据的双空间交换策略，而不只是多加几次 cross-attention。

这里还要区分网络深度上的交换与物理时间上的推进，它们不是同一个问题。

暂定题目：

Multi-Rate Point-Latent Neural Operators

我的判断：适合作为效率导向方向，也适合与前三个方向中的一个结合，但不建议一开始就加入复杂控制器。

八、方向六：显式区分背景分量与有符号波动分量

这是一个相对轻量的模块方向。

1. 可以怎样做？

借鉴 Differential Transformer，构造两组逐行归一化的编码权重：

$$E_+ \mathbf{1} = E_- \mathbf{1} = \mathbf{1}.$$

定义：

$$Z_\Delta = (E_+ - E_-)V, \quad \Delta H_\Delta = DZ_\Delta.$$

于是：

$$(E_+ - E_-)\mathbf{1} = 0.$$

这条分支会消除值场中的空间常量分量。可以让另一条普通分支负责背景与平均水平，让差分分支表达对比、变化和波动。

如果两条编码分支共享同一个 D ，该条件线性传递的秩仍至多为 M ；若分别使用独立解码器再相减，秩上界可以达到两条分支之和。

2. 为什么值得测试？

它把一种有符号结构直接放进空间传递，而不是完全交给后续通道投影间接构造。

但必须避免两个错误：

- 普通正权重 attention 的整个网络并非不能表达负值或高频，value/output 投影和多头组合本来就能产生符号变化；
- “对常数响应为零”不等于“质量守恒”。

矩阵条件分别是：

$$K\mathbf{1} = 0 \quad \text{与} \quad w^\top K = 0,$$

二者不同。如果要做守恒，需要另行构造与物理变量、测度和边界条件一致的约束。

3. 我如何定位它？

适合验证：

显式分离背景与波动，是否能以更少 latent 或更少 head 达到相同精度？

必须与相同参数、相近成本的普通多头设计对照。

我的判断：**适合作为辅助模块；单独移植 differential attention，研究贡献通常不够。**

九、任意输出查询和反问题，应当怎样纳入这些方向？

我建议把它们作为架构接口要求，而不是独立的新颖性来源：

$$Z = \text{Encode}(X_{\text{obs}}, a_{\text{obs}}), \quad \hat{u}(Y) = \text{Decode}(Y, Z).$$

也就是输入观测点集与输出查询点集分离，解码器不能依赖查询位置上未知的真实场值。

但要保持此前讨论的来源边界：

- Cross-LinearNO 已在你提供的 [LinearNO 附录](#) 中展示；
- LRSA 的 coordinate-only up-query、FLARE 的独立跨点集编码–解码，是我们讨论的扩展，不能写成原论文已展示；
- 任意坐标查询本身还有 [IPOT](#) 等先例。

更值得加入新研究的是：**稀疏观测和完整输入是否应采用不同容量分配、不同残差通路与不同解码结构。**

尤其要注意，跨点集预测能力不等于逆问题可辨识性。多个真实场可能对应相同观测，这种不确定性不能靠增加 latent mixing 自动消除。

十、我建议怎样选题、怎样开始？

综合你目前的问题，我会这样排序：

方向	最值得回答的问题	我的建议定位
表示预算与交互预算分离	需要多少 latent，和需要多少 mixing，是否是两回事？	最先做的机制实验
条件感知与纠错编码	mixing 在学习交互，还是补偿表示问题？	最贴近你的原始疑问，理论潜力较强
误差驱动的异质秩	计算容量应当分给什么区域、尺度与样本？	优先考虑的完整架构主线
残差引导的稀疏细节通路	哪些信息不应只通过低秩瓶颈？	有潜力的另一条主线，工程风险较高
双空间按需交换	什么时候必须重新访问点空间？	效率方向或后续扩展
有符号背景–波动分离	能否用结构约束提高表示效率？	轻量辅助模块

第一步：先做一个能真正解释矛盾的统一实验

不要直接比较五篇论文各自最优配置。先固定同一个 backbone，控制：

轴	对照选项
压缩与重建	共享／解耦
重建路由	只依赖点状态／同时依赖处理后的 latent
latent 模块	恒等／通道 MLP／静态 token 混合／低秩残差混合／residual self-attention

轴	对照选项
编码	普通聚合／条件改善／有限步纠错
表示规模	扫描 M

通道 MLP 不等于跨 latent mixing；“恒等连接”也不等于“可学习线性矩阵”。这两点必须在实验命名中分清。

不必一开始把所有组合跑完。先做小型机制实验，确定真正有影响的轴，再扩大到实际 PDE 任务。

第二步：让每个方向都有可能被实验否定

例如：

- 校正编码后 mixing 仍然同样有用：说明“表示纠错”不足以解释它；
- 自适应秩不优于固定分配：说明误差指标或动态分配假设不成立；
- 残差稀疏通路只改善隐藏特征重建、不改善最终解：说明保留的信息不够任务相关；
- 减少交换次数反而提高精度：提示频繁重路由可能存在优化或信息干扰问题。

这样的结果同样能帮助形成可靠的架构判断。

第三步：同时评价表达、泛化与真实成本

至少应包括：

- 相同数据划分、预处理、训练协议及多随机种子；
- 参数量、训练成本、显存与实测推理时间；
- 场误差之外，考察梯度、边界或任务重要区域；
- 跨分辨率、非均匀重采样和噪声观测；
- 路由、残差估计、排序和稀疏索引的额外耗时。

不规则网格上的“高频表现”也不能通过任意展平后做 FFT 判断，应使用合适的几何或物理度量。

我的最终建议是：先用“表示几何与 mixing 必要性”建立机制认识，再把最有证据的结论发展成“误差驱动的异质秩架构”。

这条思考链能够真正延续你的问题：

解耦让什么变得不再必要 → 剩余误差究竟来自表示还是交互 → 应增加 latent、增加 mixing，还是读取未压缩细节 → 在固定预算下自动做出选择。

它比单独寻找一种新的 attention 公式更有机会形成连续、可解释的研究主题。

- :codex-followup[展开首选架构]{prompt="把误差驱动的异质秩神经算子展开成完整架构，给出逐层张量形状、所有映射、复杂度与训练方式，并严格区分已有机制和拟议创新。"}
- :codex-followup[设计关键消融]{prompt="围绕 LinearNO 与 LRSA 的 mixing 矛盾，设计一个最小但有解释力的统一消融方案，检验表示条件数、压缩重建解耦和重建路由依赖分别起什么作用。"}
- :codex-followup[进一步筛选选题]{prompt="进一步比较条件感知低秩算子、误差驱动自适应秩、残差引导稀疏低秩三个方向，重点检索最近邻工作并判断各自尚缺少什么才能形成独立论文。"}

2026-09-11 19:03:36 You 

我打算在：四、方向二：mixing 是否在替压缩过程“纠错”？这方面进行研究，但不考虑“4. 从语言模型迁移过来的另一种实现”，请继续完善此方案，给出完整的方案后还要对其中的每一个设计进行具体的解释，同时先给出一些其中具体的数学原理

我会把方案限定为显式的正则化最小二乘编码，去掉有限步迭代纠错。先推导“加权聚合为什么不一定得到基底系数”，再据此设计完整模型，并逐项说明编码、重建、正则化和 mixing 对照各自解决什么问题。

我也会使用 Library 文件技能，把完整推导和方案保存成可继续修改的 Markdown 文档。这里需要先把“纠错”定义清楚：普通加权聚合并不天然是错误的；我们要检验的是，在明确的基底和系数定义下，Gram 校正是否能减少后续 mixing 的工作。

推导中找到了一个更直接的切入点：Transolver 的 slice 归一化，可以写成用 Gram 矩阵的“行和对角矩阵”代替完整 Gram 矩阵。这样就能构造一个连续对照：一端是原来的加权聚合，另一端是完整最小二乘求解。它仍然只做一次显式求解，也能更清楚地检验 mixing 是否在补偿这项近似。

关键公式已通过数值核对：新编码能精确退回原聚合端点；同基底重建时，增加校正强度会减轻部分系数方向的衰减。还有两个需要在方案中明确的边界：Gram 求解本身也是一种跨 latent 混合；它新增了 $O(NM^2)$ 的计算，因此研究目标应是分离“解析校正”与“可学习交互”，不能预设删掉 self-attention 后就一定更快。

完整架构已确定。我保留了一个影响公平比较的细节：归一化只用于稳定求解，求解后要恢复原来的 latent 系数，再送进相同的 mixing 模块。否则 self-attention 和 LayerNorm 看到的尺度也变了，就无法判断收益究竟来自 Gram 校正还是坐标变化。实验方案也会分别检验固定基底下的机制和端到端训练中的实际收益。

我建议把这个课题明确为：

显式校正低秩编码中的基底重叠，能否降低后续可学习 latent mixing（潜空间混合）的必要性？

按照你的要求，方案只采用**直接线性系统求解**，不包含有限步迭代、递归纠错或语言模型式的内部状态更新。

完整推导、架构、逐项解释和实验方案已整理成文档：

这次完善后，我建议对上一版做一个关键细化：以“普通聚合–完整最小二乘”之间的连续校正作为主方案，标准岭回归作为对照。这样能与 Transolver 式聚合建立精确联系，研究对象也更清楚。

下面先讲数学原理，再给出完整模型和各项设计的理由。

一、数学起点：聚合得到的是什么？

先只考虑一个样本、一个注意力头。

符号	形状	含义
V	$N \times d$	输入点上的值特征
A	$N \times M$	输入点到 latent 的软分配，同时视作一组空间基底
Z	$M \times d$	latent 系数
B	$N \times M$	输出重建基底
$W = \text{diag}(w_i)$	$N \times N$	采样积分权重

这里 N 是点数， $M \ll N$ 是 latent 数量。令 $w_i > 0$ 、 $\sum_i w_i = 1$ ，并定义：

$$\|R\|_W^2 = \text{tr}(R^\top W R) = \sum_i w_i \|R_i\|_2^2.$$

均匀采样时可以取 $w_i = 1/N$ ；已有网格面积、体积权重时，应使用相应权重。

1. 内积摘要不一定是展开系数

假设输入特征恰好可以写成：

$$V = AZ_*$$

也就是说， Z_* 是在**固定基底** A 下能够重建 V 的系数。

直接聚合得到：

$$b = A^\top W V.$$

代入 $V = AZ_*$ ：

$$b = \underbrace{A^\top W A}_C Z_*.$$

其中：

$$C = A^\top W A$$

就是 Gram matrix（格拉姆矩阵），它记录基底之间的加权内积：

- C_{mm} ：第 m 个基底的大小；
- C_{mn} ：第 m, n 个基底的重叠程度。

因此，除非 $C = I$ ，否则：

$$b \neq Z_*$$

直观地说：当两个 latent 覆盖了很多相同位置时，分别对它们加权求和，会把它们对应的信息混在一起。

但要特别强调： b 仍然是合法的神经网络特征。不能直接说它“错了”；它只是通常不等于我们指定的展开系数。

2. 最小二乘怎样得到系数？

考虑：

$$Z_{\text{LS}} = \arg \min_Z \frac{1}{2} \|AZ - V\|_W^2.$$

它表示：寻找一组 latent 系数，使重建 AZ 尽量接近输入 V 。

对 Z 求导：

$$A^\top W(AZ - V) = 0.$$

因此：

$$CZ = A^\top W V = b.$$

当 A 满列秩时：

$$Z_{\text{LS}} = C^{-1}b.$$

实现时不会真的构造 C^{-1} ，而是直接求解线性系统 $CZ = b$ 。

这一步的作用是：**在指定基底**下，把内积摘要转换为最小二乘系数。

3. 什么情况下它才是“投影”？

若重建也使用 A ，则：

$$\widehat{V} = A(A^\top W A)^{-1} A^\top W V.$$

记：

$$P = A(A^\top W A)^{-1} A^\top W,$$

有：

$$P^2 = P, \quad P^\top W = W P.$$

这才是 W 内积下投向 $\text{span}(A)$ 的正交投影。

如果输出使用独立的 B ：

$$\widehat{V} = B(A^\top W A)^{-1} A^\top W V,$$

它通常只是“输入分析–输出合成”映射。**不能仅因为输入、输出基底独立，就把它称为 Petrov–Galerkin 投影。**

另外，最小二乘式 Key–Value–Query 模块已经有 [Intention](#) 先例。因此，研究贡献必须落在神经算子中的具体机制和证据上，而不是这个求解公式本身。

二、进一步推导：Transolver 的聚合与 Gram 矩阵有什么关系？

这一部分是本方案最值得抓住的数学切入点。

1. 写出 Transolver 式聚合

设分配矩阵满足：

$$A_{im} \geq 0, \quad \sum_{m=1}^M A_{im} = 1.$$

定义每个 latent 接收到的总权重：

$$s = A^\top w, \quad S = \text{diag}(s).$$

Transolver 式聚合可以写为：

$$\boxed{Z_0 = S^{-1} A^\top W V = S^{-1} b}.$$

均匀权重下，分子、分母中的 $1/N$ 抵消，就得到原来的 slice 加权平均。

现在注意：

$$C\mathbf{1} = A^\top W A \mathbf{1} = A^\top W \mathbf{1} = s.$$

所以：

$$\boxed{S = \text{diag}(C\mathbf{1})}.$$

也就是说， S 是 Gram 矩阵 C 的“行和对角矩阵”。

注意它不是 $\text{diag}(C)$ ：

$$S_{mm} = \sum_i w_i A_{im},$$

而：

$$C_{mm} = \sum_i w_i A_{im}^2.$$

两者不能混淆。

这种把完整矩阵替换成行和对角矩阵的形式，与数值方法中的 mass lumping（质量矩阵集总）存在严格的代数联系。这里是我们对 Transolver 聚合的推导性解释，不是原论文已经提出的结论。相关数值背景可参考[质量矩阵集总与对偶基研究](#)。

2. 为什么这种聚合可能产生“可校正的衰减”？

仍令 $V = AZ_*$ ，则：

$$Z_0 = S^{-1} C Z_*.$$

它与真实展开系数之间，多了：

$$S^{-1} C.$$

如果一个后续线性 mixer 想恢复 Z_* ，它需要补偿：

$$G_{\text{correction}} = C^{-1} S.$$

进一步，若任务真正需要的系数映射是 T ，即目标为：

$$Y = B T Z_*,$$

那么普通聚合后的理想 mixer 为：

$$\boxed{G_0^* = T C^{-1} S}.$$

完整最小二乘之后则只需要：

$$\boxed{G_{\text{LS}}^* = T}.$$

这就把两种工作分开了：

- $C^{-1}S$ ：指定表示方式下的系数校正；
- T ：真正需要学习的任务交互。

这是固定基底、指定任务下可以严格推导的分解。它还不能证明真实模型里的 **self-attention** 就是在做这两部分工作。

3. 一个很小的例子

取：

$$A = \begin{pmatrix} 1 & 0 \\ 1/2 & 1/2 \\ 0 & 1 \end{pmatrix}, \quad W = \frac{1}{3}I.$$

三个点中，中间点同时属于两个 latent。

此时：

$$S = \frac{1}{2}I, \quad C = \frac{1}{12} \begin{pmatrix} 5 & 1 \\ 1 & 5 \end{pmatrix}.$$

指定：

$$Z_* = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \quad V = AZ_* = \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}.$$

普通聚合得到：

$$Z_0 = \begin{pmatrix} 2/3 \\ -2/3 \end{pmatrix}.$$

重建后：

$$AZ_0 = \begin{pmatrix} 2/3 \\ 0 \\ -2/3 \end{pmatrix}.$$

结构保留了，但幅度被缩小。完整最小二乘则能恢复 Z_* 。

这个例子说明：在表示空间没有丢掉目标方向的情况下，仅仅聚合方式本身，也可能造成后续模块需要补偿的变化。

三、推荐的核心机制：从普通聚合连续过渡到完整求解

我不建议第一版直接强制使用完全无正则的 C^{-1} 。学习得到的基底可能高度重叠，甚至线性相关。

建议引入校正强度：

$$0 \leq \alpha < 1,$$

定义：

$C_\alpha = (1 - \alpha)S + \alpha C$

并求解：

$Z_\alpha = C_\alpha^{-1}b.$

它有清晰的端点：

设置	含义
$\alpha = 0$	精确回到原来的加权聚合
$0 < \alpha < 1$	部分校正基底重叠造成的变化
$\alpha \rightarrow 1$	满列秩时趋近完整最小二乘

α 控制校正强度，不是迭代次数。整个编码仍然只做一次直接求解。

1. 它对应什么优化问题？

首先可以证明：

$$S - C \succeq 0.$$

因为对任意向量 z ：

$$z^\top (S - C) z = \sum_i w_i \left[\sum_m A_{im} z_m^2 - \left(\sum_m A_{im} z_m \right)^2 \right] \geq 0.$$

括号里就是一个加权方差。

于是，新编码恰好是：

$$Z_\alpha = \arg \min_Z \left[\frac{1}{2} \|AZ - V\|_W^2 + \frac{1-\alpha}{2} \text{tr}(Z^\top (S - C) Z) \right].$$

第二项还可以解释为：

$$\text{tr}(Z^\top (S - C) Z) = \sum_i w_i \sum_{m < n} A_{im} A_{in} \|Z_m - Z_n\|_2^2.$$

它惩罚：**共同覆盖同一点的两个 latent，给出差异很大的系数。**

因此，更准确的理解是：

- 普通聚合对应较强的重叠感知平滑；
- 增大 α ，逐渐减弱这种平滑；
- 完整最小二乘尽可能恢复当前基底可以表示的细节。

这也意味着， α **并非越接近 1 越好**。某些任务需要保留平滑，以抑制噪声或改善泛化。

2. 为什么这样设计比直接加 λI 更适合本课题？

标准岭回归：

$$(C + \lambda I)^{-1} b$$

当然可以用，但它：

- 没有原聚合的精确端点；
- 正则效果受基底尺度影响；
- 一般会收缩常数分量。

当前设计具有一个特别清楚的实验含义：

在保持同一组基底、同一套系数坐标和同一个后续网络时，只改变从集总矩阵到完整 Gram 矩阵的校正程度。

它仍然属于显式正则化最小二乘，只是正则矩阵从 I 换成了由重叠结构决定的 $S - C$ 。

3. 怎样稳定求解？

定义：

$$Q = S^{-1/2} C S^{-1/2},$$
$$L_\alpha = (1 - \alpha) I + \alpha Q.$$

实际计算：

$$\bar{Z} = \text{Solve} \left(L_\alpha, S^{-1/2} b \right),$$

再恢复：

$$Z_\alpha = S^{-1/2} \bar{Z}.$$

这里有一个必须保留的细节：

归一化只用于求解；恢复 Z_α 后，再送进原来的 latent 模块。

如果直接让 self-attention 接受 $\bar{Z}$ ，它看到的系数尺度也变了，就不再是只研究“是否进行了 Gram 校正”。

由于：

$$0 \preceq Q \preceq I,$$

可以得到：

$$\kappa_2(L_\alpha) \leq \frac{1}{1-\alpha}.$$

例如：

- $\alpha = 0.9$ ：条件数上界为 10；
- $\alpha = 0.95$ ：上界为 20；
- $\alpha = 0.99$ ：上界为 100。

因此，第一版可以把 $\alpha = 0.9$ 作为候选起点，随后扫描；不能直接把它称为最优设置。

4. 校正到底增加了什么表达能力？

固定 A 时：

$$E_\alpha = C_\alpha^{-1} A^\top W, \quad E_0 = S^{-1} A^\top W.$$

两者只差一个可逆的 latent 变换，因此具有相同零空间。

所以它：

- 不增加 latent 数量；
- 不扩大该分支的空间秩；
- 不恢复两种编码共同丢失的信息；
- 改变的是保留信息在 latent 中的表示方式与衰减程度。

当 $A = A(H)$ 随输入变化时，校正矩阵也随输入变化，因此这是一个**输入相关的解析 mixing**。研究目标是减少额外可学习 mixing 的负担，而不是消除一切 mixing。

四、完整模型方案

建议先使用一个**Transolver 式软分配、独立读写基底的受控原型**。

选择它，是因为上述数学关系与它精确对应。它不是直接把 LinearNO 或 LRSA 改名；三者仍应作为外部基线。

1. 输入与整体结构

输入：

$$X, \quad a(X), \quad \text{已知边界、几何或全局参数}.$$

先做逐点特征提升：

$$H^0 = \text{MLP}_{\text{lift}}([X, a, \text{已知条件}]).$$

然后堆叠 L 个 block，最后用逐点输出 MLP 得到目标场。

第一版保留完整点状态，每层重新计算路由。先不加入持续 latent、多尺度、局部分支或自适应秩，以便集中验证当前假设。

2. 每个 block 的计算

步骤①：点级预归一化

$$U = \text{LN}(H).$$

LayerNorm 沿通道计算，不沿空间点归一化。

步骤②：生成输入分配

$$A = (1 - \varepsilon) \text{softmax}_M([U, X]W_A + b_A) + \frac{\varepsilon}{M}.$$

多头实现时，先把路由输出整理成：

$$[\text{batch}, h, N, M],$$

再在**每个 head 内**沿 M 维做 softmax。

步骤③：生成独立输出分配

$$B = (1 - \varepsilon) \text{softmax}_M([U, X]W_B + b_B) + \frac{\varepsilon}{M}.$$

W_A 、 W_B 独立。

第一版让 B 只依赖当前点状态和坐标，暂时不依赖处理后的 latent。否则改变 mixing 时，重建路由也会同时变化。

步骤④：生成值特征

$$V = UW_V.$$

各 head 分别处理自己的值特征。

步骤⑤：计算质量与重叠统计

$$\begin{aligned} s &= A^\top w, & S &= \text{diag}(s), \\ C &= A^\top W A, & b &= A^\top W V. \end{aligned}$$

每个样本、每层、每个 head 独立计算。

步骤⑥：直接 Gram 校正

按前面的标准化方式求解：

$$C_\alpha Z_\alpha = b.$$

使用 Cholesky 分解和三角求解，不构造逆矩阵，也没有内部迭代。

步骤⑦：latent 处理

为了区分“通道处理”与“跨 latent 交互”，建议使用相同的 FFN 骨架：

$$\begin{aligned} Z_1 &= Z_\alpha + \text{FFN}_{\text{in}}(\text{LN}(Z_\alpha)), \\ Z_2 &= Z_1 + \mathcal{M}(\text{LN}(Z_1)), \\ Z' &= Z_2 + \text{FFN}_{\text{out}}(\text{LN}(Z_2)). \end{aligned}$$

其中 $\mathcal{M}$ 分别取：

- 0：没有额外跨 latent mixing；
- 静态 token 线性混合；
- self-attention。

另设一个严格的 $Z' = Z_\alpha$ 组，用于数学验证。

“没有 self-attention”与“整个 latent 模块为恒等”必须分别命名。

步骤⑧：重建点特征

每个 head：

$$Y_j = B_j Z'_j.$$

步骤⑨：合并各 head 并加入点残差

$H' = H + \text{Concat}(Y_1, \dots, Y_h)W_O.$

步骤⑩：点级 FFN

$H_{\text{next}} = H' + \text{FFN}_{\text{point}}(\text{LN}(H')).$

最后：

$\hat{u} = \text{MLP}_{\text{out}}(H^L).$

3. 一个具体张量例子

以下只是建议的原型起点，并非验证过的最优配置：

$\text{batch} = 2, N = 4096, C_h = 128, h = 4, d = 32, M = 64.$

张量	形状
点状态 H	$2 \times 4096 \times 128$
分配矩阵 A, B	$2 \times 4 \times 4096 \times 64$
值特征 V	$2 \times 4 \times 4096 \times 32$
质量 s	$2 \times 4 \times 64$
Gram 与求解矩阵	$2 \times 4 \times 64 \times 64$
latent Z_α	$2 \times 4 \times 64 \times 32$
合头后的点特征	$2 \times 4096 \times 128$

求解对象是每个 head 的 64×64 矩阵，而不是 4096×4096 矩阵。

五、每项设计为什么这样选？

设计	具体理由
坐标与已知输入共同进入模型	同样的场值在不同位置可能有不同作用；路由需要区分空间位置和输入状态
输入基底 A 使用 softmax	保证非负、每点分配和为 1，使质量矩阵、集总端点和谱分析成立
加入小的 ε/M	保证每个 latent 获得非零质量，避免空 latent；建议从 $\varepsilon = 10^{-4}$ 验证，所有对照保持一致
不直接 clamp 质量 s	随意截断 s 会破坏 $C\mathbf{1} = s$ 的精确关系；均匀底座更容易保持理论与实现一致
使用独立输出基底 B	允许读取与写回不同空间模式，避免把共享路由的限制误认为 mixing 的必要性
第一版 B 不依赖 Z'	把“改变 latent 值”和“改变重建路由”分开；LRSA 风格的动态重建留作后续独立实验
独立 value 投影 W_V	A 决定从哪里读取， V 决定读取什么；避免把这两类功能绑在一起
明确使用积分权重 W	区分物理测度与点密度；非均匀采样时不能无条件把平均视作积分
分别计算 S 与 C	S 记录总质量， C 记录重叠；这是本课题要分离的两个因素
采用固定 α 扫描	获得可解释的“校正强度–mixing 收益”关系；第一版不让可学习 α 掩盖这种关系
标准化后求解，再恢复原系数	同时满足数值稳定与公平比较，避免 latent 坐标变化成为额外变量
直接 Cholesky 求解	利用小型对称正定系统的结构；数学求解与实际实现一致。 官方接口
保持求解路径可微	最终任务损失仍能更新路由和值特征；主训练不对 Gram 或质量统计停止梯度
各组保留相同通道 FFN	避免把减少一般非线性容量误解释为减少跨 latent 交互
LN 只放在残差支路内	保留未经归一化的系数幅度路径；数学合成实验则完全去掉 LN
保留多头与输出投影	多组基底能表达不同模式，输出投影负责组合它们；这些容量在对照中保持一致
保留点空间残差	防止全分辨率信息被一层压缩永久抹去，同时允许全局更新与点级非线性交替

设计	具体理由
不默认加入隐藏重建损失或正交损失	更好重建隐藏特征不一定改善 PDE；强制正交还可能直接消除准备研究的重叠因素

其中最后一项尤其重要。

如果训练时额外强制：

$$A^{\top}WA \approx I,$$

那么你研究的“重叠是否导致 mixing 需求”可能被正则项直接抹掉。因此，主实验应当让基底自由学习，**观察并处理它的重叠，而不是先把重叠消除。**

六、怎样把它做成可信的研究，而不是一次模块替换？

建议分成三个阶段，它们回答不同强度的问题。

阶段一：固定基底，验证数学机制

先关闭所有可能吸收校正的部分：

- 不学习 A, B, V ；
- 不加点残差；
- 不加 LayerNorm、FFN；
- 仅比较编码与规定的 mixer。

设置两类任务：

$$V = AZ_*, \quad Y = AZ_*,$$

以及：

$$V = AZ_*, \quad Y = BTZ_*.$$

第一类只有表示恢复需求；第二类同时存在表示校正与真实交互。

然后改变：

- 基底重叠程度；
- 条件数；
- 噪声水平；
- 任务映射 T 的复杂度。

还要补充“保持物理输入、输出和子空间不变，只改变基底坐标”的实验。否则增加重叠时，可能顺便改变了任务难度。文档中给出了具体的可逆换基构造。

这一阶段才能严格区分：哪些 mixing 用于校正表示，哪些用于完成任务。

阶段二：冻结真实模型的表示，研究 mixing 收益

从共同 checkpoint 提取并冻结 A, V, B ，不同组使用完全相同的隐藏特征。

最小核心实验可以先做四组：

编码	相同 FFN、无 self-attention	相同 FFN、有 self-attention
$\alpha = 0$	✓	✓
$\alpha = 0.9$	✓	✓

然后补充：

- $\alpha = 0.5$ ；
- 静态 token 线性混合；
- 严格恒等 latent 模块；

- 标准 ridge;
- $\text{diag}(C)$ 对角校正。

定义 self-attention 的收益：

$$\Delta(\alpha) = \mathcal{L}(\alpha, \text{无SA}) - \mathcal{L}(\alpha, \text{有SA}).$$

重点考察：

$$\Xi = \Delta(0) - \Delta(0.9).$$

如果：

1. 校正编码本身改善结果；
2. $\Xi > 0$;
3. 在控制基底重叠的实验中可重复观察；
4. 不能仅由尺度变化或误差下限解释；

才支持“可学习 mixing 部分承担表示校正”的假设。

还有一个容易遗漏的控制：**Gram 模块使用了 $C(A)$ 这份路由统计，而普通 SA 通常只接收聚合后的 Z_0 。** 对输入相关基底，应补一个让 learned mixer 也访问 Q, s 的小型对照，区分“解析求解更合适”与“拿到了额外信息”。

阶段三：端到端 PDE 验证

最后才让整个模型共同训练。

建议先选：

- 一个相对平滑的椭圆任务，例如 Darcy；
- 一个结构更复杂的任务，例如 Elasticity；

通过后再扩展 Navier–Stokes 等任务。

必须分别报告：

1. 原版 Transolver、LinearNO、LRSA 的外部比较；
2. 同一主干、同一训练协议下的受控消融。

其中：

- $\alpha = 0, B = A$ ：能够精确退回 Transolver 式聚合–处理–重建子模块；
- $\alpha = 0, B$ 独立：是我们的解耦聚合原型；
- 后者**不是**原版 LinearNO，**也不是**原版 LRSA。

这一阶段回答“架构是否有效”，不再能唯一识别每个模块内部在做什么。

七、需要同时记录什么？

至少把三种误差分开：

1. 求解误差

$$\frac{\|C_\alpha Z_\alpha - b\|_F}{\|b\|_F + \delta}.$$

用于确认线性系统实现正确。

2. 隐藏表示重建误差

$$\|AZ_\alpha - V\|_W.$$

用于观察校正是否改善当前表示。

3. 最终任务误差

$$\|\hat{u} - u\|.$$

用于判断这种表示改变是否适合 PDE。

另外记录：

- Q 的谱与求解矩阵条件数；
- 不同 head 的质量分布；
- 校正前后 self-attention 的边际收益；
- 跨采样密度、非均匀采样和噪声表现；
- Gram、求解、latent 模块及整网的实际耗时。

不能把第二项改善直接写成第三项一定改善。

计算上，单 head 的主要成本为：

$$\begin{aligned} O(NMd) & \text{ 聚合与重建,} \\ O(NM^2) & \text{ Gram 构造,} \\ O(M^3 + M^2d) & \text{ 分解与求解.} \end{aligned}$$

因此，即使删掉了 $O(M^2d)$ 的 latent self-attention，新增加的 $O(NM^2)$ 也可能更贵。这个方案首先应检验表达与归纳偏置，速度优势需要实测。

八、这个方案可以形成什么贡献？

最有说服力的贡献组合是：

- **数学解释：**把软聚合解释为一种与基底重叠相关的系数平滑，明确它和最小二乘之间的关系；
- **受控架构：**构造有精确聚合端点、有稳定性界的直接校正模块；
- **机制证据：**通过冻结基底、改变重叠、控制解码器的实验，判断可学习 mixing 是否部分承担该校正；
- **实际验证：**说明在哪些任务、噪声和采样条件下，解析校正与可学习 mixing 应当怎样组合。

与此同时，[Intention](#)、[ONO](#)、[SVD-NO](#) 和 [Galerkin Transformer](#) 已覆盖最小二乘模块、正交基与变分注意力等相邻思想。不能把“使用 Gram 求解”“独立双基”或“正交分析”单独作为新颖性主张。

我建议第一轮就围绕一个可被否定的假设展开：

在固定读写表示后，减轻聚合的重叠平滑，会降低某些任务对额外 self-attention 的需求；但真实跨模式交互复杂时，两者仍然互补。

本轮已完成公式的数值核对，包括聚合端点、标准化求解等价性、变分驻点和谱界；这些检查通过。尚未进行 PDE 训练，所以目前得到的是数学一致、可实施且可检验的研究方案，而不是性能结论。

2026-09-11 20:08:19 You 

那按照你的计划修改后的模型应该是什么样的，是跟transolver一样反复slice和deslice并且每一层都要计算attention还是像linearno一样变成线性注意力，还是像LNO一样后续不再需要压缩和重建而是直接在潜空间中计算，总之给出完整的对模型架构的描述，说明修改后的模型小教育baseline有什么优势（准确性，计算量，推理延迟，参数量等多方面考虑），先不要修改我的方案只会打我的问题

 ChatGPT gpt-6-astra-wm

我会按现有方案说明：先明确每层的数据流和 attention 的位置，再区分“保留 latent self-attention”与“不保留”的对照版本，比较它们相对 baseline 的精度潜力、计算量、参数量和延迟。

按现有方案，修改后的模型采用 Transolver 式的逐层“压缩—处理—重建”主干：每一层都从 N 个点压缩到 M 个 latent，再重建回 N 个点。区别在于：压缩时加入显式 Gram 校正，并采用独立的输入、输出分配矩阵。

现有方案中，是否保留 latent self-attention 仍是需要比较的变量，因此包含两个版本：

- **校正+SA**：Gram 校正后继续进行 latent self-attention。
- **校正-SA**：Gram 校正后只保留相同的 latent 通道 FFN，不进行 self-attention。

它不是 LNO 的“一次编码、长期在 latent 中计算”，也不是直接变成原版 LinearNO。

下面只解释已经确定的方案，不增加新模块或改变设计。

一、先明确：它究竟属于哪一类架构？

问题	现有方案的答案
是否始终保留 N 个点的隐藏状态？	是
是否每层重新生成 slice 分配？	是
是否每层都压缩、重建？	是
是否每层重新计算 Gram 矩阵？	是，校正版本需要
是否每层进行直接线性系统求解？	是，不是仅训练时求一次
是否计算原始点之间的 $N \times N$ self-attention？	否
是否计算 $M \times M$ latent self-attention？	校正+SA 版本计算；校正-SA 版本不计算
不使用 SA 后是否完全没有 latent mixing？	不是，Gram 求解本身就包含输入相关的 latent 混合
是否固定 M 时对 N 保持线性复杂度？	是，但成本高于普通聚合，不能与 LinearNO 的成本等同

这里最重要的是区分：

每层重新计算压缩和重建路由，与每层是否使用 latent self-attention，是两个不同的问题。

我们的两个版本都反复压缩和重建，只在中间是否有额外的可学习 self-attention 上不同。

二、完整的数据流是什么？

设第 ℓ 层输入为：

$$H^\ell \in \mathbb{R}^{N \times D}.$$

N 是空间点数， D 是隐藏通道数。

整体流程是：

```
mermaid
graph TD
    H["N 个点的状态 H"] --> R["生成 A、B、V"]
    R --> S["计算质量、Gram 与聚合量"]
    S --> Z["直接求解 latent 系数"]
    Z --> F["latent FFN 与可选 SA"]
    F --> D["使用 B 重建 N 个点"]
    R --> OutB["输出分配 B"]
    D --> P["合头、残差与点级 FFN"]
    H --> Res["点状态残差"]
    P --> N["下一层 N 个点的状态"]
```

1. 输入提升：先建立点空间状态

输入坐标、已知物理场和其他已知条件：

$$H^0 = \text{MLP}_{\text{lift}}([X, a, \text{已知条件}]).$$

此时仍然有 N 个点，没有提前把整个输入压缩成 latent。

2. 每层先生成新的输入、输出分配

对当前点特征进行归一化：

$$U^\ell = \text{LN}(H^\ell).$$

然后分别生成：

$$A^\ell = (1 - \varepsilon) \text{softmax}_M([U^\ell, X]W_A^\ell + b_A^\ell) + \frac{\varepsilon}{M},$$

$$B^\ell = (1 - \varepsilon) \text{softmax}_M([U^\ell, X]W_B^\ell + b_B^\ell) + \frac{\varepsilon}{M}.$$

其中：

- A^ℓ ：决定如何从点空间读取信息；
- B^ℓ ：决定如何把 latent 信息写回点空间；
- 两者分别学习；
- 每个 head 内分别沿 M 维归一化。

它们都根据当前层更新后的 H^ℓ 重新生成，不是第一层算完后一直复用。

值特征也重新生成：

$$V^\ell = U^\ell W_V^\ell.$$

3. 每层计算聚合量和基底重叠

对每个 head：

$$s^\ell = (A^\ell)^\top w, \quad S^\ell = \text{diag}(s^\ell),$$

$$C^\ell = (A^\ell)^\top W A^\ell,$$

$$b^\ell = (A^\ell)^\top W V^\ell.$$

它们分别表示：

- s^ℓ ：每个 latent 接收到的总质量；
- C^ℓ ：不同 latent 基底之间的重叠；
- b^ℓ ：尚未校正的内积聚合量。

4. 每层进行直接 Gram 校正

求解：

$$\boxed{[(1 - \alpha)S^\ell + \alpha C^\ell] Z_\alpha^\ell = b^\ell}.$$

实际实现采用标准化后的 Cholesky 求解，再恢复原始基底下的系数坐标。

这里的结果为：

$$Z_\alpha^\ell \in \mathbb{R}^{M \times d}$$

——这是一个 head 内的 latent 表示。

需要强调：**推理时也要执行这一步**。因为 A^ℓ 随当前输入和当前层特征变化， C^ℓ 就不能预先算好，作为一个训练后固定的矩阵使用。

5. latent 中的处理：两个版本在这里分开

两个版本都保留相同的通道处理：

$$Z_1 = Z_\alpha + \text{FFN}_{\text{in}}(\text{LN}(Z_\alpha)).$$

校正+SA：

$$Z_2 = Z_1 + \text{SA}(\text{LN}(Z_1)).$$

校正-SA：

$$Z_2 = Z_1.$$

然后两者都计算：

$$Z' = Z_2 + \text{FFN}_{\text{out}}(\text{LN}(Z_2)).$$

所以，校正—SA 版本仍然有：

- Gram 校正带来的跨 latent 混合；
- 每个 latent 内部的通道非线性；
- 残差连接。

它不是简单的“压缩后什么也不做”。

此前方案中的严格 $G = I$ 组，是另外设置的数学验证和消融组，不能与“保留 FFN、去掉 SA”混为一谈。

6. 每层重建回原始点空间

对每个 head：

$$Y_j^\ell = B_j^\ell Z_j'.$$

合并各 head：

$$\widetilde{H}^\ell = H^\ell + \text{Concat}(Y_1^\ell, \dots, Y_h^\ell) W_O^\ell.$$

再进行点级 FFN：

$$H^{\ell+1} = \widetilde{H}^\ell + \text{FFN}_{\text{point}}\left(\text{LN}(\widetilde{H}^\ell)\right).$$

此时重新得到：

$$H^{\ell+1} \in \mathbb{R}^{N \times D}.$$

下一层再用这些新点特征生成新的 A, B, V ，重复整个过程。

7. 最后一层输出物理场

经过 L 层后：

$$\hat{u} = \text{MLP}_{\text{out}}(H^L).$$

因此，整体结构可以概括为：

点特征提升 → [重新分配 → Gram 校正压缩 → latent 处理 → 重建与点级更新] ^{L} → 输出场

三、它和 Transolver、LinearNO、LNO 分别有什么结构区别？

维度	Transolver	LinearNO	LNO	当前方案
深层主状态	点空间	点空间	latent 空间	点空间
压缩—重建频率	每层	每层	主要在首尾	每层
输入、输出分配	共享 slice 权重	解耦	使用其编码—解码机制	独立 A, B
压缩核心	质量归一化加权平均	双归一化低秩乘法	输入到 latent 的编码	聚合 + Gram 直接校正
中间 latent SA	有	无	有	保留与移除两个版本
每层显式 Gram 求解	无	无	无	有
是否每层访问更新后的点状态	是	是	不采用同样的逐层往返方式	是

所以：

- 宏观主干最接近 Transolver。
- 读写解耦的思想与 LinearNO 相近。
- 编码计算本身是新的受控 Gram 校正形式。

- 没有采用 LNO 的长期 latent 主干。

而且，“校正－SA”虽然固定 M 时仍对 N 线性，但它不等于 LinearNO 的线性注意力实现。

即便完全忽略 latent FFN，当前分支也形如：

$$Y = B [(1 - \alpha)S + \alpha C]^{-1} A^\top W V,$$

其中包含输入相关的矩阵求解；这与普通的两次低秩注意力乘法有明显计算区别。

四、准确性方面，可能有什么优势？

这里需要区分“结构上确定的变化”与“尚未实测的收益”。

1. 更明确地处理基底重叠

普通聚合只利用每个 latent 的总质量。当前方案额外使用：

$$C = A^\top W A,$$

处理不同 latent 之间的重叠。

因此，它可能减少以下情况：

多个 latent 读取了大量相同信息，导致系数混合与衰减，后续网络不得不再学习补偿。

这是当前方案最直接的潜在准确性优势。

但它并没有增加空间秩，也不能找回固定编码共同丢失的信息。

2. 可能降低可学习 mixing 的负担

在校正+SA 版本中，SA 可以把更多容量用于任务交互，而不必同时学习某些表示校正。

在校正－SA 版本中，如果任务主要需要灵活读写和通道非线性，解析校正可能使额外 SA 变得不那么必要。

但也可能出现：

- 校正有用，SA 仍然同样有用；
- 校正改善隐藏重建，却不改善目标场；
- 更强校正保留了更多噪声，反而降低精度。

所以目前不能说它一定优于任何一个 baseline。

3. 相对 LNO，保留了逐层重新组织点特征的能力

由于每层都返回点空间，模型可以：

1. 利用上一层全局信息更新每个点；
2. 对更新后的点特征施加非线性；
3. 重新决定下一层的压缩分配；
4. 继续访问尚未充分提取的点点信息。

这是与一次压缩后主要在 latent 中计算的结构差异。

但这不是“证明 LNO 表达能力更弱”。LNO 的 latent backbone 和解码器也有自身表达能力；我们保留这种逐层访问能力，需要付出重复通信的成本。

五、计算量方面：它没有天然优势，尤其不能认为“去掉 SA 就更省”

先比较同一个受控主干，避免混入原论文其他结构差异。

下面只列单 head 的主要非局部成本，省略共同投影和 FFN：

版本	聚合与重建	Gram 与求解	latent SA
未校正+SA	$O(NMd)$	无	$O(M^2d)$

版本	聚合与重建	Gram 与求解	latent SA
校正+SA	$O(NMd)$	$O(NM^2 + M^3 + M^2d)$	$O(M^2d)$
校正−SA	$O(NMd)$	$O(NM^2 + M^3 + M^2d)$	无

因此：

- **校正+SA**：相对相同主干的未校正+SA，明确增加计算。
- **校正−SA**：虽然删除 SA，但新增 Gram 构造通常不能忽略。

一个具体例子

采用前面讨论的：

$$N = 4096, \quad M = 64, \quad d = 32.$$

按单 head 的主要 FLOPs 估算，乘加计两次：

运算	约需计算量
原有聚合+重建	33.55 MFLOPs
新增 Gram 构造	33.55 MFLOPs
Cholesky 分解	0.087 MFLOPs
两次三角求解	0.262 MFLOPs
latent SA 的 $QK^\top$ 与 AV 两次交互乘法	0.524 MFLOPs

这个例子说明：

新增 Gram 构造的计算量，与原有聚合和重建两次乘法相当。

而且：

$$\frac{\text{Gram 构造}}{\text{SA 的两次交互乘法}} = \frac{N}{2d} = 64.$$

这里的 64 只比较这两类矩阵乘法，**不是说整网慢 64 倍**。SA 投影、FFN、访存等尚未包含。

但它足以说明：在点数很大的任务中，删除一个小 latent 空间上的 SA，未必能抵消新增 Gram 的成本。

相对 LNO 呢？

当前方案在 L 层中重复承担点−latent 通信和 Gram 构造；LNO 主要把深层计算放在较小的 latent 空间。

因此，在相近宽度、深度和 latent 数量下，**当前方案通常更难获得 LNO 那样的计算压缩收益**。它选择保留的是逐层点空间更新能力，而不是一次编码后的最低计算成本。

六、参数量会不会更少？

最确定的结论是：

固定 α 的 Gram 校正本身，新增可训练参数为 0

S, C, b 都是从当前输入计算出来的，不是可训练矩阵。

但是，“Gram 模块零参数”不等于“整个新模型参数更少”。

比较对象	参数量判断
同一解耦主干：未校正+SA → 校正+SA	参数基本相同
同一解耦主干：校正+SA → 校正−SA	减少被删除的 SA 参数
原版 Transolver → 当前方案	不保证减少：新增独立输出路由，并且 latent FFN 配置可能不同
原版 LinearNO → 当前方案	没有确定优势：本方案保留的 latent FFN 等也需要计入

比较对象	参数量判断
原版 LRSA/LNO → 当前方案	需要按具体宽度、层数和投影实现统计，不能仅凭模块名称判断

例如，若独立输出路由直接从 $D + q$ 维输入生成 hM 个 logits，包含偏置，则额外路由参数为：

$$(D + q + 1)hM.$$

另外，不能把重建后的共同输出投影 W_O 也算作删除 SA 省下的参数：**它在当前两个版本中都仍然存在。**

七、推理延迟和显存可能怎样变化？

推理延迟

目前没有理由承诺更低延迟。

影响包括：

- 每层新增一次 Gram 构造；
- 每个样本、head 都有小矩阵分解和求解；
- 求解区域采用 FP32；
- 这些操作之间存在依赖关系；
- 少量参数并不能抵消上述运行成本。

因此，我对现有方案的判断是：

- **校正 + SA**：相对相同未校正主干，更可能增加延迟。
- **校正 − SA**：是否变快取决于点数、宽度、head 数和实现；在大 N 、小 M 设置中，也很可能仍然更慢。
- **相对 LNO**：重复点−latent 往返通常不利于低延迟。

这些是根据计算结构作出的判断，不是实测结果。

显存

新增的主要小矩阵包括：

$$C, \quad Q, \quad L_\alpha, \quad \text{Cholesky 因子},$$

规模大致为：

$$O(hM^2).$$

但每层仍保留：

- $N \times D$ 点状态；
- $N \times M$ 输入、输出路由；
- 训练反向传播所需中间量。

因此，当前方案没有获得 LNO 那种“深层主要只维护 latent 状态”的存储结构。

删除 SA 可以省去对应模块的一部分中间量，但是否抵消新增求解缓存，需要测量；不能直接推断显存一定下降。

八、对现有方案的整体评价

方面	当前方案最可信的判断
准确性	有潜力改善重叠基底下的表示，但尚未证明优于 baseline
可学习交互需求	可能减少部分任务对 SA 的需求，是本研究要验证的核心
表达秩	不提高固定低秩分支的空间秩上界
参数效率	Gram 校正零新增参数；整网参数是否减少取决于比较对象
计算量	新增 Gram 与直接求解，通常增加主项计算

方面	当前方案最可信的判断
推理延迟	没有天然优势，较大点数下尤其需要谨慎
显存	增加求解中间量，仍保留每层点状态与路由
数值可分析性	有明确的聚合端点和标准化求解条件数界
研究解释力	能更清楚地分离表示校正与可学习任务交互

所以，按你目前的方案，这是一种“每层反复压缩和重建、带显式 Gram 校正的低秩神经算子”。它的主要潜在优势是更合理地使用已有表示、减少需要学习的代数补偿，以及提高机制可解释性；目前不能把更快、更省计算或总参数更少列为已经成立的优势。

2026-09-11 20:30:22 You 

我的目的是为了在对latent的修正混叠问题进行修正后（我把加入你上述说的修改后的层的新构造起名为校正层）可以在后续的空间中（比如在两层这样的校正层之后）可以不再进行每层分片或不分片的操作而是直接在潜空间中运算（有两种方案，一种是像LNO一样依然在latent空间中每层都做attention计算，第二种是在latent空间中也像linearno或Irsa的消融版本一样做mlp计算（这两个中也要挑一个方式）），因为我的目的就是按照你提出的方向2通过几次对latent软分配的矫正后可以不用反复进行压缩和重建，而是可以直接像LNO一样直接在固定的潜空间中运算，如果上述方案兼容我的想法，就把这个想法补充进去形成完整的方案，如果不兼容，先告诉我原因，并思考可不可以通过适当的调整来适应我的额想法，如果是这样先不用直接产生新方案可以先告诉我修改的想法

 ChatGPT gpt-6-astra-wm

我会先检查两点：校正层得到的系数能否作为后续固定 latent 空间的状态，以及停止压缩–重建后，attention 或 MLP 分别还能完成哪些交互。尤其需要区分“校正了基底重叠”和“已经提取了任务所需的全部信息”，后者不能由前者直接推出。

你的目标可以实现，Gram 校正与“后续长期在 latent 空间中计算”并不冲突。但现有方案需要调整校正层的输出接口、点空间残差和最终解码方式，才能准确实现你的想法。

我之前把校正安排在每一层，重点放在“改善每次压缩”。你现在明确的目标则是：

用少数前端校正层建立足够好的 latent 表示，随后停止反复压缩–重建，把主要深度计算放在 latent 空间。

这是一个不同、也更直接面向效率的研究目标。

它属于你说的“可以通过适当调整来适应”的情况。因此，下面先说明兼容性、需要调整的地方，以及 attention/MLP 的选择，不直接改写成完整新方案。

一、核心机制兼容，但需要修正一个推理前提

你的设想包含两个命题：

- 1. Gram 校正能够减轻软分配基底重叠造成的系数串扰。
- 2. 经过少数校正层后，latent 已经足够完整，可以不再访问点空间。

第一个有前面的数学依据；第二个是需要实验验证的新假设，不能由第一个直接推出。

这里的“混叠”，更准确地说是**软基底重叠引起的系数混合与衰减**，不是采样不足导致的频谱混叠。

1. 校正的是系数，不是自动把基底变成正交基

现有校正为：

$$Z_{\alpha} = [(1 - \alpha)S + \alpha C]^{-1} A^{\top} W V,$$

其中：

$$C = A^{\top} W A, \quad S = \text{diag}(A^{\top} w).$$

求解以后，得到的是更接近指定基底下展开系数的 Z_α 。

但这一步并没有让：

$$A^\top W A = I.$$

它也没有直接保证：

- 分片已经覆盖所有任务相关结构；
- latent 数量 M 已经足够；
- 后续处理不再需要其他输入信息。

2. 校正不会恢复共同遗漏的信息

固定 A 时：

$$E_\alpha = [(1 - \alpha)S + \alpha C]^{-1} A^\top W,$$

与普通编码：

$$E_0 = S^{-1} A^\top W$$

具有相同零空间。

也就是说，如果某个信息方向在 $A^\top W V$ 中已经完全消失，后面的 Gram 求解不能把它找回来。

因此，你的研究假设应当表述为：

校正是否能让有限容量的 latent 表示更适合长期计算，从而减少重新访问点空间的必要性？

而不是：

校正两次以后，信息就一定完整了。

3. “两层”可以作为起点，但不是理论上的充分次数

如果两个校正层之间有点级非线性、残差和重新学习的分配，那么第二层确实可能形成更适合任务的表示。

但收益来自整个前端的表示学习，不是“同一个最小二乘问题多修正一次就更充分”。

例如，在固定基底、去掉外围非线性和残差的简化情况下，完整最小二乘重建是投影：

$$P^2 = P.$$

重复投影不会增加信息。正则化版本重复执行，还可能继续衰减某些方向。

所以，可以研究两次校正，但应当把校正次数视为结构超参数。

二、为什么不能直接把“两层原样校正层”接到 latent 主干？

问题在于：当前完整校正层的输出仍然是点状态 H ，不是 latent 状态 Z 。

其结构是：

$$H^{k-1} \rightarrow Z_k \rightarrow Z'_k \rightarrow B_k Z'_k \rightarrow \text{点残差与点级 FFN} \rightarrow H^k.$$

如果做完两层后，直接取第二层内部的 Z'_2 作为后续输入，那么：

- 第二层最后的重建没有用于建立后续 latent；
- 第二层点残差的信息没有进入这个 Z'_2 ；
- 第二层最后的点级 FFN 也被跳过。

因此，它并不等价于“完整执行两层现有校正层，再进入 latent”。

这不是根本障碍，但必须把交接位置定义清楚。

我更倾向的最小调整

前面若干层完整往返，最后一个前端层改成只输出 latent 的过渡编码层。

以你说的“两层”为例：

前端部分	作用	输出
第一次校正	压缩、校正、处理、重建、点级更新	N 个点状态
第二次校正	根据更新后的点状态重新压缩并校正，到 latent 处结束	M 个 latent
后续主干	始终在这 M 个 latent 上计算	M 个 latent
最终输出	一次性解码到目标点	物理场

准确地说，这是：

两次校正编码，其中第一次包含前端重建，第二次负责进入长期 latent 主干。

这样不会为了“先完成第二层重建”，随后又额外进行第三次编码。

另一种做法当然是保留两层完整校正层，再增加一次入口编码。但那实际是三次编码，计算量也必须按三次计算。就你的目标而言，我更倾向前一种接口调整。

三、停止往返之后，还需要调整哪些地方？

主要有三处。

1. 明确“固定 latent 空间”固定的是什么

建议你的“固定”理解为：

- latent 数量 M 固定；
- 进入主干后不再生成新的点空间分片；
- 不再根据深层点状态重新压缩；
- 深层更新的是 $Z^0, Z^1, \dots, Z^L$ 。

但它不意味着不同输入样本必须共享完全相同的物理分区。

最后一次编码仍可以根据当前输入生成 A_* 。只是进入 latent 主干后，不再反复重新生成它。

因此：

输入相关的前端表示 + 固定接口的 latent 主干

是兼容的。

LNO、IPOT 已经说明“编码—latent 主干—解码”这种宏观拓扑可行；你的研究价值应当放在校正能否改善这种拓扑中的入口表示。[LNO](#)、[IPOT](#)

2. 最终解码需要与最后一次编码衔接

现有方案每层都有自己的 B^ℓ 。停止逐层重建后，需要明确最终使用哪个输出基底。

最小的调整思路是：

在前端交接时生成与最后一次编码相配套的 B_* ，随后保留它，等 latent 主干完成后再解码一次。

例如：

$$\widehat{H} = B_* Z^L.$$

这不要求后续每一层重新计算 B_* 。

但要准确区分：

- 深层主要计算只发生在 latent 空间；
- 是否只存储 latent。

如果缓存了 $B_* \in \mathbb{R}^{N \times M}$ ，仍然保留了与输出点有关的存储。它也可能携带输入相关信息，不能宣称整个输入只通过一个 $M \times d$ 张量传递。

3. 原来的点残差不能默默丢掉

之前每层都有：

$$H_{\text{next}} = H + \text{全局更新} + \text{点级非线性更新}.$$

它承担了保留点级细节的作用。

转入长期 latent 后，后续当然不能继续照搬这个 N 点残差主干。需要明确：

- 哪些信息已经进入 latent；
- 最后一个完整前端点状态是否直接结束使用；
- 是否仅保留一次最终解码用的点状态旁路。

保留一次旁路不会导致深层反复 slice/deslice，但会保留额外的点状态存储；不保留则构成更严格的信息瓶颈。

这是需要明确选择的接口问题，不能把原方案的点残差优势自动算到新主干上。

四、两种 latent 主干，哪个更适配？

方案 A：在 latent 中继续使用 attention

这一种与校正编码最直接兼容。

其逻辑是：

1. 前端负责形成和校正表示；
2. 后续 attention 负责不同 latent 之间的任务交互；
3. 最后统一解码。

这不会违背你的研究目标。你想删除的是反复点-latent 往返，并不必然要同时删除 latent 内部的 attention。

事实上，这样还能把研究问题分开：

先验证校正是否允许停止往返，再验证能否进一步取消 latent attention。

这是两种不同强度的简化。第一种成立，并不保证第二种也成立。

我更倾向把 LNO 式 latent attention 主干作为首先验证的版本，因为它保留了深层跨 latent 通信，可以较干净地检验“不再返回点空间”本身的影响。

方案 B：在 latent 中只使用 MLP

可以做，但这里必须先纠正论文对应关系。

如果在 LinearNO 和 LRSA 之间选择 MLP 分支的参考，我选择 LRSA 的去 intra-attention 消融思路。

原因是：

- LinearNO 的简化是把中间 G 设为恒等映射，不能称作“LinearNO 的 latent MLP”；
- LRSA 的消融确实描述了用 MLP 替换 latent processing、移除 self-attention。

不过，LRSA 论文并没有把该消融 MLP 的所有实现细节写清楚，不能直接假定它采用了沿 latent 轴混合的 MLP。[LinearNO，第4页](#)、[LRSA，第5页与第7页](#)

这里最关键的是：MLP 究竟作用在哪个维度？

MLP 类型	计算含义	是否在深层产生跨 latent 交互
逐 latent 的通道 MLP	每个 latent 单独更新自己的通道	否
沿 latent 轴的 MLP	用不同 latent 的信息共同更新输出	是

如果采用普通逐 token FFN：

$$Z_m^{\ell+1} = Z_m^\ell + f_\ell(Z_m^\ell),$$

那么：

$$\frac{\partial Z_m^{\ell+1}}{\partial Z_n^\ell} = 0, \quad m \neq n.$$

这意味着进入固定 latent 后，不同 latent 不再新增通信。

整个模型仍然可以具有全局作用——前端编码本来就是全局聚合——但深层主干不能再根据各 latent 的新状态，建立新的跨 latent 交互。

所以，LRSA 式逐 token MLP 可以作为第二个版本，但它检验的是更强的假设：

前端是否已经完成了足够的跨 latent 信息组织，使后续只做通道非线性也能完成任务？

如果改用沿 M 轴的 token-mixing MLP，则能保留跨 latent 交互。但那属于额外的适配设计，不能说成已经核实的 LinearNO/LRSA 原文构造。本轮不把它自动加入你的方案。

五、为什么 LinearNO 或 LRSA 的“去 attention”结果不能直接保证你的 MLP 版本有效？

这是你这个新目标中最需要注意的区别。

它们每层仍有点空间往返。即使没有显式 latent self-attention，仍然可能存在：

$$Z^{\ell+1} = E_{\ell+1} \left[\sigma(B_{\ell} Z^{\ell}) \right],$$

这里暂时省略其他投影和残差。

这条路径包括：

- 1. 多个 latent 的信息在点空间重建时相遇；
- 2. 点级非线性 σ 对组合后的信息进行处理；
- 3. 下一层再次聚合成 latent。

这个过程本身可以产生跨 latent 的间接非线性交互。

当你停止压缩—重建后，这条交互路径也被删除了。

因此：

- 原模型中“没有 SA，但还有反复往返”；
- 新模型中“没有 SA，也没有反复往返”；

两者的表达能力来源并不相同。

这并不否定纯 MLP 主干，而是说明它必须单独验证，不能直接借用已有消融结论。

六、这样的调整是否更符合你的效率目标？

是，明显比“每层都加 Gram 校正”更符合。

原方案的主要问题是：每层都新增一次

$$O(NM^2)$$

的 Gram 构造。

若只有前端 k 次校正，随后有 J 层 latent 计算，那么主要成本分为：

部分	主要成本，省略 head 等共同系数
前端校正	约 $O(kNM^2 + kNMd + kM^3)$
latent attention 主干	约 $O(JM^2d + JMd^2)$
逐 token MLP 主干	约 $O(JMd^2)$
最终解码	约 $O(NMd)$

还需要计入前 endpoint 级网络、投影、输出层等成本。

这种改变把与大点数 N 有关的昂贵操作集中到少量前端层。真正的效率来源是减少深层访问点空间的次数，而不是假设 Gram 求解比 attention 更便宜。

不过，相对“一次普通编码+同样 latent 主干”，你的模型仍然新增了前端校正成本。它需要通过准确性、稳定性，或者更小的 latent 容量证明这笔成本值得。

七、我对兼容性与调整方向的最终判断

你的目标	判断
少数校正层后长期留在 latent 空间	可以实现，与校正公式兼容
两个完整原样校正层直接输出 latent	不能原样衔接，需要调整最后一层出口
两次校正就保证不再需要读点空间	没有这个数学保证，应作为研究假设
后续使用 LNO 式 latent attention	直接适配，适合首先验证
后续只使用 MLP	可以研究，但同时删除了深层跨 latent 通信时，假设更强
MLP 在 LinearNO/LRSA 中选参考	选 LRSA 去 intra-attention 的思路；LinearNO 的 G 是恒等

我建议的最小调整方向是：保留原来的校正公式，把前端最后一个校正层改为 latent 入口，明确一次性的最终解码与点残差处理，然后分别研究 attention 主干和 LRSA 式 MLP 主干。

相应地，研究主题也应更准确地变为：

校正软分配的系数串扰，能否使低秩表示更适合持续的潜空间计算，从而减少点空间—潜空间之间的反复转换？

这比原来的“校正能否替代部分 mixing”更贴近你的实际目标。至于是否还能进一步移除 latent attention，应作为第二层问题，而不是校正成功后自动成立的结论。

2026-09-11 21:03:34 You 

我希望能实现直接在后续的latent空间中进行线性映射，也就是linearno的做法，linearno中说明transolver的能力主要来自反复的silce和deslice，我的根本目的是为了设计更高效的slice和deslice，使得更少的slice和deslice就可以获得与transolver,linearno式的slice和deslice一样甚至更好的效果，另外，在这里还可以补充思考一个问题：之所以linearno可以通过slice和deslice完全替代transolver的attention，除了论文本社的分析外，还有一个架构上的原因就是linearno解耦了压缩和重建的过程，我再思考可不可以通过更优的解耦方式/压缩与重建之间更好的关系（比如让两者正交，或是其他的关系，总之要有一定的数学依据），进一步提高压缩和重建的效率（注意由于本方案准备采用transolver风格的slice和deslice方式，所以这里的压缩和重建就是slice和deslice你打算通过其他类型的架构（比如Irsa,ipot风格的）替换压缩/重建的架构），最终目的是以更少的slice和deslice次数换取与transolver或linearno中多次slice和deslice一样甚至更好的效果，对于这个方案的研究还需要考虑以下几点：1.我用更好的slice和deslice替换baseline模型较多的slice和deslice本质上是想要降低计算量/降低推理延迟/训练/推理速度，至少要有一方面效率上的优势，所以在设计模型时要充分考虑这一点，不能时为了减少层数而设计；2.我现在的想法可能是几词slice和deslice后就输出（就是直接减少slice和deslice的次数），还有一种思路是在少数几次slice和deslice之后依然在潜空间中运算，可以考虑以下哪种更合适，或者是不是想要做对比；3.还有一个问题就是在改进的模型后也要考虑改进后的slice和deslice间是像transolver一样做attention运算，还是像linearno一样做线性注意力计算也要考虑清楚；4，始终注意数学原理解释，想法要合理；5.可以再多阅读一些参考文献，并了解所选取baseline可以使用的数据集的具体格式，一遍估计改进模型的效率优势

 ChatGPT gpt-6-astra-wm

我会把目标按“更少的 slice/deslice，且有真实效率收益”重新审查：先核对 LinearNO 中线性映射的准确含义，再比较有数学依据的读写关系，并结合 Transolver、LinearNO 的代码与数据张量规模估算成本。之前的完整 Gram 求解也会重新评估，不预设它值得保留。

代码核对带来了一个重要修正：这些基线的计算不只在 slice/deslice。规则网格版本还反复执行点空间卷积和 FFN，所以减少往返可能省下这些大开销；但实验也必须区分收益究竟来自更好的分片，还是单纯删掉了卷积和 FFN。另外，严格要求非负软分片的读写映射“互为逆”，会迫使它们趋向相同的硬分区，可能抵消你希望保留的解耦自由度。

目前较合理的路线是：保留独立的 slice/deslice，用有界的小矩阵校正处理分片重叠，再检验它能否减少往返次数。推理时可以复用训练统计，避免每个输入都构造 Gram 矩阵。但这里必须区分两件事：校正能改善系数的表示，不等于已经证明后续只用线性映射就足够；后者需要

单独验证。

还有一个会直接影响架构选择的结论：如果最后一次 slice 后只连续做固定的 token 线性映射，那么这些映射可以合并成一个矩阵。因此，“浅层模型的最后一个块加入线性 mixing”和“最后一次编码后留在 latent 中做多层线性计算”，在这种设定下其实是同一个模型。值得检验的是少量往返加一个有效的线性混合，能否保住深层基线的精度。

你的目标已经很明确：**提高每次 slice/deslice 的有效性，用更少的点空间–潜空间往返，在保持精度的同时获得实际效率收益。**

我建议把主方案确定为：

保留 Transolver 风格的软分片；独立学习 slice 与 deslice；在小规模 latent 上校正软分片的重叠；主干只保留少量往返；不默认使用 latent self-attention，最后允许加入一个低成本的线性 token mixing。

但这里有两个必须先讲清楚的地方：

- 改善软分片的系数表示，不等于已经证明可以替代深层重新分片。后者正是需要实验回答的研究问题。
- 如果最后一次 slice 后只有连续的固定线性映射，那么“浅层模型”与“长期停留在 latent 的深层线性模型”可能只是同一个函数的不同写法，不应该人为分成两个架构创新。

下面依次说明数学依据、具体架构、效率预算和实验安排。**以下改进是研究设计，尚没有训练结果证明它优于基线。**

先修正此前对 LinearNO 的一个表述：它去掉的是显式 latent mixing，而不是加入一个可学习的 latent 线性矩阵。

LinearNO 的核心写成：

$$Y = \phi(Q) [\psi(K)^\top V].$$

括号内部得到 latent，随后直接重建，中间的 G 是恒等映射：

$$G = I.$$

官方代码也只有 `k.T @ v` 和 `q @ kv`，两者之间没有可学习的 latent mixing 模块。其深度仍然来自多次点空间更新，包括重新生成路由、非线性、残差，以及部分任务中的卷积。[LinearNO 论文](#)、[官方模型代码](#)

因此，我们需要区分：

选择	latent 内部做什么	在本研究中的定位
$G = I$	不增加任务混合模块	首要版本，检验校正后的读写是否足够
$G(Z) = TZ$	可学习的固定 token 线性映射	首要对照，也是可选的最终架构
$G(Z) = \text{SA}(Z)$	输入相关的 self-attention	机制对照，检验校正后是否仍需要 attention

还要注意：**校正矩阵本身已经进行 latent 之间的线性混合**。因此，“校正后 $G = I$ ”准确的含义是“不再增加额外的任务 mixing”，不能说整个模块完全没有 latent mixing。

你提出的“解耦可能使 attention 变得不必要”，有合理的架构依据，但它不是无条件成立的等价定理。

下面先讨论单个 head。设：

- 点数为 N ，latent 数为 M ，每个 head 的通道数为 d ；
- $A, B \in \mathbb{R}^{N \times M}$ 分别是 slice 和 deslice 的软分配；
- 两者每行经过 softmax，因此 $A\mathbf{1} = B\mathbf{1} = \mathbf{1}$ ；
- $W = \text{diag}(w)$ ， $w_i > 0$ ， $\sum_i w_i = 1$ 。

定义：

$$s = A^\top w, \quad S = \text{diag}(s), \quad E_A = S^{-1} A^\top W.$$

Transolver 风格的编码为：

$$Z = E_A V.$$

绑定读写、不使用 attention 时，空间分支是：

$$Y = A E_A V.$$

解耦后则是：

$$Y = BE_A V.$$

前者在加权内积下具有自伴、半正定的结构；后者可以表示更一般的非对称空间映射。**区别不是前者没有全局交互——软分配下 AE_A 本来就可以是稠密全局核——而是读写绑定限制了核的结构。**

如果把 attention 的空间混合暂时记为 G ，冻结当前输入对应的所有矩阵，则：

$$BGE_A = (BG)E_A.$$

这说明：**独立的解码器有机会承担原本由部分 latent mixing 完成的工作。** 当 B, G 都是行随机矩阵时， BG 仍然可以作为合法的非负软分配矩阵。

但独立的逐点路由网络是否能表示这个 BG ，仍取决于它能访问哪些信息、容量是否足够。加入多头、通道映射和非线性后，也不能直接沿用这个简单等式证明完整网络等价。

另外，LinearNO 的 key 沿点轴归一化，而这里保留的是 Transolver 的“沿 latent 轴 softmax，再按列质量归一化”。**我们的解耦版本与原版 LinearNO 思想相关，但并不完全相同。**

关于“让 slice/deslice 正交”：需要先区分几种不同关系，不能直接把正交性设为目标。

关系	数学含义	对当前软分片是否合适
$A^\top W B = 0$	读写两组基相互正交	不合适；非负、归一化条件下甚至不可能整体成立
$A^\top W A$ 接近对角	减少读取基自身的重叠	有意义，但过强会逼近硬分区
$E_A B = I_M$	latent 经解码再编码后保持不变	对当前非负软分配施加精确条件，会迫使读写趋向相同的硬分区
有符号分析、合成基满足对偶关系	稳定表示某个低维子空间	数学上合理，但不能直接等同于原始非负 soft slice

为什么 $E_A B = I$ 会有问题？对 $m \neq n$ ：

$$(E_A B)_{mn} = \frac{1}{s_m} \sum_i w_i A_{im} B_{in} = 0.$$

每一项都非负，所以只要 $A_{im} > 0$ ，就必须有 $B_{in} = 0$ 。结合 B 每行和为 1，可以进一步推出 A, B 必须对应相同的硬分区；有限 softmax 的严格正值分配甚至不能精确满足它。

还有一个容易忽略的限制：如果真的实现了 $EB = I$ ，同时中间没有任务变换，那么

$$K = BE, \quad K^2 = BEBE = BE = K.$$

这一冻结的空间分支变成了投影算子。 投影适合表达和重建子空间，却不是任意 PDE 输入—输出映射。

这不意味着完整残差非线性网络只能做投影，但意味着：“**读写尽可能互逆**”未必是最有利于任务表达的关系。

因此，我建议采用的关系是：

读取端尽量获得稳定、低串扰的系数；写入端保留任务所需的独立性。

它改善的是“分析系数的可靠性与任务合成的分工”，目前还不能宣称解决了 A, B 之间的最优交叉关系。

具体的校正，先从一个可解释的精确版本出发，再设计便宜的近似。

记：

$$C = A^\top W A, \quad b = A^\top W V.$$

普通 slice 得到：

$$Z_0 = S^{-1} b.$$

假设当前值特征恰好可以写为 $V = AU$ ，那么：

$$Z_0 = S^{-1} C U.$$

除非 $C = S$ ，否则读出的系数不是 U ，而是被基底重叠矩阵 $S^{-1}C$ 混合过的系数。这就是本研究中“串扰”的明确数学含义；它不等同于采样理论中的所有混叠现象。

可以定义：

$$Z_\alpha = [(1 - \alpha)S + \alpha C]^{-1}b, \quad 0 \leq \alpha < 1.$$

它有三个直接性质：

- $\alpha = 0$ ：退化为原来的质量归一化 slice。
- α 增大：逐步考虑软基之间的重叠。
- 当 $\alpha \rightarrow 1$ 且 C 可逆：趋近于加权最小二乘系数。

它还是下面优化问题的唯一解：

$$\min_Z \frac{1}{2} \|AZ - V\|_W^2 + \frac{1 - \alpha}{2} \text{tr}[Z^\top (S - C)Z].$$

这里 $S - C \succeq 0$ 。第二项会抑制高度重叠的 latent 获得差别过大的系数。因此， α 控制的是：**保留平滑、稳健的平均表示，还是更充分地分离重叠基的系数。**

但必须保留两个边界：

1. 这不会增加编码的秩，也不能恢复被编码丢掉的方向。
2. B 独立时，校正并不保证 $E_{\text{corr}}B = I$ 。

它是有数学依据的**读取端校正**，并不是已经证明“所有 attention 都在求这个逆”。

为满足效率目标，我更建议把“训练统计驱动的校正”作为部署候选，把逐输入求解作为机制对照。

精确版本需要构造 C ，成本为 $O(NM^2)$ ，还需要小矩阵求解。虽然它未必昂贵到不可用，但没有必要预先要求每次推理都承担这些操作。

先定义标准化 Gram：

$$Q = S^{-1/2}CS^{-1/2}, \quad u = \sqrt{s}.$$

由软分配的归一化性质：

$$0 \preceq Q \preceq I, \quad Qu = u, \quad u^\top u = 1.$$

精确校正对应：

$$R_{\text{input}} = [(1 - \alpha)I + \alpha Q]^{-1}.$$

训练时，为每层、每个 head 维护 Q 的移动平均 $\bar{Q}$ ，部署时固定：

$$\bar{R} = [(1 - \alpha)I + \alpha \bar{Q}]^{-1}.$$

训练协方差、推理复用统计的思路已有 ONO 的先例；这里将其用于质量归一化软分片，并加入下面的常数模式处理。ONO

由于不同输入的 s 会变化，直接使用 $\bar{R}$ 未必保留当前常数模式。可以令：

$$P = I - uu^\top, \quad R_s = uu^\top + P\bar{R}P.$$

最终编码为：

$$Z_{\text{corr}} = S^{-1/2}R_sS^{-1/2}A^\top WV.$$

实际实现无需构造 P 。令

$$\bar{b} = S^{-1/2}A^\top WV, \quad \mu = u^\top \bar{b},$$

依次计算：

$$\begin{aligned} r &= \bar{b} - u\mu, & q &= \bar{R}r, \\ Z_{\text{corr}} &= S^{-1/2}[u\mu + q - u(u^\top q)]. \end{aligned}$$

这样新增的推理计算主要是：

$O(M^2d)$

而不是 $O(NM^2)$ ，也不需要逐输入分解矩阵。

这个设计的每一部分都有明确作用：

设计	作用
$S^{-1/2}$ 标准化	分离 latent 占用量与基底相关性
$\bar{R}$	用训练统计近似重叠校正
当前输入的 $u = \sqrt{s}$	保留当前分片的质量信息
$uu^\top + P\bar{R}P$	精确保留常数模式
独立 B	保留任务相关的合成自由
有界的 $\alpha < 1$	避免接近奇异的完全逆

在上述谱条件下：

$$I \preceq R_s \preceq \frac{1}{1-\alpha}I.$$

而且：

$$\|R_s - R_{\text{input}}\|_2 \leq \frac{\alpha}{(1-\alpha)^2} \|\bar{Q} - Q\|_2.$$

这个界直接说明了它什么时候可能有效：**当前样本的基底重叠结构不能与训练统计偏离太远。**

对常值 $V = \mathbf{1}c^\top$ ，还可以得到：

$$Z_{\text{corr}} = \mathbf{1}c^\top, \quad BZ_{\text{corr}} = \mathbf{1}c^\top.$$

这是该空间分支的常值保持性质，不是完整网络的物理守恒定理。

需要特别诚实地指出：

- 这不是完整的逐样本 Gram 校正：两个输入若具有相同的 s 、不同的重叠结构，会使用相同的校正。
- 当 s 几乎固定时，它接近一个固定 token 线性层。因此必须与普通可学习 T 比较。
- 它不继承精确版本的全部收缩性质，可能过度放大部分模式。
- 很小的 s_m 仍会带来问题。若设置质量下界，最好在软分配本身加入微小均匀分量，并让对照使用相同处理。
- 训练仍需收集 Gram 统计，所以首要承诺应是**推理成本较低**；训练速度另行验证。

训练前向也应使用同一种统计校正形式，不能训练时全部用精确求解，推理时突然换成固定统计。

完整模型可以这样构造：先用两个校正块作为最小候选，同时保留四块版本。

设 $H_\ell \in \mathbb{R}^{N \times D}$ 为点特征。每个校正块依次执行：

$$X_\ell = \text{LN}(H_\ell),$$

从 X_ℓ 经过基线已有的点映射或局部卷积，生成 V_ℓ 和两组独立的路由：

$$A_\ell = \text{softmax}_M(f_\ell^A(X_\ell)),$$

$$B_\ell = \text{softmax}_M(f_\ell^B(X_\ell)).$$

然后：

$$Z_\ell = \text{CorrectedSlice}(A_\ell, V_\ell),$$

$$\widehat{H}_\ell = H_\ell + \mathcal{O}_\ell(\text{ConcatHeads}[B_\ell G_\ell(Z_\ell)]),$$

$$H_{\ell+1} = \widehat{H}_\ell + \text{FFN}_\ell \left(\text{LN}(\widehat{H}_\ell) \right).$$

其中 $\mathcal{O}_\ell$ 保留所选基线的输出处理。

第一轮实验保持这些外围模块一致，只修改读写关系和校正。 否则同时改变卷积、FFN、宽度、路由，结果将很难归因。

这里每个模块的表达来源不同：

部分	提供的能力
输入提升、局部卷积	编码坐标、条件、局部几何和局部模式
输入相关 A_ℓ	根据当前样本选择如何汇聚信息
校正	调整重叠软基下的系数表示
独立 B_ℓ	将读取的模式用于不同的空间输出组合
可选 T	低成本的跨 latent 模式耦合
点级 FFN	非线性通道变换
点级残差	保留低秩分支未承载的信息
下一块重新生成 A, B	在更新后的特征上重新组织空间表示

最后一项尤其关键：**深层模型不只是重复计算同一套分片，它可能在不断改变分片依据。** 校正能否减少这种重新组织的需求，不能由条件数改善直接推出。

对于“直接输出还是持续 latent”，建议按下面方式准确计数：

结构	slice 次数	deslice 次数	latent 处理
两个完整校正块后输出	2	2	每块 $G = I$
第一块完整往返；第二次编码后做 T ，最后重建	2	2	一次线性 mixing
两个完整块后，再编码进入 latent，最后重建	3	3	增加了一次往返
第二次编码后堆叠 latent attention	2	2	多层非线性、输入相关处理

我推荐比较前两行，不建议为了制造“持久 latent 阶段”额外加入第三次编码。

第二行的实现还应保留最后一次编码前的点特征作为最终残差，并在末端进行一次输出处理。它不再逐层更新点空间，但仍可能保存一份点特征或解码路由，因此不能宣称内存从此只剩 $M \times D$ 。

如果后续只使用固定的 token 线性映射，那么不需要堆叠多层。

假设：

$$Z_{\ell+1} = T_\ell Z_\ell.$$

则：

$$Z_L = T_{L-1} \cdots T_0 Z_0 = T_* Z_0.$$

即使使用同型线性残差：

$$Z_{\ell+1} = Z_\ell + T_\ell Z_\ell,$$

也能合并为 $\prod_\ell (I + T_\ell)$ 。

所以，对你当前强调的纯 token 线性处理：

“第二次 slice 后在 latent 中做多层固定线性映射，再 deslice”，等价于“第二个校正块的中间放一个有效矩阵 T_* ”。

它并不会自动获得类似 LNO 多层 attention 的深度表达能力。

如果加入 GELU、LayerNorm 或输入相关门控，深度就可能有额外作用，但那已经是“线性 token mixing 加非线性处理”的另一种假设。

视觉领域的 TokenFuser 确实使用过 token 轴线性矩阵，再结合独立的空间重建权重；它比“LinearNO 中有 latent 线性层”这个说法更接近你设想的具体算子。不过原论文还包含 Transformer，不能据此推断纯线性 latent 主干已经被验证充分。[TokenLearner / TokenFuser](#)

读写关系还应保留一个很便宜、很有价值的结构对照：绑定分片，加固定的 latent 线性矩阵。

比较：

$$AE_AV, \quad ATE_AV, \quad BE_AV.$$

它们分别对应：

- 1. 绑定读写，无额外 mixing；
- 2. 绑定读写，固定 token mixing；
- 3. 独立读写，无额外 mixing。

第二种可以理解为有效合成矩阵 $B_{\text{eff}} = AT$ 。若 T 行随机，它仍是软分配；若 T 无约束，则有效合成可以有符号。

实现时应计算：

$$A[T(E_AV)],$$

不要显式构造 AT ，否则会引入 $O(NM^2)$ 的额外工作。

这组对照能直接回答你的架构猜想：究竟是“独立解码”重要，还是一个便宜的固定 latent mixing 就能补足大部分能力？

但它不是完全独立的 B ，其合成空间仍受 A 限制。并且它不一定减少参数：官方实现的路由权重跨 head 共享，新增路由参数约为 dM ，而一个共享 T 已有 M^2 个参数；当 $M > d$ 时，后者反而更多。

从实际代码看，减少往返确实存在明显的计算预算空间，但主要收益不能只归因于删掉 attention。

我核对了两份官方仓库及数据加载代码。下面是单次 forward 的数据格式，尚未下载数据或进行速度实测。

数据集	点数 N	单次输入 → 输出	原始文件与字段
Darcy	$85^2 = 7,225$	坐标 2 + 系数 1 → 解 1	.mat : coeff, sol ; 原 421^2 ，步长 5 下采样
Airfoil	$221 \times 51 = 11,271$	坐标 2 → Mach 1	NACA_Cylinder_X/Y/Q.npy，目标 Q[:,4]
Pipe	$129^2 = 16,641$	坐标 2 → 水平速度 1	Pipe_X/Y/Q.npy，目标 Q[:,0]
Elasticity	972	坐标 2 → 应力 1	Random_UnitCell_XY_10.npy、sigma_10.npy
Navier–Stokes	$64^2 = 4,096$	坐标表示 + 过去 10 帧 → 下一帧 1	.mat : u
Plasticity	$101 \times 31 = 3,131$	坐标 2 + 输入 1 + 时间条件 → 4 通道	.mat : input, output

来源：[Darcy](#)、[Airfoil](#)、[Pipe](#)、[Elasticity](#)、[NS](#)、[Plasticity](#)。

这些格式对效率评估有几个直接影响：

- NS 完整预测需要调用整个模型 **10 次**；Plasticity 按时间条件调用 **20 次**。要同时报告单次与完整任务延迟。
- Airfoil、Pipe 是曲线结构网格，不能简单等同于均匀笛卡尔网格。
- 现有加载器没有提供物理积分权重。直接设 $W = I/N$ 对应离散经验测度，不能声称已经使用面积或体积分。
- Darcy/NS 的部分配置使用到参考点的距离表示坐标，输入提升的实际通道数不只是表中的 2。
- 后续大 N 测试可使用 AirFRANS：LinearNO 训练代码每个样本采样 32,000 点，完整网格测试点数可变。[AirFRANS 训练代码](#)

最重要的代码发现是：

Transolver 的结构网格版本每块有两个 3×3 卷积；LinearNO 对应版本每块也有局部卷积、点级输出处理和 FFN。 因此减少完整块，通常会同时减少大量点空间计算。[Transolver 实现](#)、[LinearNO 实现](#)

以 LinearNO 的点模块配置估算，记总通道 $D = hd$ ，FFN 扩展比例为 r ，忽略 softmax、归一化等操作，单块主要 MAC 为：

非卷积版本： $N[(2 + 2r)D^2 + 4DM + Dd],$

卷积版本： $N[(11 + 2r)D^2 + 4DM + Dd].$

逐输入 Gram 的额外项约为：

$$NhM^2,$$

统计校正的额外项约为：

$$M^2D.$$

对应官方常用配置：

配置	原块 MAC/点	完整 Gram 增加 MAC/点	相对增加
卷积, $D = 128, h = 8, M = 64, r = 1$	247,808	32,768	13.2%
非卷积, 同样宽度	100,352	32,768	32.7%
NS 非卷积, $D = 256, h = 8, M = 32, r = 2$	434,176	8,192	1.9%

这也修正了此前可能过强的成本判断：**不能因为 Gram 比小规模 latent attention 贵，就推断整个块会贵很多倍。** 卷积和点级 FFN 往往才是主要运算。

以 Darcy 为例，保留 LinearNO 的点模块、按上述主项估算：

模型预算	主干 GMAC
8 块原始主干	14.32
2 块+统计校正	3.58
2 块+逐输入 Gram 校正	4.06, 尚未计小矩阵求解

这些数字排除了输入提升、最终输出、部分逐元素操作等，**不是延迟测试结果，更不代表精度已经相同。** 它们说明：只要少量校正块确实保住精度，存在可观的效率空间。

参数量方面，统计矩阵是 buffer，不增加训练参数；两个块相较八个块会减少大量主干参数。但 stem、输出头、独立路由、可选 T 都要计入，不能直接把总参数量写成原来的四分之一。

补充文献对方案的影响，主要是帮助我们划清已有工作与待验证贡献。

文献	与本方案最相关的内容	对研究定位的影响
Transolver++	分片同质化、逐点温度、Gumbel-Softmax 调整分配	单独强调更尖锐、更多样的 slice 已不够构成新贡献
Transolver-3	将可交换的线性投影移到 latent；分析计算与访存	应让基线和新模型共享适用的等价优化
FLARE++	明确研究更好的动态路由替代部分深度	“更好路由换更少层”已有直接相关研究
ONO	协方差统计、基底标准化、推理复用	支持统计校正路线，但统计复用本身不是新点
SVD-NO	分别约束左右基底	提醒我们区分各自正交与交叉对偶
TokenLearner / TokenFuser	少量 token、线性 token mixing、独立空间重建	提供直接相关的视觉架构参考

尤其是 FLARE++ 已经把路由质量与深度替代联系起来。因此，本研究更有辨识度的问题应是：

在保留 Transolver 软分片形式的条件下，对非正交读取系数的校正，是否能降低重复读写的需求？这种收益是否仍然成立于没有 self-attention、只有低成本线性 mixing 的模型？

实验应分阶段进行，首先验证机制，再讨论最终架构。

第一阶段只用固定深度，例如 2 块和 4 块，比较：

对照	要回答的问题
普通绑定读写, $G = I$	没有解耦、没有校正时怎样

对照	要回答的问题
普通独立读写， $G = I$	解耦本身贡献多少
独立读写 + 逐输入校正	重叠校正是否有效
独立读写 + 统计校正	便宜近似能保留多少收益
独立读写 + 自由线性矩阵	收益是否只是增加了普通 mixing
绑定读写 + 固定 T	固定 mixing 能否替代解耦

比较统计校正与自由矩阵时，应给它们相同的质量归一化、相同的常数模式处理，避免包装不同造成混淆。二者推理主成本一致，但自由矩阵的训练参数更多，需要如实报告。

第二阶段再比较 $G = I$ 、固定 T 、self-attention，重点观察：

attention 的收益 是否在加入校正后明显下降。

如果下降，支持“attention 的部分作用与表示校正有关”；如果没有下降，则说明它仍提供重要的任务相关动态交互。单个消融还不能证明 attention 原来就在近似 Gram 逆。

第三阶段才把最佳浅模型与官方 8 层 Transolver、LinearNO 比较。**必须同时保留普通的 2 层、4 层基线**，否则无法判断成果来自校正还是简单删层。

建议先从 Darcy、Elasticity、Pipe 开始：分别覆盖条件场、非规则点集、较大结构网格。随后加入 NS，检查误差在多步预测中的累积；最后用较大网格验证效率随 N 增长的趋势。

最终评价目标可以写成：

$$\min t_{\text{inference}} \quad \text{s.t.} \quad \mathcal{E}_{\text{new}} \leq \mathcal{E}_{\text{baseline}} + \delta,$$

其中误差容差 δ 预先确定。

至少报告任务误差、batch 1 延迟、吞吐、训练时间或达到目标误差的时间、峰值显存、参数与 buffer 大小。比较时固定精度、硬件、编译选项与计时方式；NS、Plasticity 同时报告完整预测耗时。训练损失也要对齐，例如官方 LinearNO 的 Darcy 训练包含梯度损失项。

我的实际优先级是：先做“独立读写 + 校正 + 2/4 次往返 + $G = I$ ”；再判断一个固定 T 是否值得加入。 只有当校正相较同成本自由 mixing 确有优势，并且少次往返在匹配误差下更快，才能把结论归于“更高效的 slice/deslice”。如果这些证据成立，你得到的就不只是一个浅模型，而是对“低秩神经算子为什么需要反复读写、哪些读写可以省掉”的具体回答。

2026-09-11 21:26:37 You 

解释一下上面的架构的一个block到底是什么架构，其中的slice和deslice采用的是何种方式

 ChatGPT gpt-6-astra-wm

上面方案中，一个 block 的核心是：

点特征 → 软 slice → latent 系数校正 → 独立软 deslice → 点特征更新

slice 和 deslice 都保留 Transolver 的加权汇聚、加权重建方式。改变的是：两者使用独立的分配权重，并在中间加入系数校正。主版本不再加入 self-attention。

下面按 Transolver 的 block 外壳，把每一步写完整。上一条借用 LinearNO 点模块计算的成本预算，与这里的具体外壳需要区分。

1. 输入经过归一化，生成用于分片的特征和值特征。

设输入为：

$$H \in \mathbb{R}^{N \times D},$$

其中 N 是点数， D 是特征维度。先计算：

$$X = \text{LN}(H).$$

然后使用两个映射：

$$U = P_{\text{route}}(X), \quad V = P_{\text{value}}(X).$$

它们的职责不同：

- U ：决定每个点如何分配到 latent，以及如何从 latent 读取信息。
- V ：实际需要被汇聚、处理和重建的特征。

沿用 Transolver 时，规则结构网格上的这两个映射可以保留原来的 3×3 卷积；不规则点集上保留原来的逐点线性映射。

随后划分成 h 个 head，每个 head 的通道数为 $d = D/h$ 。以下公式先描述一个 head：

$$U, V \in \mathbb{R}^{N \times d}.$$

2. slice 使用 Transolver 风格的“软分配+按分片质量归一化”。

首先由点特征生成分片权重：

$$A = \text{softmax}_M(UW_A + b_A), \quad A \in \mathbb{R}^{N \times M}.$$

这里的 softmax 沿 M 个 latent 的方向计算，所以：

$$\sum_{m=1}^M A_{im} = 1.$$

A_{im} 表示“第 i 个点，有多大比例被分配给第 m 个 latent”。

接着，对每个 latent 做加权平均：

$$(Z_0)_m = \frac{\sum_{i=1}^N A_{im} V_i}{\sum_{i=1}^N A_{im}}.$$

这就是普通的 Transolver 风格 slice。

它不是硬分区：一个点可以同时参与多个 latent；也没有用 LRSA/IPOT 的 cross-attention 编码器替换这一步。

需要区分两次归一化：

操作	归一化对象	目的
生成 A 时的 softmax	同一个点对应的 M 个 latent	决定这个点如何分配
汇聚时除以 $\sum_i A_{im}$	同一个 latent 接收的所有点	得到加权平均，避免点数多的分片仅因规模而幅值更大

3. 对 slice 得到的 latent 系数进行校正。

这一步输入、输出都在潜空间：

$$Z_0 \in \mathbb{R}^{M \times d} \longrightarrow Z_{\text{corr}} \in \mathbb{R}^{M \times d}.$$

为什么需要它？如果两个 latent 的软分配高度重叠，它们汇聚的点也高度重叠，得到的系数就可能包含较强的重复和串扰。

按照上一条推荐的统计校正版本，取：

$$w_i = \frac{1}{N}, \quad s = A^\top w, \quad S = \text{diag}(s), \quad u = \sqrt{s}.$$

由训练统计得到固定的小矩阵 $\bar{R}$ ，再结合当前输入的分片质量，构造：

$$R_s = uu^\top + (I - uu^\top)\bar{R}(I - uu^\top).$$

校正为：

$$Z_{\text{corr}} = S^{-1/2} R_s S^{1/2} Z_0.$$

这里：

- $\bar{R}$ 根据训练期间的分片重叠统计得到；
- s 来自当前输入，因此质量归一化仍然随输入变化；
- 常数模式的投影处理用于保留常值特征；
- 实现时不需要显式构造两个 $I - uu^\top$ ，只需小矩阵乘法和向量运算。

这一步本身就是一个 **latent 线性变换**，但它不是 **self-attention**。给定分配和校正矩阵后，它对 Z_0 是线性的；整个 block 因为路由、归一化和 FFN 等操作，仍然是非线性的。

主版本在校正后直接进入 deslice，也就是：

$$G = I.$$

此前讨论的可学习 TZ_{corr} 是额外对照，不是这个主版本必需的组成部分。

4. deslice 仍然是 Transolver 风格的加权重建，但权重独立学习。

从同一组路由特征 U 生成另一套权重：

$$B = \text{softmax}_M(UW_B + b_B), \quad B \in \mathbb{R}^{N \times M}.$$

其中：

$$W_B \neq W_A$$

表示两套参数独立学习，并非要求它们数值上必须不同。与 A 一样：

$$\sum_{m=1}^M B_{im} = 1.$$

然后重建点特征：

$$Y_i = \sum_{m=1}^M B_{im}(Z_{\text{corr}})_m.$$

矩阵形式就是：

$$Y = BZ_{\text{corr}}, \quad Y \in \mathbb{R}^{N \times d}.$$

deslice 不再除以分片质量。因为它是在每个点上，对不同 latent 进行组合，而 B 在这个方向上已经归一化。

这里“解耦”的具体含义是：

A 决定点特征如何被读取到 latent； B 决定 latent 如何被写回各个点。两者由独立参数产生，但保持一致的 latent 编号和通道接口。

原版 Transolver 在重建时复用 A ，即：

$$Y_{\text{Transolver}} = AZ_{\text{processed}}.$$

当前方案改为：

$$Y_{\text{new}} = BZ_{\text{corr}}.$$

5. 合并多头，经过输出映射和两次残差更新。

把所有 head 的 Y 拼接回 $N \times D$ ，然后：

$$H_{\text{mid}} = H + W_{\text{out}} \text{ConcatHeads}(Y).$$

再进行点级 FFN：

$$H_{\text{out}} = H_{\text{mid}} + \text{FFN}(\text{LN}(H_{\text{mid}})).$$

因此，一个完整 block 可以写成：

$$\begin{aligned}
&X = \text{LN}(H), \\
&U = P_{\text{route}}(X), \quad V = P_{\text{value}}(X), \\
&A = \text{softmax}_M(UW_A + b_A), \\
&B = \text{softmax}_M(UW_B + b_B), \\
&Z_0 = \text{MassNormalizedSlice}(A, V), \\
&Z_{\text{corr}} = \text{Correction}(A, Z_0), \\
&Y = BZ_{\text{corr}}, \\
&H_{\text{mid}} = H + W_{\text{out}} \text{ConcatHeads}(Y), \\
&H_{\text{out}} = H_{\text{mid}} + \text{FFN}(\text{LN}(H_{\text{mid}})).
\end{aligned}$$

以 Darcy 常用的 $N = 7225, D = 128, h = 8, M = 64$ 为例，设 batch size 为 b :

张量	形状
输入 H	$[b, 7225, 128]$
每头路由、值特征 U, V	$[b, 8, 7225, 16]$
slice 权重 A	$[b, 8, 7225, 64]$
deslice 权重 B	$[b, 8, 7225, 64]$
普通与校正后的 latent	$[b, 8, 64, 16]$
重建后的多头点特征	$[b, 8, 7225, 16]$
block 输出	$[b, 7225, 128]$

最后有一个与研究目标直接相关的准确表述：**当前这个“校正层”校正的是给定 A 后的 latent 系数，不是在一次前向中反复修改 A 的软分配。** A 会通过训练间接改变；下一 block 也会根据更新后的点特征重新生成 A, B 。因此，目前设计是“软分片+系数校正+独立重建”，还不是“显式迭代优化软分配”的模块。

这个校正的方向我暂时放弃，再在前面阅读的论文的基础上补充阅读一篇IPOT的论文，再思考一下几个问题：1.LRSA与IPOT最大的区别就在于LRSA的每层压缩于重建，而IPOT是只压缩和重建以此，问题在于LRSA反复的压缩和重建会带来表达能力怎样的损失，而IPOT一直在潜空间中做注意力计算会造成表达能力的损失吗

我会先暂停校正方案，重新核对 LRSA 和 IPOT 的信息流，重点区分“压缩丢掉的信息”与“反复重建、重新编码带来的表达能力”，再分析长期停留在潜空间的限制。

我会用 PDF 阅读流程核对架构图和残差路径。这里最关键的是：压缩后，原来的点特征是否仍通过残差保留下来——这会直接改变“反复压缩是否反复丢失信息”的判断。

论文确认了一个关键区别：LRSA 的点特征始终沿残差主干保留，而且重建时还直接用当前点特征作为 query；IPOT 则把后续计算的主状态放在 latent 中。因此，不能把 LRSA 简化为“每层都把全部信息压缩一次”。另外，IPOT 的 token 数少，也不必然意味着总信息维度更低，还要看每个 token 的通道数。

好的，先暂停校正方向。这次比较有一个需要首先纠正的前提：

LRSA 反复压缩、重建，并不意味着它每层都把全部点信息压缩后丢弃；IPOT 一直在潜空间计算，也不意味着它每层都在进一步损失信息。

两者更根本的区别是：

LRSA 把点特征作为持续保留的主状态，latent 是每层进行全局交互的中间表示；IPOT 把 latent 作为持续保留的主状态，后续计算依赖最初编码得到的信息。

因此，要分别讨论“全局交互受到什么限制”和“整个模型保留了哪些信息”。下面的架构事实来自论文及代码，关于表达能力的判断则是据此进行的数学分析。

1. 先看真实信息流：LRSA 有两条容易被忽略的点特征通路。

LRSA 的一个 block 可以概括为：

$$\begin{aligned} X_\ell &= \text{Norm}(H_\ell), \\ Z_\ell &= \text{DownAttention}(P_\ell, X_\ell, X_\ell), \\ Z'_\ell &= \text{LatentProcessor}_\ell(Z_\ell), \\ \Delta H_\ell &= \text{UpAttention}(X_\ell, Z'_\ell, Z'_\ell), \\ \widetilde{H}_\ell &= H_\ell + \Delta H_\ell, \\ H_{\ell+1} &= \widetilde{H}_\ell + \text{FFN}(\text{Norm}(\widetilde{H}_\ell)). \end{aligned}$$

其中：

- $H_\ell \in \mathbb{R}^{N \times D}$ 始终是点空间主状态。
- P_ℓ 是该层的可学习 latent queries（潜在查询）。
- latent 内部包含 FFN、self-attention、FFN，以及对应残差。
- 不同层重新生成自己的 latent 表示。

关键在于下面两条路径。

第一条是点级残差：

$$H_\ell \longrightarrow H_\ell + \Delta H_\ell.$$

没有进入本层 latent 的信息，仍然可能保留在 H_ℓ 中。

第二条是重建时的 query：

$$Q_{\text{up}} = X_\ell W_Q^\uparrow.$$

每个点直接根据自己的当前特征，决定从 latent 中读取什么。因此，即使两个输入产生相同的 latent，它们也可能因为点 query 不同而得到不同的全局更新。

所以 LRSA 的全局分支更准确地写成：

$$\Delta H_\ell = D_\ell(H_\ell, G_\ell(E_\ell(H_\ell))),$$

而不是只有：

$$D_\ell(G_\ell(E_\ell(H_\ell))).$$

这些结构在论文式（2）、（8）－（10）及官方实现中都有明确体现。[LRSA 论文](#)、[点特征残差实现](#)、[压缩与重建实现](#)

IPOT 则是：

$$Z_0 = E(a), \quad Z_{\ell+1} = F_\ell(Z_\ell), \quad \hat{u}(y) = D(y, Z_L).$$

编码使用可学习 queries 与输入做 cross-attention；中间在 latent 上进行带残差和 FFN 的 self-attention；解码 query 来自输出坐标及其编码。解码器自己的残差来自 query，**不是输入场的点特征残差**。[IPOT 图1](#)、[式（5）－（8）及附录式（12）](#)

比较项	LRSA	IPOT
持续保留的主状态	$N \times D$ 点特征	$M \times d_z$ latent
后续层能否重新读取点特征	能	标准 processor 不能
latent 是否跨层直接延续	每个 block 重新形成	持续更新
重建 query 来自哪里	当前点特征	输出坐标及其编码
输入场是否有点级残差通路	有	标准架构没有这条通路

比较项	LRSA	IPOT
低秩表示承担的职责	每轮全局信息交换	后续任务计算的主要信息载体

这里也需要细化“一次重建”：IPOT 对一个最终输出可以只解码一次，但在时序预测中可以每个时间步解码一次，同时继续在 latent 中推进，**不必把解码结果重新编码**。

2. LRSA 的表达限制，首先是“每次全局通信能传递多少信息”，而不是“整个状态只能剩多少信息”。

为了看清这一点，暂时冻结路由，忽略通道和非线性，把一次全局更新简化为：

$$K_\ell = B_\ell G_\ell E_\ell, \quad \text{rank}(K_\ell) \leq M.$$

这个条件表示：**该次全局更新受到有限空间模式数量的约束**。

如果某个细小涡结构、局部材料变化或尖锐界面没有被当前 latent 充分区分，它在这一轮向其他位置传递的消息可能就不够准确。

但是，真实主干还有残差：

$$H_{\ell+1} = (I + K_\ell)H_\ell.$$

即使 K_ℓ 低秩， $I + K_\ell$ 也完全可能满秩。经过多层：

$$H_L = (I + K_{L-1}) \cdots (I + K_0)H_0.$$

不能因为每层的全局分支低秩，就说完整网络一直只有 M 个表达方向。

在这个简化线性模型中，还可以得到：

$$\text{rank} \left[(I + K_{L-1}) \cdots (I + K_0) - I \right] \leq \min(N, LM).$$

这说明，不同层可以累积不同方向的全局更新，覆盖范围可能超过单层的 M 个方向。它只是可能性和上界，并不保证实际达到。

真实 LRSA 的路由依赖输入，还有多头、FFN 和点 query，因此上述公式用于解释残差的作用，不能直接当成完整非线性网络的秩定理。

同样，**有残差通路也不等于整个模型必然无损或可逆**：训练得到的更新仍可能抵消、覆盖某些信息。

3. LRSA 反复重建、重新压缩，既可能造成不充分的信息传递，也提供了新的表达机会。

可能的限制主要有三类。

其一，当前汇聚没有区分开的细节，无法充分参与这一轮全局交互。

多个不同区域如果被汇聚为相似的 latent 特征，后续 latent attention 只能处理这些汇总结果。这属于当前通信分支的瓶颈。

但这些细节仍可能留在点状态中，下一层可以换一种方式读取。

其二，跨层的表示传递没有自动的无损保证。

某层形成的 latent 会被写回点空间，下一层再重新读取。前后编码器、解码器并不要求互逆，所以某种中间表示不一定能完整传递。

不过，LRSA 的目的也不是逐层复原旧 latent。下一层可以主动重组信息，因此“没有保持同一套 latent 系数”不能直接认定为表达损失。

其三，错误的全局消息可能影响后续路由。

如果某层给点状态加入了不准确的上下文，后续压缩和重建又依赖这些更新后的特征，误差可能被放大。这是可能发生的学习和传播问题，不是反复往返必然发生的退化。

另一方面，多次往返提供了一种 IPOT 标准 processor 没有的计算过程：

读取全局信息
 → 更新每个点
 → 逐点非线性加工
 → 重新读取全局信息

一个极简例子可以说明它的意义。假设某次编码只保留均值：

$$E(a) = \frac{1}{N} \sum_i a_i.$$

对于：

$$a^{(1)} = (1, -1), \quad a^{(2)} = (0, 0),$$

编码结果都是零。但是它们的均方分别是 1 和 0。

如果保留点状态，模型可以先通过逐点非线性得到 a_i^2 ，下一次汇聚再读取均方。**重复读取点状态，允许模型先改变“要汇聚什么”，再进行汇聚。**

这个例子不表示 IPOT 学不会均方——一个合适的 encoder 完全可以一开始就编码均方。它说明的是：**一次编码遗漏的信息，不能靠后续处理补回来；多次访问点状态则保留了重新提取的机会。**

因此，把 LRSA 的反复往返全部看成负担，会漏掉它的一部分表达能力来源。

4. IPOT 的主要风险是最初的编码是否充分，而不是后续 attention 是否又做了一遍压缩。

固定输入、输出坐标和其他外部条件后，IPOT 关于输入场 a 的预测可以写成：

$$\widehat{\mathcal{G}}(a) = D_Y(F(E(a))).$$

如果两个输入满足：

$$E(a_1) = E(a_2),$$

那么无论中间有多少层 self-attention，都有：

$$\widehat{\mathcal{G}}(a_1) = \widehat{\mathcal{G}}(a_2).$$

如果真实解不同：

$$\mathcal{G}(a_1) \neq \mathcal{G}(a_2),$$

模型就不可能同时准确预测两者。由三角不等式：

$$\max_{j=1,2} \left\| \widehat{\mathcal{G}}(a_j) - \mathcal{G}(a_j) \right\| \geq \frac{1}{2} \left\| \mathcal{G}(a_1) - \mathcal{G}(a_2) \right\|.$$

这给出了一个明确限制：

更深的 latent attention 可以更充分地处理已经编码的信息，但无法重新区分编码器已经混为一谈的输入。

这里必须固定输出坐标等条件。IPOT 的 decoder 仍直接接收坐标，所以坐标提供的信息并没有全部经过 latent；不能把所有几何信息也一概说成被瓶颈截断。

还要区分“输入信息损失”和“任务所需信息损失”。如果任务只依赖某些统计量，那么 encoder 只保留这些统计量就足够了，不需要复原全部输入点值。

5. 一个非常重要的细节： $M \ll N$ ，本身不能证明 IPOT 必然丢失输入信息。

真正的 latent 标量维数是：

$$r = M d_z.$$

应该把它与原始输入的独立自由度，或者数据分布的内在维数比较，不能只比较 token 数量。

例如 IPOT 论文中 Darcy 使用：

$$N = 7225, \quad M = 256, \quad d_z = 64.$$

因此 latent 包含：

$$256 \times 64 = 16384$$

个标量，比固定网格上单通道系数场的 7225 个标量还多。**它压缩了 token 数量，却不能仅凭维数断言它一定丢掉原始系数场的信息。**当然，维数足够也不保证 attention 编码器实际上学到了充分表示。[IPOT 附录表 5](#)

更严格地，设原始输入自由度为 p ，网络可微、输出坐标固定、没有额外输入场旁路，则：

$$J_{\widehat{\mathcal{G}}} = J_D J_F J_E,$$

所以：

$$\text{rank}(J_{\hat{G}}) \leq r.$$

当目标映射需要超过 r 个独立输入变化方向时，这个瓶颈才构成明确的表达限制。

因此，需要同时考虑：

- latent 数量 M ；
- 通道数 d_z ；
- encoder 能否有效利用这些通道；
- 任务究竟需要多少独立信息。

6. IPOT 一直在 latent 中做 attention，不会因为“待得久”就必然越来越损失信息。

其 processor 始终处理：

$$Z_{\ell} \in \mathbb{R}^{M \times d_z}.$$

每层没有继续减少 M 或 d_z ，并且包含残差与 FFN。因此，它不是一个不断缩小的瓶颈。

后续层可以在相同状态维度中学习更复杂的非线性关系。甚至把残差分支设为零，就可以保留原来的 latent，这已经说明“每过一层一定丢一些信息”不是架构上的必然结论。

实际可能出现的问题包括：

- token 越来越相似；
- 部分变化方向被网络抑制；
- 有限状态容量无法同时承载所有任务信息；
- 深层计算出现优化或误差传播问题。

但这些需要实验验证，不能简单套用“反复加权平均必然过度平滑”的结论，因为 IPOT 还有输入相关权重、多头、通道投影、非线性和残差。

此外，**固定 latent 大小不等于固定物理基底**。IPOT 每层的 latent 内容、Q/K/V 和 attention 权重都在变化，decoder 也根据坐标和当前 latent 产生输出。

因此，小 latent 也不等于只能输出低频。例如：

$$u(y) = a \sin(\omega y) + b \cos(\omega y)$$

即使 ω 很大，只要频率已知，两个系数也能描述整个高频场。真正困难的是大量彼此独立的细节，而不是“高频”这个标签本身。能生成高频形状，也不代表能保留任意输入中的高频细节。

7. 对时间推进，IPOT 还有一个比“压缩次数”更深的条件：latent 是否足以决定未来。

IPOT 论文允许初始编码一次，随后反复推进 latent：

$$Z_0 = E(u_0), \quad Z_{t+1} = F_{\Delta t}(Z_t).$$

如果希望它对应真实动力学在编码空间中的演化，需要近似满足：

$$E(\Phi_{\Delta t}(u)) \approx F_{\Delta t}(E(u)).$$

其中 $\Phi_{\Delta t}$ 是真实时间推进。

如果两个物理状态当前编码相同：

$$E(u_1) = E(u_2),$$

但经过真实演化后：

$$E(\Phi_{\Delta t}(u_1)) \neq E(\Phi_{\Delta t}(u_2)),$$

就不存在一个仅依赖当前 latent 的确定性更新，能够同时正确处理两者。这叫 latent closure（潜空间闭合）问题。

直观地说：**某些当前看似不重要、被编码忽略的小尺度信息，可能影响未来的大尺度行为**。此时增加 latent processor 的深度，也无法弥补输入信息的缺失。不过这是一项需要检验的条件，并不是 IPOT 已被证明存在的普遍失败模式。

两者的表达能力取舍可以这样判断：

问题	LRSA	IPOT
某项细节没有进入当前 latent	仍可能保留在点状态，供后层读取	若最初未编码、也无其他输入旁路，后续不能重新获取
主要容量限制	每轮全局交互的容量	持续 latent 状态的容量与编码充分性
多层计算的主要作用	多轮全局读取、点状态更新、重新提取	对已有 latent 进行深入非线性处理
是否必然逐层损失更多信息	否	否
主要效率代价	每层都要访问、处理大量点	编解码集中在边界，深度主要作用于小 latent
同预算下的潜在优势	更充分保留点级细节与重新读取机会	可以把更多计算用于 latent 容量或处理深度

IPOT 自身的 ERA5 消融也显示，latent 数量从 64 增加到 256 时，误差从 1.45×10^{-2} 降到 6.87×10^{-3} 。这说明 latent 容量确实影响结果，但不能单独判断改善来自编码、处理中间状态还是解码，也不能据此与 LRSA 直接比较。[IPOT 表 3](#)

对你当前这个问题，我认为最值得保留的判断是：LRSA 多次往返的意义，在于能反复访问并更新点状态；IPOT 的关键要求，是一次编码形成的表示足以支撑后续任务。只有把这两个因素分开，才能判断哪些往返只是计算开销，哪些往返实际承担了不可替代的信息提取。

2026-09-12 10:36:19 You 

IPOT一直在潜空间中计算，会不会获得相较于每层都压缩重建效率/推理延迟/参数量上的优势，另外思考一个问题：transolver和linearNO的架构，现在的IPOT和LRSA仓库中对于规则网格的处理有卷积操作吗，无论有没有考虑这样一个问题为IPOT或LRSA在处理规则网格时加入卷积是否会提升表达能力（像transolver或linearNO代码仓库中一样的方式，一样的结构加入卷积）如果这样是不是像IPOT一样一直在潜空间中的效率优势更明显

 ChatGPT gpt-6-astra-wm

我会核对 IPOT 和 LRSA 仓库中规则网格分支的实际实现，再分别比较计算量、推理延迟和参数量。卷积放在编码前、每个点空间块内，还是 latent 内，会产生不同的成本和表达能力，需要分开讨论。

代码里有两个重要细节：IPOT 的公开实现没有空间卷积；LRSA 有卷积分支，但当前推荐训练配置中是 Darcy 启用，并非所有规则网格任务都启用。另外，IPOT 的 latent processor 在深度循环中复用了同一组权重，所以它的参数量优势也包含权重共享的影响，不能全部归因于常驻 latent。

卷积的位置也已核清：Transolver 在压缩前使用两条卷积分支，LinearNO 的卷积版本在生成点侧特征前使用一条共享卷积；LRSA 的 Darcy 分支则在重建后的点空间 FFN 中做卷积。这三种方式作用不同。只在 IPOT 编码端做少量卷积，确实能省下后续的重复卷积，但也少了“全局交互后再处理局部特征”的机会。

会有优势，尤其是计算量和中间特征的存储成本；但推理延迟、参数量需要分别判断。加入卷积后，如果 IPOT 只在编码端进行少量卷积，而另一模型每层都在点空间卷积，省下的计算通常会更多。

不过，当前仓库有两个会影响结论的重要细节：

- IPOT 没有空间卷积；LRSA 已有卷积分支，但当前推荐配置并非所有规则网格任务都启用。
- IPOT 的公开代码在 latent processor 的深度循环中共享权重。所以它参数少，不能全部归因于“一直在 latent 中计算”。

我重新核对了四个官方仓库。以下区分实际代码、理论成本和可能的表达收益；目前没有进行模型测速。

1. 常驻 latent 的主要优势，是避免每层处理全部 N 个点。

为了单独比较架构，先假设两种模型具有相同的：

- 点数 N 、latent 数 M ，且 $M \ll N$ ；
- 隐藏维度 D ；
- latent self-attention 总次数 L 。

对于 LRSA 式结构，每层都执行：

$$N \rightarrow M \rightarrow M \rightarrow N.$$

对于 IPOT 式结构，则是：

$$N \rightarrow \underbrace{M \rightarrow M \rightarrow \cdots \rightarrow M}_{L \text{ 次处理}} \rightarrow N.$$

只看注意力矩阵乘法的量级，省略共同常数：

$$C_{\text{LRSA}} = O[L(2NMD + M^2D)],$$
$$C_{\text{IPOT}} = O[2NMD + LM^2D].$$

因此，IPOT 避免了约 $L - 1$ 轮完整的点-latent 往返。

但收益不止这些矩阵乘法。LRSA 还在每层执行点侧投影、归一化和点级 FFN；在统一隐藏宽度的比较下，这部分包含：

$$O(LND^2).$$

IPOT 则将大部分深层处理放在 M 个 latent 上，对应：

$$O(ND^2 + LMD^2).$$

这才是常驻 latent 很有吸引力的地方：深度增加时，主要增加的是小规模潜空间计算，而不是整张网格上的计算。

指标	IPOT 式常驻 latent 的潜在优势	需要保留的条件
计算量	减少重复跨注意力、点侧投影、点级 FFN	在宽度、处理深度等条件相近时比较
训练激活内存	深层状态主要为 $M \times D$	编解码 attention 的实现仍可能影响峰值
推理延迟	减少对全部点的重复处理和访存	不能直接由 FLOPs 推出实际加速倍数
参数量	可减少重复的编解码模块	token 数变少本身不减少线性层参数

这也不代表 IPOT 在相同误差下必然更便宜。如果它需要显著增大 M 或 D 才能弥补一次编码的限制，部分优势会被抵消。

2. 当前 IPOT 与 LRSA 仓库的推理延迟，不能直接当成纯架构对比。

IPOT 的公开实现用：

```
python

sim = einsum(..., q, k)
attn = sim.softmax(...)
out = einsum(..., attn, v)
```

显式形成 attention 矩阵。LRSA 则支持调用 PyTorch SDPA 或 xFormers 的高效注意力实现。[IPOT attention 实现](#)、[LRSA attention 实现](#)

因此，有可能出现：

IPOT 理论计算更少，但旧式 attention 实现没有充分兑现延迟优势。

如果要判断“常驻 latent 究竟快多少”，应同时看：

- 1. 原仓库的实际速度；
- 2. 使用相同 attention 后端、精度和硬件后的架构对照。

小矩阵的 GPU 利用率、kernel 启动、张量排列和访存，都会影响最终延迟。卷积本身又是 GPU 上优化较充分的操作，因此不能把卷积 FLOPs 的变化直接换算为相同比例的耗时变化。

3. 参数量优势要拆开看，IPOT 当前代码的权重共享尤其重要。

一个线性层：

$$Y = XW, \quad W \in \mathbb{R}^{D \times D},$$

无论作用在 N 个点还是 M 个 latent 上，参数量都是：

$$D^2.$$

变化的是运行次数和计算量：

$$ND^2 \quad \text{与} \quad MD^2.$$

所以：

token 数减少，不等于该层参数按 M/N 减少。

IPOT 式结构可以因为只保留一套 encoder、decoder，而少于每层都有 down/up 模块的结构；但当前代码还有另一项因素：

```
python

self.processor_self_attn = ...
self.processor_ff = ...

for i in range(self_per_cross_attn):
    self.layers.append([
        self.processor_self_attn,
        self.processor_ff
    ])
```

循环加入的是**同一组模块对象**。因此，processor 多执行几次，计算量增加，但独立参数不随之增加。构造函数中的 `weight_tie_layers` 参数在这份代码中并没有控制这段行为。[IPOT processor 实现](#)

LRSA 则逐层创建独立 block。

因此，参数量至少要拆成：

$$\text{编解码模块数量} + \text{processor 是否共享权重} + \text{宽度、FFN 和 latent queries 的配置}.$$

不能将所有差异都归因于常驻 latent。

4. 四个仓库中的卷积情况，实际是下面这样。

这里说的是具有空间邻域作用的卷积，例如 3×3 Conv2d，不把逐点 Linear 也算成空间卷积。

模型	当前公开实现中的空间卷积	放置位置
Transolver	结构网格版本有	压缩前，两条独立卷积分支分别生成路由特征和值特征
LinearNO	Conv 版本有	压缩前，一条共享卷积生成后续点侧 Q/K/V 所需特征
IPOT	当前模型路径没有	输入网格展平，结合坐标编码后做 cross-attention
LRSA	有可选结构网格版本	完整全局交互结束后，在点空间 FFN 分支中卷积

Transolver 的两条分支是：

$$U = \text{Conv}_{3 \times 3}^{\text{route}}(H), \quad V = \text{Conv}_{3 \times 3}^{\text{value}}(H).$$

前者影响分片权重，后者提供被汇聚的值。[Transolver 实现](#)

LinearNO 的卷积版本则先得到：

$$U = \text{Conv}_{3 \times 3}(H),$$

再从 U 生成点侧 Q/K/V。其官方脚本中，Darcy、Airfoil、Pipe、Plasticity 使用卷积版本；**NS 虽然是规则网格，使用的却是非卷积版本**，Elasticity 也没有卷积。[LinearNO 实现](#)、[NS 配置脚本](#)

IPOT 的网格预处理会展平空间维度，并加入 Fourier 坐标特征，但没有邻域卷积。[IPOT 预处理](#)

LRSA 当前推荐训练脚本中：

任务	是否启用空间卷积
Darcy	是
NS	否
Airfoil	否
Pipe	否
Plasticity	否
Elasticity	否

Darcy 脚本调用 `darcy-avoid-overfit.yaml`，其中明确设置 `enable_conv: true`；其他上述任务的当前脚本没有统一启用这一路径。[Darcy 脚本](#)、[Darcy 配置](#)

LRSA 已有的卷积块可以概括为：

$$\begin{aligned}\widetilde{H} &= H + \text{LRSA}(\text{Norm}(H)), \\ H' &= \widetilde{H} + \text{ConvFFN}(\text{Norm}(\widetilde{H})),\end{aligned}$$

其中：

$$\text{ConvFFN}(X) = W_2\sigma\big(W_1 \text{LN}\big(\text{Conv}_{3\times 3}(X)\big)\big).$$

它的卷积发生在重建之后，和 **Transolver**、**LinearNO** 的压缩前卷积不是同一个位置。代码中的变量虽然叫 `dwconv`，但没有设置 `groups=D`，实际是普通卷积，不能按 `depthwise convolution`（逐通道卷积）的低成本来估算。[LRSA 结构网格实现](#)

5. 加入压缩前卷积，有合理的表达收益，尤其可能改善 IPOT 的一次编码。

其作用是让编码器读取的对象从：

某个点自身的特征

变成：

这个点及其邻域的特征.

例如，在汇聚到 `latent` 之前，先提取局部变化、界面附近的模式、方向性和梯度相关特征。这样，一次编码就有机会汇聚已经加工过的局部信息。

如果采用接近 **Transolver** 的双分支方式，可以对 IPOT 的 `encoder` 写成：

$$\begin{aligned}K_{\text{in}} &= \text{Conv}_K(H), \quad V_{\text{in}} = \text{Conv}_V(H), \\ Z_0 &= \text{CrossAttn}(Q_{\text{latent}}, K_{\text{in}}, V_{\text{in}}).\end{aligned}$$

这里卷积定义在原始网格上，`latent query` 仍按 IPOT 的方式产生。这是把“路由特征、值特征分别局部化”的思想迁移过来，**没有把 IPOT 改成 Transolver 的 `soft slice`**。

接近 **LinearNO** 的方式则是：

$$\begin{aligned}U &= \text{Conv}_{3\times 3}(H), \\ Z_0 &= \text{CrossAttn}(Q_{\text{latent}}, UW_K, UW_V).\end{aligned}$$

两者区别是局部特征是否共享。

对 LRSA，也可以在 `down-attention` 前做相同处理。但它已有的 Darcy 卷积在重建后，因此：

- 压缩前卷积**：影响编码器读入哪些局部模式；
- 重建后卷积**：让点特征结合全局信息后，再进行局部加工。

这两种作用不能互相同。如果在现有 LRSA Darcy 模型上再加两条前置卷积，就可能变成每个 block 三个普通 3×3 卷积，成本需要重新计算。

数学上，在输入输出通道一致时， 3×3 卷积可以通过将非中心位置权重设为零，退化为逐点线性投影。因此，这种替换扩大了局部投影模块可选择的映射集合。

但它不保证测试精度提高，原因包括：

- 更大的参数量可能带来过拟合；
- 局部特征不一定是当前任务缺少的信息；
- 只给 value 加线性卷积，可能仍被后续汇聚抵消，未必产生新的有效统计；
- 固定索引网格上的 3×3 邻域，其物理尺度随分辨率变化。

我认为“让 key/路由也感知邻域”比单纯给 value 增加一个线性滤波，更值得认真验证。前者能改变“如何汇聚”，而不仅是“汇聚的值经过什么线性处理”。

还有一个结构边界：IPOT 的 learned latent 没有天然的二维物理邻接。不能把 M 个 latent 随意排成 $\sqrt{M} \times \sqrt{M}$ ，再说上面的卷积等同于原网格卷积。

6. 加卷积以后，常驻 latent 能省下多少计算，可以明确估算。

普通 $D \rightarrow D$ 的 3×3 卷积，在忽略 bias 时：

$$P_{\text{conv}} = 9D^2,$$
$$\text{MAC}_{\text{conv}} = 9ND^2.$$

以：

$$N = 85^2 = 7225, \quad D = 128$$

为例：

卷积配置	卷积 MAC	卷积权重参数量
一条卷积，执行一次	1.065 GMAC	0.147M
八层，每层一条独立卷积	8.523 GMAC	1.180M
两条卷积，执行一次	2.131 GMAC	0.295M
八层，每层两条独立卷积	17.046 GMAC	2.359M

这张表只统计卷积，不是整网成本，也不是测速结果。若同一个卷积共享权重、反复执行八次，参数量不会乘八，但计算量仍然乘八。

如果是将已有的同维逐点线性投影替换为卷积，而不是直接新增卷积，那么**净增加**的主要成本应按：

$$(9 - 1)ND^2 = 8ND^2$$

计算，参数净增加按 $8D^2$ 计算。

所以，如果 LRSA 式模型每层都卷积，而 IPOT 只在编码端卷积一次，确实可以避免大量重复的原网格局部计算。

但“优势更明显”还要分清绝对节省和加速倍数。设原成本为：

$$C_{\text{LRSA}} = A, \quad C_{\text{IPOT}} = B,$$

每次卷积成本为 c ，LRSA 做 L 次，IPOT 做 k 次，则：

$$C'_{\text{LRSA}} - C'_{\text{IPOT}} = (A - B) + (L - k)c.$$

当 $L > k$ 时，绝对省下的 MAC 增加了。

然而成本比：

$$\frac{A + Lc}{B + kc}$$

不一定比原来的 A/B 更大。因此不能无条件说相对加速倍数也一定上升，实际延迟更需要测量。

7. 少做卷积省下的计算，对应着少做某种计算过程。

IPOT 加输入卷积后的过程是：

局部特征提取 → 一次全局编码 → 多层 latent 处理 → 输出。

逐层回到点空间的模型则可以反复进行：

局部处理 → 全局交互 → 更新局部状态 → 再次局部处理。

后者的下一次局部卷积，看到的是已经包含全局上下文的新特征。一开始的卷积不能自动替代这种过程。

例如，某个局部模式的处理方式，可能要等模型获得整体边界条件或全局流动状态后才能决定。IPOT 可以尝试在 latent 中完成相应推理，但并没有天然保证它与点空间的反复局部处理等价。

另外，IPOT 若在解码后增加输出卷积：

- 对单个最终输出，可以只运行一次；
- 对逐帧输出的时间预测，输出卷积需要每帧运行；
- 输入端卷积仍可只在初始编码时运行一次。

因此，我认为你的推断成立于一个明确条件下：**少量网格卷积已经提取了足够的局部信息，后续主要依赖全局 latent 计算即可完成任务。**在这个条件下，常驻 latent 能同时省下反复读写和反复卷积的成本。

值得检验的比较是“少量局部编码+持续 latent 处理”与“多轮局部-全局交替”，并在相同误差下比较延迟。当前 LRSA 的 Darcy 分支已经提供了后者的一个实际参照；IPOT 的公开实现则提供了前者的无卷积起点。

2026-09-12 10:49:46 You 

如果IPOT相较于LRSA起一个额外的修饰词，你觉得一下哪种前缀词或额外的修饰词更合理呢（**Efficient**：最通用，强调 FLOPs、延迟、内存、能耗更低。
例：EfficientNet、EfficientDet、EfficientFormer、EfficientViT、EfficientSAM。

- **Lightweight / Light-weight / Lite**：强调模型小、参数少。
例：Lite-HRNet、MobileViT 副标题里的“Light-weight”。

- **Mobile / Edge / On-device / Embedded / IoT**：强调部署场景。
例：MobileNet、MobileViT、EdgeNeXt、MCUNet。

- **Tiny / Micro / Nano / Compact / Small**：强调极小模型。
例：TinyViT、MicroNet、CompactNet。

- **Fast / Real-time / Accelerated / Speed**：强调推理速度。
例：FastViT、FasterNet、YOLO、ThunderNet。

- **Green / Energy-efficient / Low-power**：强调能耗友好。

- **Sparse / Pruned / Quantized / Distilled / Low-rank**：强调压缩/加速技术，不一定是架构名，但常出现在标题。），或者你有更好的其他修饰词

 ChatGPT gpt-5.6-sol-wm

我会按“这个词是否准确描述架构，而不是提前宣称实验结果”来筛选，并检查候选名称是否已被相近模型大量使用。当前最可能合适的方向不是 Lite/Tiny/Fast，而是突出“一次编码后长期驻留 latent”的结构属性。

命名上我建议把“结构事实”和“实验结果”分开： Persistent-Latent 描述的是已经确定的计算路径； Efficient/Fast/Lightweight 则需要分别用 FLOPs、实测延迟或参数量来证明。这样标题和模型名更不容易被审稿人抓住表述过度的问题。

我最推荐的不是 Efficient，而是：

Persistent-Latent IPOT，缩写 PL-IPOT

中文可译为“持续潜空间 IPOT”或“潜空间驻留式 IPOT”。

它最准确地描述了 IPOT 相较 LRSA 的核心结构差异：

Encode once → process repeatedly in latent space → Decode.

主干状态持续驻留

LRSA 则是：

$$\underbrace{\text{Compress} \rightarrow \text{latent process} \rightarrow \text{reconstruct}}_{\text{每个 block}} \times L.$$

LRSA 论文也明确把空间点特征作为主状态，把 latent 当作每层内部的低秩通信媒介；IPOT 则把 latent 作为 processor 的主状态，并使处理深度与输入、输出点数解耦。[LRSA](#)、[IPOT](#)

候选词可以这样排序：

修饰词	推荐度	适合表达什么	主要问题
Persistent-Latent	最高	主状态持续位于潜空间	“persistent”要说明是跨网络深度，而不一定是跨时间记忆
Latent-Core	很高	主干计算发生在 latent 中	没有直接强调只编码、重建一次
Encode-Once	较高	只进行一次输入编码	可能被误解为 encoder 只有一层
Depth-Decoupled	较高	网络深度与点数 N 的计算解耦	偏理论描述，作为模型名略生硬
Efficient	有结果后推荐	FLOPs、显存、延迟等综合效率	需要实验支撑，而且含义太宽
Scalable	一般	大规模点集上的扩展能力	LRSA 同样对 N 近线性，区分度有限
Lightweight/Lite	暂不推荐	参数量较小	IPOT 参数少还受 processor 权重共享影响
Fast/Accelerated	暂不推荐	推理延迟低	必须有统一实现后的实测结果
Compact/Tiny	不推荐	极小参数模型	不能体现一次编码、持续 latent 的机制
Low-rank	不推荐	低秩瓶颈	LRSA 本身也是低秩模型，无法区分
Mobile/Edge/Green	不推荐	部署或能耗	需要专门的设备与能耗实验

如果希望名称更像一个自然的模型名，我认为下面两个版本最好。

方案一：最准确地描述架构

Persistent-Latent IPOT (PL-IPOT)

论文中第一次出现时定义为：

Persistent-Latent denotes that the pointwise input is encoded once, while the main hidden state remains in the latent space throughout the processor depth and is decoded only at the output boundary.

中文可以写：

“Persistent-Latent”表示输入场只在网络边界被编码一次，随后主状态持续驻留在潜空间中完成多层计算，最终再映射到输出空间。

为了避免把它理解成时序记忆，可以进一步写成：

depth-persistent latent state（跨网络深度持续保留的潜状态）

方案二：更像简洁的架构名

Latent-Core IPOT (LC-IPOT)

这里的 “Latent-Core” 表示：

- encoder 和 decoder 位于网络边界；
- 深层计算核心位于 latent；
- 与 LRSA 的 point-state backbone（点状态主干）形成清楚对比。

这个名字比 Persistent-Latent 更顺口，也较少产生“跨时间持久记忆”的歧义。因此：

- 做论文中的机制分类时：用 Persistent-Latent ；
- 给具体模型命名时：可以用 Latent-Core IPOT 。

Encode-Once IPOT（EO-IPOT）也很直观，但只能强调输入侧：

$$Z_0 = E(a)$$

没有充分表达后续多层都在 latent 中计算这一特点。时序 IPOT 还可能每个时间步都解码一次，所以 Encode-Once-Decode-Once 并不适用于所有任务。

如果必须从你列出的传统前缀中选，我会选择：

Efficient

但应该在实验后再正式使用。例如只有在匹配精度下证明确实降低了推理时间、FLOPs 或峰值显存，才命名为：

Efficient Persistent-Latent Operator Transformer

这里两个词的职责不同：

- Persistent-Latent 说明为什么可能更高效；
- Efficient 表示实验已经证明效率优势。

还有一个研究定位上的边界：IPOT 原始架构本身已经是一次编码后持续在 latent 中处理。因此 Persistent-Latent IPOT 可以作为与 LRSA 对比时的类别名称，但不能单独作为新模型的创新命名。如果你的新模型进一步加入压缩前卷积，那么真正体现新增设计的名称应当是：

Locality-Enhanced Persistent-Latent IPOT

或更简洁地：

Locality-Enhanced IPOT（LE-IPOT）

此时：

- Locality-Enhanced 对应你的新增局部卷积编码；
- Persistent-Latent 对应继承自 IPOT 的计算拓扑；
- Efficient 留到完成效率实验后再放入论文标题。

综合准确性、可解释性和审稿风险，我的最终选择是：

Persistent-Latent IPOT (PL-IPOT)

如果作为你后续新模型的正式名称，并且主要改进确实是卷积增强的一次编码器，则使用：

Locality-Enhanced Persistent-Latent Operator Transformer

会比 Efficient-IPOT 更能直接说明模型到底改变了什么。

我现在的打算是在前面几层使用LRSA风格的block（decoder的query暂定保持LRSA形式的，即query与输入点有关，LRSA的设置，总之完全使用LRSA的单个block），在后续的block使用IPOT风格持续在潜空间中计算，同时在decoder部分也保持IPOT的处理方式，即query与输出位置有关，保持其超分辨率的能力，你可以综合这两种模型的优势，说明两个阶段分别有什么功能，由什么优势，两个阶段又分别由什么瓶颈，相较于LRSA的每层压缩和重建会获得什么优势，从上面的命名参考中为这个新模型起一个新名字

我会把这个模型明确为“前段完整 LRSA 块、一次进入持续潜空间的编码、后段 IPOT 处理与位置查询解码”，重点检查阶段衔接、超分辨率条件，以及它相对全程 LRSA 究竟省掉哪些计算。

这里有一个关键链接：前面的 LRSA 块既然完整保留，输出仍是点特征，因此进入后段前还需要一次编码。这个模型的合理分工是“先保留点状态做几轮特征更新，再进入持续潜空间做深层计算”；省下的是后段反复访问点空间的成本，但最终编码的信息瓶颈仍然存在。

这个组合在架构上是兼容的。它的核心价值，是先用少量 LRSA 块充分加工点特征，再把后续深度放到较小的潜空间中。 前段负责提高“进入潜空间的信息质量”，后段负责降低“继续增加计算深度的成本”。

我建议暂命名为 **S2LNO: Spatial-to-Latent Neural Operator，空间到潜空间神经算子**。这里的 Spatial-to-Latent 强调的是**网络主状态从点空间处理转入持续潜空间处理**。从你之前列出的修饰词中，**Efficient** 最适合它的研究目标；是否正式写进标题，应由相同精度下的效率结果支持。

下面严格按照你现在指定的结构分析：前段保留完整 LRSA block，后段采用 IPOT 的 latent processor，最终采用位置 query 解码。

首先，两个阶段之间需要一次明确的编码衔接。

设输入位置为 $X = \{x_i\}_{i=1}^{N_{\text{in}}}$ ，输入场为 $a(X)$ ，输出查询位置为 $Y = \{y_j\}_{j=1}^{N_{\text{out}}}$ 。模型可以写成：

$$\begin{aligned} H_0 &= \text{Lift}(a(X), X), \\ H_\ell &= \text{LRSAblock}_\ell(H_{\ell-1}), \quad \ell = 1, \dots, k, \\ Z_0 &= E_{\text{IPOT}}(H_k, X), \\ Z_{j+1} &= \text{Processor}_j(Z_j), \quad j = 0, \dots, r-1, \\ \hat{u}(Y) &= D_{\text{IPOT}}(\text{PE}(Y), Z_r). \end{aligned}$$

其中：

$$H_\ell \in \mathbb{R}^{N_{\text{in}} \times d_p}, \quad Z_j \in \mathbb{R}^{M \times d_z}.$$

部分	主状态与操作	query 来源
前 k 个完整 LRSA 块	点特征 $\rightarrow$ 临时 latent $\rightarrow$ 点特征	down 为可学习 latent query；up 为当前点特征
阶段衔接 encoder	将最终点特征 H_k 编码为 Z_0	可学习 latent query
后 r 个 IPOT processor 块	持续更新 M 个 latent token	当前 latent 特征
最终 IPOT decoder	从 latent 查询输出场	输出位置的编码 $\text{PE}(Y)$

这里有一个容易遗漏的细节：**完整 LRSA 块输出的是点特征，所以前段结束后，还要额外编码一次。**

不能直接取最后一个 LRSA 块内部的 latent 作为后段输入，因为这样会绕过该块随后发生的 up-attention 和点级 FFN；它们的更新便无法影响最终结果。那就不再是你指定的“完整使用 LRSA 单个 block”。

因此，对一次最终场输出，整个模型包含：

压缩 $k + 1$ 次，重建 $k + 1$ 次

前 k 次重建发生在输入点集 X ，最后一次重建发生在输出点集 Y 。

前段的功能，是在仍能访问完整点状态时，进行几轮全局信息交互和点特征更新。

用 $\text{CA}(Q, K, V)$ 表示包含投影的 cross-attention，一个完整 LRSA 块的主要结构可以概括为：

$$\begin{aligned} \bar{H}_\ell &= \text{Norm}(H_\ell), \\ T_\ell &= \text{CA}_\ell^\downarrow(P_\ell, \bar{H}_\ell, \bar{H}_\ell), \\ T'_\ell &= G_\ell(T_\ell), \\ U_\ell &= H_\ell + \text{CA}_\ell^\uparrow(\bar{H}_\ell, T'_\ell, T'_\ell), \\ H_{\ell+1} &= U_\ell + \text{FFN}_{p,\ell}(\text{Norm}(U_\ell)). \end{aligned}$$

其中 G_ℓ 保留 LRSA 原来的 **latent FFN $\rightarrow$ latent self-attention $\rightarrow$ latent FFN**，以及相应归一化和残差。这些模块、down/up 投影和 latent query 均按原 LRSA 块的设置保留。[LRSA §3.4](#)

它有三项直接作用。

- **让后一次读取依赖前一次获得的全局信息。** 第一层完成全局汇聚和回写后，第二层面对的是更新后的点特征。后续 down-attention 因而可以采用不同的读取权重，提取新的任务相关信息。
- **利用点特征决定如何接收全局信息。** up-query 来自当前点状态，因此同一个 latent 信息可以根据不同点的状态，被不同程度地读取。

- **保留再次访问点级细节的机会。** 点残差保留了一条不经过当前 latent 瓶颈的路径。当前 down-attention 没有提取的信息，后续块仍可能从点状态中访问。

因此，前段可以称为 **spatial refinement，点空间特征细化阶段**。这里的“空间细化”指更新点状态；LRSA 的全局注意力本身并非局部算子，普通逐点 FFN 也不等于邻域卷积。

它相较直接使用 IPOT encoder 的潜在优势是：**最终编码器读取的 H_k ，已经包含若干轮全局上下文和非线性加工**。这可能让同样大小的 latent 更容易承载预测所需的信息。

但这仍然是待验证的假设。端到端训练并不保证前 k 层一定形成充分表示，也不保证它们比扩大原 IPOT encoder 的容量更划算。

前段的瓶颈也很明确：每一层仍要处理全部 N_{in} 个点，执行 down/up-attention、点特征投影和点级 FFN。随着 k 增加，计算量、训练激活和访问点状态的开销都会上升。

后段的功能，是对已经形成的全局表示进行更深的潜空间计算。

IPOT 风格的 processor 可以写成：

$$\begin{aligned}\tilde{Z}_j &= Z_j + \text{SA}_j(\text{Norm}(Z_j)), \\ Z_{j+1} &= \tilde{Z}_j + \text{FFN}_j(\text{Norm}(\tilde{Z}_j)).\end{aligned}$$

这里每层继续做 latent self-attention 和非线性 FFN。固定的是 token 数量 M ，而 latent 内容、Q/K/V 和注意力权重都会随输入和层状态变化。IPOT，模型结构

它的优势是：**增加后段深度时，不必重新读取和更新全部输入点。**

在固定宽度和 FFN 扩张比例下，每增加一个后段块，主要计算量为：

$$O(M^2 d_z + M d_z^2).$$

当 $M \ll N_{\text{in}}$ 时，这通常比再增加一个完整 LRSA 块便宜，也减少了大点特征张量的反复读写。

不过，进入后段后，模型也失去了重新访问点状态的机会。把前段和最终编码器合起来记为：

$$E_{\text{total}} = E_{\text{IPOT}} \circ \text{LRSAblock}_k \circ \cdots \circ \text{LRSAblock}_1 \circ \text{Lift}.$$

固定输入、输出坐标和其他条件，如果两个输入满足：

$$E_{\text{total}}(a_1) = E_{\text{total}}(a_2),$$

那么后续无论增加多少 latent 层，都有：

$$\hat{u}(a_1, Y) = \hat{u}(a_2, Y).$$

前段改善的是最终编码可能保留下来的信息，后段无法补回最终编码已经无法区分的信息。

对于可微网络，还可以从链式法则得到：

$$\text{rank}\left(\frac{\partial \hat{u}(Y)}{\partial a(X)}\right) \leq M d_z.$$

这说明整个输入场到输出场的依赖必须经过 $M d_z$ 维 latent 状态。但不能仅凭 $M < N_{\text{in}}$ 就断言一定丢失信息：还要考虑通道数，以及任务实际需要多少独立信息。

两阶段的功能和瓶颈因此可以对应起来：

阶段	主要功能	主要优势	主要瓶颈
前段 LRSA	反复读取全局信息，并更新点状态	保留点级旁路；根据更新后的特征重新提取信息	每层仍承担点空间计算和访存
后段 IPOT	在持续 latent 中深化全局表示	新增深度的成本主要取决于 M	无法重新读取最终编码之外的信息
位置 decoder	将 latent 表示转换为指定位置的场值	输出位置与输入点集可以独立选择	查询精度受 latent 信息和解码能力限制

最终保留位置 query，可以保留 IPOT 的输出位置查询能力。

最终 decoder 的注意力权重可以概括为：

$$A_{\text{out}}(Y, Z_r) = \text{softmax}\left(\frac{Q(\text{PE}(Y))K(Z_r)^\top}{\sqrt{d}}\right).$$

虽然 query 只由输出位置构造，但 K/V 来自当前样本的 latent，所以解码仍然依赖输入场。

前段残差在输入点集 X 上完成更新，最终 decoder 在输出点集 Y 上查询，两者没有形状冲突。于是可以保持同一个输入，换用更密集的 Y 输出预测，而不需要为新增输出点提供 LRSA 风格的输入点特征。

这里应当准确区分两件事：

- 结构上支持更密集或不同位置的输出查询。
- 在这些位置上取得可靠的超分辨率精度。

前者由位置 decoder 提供；后者需要实验验证。位置 query 本身不保证恢复输入采样无法确定的细节。LRSA 原模型也展示了分辨率泛化，因此新模型更明确的接口优势是**输入与输出点集可以独立选择**，不能笼统表述为“只有新模型支持超分辨率”。[LRSA，分辨率泛化实验](#)

相较全程 LRSA，最直接的收益是减少后段的跨空间往返，而不是自动获得更强的表达能力。

先设输入、输出点数相同，均为 N ，统一宽度为 D ，所有 latent 大小为 M 。将总 latent self-attention 次数对齐为：

$$L = k + r.$$

忽略常数和不同 FFN 配置后，全程 LRSA 的主要成本为：

$$C_{\text{LRSA}} = O(LND^2 + LNMD + LMD^2 + LM^2D).$$

你的两阶段模型为：

$$C_{\text{S2LNO}} = O((k + 1)ND^2 + (k + 1)NMD + LMD^2 + LM^2D).$$

主要变化是，点空间相关项的重复次数从 L 降到约 $k + 1$ 。这里的公式是统一宽度下的架构估计；实际 IPOT decoder 的输出维度、LRSA 的 FFN 数量等会改变常数。

例如取 **前段 2 个 LRSA 块，后段 6 个 latent processor 块**：

项目	8 个完整 LRSA 块	2 个 LRSA + 6 个 latent 块
latent self-attention 次数	8	8
压缩次数	8	3
重建次数	8	3
cross-attention 次数	16	6
attention 模块调用总次数	24	14
原 LRSA 点级 FFN 次数	8	2，另计编解码端处理

这个例子中，减少的是 **5 次完整往返**。因为前段之后还需要一次编码，不能将减少量算成 6 次。上述计数也不等于相同参数量或相同精度下的比较。

各项收益应分别判断：

指标	可以期待的优势	需要注意的条件
计算量	减少重复的点投影、cross-attention 和点级 FFN	额外编码、最终解码以及 latent 宽度必须计入
推理延迟	减少大点张量访问和部分串行模块调用	实际收益受 attention kernel、batch size 和 GPU 利用率影响
训练显存	后段主要保存较小的 latent 激活	前段仍需保存点激活；收益受 checkpoint 和 attention 实现影响
参数量	可省去后续 LRSA 块独立的 down/up 和点级模块	参数不随 token 数直接按 N/M 缩小
准确率	同预算下可能把节省的资源用于更合适的 latent 容量或深度	同宽同深度下没有必然提升，可能因失去后续点更新而下降

指标	可以期待的优势	需要注意的条件
输出灵活性	可在独立的输出点集上查询	输出分辨率泛化与输入离散化泛化应分别测试

参数比较还有一项实现细节：**当前官方 IPOT processor 在不同深度重复使用同一组 attention 和 FFN 权重**。若直接沿用代码，这会额外减少参数；这项收益来自权重共享，需要与持续 latent 计算本身区分。[IPOT_processor 源码](#)

若前段采用 LRSA 的规则网格卷积版本，后段切换还会减少原网格卷积的重复次数。但同时也减少了后续逐层进行局部邻域加工的机会，不能只计算节省而忽略这部分表达能力的变化。

尤其不能把模型优势直接解释成“压缩重建少了，因此信息损失一定更少”。完整 LRSA 有点残差，而且 up-query 依赖点特征，主状态并没有在每层都被完全替换成低维 latent。你的模型在阶段切换后，反而明确引入了一个整个后段都必须经过的有限维信息瓶颈。

所以，这个设计最值得验证的研究假设是：

经过少量完整的点空间交互后，预测所需的信息是否已经足够集中，使后续计算主要在潜空间完成，就能以更低成本达到全程 LRSA 的精度？

如果成立，前段就为持续 latent 提供了更好的入口；后段则把继续深化表示的成本降下来。相较原始 IPOT，你支付了额外的前段成本；相较全程 LRSA，你省去了大量后段点空间往返。最佳切换深度 k 正是这一方案的核心研究变量。

对于命名，我建议采用：

S2LNO – Spatial-to-Latent Neural Operator
空间到潜空间神经算子

两个阶段分别称为 **Spatial Refinement** 和 **Persistent Latent Processing**。在效率实验得到支持后，论文标题可以写成：

S2LNO: An Efficient Neural Operator via Spatial Refinement and Persistent Latent Processing

Efficient 能覆盖你关注的计算量、延迟和显存；Lightweight 更偏参数量，目前不能充分概括这个设计。Encode-Once 则不适合作为整网名称，因为前段仍然包含多次编码与重建。

2026-09-12 11:28:51 You 

 kimi k3(4).pdf
1.7 MB

注意我对规则网格准备采用transolver/LinearNO风格的卷积处理。这样上述的效率优势会不会更明显，同时阅读一下此篇论文，我准备引入一种物理残差注意力机制，即在后续潜在空间处理中引入类似于附上的kimi k3报告一样，让潜空间中的block也可以通过kimi k3中的residual attention一样于前面反复的呀所重建的block做交互，从而避免在后续latent block中一次压缩可能造成的不可逆的信息损失

 ChatGPT gpt-6-astra-wm

我会先核对 Kimi 报告中 residual attention 的具体信息通路，再分析它怎样接入你这个两阶段模型。关键是区分两点：规则网格卷积会增加多少可节省的点空间计算，以及后段读取前段信息时，是否真正绕过了最终编码的信息瓶颈。

报告中的机制已经核对清楚：Kimi 的 Attention Residuals 是“同一个 token 在不同深度的表示之间做选择”，并不会自动从少量 latent token 读取大量物理点。你的想法可以沿这个方向扩展，但需要增加空间对齐或读取操作；只连接前面已经压缩过的 latent，仍无法找回所有压缩结果都遗漏的信息。

成本估算显示，只要后段复用已经计算好的点侧 K/V，不重新跑卷积，也不把 latent 重建回点空间，就仍能保留较大的计算节省。不过，这时应称为“潜空间主状态 + 只读物理记忆”：每次读取仍有随网格点数增长的成本，不能再沿用纯 IPOT 后段完全不访问点空间的复杂度。

加入 Transolver / LinearNO 风格的原网格卷积后，你这个两阶段模型通常会获得更大的绝对计算节省。你提出的跨阶段交互也有合理性，但需要对 Kimi 的机制做一次明确适配：Kimi 的 Attention Residuals 沿网络深度选择信息，而你的模型还需要解决“从 N 个物理点向 M 个 latent token 读取信息”的问题。

适配后，模型可以成为：

前段反复更新点特征，后段持续更新 latent，同时选择性读取前段缓存的物理空间信息。

这可以缓解对最后一次编码的依赖。不过，准确的数学表述应是保留补充读取信息的路径，绕过单次编码造成的信息遗漏，不能保证消除所有不可逆信息损失。

先看卷积为什么会使计算节省更明显。

这里按你的意思，将卷积放在规则网格上的点特征处理中，采用 stride 为 1、保持网格分辨率的空间卷积。需要区分两种参考结构：

参考方式	原仓库的主要结构	迁移时的对应思路
Transolver 风格	两条独立空间卷积分支，分别产生分配与被汇聚的特征	让压缩的路由特征与 value 特征分别包含邻域信息
LinearNO 风格	一条空间卷积产生共享中间特征，再生成不同投影	先提取共享邻域特征，再供压缩、重建相关投影使用

这些是原网格上的卷积，不能直接对没有规则物理邻接关系的 latent 编号照搬。[Transolver 源码](#)、[LinearNO 源码](#)

设原网格有 N 个点，通道宽度为 D 。一条普通、全通道的 3×3 卷积，其主要成本为：

$$C_{\text{conv}} = 9ND^2 \quad \text{MAC}, \quad P_{\text{conv}} = 9D^2$$

这里忽略 bias，一次乘加记为一个 MAC。

如果全程 LRSA 有 L 个块，每块执行 c 条这样的卷积，而你的模型只在前段及衔接编码处执行共 q 组，那么卷积部分节省：

$$\Delta C_{\text{conv}} = (L - q)c9ND^2.$$

例如，仅作参考示例，取：

$$N = 85^2 = 7225, \quad D = 128, \quad L = 8, \quad k = 2.$$

保守地假设衔接 encoder 也执行一组卷积，因此 $q = k + 1 = 3$ ：

卷积分支	全程 8 次	两阶段模型 3 次	卷积部分减少
每次一条卷积	8.523 GMAC	3.196 GMAC	5.327 GMAC
每次两条卷积	17.046 GMAC	6.392 GMAC	10.654 GMAC

这是卷积分支的总成本；如果卷积替换已有逐点线性投影，计算相对原模型的净增量时，应使用 $8ND^2$ ，避免重复计算被替换的投影。

因此，当卷积只在少数点空间阶段执行时，避免后段重复卷积会成为一个实质性的效率来源。但绝对计算节省增加，不等于实测加速倍率一定增加；延迟还受卷积实现、attention kernel、访存和 batch size 影响。

再看附件：Kimi 实际使用的是 Attention Residuals，简称 AttnRes。

我核对了附件第 6 页 §2.2、式（8）－（10），并补充阅读了独立的《Attention Residuals》报告。它的核心是：让一个 token 在不同深度产生的表示之间进行选择。[Kimi K3 报告](#)，§2.2、[Attention Residuals](#)

补回原文省略的 token 索引 t ，可以写成：

$$\alpha_{\ell,s,t} = \text{softmax}_s \left[w_\ell^\top \text{RMSNorm}(v_{s,t}) \right],$$
$$h_{\ell,t} = \sum_s \alpha_{\ell,s,t} v_{s,t}.$$

其中：

- w_ℓ ：目的层独立学习的 pseudo-query（伪查询向量），不是由当前 hidden state 投影得到的 query。
- $v_{s,t}$ ：同一个 token 在不同历史来源中的表示；Full AttnRes 使用 embedding 和历史分支输出。
- RMSNorm：用于打分，避免幅值大的来源单纯因为尺度而占据优势。
- softmax：沿历史层或块的维度进行，而非沿空间点维度进行。

实际 Kimi K3 使用 Block AttnRes：将一个块内的分支输出求和，保存块级表示，再让后续模块选择性读取这些历史块。

还有一个重要区别：原版 AttnRes 替换的是普通残差的累积方式，保存的主要是分支输出；你目前保留完整 LRSA 块，再额外访问其完整点状态，属于受它启发的新适配。

它对你的启发是“不要让所有历史信息只能通过当前一个状态传递”，但原报告没有证明低秩空间压缩可以无损。

你的模型不能直接照搬上述公式，因为两个阶段的 token 不对应。

前段的点状态是：

$$H_s \in \mathbb{R}^{N \times d_p},$$

后段的潜状态是：

$$Z_j \in \mathbb{R}^{M \times d_z}.$$

Kimi 的各层共享同一套 token 位置，因此可以在同一个 t 上聚合历史表示。你的 N 个物理点和 M 个 latent token 没有这样的逐项对应。

即使改为读取前面各 LRSA 块内部的 latent，也不能仅凭形状都为 $M \times d_z$ ，就认定第 m 个 token 在不同块中具有相同含义：这些块使用不同的可学习 query，latent 编号没有天然的跨层语义对齐。

不同连接方式的作用应分开判断：

后段读取什么	能提供什么	能否补充最后一次编码遗漏的信息
Z_0 及其后续层输出	改善后段跨深度信息访问	无法获得 Z_0 之外的信息
前段不同块的压缩 latent	利用多份可能互补的压缩结果	有可能，但所有历史 latent 都遗漏的信息仍不可访问
前段未做空间汇聚的点特征或点侧 K/V	重新读取物理点信息	可以绕过最后一次空间汇聚，但读取有额外成本

如果你的主要目标是缓解最后一次编码的信息遗漏，我更倾向于保留点侧记忆，再让后段读取。

一个具体、兼容当前两阶段主干的适配如下。以下是针对你的模型提出的设计，并非 Kimi 已经展示的 PDE 架构。

```
mermaid
graph TD
    A["前段完整 LRSA 块<br/>原网格卷积与点更新"] --> B["衔接 encoder"]
    B --> C["持续 latent processor"]
    C --> D["输出位置 query 解码"]
    A --> E["缓存少量前段点侧 K/V"]
    E --> F["选定后段块读取"]
    F --> C
```

第一步，保存少量前段点侧记忆。

选择前段若干深度：

$$\mathcal{S} = \{s_1, \dots, s_S\}.$$

对这些点状态构造：

$$K_s = \Phi_{K,s}(H_s, X), \quad V_s = \Phi_{V,s}(H_s, X).$$

其中 $\Phi_{K,s}, \Phi_{V,s}$ 包含需要的归一化、位置处理和投影；你选择的网格卷积结果应尽量复用。

关键实现原则是：

同一次前向传播中，记忆 K/V 计算一次，后段共享读取；后段不重新执行这些原网格卷积。

仅仅缓存 H_s ，却让每个后段块重新执行不同的 K/V 卷积或点投影，就无法获得同样的节省。若复用衔接 encoder 已经生成的点侧 K/V，还可以避免重复构建最后一份记忆。

缓存应保持正常的梯度连接；“只读记忆”指前向过程中不回写点状态，并不意味着训练时将其 detach。

第二步，用当前 latent 查询每份物理记忆，解决空间对齐问题。

在选定的后段块 j 中：

$$Q_j = \text{Norm}(Z_j)W_{Q,j},$$

$$R_{j,s} = \text{softmax}_N \left(\frac{Q_j K_s^\top}{\sqrt{d}} \right) V_s W_{O,j,s}.$$

于是：

$$R_{j,s} \in \mathbb{R}^{M \times d_z}.$$

这里所有来源的第 m 行，都是由**同一个当前 latent query** 发起的读取。因此，它们可以在这个 query 对应的位置上进行融合，不必假设不同前段层的 latent 编号已经对齐。

这一步回答：

当前这个 latent token，从某个深度保存的物理点特征中，需要读取哪些位置、哪些内容？

第三步，借鉴 AttnRes，在读取结果之间选择信息来源。

将当前状态也作为候选：

$$B_{j,0} = Z_j, \quad B_{j,s} = R_{j,s}, \quad s = 1, \dots, S.$$

对每个 latent token m ：

$$\alpha_{j,s,m} = \text{softmax}_s \left[w_j^\top \text{RMSNorm}(B_{j,s,m}) \right],$$

$$\bar{Z}_{j,m} = \sum_{s=0}^S \alpha_{j,s,m} B_{j,s,m}.$$

然后继续执行 IPOT 风格的 latent self-attention 和 FFN：

$$\begin{aligned} \tilde{Z}_j &= \bar{Z}_j + \text{SA}_j(\text{Norm}(\bar{Z}_j)), \\ Z_{j+1} &= \tilde{Z}_j + \text{FFN}_j(\text{Norm}(\tilde{Z}_j)). \end{aligned}$$

这一步回答：

当前 latent 更应该依赖自身状态、较早的点特征，还是经过更多全局处理的点特征？

这里存在两种不同的 query：

query	来源	功能
Q_j	当前 latent 特征	在每份物理记忆内部选择空间信息
w_j	当前聚合模块的可学习向量	在不同深度来源之间选择信息

这样就把**空间读取**与**跨深度选择**分开了。它是一个两级注意力结构，也不同于直接将所有历史点拼接起来做一次普通 cross-attention。

保留多个深度并不意味着获得了更多独立的原始信息——它们都来自同一个输入。潜在收益在于：不同深度提供不同加工程度的特征，使后段更容易读取所需内容。

这种连接为什么有机会补充最终编码遗漏的信息？

原模型的后段是：

$$Z_{j+1} = F_j(Z_j).$$

因此，两个输入只要得到相同的 Z_0 ，此后的输出就必然相同。

引入记忆后变为：

$$Z_{j+1} = F_j(Z_j, \mathcal{M}(a)),$$

其中：

$$\mathcal{M}(a) = \{K_s(a), V_s(a)\}_{s \in \mathcal{S}}.$$

现在，即使两个输入的 Z_0 相同，只要记忆仍然不同，而且读取与后续计算保留了这种差异，模型就可能重新区分它们。

一个简单例子是：

$$E(x_1, x_2) = \frac{x_1 + x_2}{2}.$$

输入 $(1, -1)$ 和 $(0, 0)$ 都编码成 0。仅处理这个编码，无法区分二者。

如果记忆保留两个点，并允许按位置相关权重读取：

$$R = \beta x_1 + (1 - \beta)x_2,$$

那么两个输入分别得到：

$$R_1 = 2\beta - 1, \quad R_2 = 0.$$

当 $\beta \neq 1/2$ 时，它们可以重新被区分。如果读取始终也是均匀平均，缓存再完整也没有帮助。

所以，**有记忆只是必要的信息通路；有效的读取方式同样重要。**

在冻结注意力权重的线性示意下，如果最终编码为 Ch ，补充读取为 Rh ，那么编码遗漏的某个方向 δh 可以通过记忆重新进入计算，条件是：

$$C\delta h = 0, \quad R\delta h \neq 0.$$

反之，落在所有编码、读取映射共同零空间中的信息，依然无法被利用。

同时需要保留上一轮分析中的限制：只要最终输出仍然仅依赖一个 $M \times d_z$ 的最终 latent，

$$\hat{u}(Y) = D(\text{PE}(Y), Z_r(a)),$$

固定坐标和其他条件，在可微处仍有：

$$\text{rank}\left(\frac{\partial \hat{u}(Y)}{\partial a}\right) \leq Md_z.$$

新机制改善的是“怎样形成最终 latent”，不是取消最终 latent 的有限容量。 它允许根据逐步形成的任务状态多次读取信息，而不要求第一次 encoder 就一次性提取所有必要内容。

加入这种读取后，效率优势还在，但复杂度必须重新计算。

对一份点记忆，统一宽度为 D ，使用共享的 K/V 投影，并为读取配置 Q/O 投影，主要额外成本约为：

$$C_{\text{cache}} = 2ND^2,$$

$$C_{\text{read}} = 2NMD + 2MD^2.$$

后段读取 t 次：

$$C_{\text{extra}} \approx 2ND^2 + t(2NMD + 2MD^2).$$

这个估算不含 softmax、归一化等操作，也不包含额外新加的卷积。已有 encoder K/V 能直接复用时，缓存构建的增量还可以减少。读取多份点记忆时，主要读取成本会随来源数增加。

它仍然省去了后段每层的：

- 原网格卷积；
- latent 到点空间的 up-attention；
- 点级 FFN；
- 点状态更新及相应中间激活。

但它重新引入了随 N 增长的点记忆读取成本。因此，后段应称为**“latent 主状态 + 只读物理记忆”**，不能继续说后段计算完全与原网格点数无关。

而且，每次读取在数学上仍是一次 $N \rightarrow M$ 汇聚。以 $k = 2, r = 6$ 、一份记忆为例：

架构	$N \rightarrow M$ 读取次数	$M \rightarrow N$ 重建次数
8 个完整 LRSA 块	8	8
原两阶段模型	3	3

架构	$N \rightarrow M$ 读取次数	$M \rightarrow N$ 重建次数
两阶段模型+后段读取 2 次	5	3
两阶段模型+后段每层读取	9	3

最后一种甚至有更多的读取次数，但每次读取复用点侧 K/V，而且没有对应的完整重建和点空间计算，仍可能更便宜。**这也说明，应该优化实际计算与延迟，而不能仅以“压缩次数更少”判断效率。**

继续使用前面的假设置置，并取 $M = 64$ ：

新增子路径，一份记忆	计算量
构建一次点侧 K/V	0.237 GMAC
一次读取，含 Q/O 投影	0.120 GMAC
构建记忆+读取 2 次	0.478 GMAC
构建记忆+读取 6 次	0.960 GMAC

与前面少执行五组单分支卷积所节省的 **5.327 GMAC** 相比，这个读取预算具有吸引力。即使后段每层读取，也可能保留相当一部分计算优势。但这是子路径估算，不是整网速度结果。

显存方面也有交换：需要让点侧 K/V 在后段持续驻留；SDPA 可以避免显式保存完整注意力矩阵，却不能消除这份记忆。因此，加入记忆后的显存优势通常会小于纯 latent 后段的优势。

我建议把这项研究拆成两个可分别验证的贡献，而不是直接将所有收益归于 AttnRes。

首先验证“后段重新读取物理点记忆”是否有效，再验证“选择不同前段深度”是否进一步有效：

对照	主要回答的问题
原两阶段模型，无记忆读取	单次编码的基线
只读取最后点状态 H_k ，普通 cross-attention	收益是否主要来自重新访问点信息
读取多份前段点状态，简单融合	多深度特征是否提供额外帮助
多份点状态+上述 AttnRes 式选择	内容相关的跨深度选择是否值得额外成本

读取频率可以比较 $t = 1, 2, r$ 。结合你的效率目标，我更倾向于**先在少数后段块插入读取，其他块继续纯 latent 计算**；是否需要每层读取，由精度–延迟曲线决定。最终保持 IPOT 的输出位置 query decoder，即可保留独立输出点集的查询接口。

命名上，可以将这个模块称为 **Physical-Space Residual Attention（物理空间残差注意力，PSRA）**。这里的“物理空间”说明记忆来自物理点，“残差”指网络信息路径；目前并没有使用 $\mathcal{L}(u) - f$ 这样的 PDE 方程残差。

这项机制最准确、也最容易通过实验检验的研究目标是：

在避免后段反复重建和原网格卷积的同时，通过对前段物理记忆的选择性读取，降低模型对一次性编码质量的依赖。

我的目的不是要将点的性质通过类似kimi k3中的attenres结构连接到后续的latent空间中（单词编码的部分），而是要把前面反复压缩重建中的latent空间（即前面LRSA风格的压缩重建block中中间的Latent表示（可以先假设前面LRSA风格的block重复了2次），把这两层中间的latent表示与后续IPOT风格的persistant latent空间中的首次压缩后的表示通过kimi k3中的attenres的形式连接起来，目的是弥补单词压缩可能造成的表达能力损失（参考LinearNO的说法，它认为反复压缩和重建才是表达能力的来源，但如果我不再反复压缩和重建就可以用过attenres把以前不同的压缩方式连接到现在的压缩中（也就是后面IPOT风格的一次性压缩过程中），这样就可以通过单次压缩获得多次压缩的表达能力，这样做还有一个出发点就是与其不断重复压缩重建的过程可不可以通过利用以前的压缩来直接获得多次压缩的优势，而不是再次压缩，而且由于当前压缩和重建是解耦的（且当前设计中重建采用的是IPOT风格的，与输出

明白，是我上一轮把你的意思扩展错了。你要复用的是前两个 LRSA 块内部已经形成的 latent 表示，并用 `AttnRes` 将它们与后段首次编码的 latent 连接起来。我会按这条信息路径分析它能补充哪些表达能力，以及“复用多次压缩的结果”与“继续做新的压缩重建”之间的区别。

按你这条消息，三份表示是：第一轮 LRSA 的中间 latent、第二轮 LRSA 的中间 latent，以及衔接 encoder 的输出 Z_0 。这个方案能保持后段不访问物理点的效率优势。还需要区分一个关键点： Z_0 本来就间接依赖前两轮压缩重建；`AttnRes` 增加的是对历史结果的直接、可选择访问，而“能否因此省去更多后续轮次”正是需要实验验证的假设。

明白，上一轮我把你的意思理解错了。你要复用的是前段已经计算出的 latent 表示，通过 `AttnRes` 将不同压缩阶段的信息直接送入后段，整个后段仍然不访问物理点。

这个方向与两阶段架构兼容，也能保留原先的效率优势。我的判断是：

多次压缩产生的表示，可能通过直接复用而持续发挥作用，从而减少继续执行压缩重建的需要。

这是一个合理、可验证的研究假设。但“能够完全获得继续多轮压缩重建的表达能力”，以及“decoder 的表达能力自然提高”，需要进一步区分。下面按照你的真实方案解释。

先把信息路径准确固定下来。

前面两个完整 LRSA 块保持原有计算。记：

$$\begin{aligned}T_1 &= G_1(E_1(H_0)), & H_1 &= R_1(H_0, T_1), \\T_2 &= G_2(E_2(H_1)), & H_2 &= R_2(H_1, T_2), \\Z_0 &= E_3(H_2).\end{aligned}$$

其中：

- E_1, E_2 ：两个 LRSA 块的 down-attention。
- G_1, G_2 ：原 LRSA 的 latent FFN、self-attention 等处理。
- R_1, R_2 ：包含 up-attention、点残差和点级 FFN 的剩余计算。
- E_3 ：进入持续潜空间的衔接 encoder。
- T_1, T_2 ：这里先明确为 **latent mixing 完成后、up-attention 之前** 的表示，直接缓存，无须额外计算。

如果希望研究“刚完成压缩的表示”，可以将缓存位置改为 G_i 之前；这是取点位置的消融，不影响下面的主逻辑。

你的直接连接方式是：

```
mermaid
graph TD
    A["完整 LRSA 块 1"] -->|点状态 H1| B["完整 LRSA 块 2"]
    B -->|点状态 H2| C["衔接 encoder"]
    A -->|缓存 latent T1| D["AttnRes 融合"]
    B -->|缓存 latent T2| D
    C -->|首次编码 Z0| D
    D --> E["持续 latent processor"]
    E --> F["输出位置 query 解码"]
```

整个网络仍然发生三次压缩、三次重建。`AttnRes` 复用已有结果，不增加空间压缩次数。

还有一个对理解创新点很重要的事实： Z_0 本身已经间接依赖前两轮压缩重建：

$$T_1 \longrightarrow H_1 \longrightarrow T_2 \longrightarrow H_2 \longrightarrow Z_0.$$

因此，`AttnRes` 新增的是：

让后段直接访问不同阶段的 latent，而不必要求这些历史信息都经过后续重建、点更新和再次压缩后，仍完整保留在 Z_0 中。

这比“让一次压缩首次具有多次压缩的信息”更准确，也更接近 Kimi `AttnRes` 的出发点。

按 Kimi 的形式，三份 latent 可以怎样连接？

先设：

$$T_1, T_2, Z_0 \in \mathbb{R}^{M \times d}.$$

令三个来源为：

$$B_1 = T_1, \quad B_2 = T_2, \quad B_3 = Z_0.$$

对每个 latent token m ，使用可学习伪查询 $w \in \mathbb{R}^d$ ：

$$e_{s,m} = w^\top \text{RMSNorm}(B_{s,m}),$$
$$\alpha_{s,m} = \frac{\exp(e_{s,m})}{\sum_{q=1}^3 \exp(e_{q,m})},$$

$$U_{0,m} = \sum_{s=1}^3 \alpha_{s,m} B_{s,m}.$$

随后：

$$U_{j+1} = \text{IPOTProcessor}_j(U_j), \quad \hat{u}(Y) = D_{\text{IPOT}}(\text{PE}(Y), U_r).$$

这个融合是在**压缩阶段／来源深度**上进行 softmax，不是在物理点或不同 latent token 之间进行注意力。

它沿用了 Kimi 的几个关键设计：

- query 是目的模块的可学习向量；
- 历史表示同时提供 key 和 value；
- RMSNorm 用于打分；
- 根据历史内容，为不同来源分配权重。

但它仍属于适配版本：Kimi 原版聚合历史分支输出或块内分支输出之和，这里聚合的是完整的历史 latent 状态。[Kimi K3, §2.2](#)

这里确实存在合理的表达能力增益。

不同来源可以保留不同的信息：

表示	来自什么状态	可能提供的互补信息
T_1	初始点特征 H_0	较早阶段提取的结构与输入特征
T_2	经第一轮重建、点更新后的 H_1	根据已有全局上下文重新提取的信息
Z_0	经两轮完整更新后的 H_2	更接近后段任务需求的综合表示

第二轮压缩面对的是更新后的点特征，所以 T_2 已经包含第一轮重建带来的影响； Z_0 又受到第二轮重建的影响。**复用这些 latent，确实是在复用多轮完整计算的成果。**

但间接依赖不等于直接信息冗余。即使 T_1 影响过 H_2 ，也不能保证 Z_0 能重新恢复 T_1 。AttnRes 提供的直接连接，可能让这部分信息重新变得可用。

用一个冻结注意力、线性化后的示意可以看得更清楚。将输入向量化为 a ，记：

$$T_1 = C_1 a, \quad T_2 = C_2 a, \quad Z_0 = C_0 a.$$

只使用 Z_0 时，模型无法区分的输入变化属于：

$$\ker C_0.$$

同时保留三份表示时，共同无法区分的变化属于：

$\ker C_1 \cap \ker C_2 \cap \ker C_0.$

这个交集可能比 $\ker C_0$ 小。

也就是说，某种输入差异虽然没有进入 Z_0 ，却可能已经被 T_1 或 T_2 保存。你不需要再次访问物理点，就可能利用它。

这是这个方向最直接的数学依据。

不过，“已经拥有多份压缩结果”和“等价于继续执行新的压缩重建”之间，还差一个条件。

新的 LRSA 块不只是产生另一份摘要，它还会：

1. 根据当前点特征形成新的压缩权重；
2. 处理压缩后的表示；
3. 根据点特征 query 重建；
4. 更新点状态，执行点级非线性或卷积；
5. 让下一轮压缩面对再次变化后的点特征。

历史 latent 复用能够使用**已经提取的信息**。新的 LRSA 块还可能从点残差中提取**所有历史 latent 都没有保存的信息**。

因此，如果存在两个输入：

$$(T_1(a), T_2(a), Z_0(a)) = (T_1(b), T_2(b), Z_0(b)),$$

那么任何只依赖这些历史表示的后段，都无法区分它们。继续访问点状态的 LRSA 则可能区分。

反过来，如果任务所需的信息已经包含在三份历史表示中，确实可能不再需要新的空间读取。

在线性示意中，假设某次新的读取 C_*a 满足：

$$C_* = A_1C_1 + A_2C_2 + A_0C_0,$$

那么：

$$C_*a = A_1T_1 + A_2T_2 + A_0Z_0.$$

这次读取在信息上就可以由历史结果构造出来，无须再次处理物理点。

但还要注意：普通 AttnRes 实现的是按 token 的 softmax 加权融合，**并不自动实现任意矩阵 A_1, A_2, A_0** 。后段的 self-attention 和 FFN 可以继续加工融合结果，但最终是否足以替代新的读取，需要验证。

所以，你的核心假设可以更精确地表述为：

后续重复压缩所提取的任务相关信息，是否已有相当部分包含在早期多份 latent 中，只是原模型没有直接复用它们？

如果答案是肯定的，历史复用就有机会用很小的成本替代一部分昂贵往返。

LinearNO 支持这个问题的研究动机，但没有直接证明上述替代关系。

LinearNO 的主要证据是：在其考察的模型和实验中，slice/deslice 可以承担重要的全局交互功能，去掉中间 slice attention 仍能得到较好的结果；解耦压缩和重建进一步提高了这种交互的灵活性。[LinearNO，方法与消融](#)

这说明压缩、重建映射值得重点研究，但不能直接推导出：

- 表达能力只由压缩重建次数决定；
- 保存几份压缩结果，就等价于继续执行更多轮次；
- AttnRes 一定可以替代这些轮次。

你提出的研究，实际上是在检验 LinearNO 结论之后的一个新问题：这些有效的压缩结果，能否被更充分地复用？

关于 latent 对齐，我会把它视为需要检查的设计条件，而不是否定方案的理由。

直接融合要求相同编号 m 的向量可以被放在一起比较和组合。但独立 LRSA 块的 latent query 不同，因此：

$$T_1[m], \quad T_2[m], \quad Z_0[m]$$

未必天然对应相同的物理区域或特征含义。

这不意味着不能训练。端到端训练可以让各来源逐渐形成适合融合的表达，新增连接也会反向约束它们。

需要准确理解的是：

- 相同形状保证计算合法；

- 联合训练可能学出兼容表示；
- RMSNorm 只处理尺度，不保证 token 或通道语义对齐。

因此，**直接 AttnRes 应当保留为最简主版本**。没有必要因为这个问题就先改变 LRSA query 共享方式，或额外加入另一层 attention。只有当实验显示融合存在问题时，才需要用单独的对齐消融定位原因。

另外，“入口融合一次”和“后续各层继续复用历史”是两个不同版本。

按你这条消息最直接的结构，是：

$$U_0 = \text{AttnRes}(T_1, T_2, Z_0),$$

然后后续只处理 U_0 。这是一个**利用压缩历史增强入口表示**的版本。

它很简洁，但一旦融合时遗漏某些信息，后续便不能再次从历史中读取。

更接近 Kimi 跨深度访问思想的版本，是保存：

$$\mathcal{M} = \{T_1, T_2, Z_0\},$$

让后续每个或部分块再次选择这些来源，同时保留当前计算状态：

$$\begin{aligned}\bar{U}_j &= \text{AttnRes}_{w_j}(T_1, T_2, Z_0, U_j), \\ U_{j+1} &= \text{IPOTProcessor}_j(\bar{U}_j).\end{aligned}$$

首次融合时只使用三份来源，避免将同一个 Z_0 重复列入。

版本	主要作用	限制
入口融合一次	改善后段的初始编码	后续仍受首次融合结果约束
后续块继续选择历史	不同深度可使用不同的历史信息	需要保留少量 latent 缓存，增加轻量聚合

两者都不增加物理点读取，也不增加 slice/deslice 次数。

按原版 AttnRes， w_j 是每个目的层的参数。不同层可以学习不同来源偏好，权重也依赖历史内容；但它不是由当前 U_j 生成 query、重新执行空间压缩。这一点在解释机制时应保持一致。

你的效率目标在这个纯 latent 版本中能够更完整地保留。

没有额外投影时，对 S 份、每份 $M \times d$ 的表示进行 AttnRes，主要成本为：

$$O(SMd).$$

它没有新的 N 相关项，也没有标准 latent attention 的 M^2d 主项。

例如：

$$M = 64, \quad d = 128.$$

只计算打分和加权求和：

- 一次三来源融合约为 $2 \times 3Md = 49,152$ MAC；
- 六个后段块进行融合，首次三个来源、之后各四个来源，约为 376,832 MAC；
- 两份额外历史 latent 共 $2Md$ 个元素，BF16 下约 **32 KiB／样本**。

归一化、softmax 和训练激活另计，但量级明显小于反复处理原网格。

因此，你上一轮提出的 Transolver／LinearNO 风格卷积仍只在前段和既定编码位置运行，**AttnRes 复用不需要重新执行它们**。这比我上一轮错误理解的物理点记忆读取，更符合你的效率目标。

最后，你关于 decoder 的判断需要拆成“重建效果”和“decoder 自身容量”。

更丰富的 latent 确实可能让原 decoder 得到更有用的输入，从而提高预测精度。端到端训练也会让 decoder 适应这种新的表示。

但如果 decoder 的结构、宽度和参数量保持不变：

$$\hat{u}(Y) = D_{\theta}(\text{PE}(Y), U_r),$$

则 decoder 自身的参数化能力并没有因为前段加了 AttnRes 而自动扩大。改善的是它所接收的信息，以及整个模型形成输出的方式。

压缩与重建解耦，使 decoder 不必沿用压缩权重；位置 query 则提供独立输出位置查询。这两点都有价值，但位置 query 并不在一般意义上比点特征 query 表达能力更强，二者利用的信息条件不同。

所以，这部分可以表述为：

历史 latent 融合提高了重建可利用的信息质量；独立的位置查询 decoder 可以学习如何利用这些信息生成输出场。

而不是“压缩能力提高，所以重建器的容量自然提高”。

我认为最有辨识力的实验，是在同一个两阶段主干上比较：只用 Z_0 、三来源平均融合、三来源 AttnRes、后续持续复用历史。再与增加完整 LRSA 块的结果比较精度、延迟和计算量。

这样能区分收益究竟来自“保留了历史压缩结果”，还是进一步来自“根据内容选择历史来源”，并真正检验你的目标：能否通过复用已有压缩，减少新的压缩重建，同时维持甚至提高预测精度。

2026-09-12 18:56:30 You 

现在设计的计算方式与attenres具体有什么区别

 ChatGPT gpt-6-astra-wm

我会把当前方案的融合公式和连接位置，与 Kimi 原版 AttnRes 逐项对照。入口融合一次和后续各层复用历史这两个版本，会分别说明。

当前设计与 AttnRes 的区别，主要在“连接哪些表示、放在哪里、怎样维护历史”；单次融合的计算公式可以完全一样。

也就是说，我们目前设计的是 AttnRes 式的 latent 历史融合，还没有将整个后段的残差结构替换成原版 AttnRes。上一轮我应该把这个区别说得更明确。

先看相同的部分：单次融合操作。

把一个 AttnRes 聚合写成：

$$\mathcal{A}_w(B_1, \dots, B_S)_m = \sum_{s=1}^S \alpha_{s,m} B_{s,m},$$
$$\alpha_{s,m} = \text{softmax}_s \left[w^\top \text{RMSNorm}(B_{s,m}) \right].$$

这里：

- w ：目的层的可学习 pseudo-query（伪查询向量）。
- $B_{s,m}$ ：第 s 个历史来源中，第 m 个 token 的表示。
- RMSNorm：用于计算注意力分数。
- softmax：沿历史来源维度计算。
- value：使用原始 $B_{s,m}$ ，不是打分时归一化后的向量。

当前方案将：

$$(B_1, B_2, B_3) = (T_1, T_2, Z_0)$$

代入这个公式即可。这一步与原版 AttnRes 使用的是同一种聚合计算。[Kimi K3, §2.2](#)

例如三份表示都是 $[B, 64, 128]$ ，堆叠后为：

$$[B, 3, 64, 128] \rightarrow \text{来源分数 } [B, 3, 64] \rightarrow \text{融合结果 } [B, 64, 128].$$

它没有计算 64×64 的 token 间注意力，也没有重新访问物理点。

真正的区别，可以概括为下面几项。

比较项	Kimi 的 Block AttnRes	当前入口融合版	当前逐块复用版
聚合对象	embedding、历史块的分支输出之和、当前块的部分和	T_1, T_2, Z_0	T_1, T_2, Z_0 和当前 U_j
融合位置	attention、FFN 等子层的输入汇聚处	进入后段前一次	每个选定 processor 块之前
与原残差的关系	改写跨深度的残差汇聚规则	保留 IPOT 原来的 SA、FFN 残差	同样保留 IPOT 原来的 SA、FFN 残差
历史是否持续增加	随计算产生新的块级历史	不增加	固定前段历史不增加，只更新当前状态
后段早期计算能否被单独重访	可以重访保存的分支／块级结果	不能	目前也不能，只能通过当前状态传递
token 对应关系	同一个 token 身份贯穿不同深度	独立压缩产生的 latent，需要学出兼容性	同左

其中有三个区别直接影响模型功能。

第一，原版主要保存“分支产生的更新”，当前方案保存“完整的中间 latent 状态”。

普通残差更新是：

$$h_{\ell+1} = h_{\ell} + f_{\ell}(h_{\ell}).$$

这里的：

$$f_{\ell}(h_{\ell})$$

是该层产生的更新，和已经包含历史信息的 $h_{\ell+1}$ 不是同一个对象。

为看清楚原理，先用 Full AttnRes 表示。设 v_0 为 embedding：

$$\begin{aligned} h_{\ell} &= \mathcal{A}_{w_{\ell}}(v_0, v_1, \dots, v_{\ell-1}), \\ v_{\ell} &= f_{\ell}(h_{\ell}). \end{aligned}$$

下一层就可以把新产生的 v_{ℓ} 作为一个独立来源。

Kimi K3 使用的 Block 版本，则把若干分支输出合并成：

$$b_g = \sum_{\ell \in \mathcal{B}_g} v_{\ell},$$

再对历史 b_g 和当前块部分和进行选择。[Attention Residuals, §3](#)

当前方案保存的：

$$T_1 = G_1(E_1(H_0)), \quad T_2 = G_2(E_2(H_1))$$

已经包含压缩、latent FFN、self-attention 以及相应残差的综合结果。

因此，两者选择的对象不同：

- 原版选择的是**哪些历史更新应该参与当前计算**。
- 当前方案选择的是**哪些压缩阶段形成的完整表示应该参与后段计算**。

这并不说明当前方案不合理。它正好对应你希望复用压缩结果的目的，但不能将两类来源视为完全相同。

第二，原版改变残差汇聚，当前方案保留了 IPOT 块内部的普通残差。

当前入口融合版是：

$$U_0 = \mathcal{A}_w(T_1, T_2, Z_0),$$

然后每个 IPOT processor 继续执行：

$$V_j = U_j + \text{SA}_j(\text{Norm}(U_j)),$$

$$U_{j+1} = V_j + \text{FFN}_j(\text{Norm}(V_j)).$$

所以，AttnRes 只改变了后段最初收到什么，后段仍通过普通加法残差传递信息。

即使采用逐块复用版：

$$\begin{aligned}\bar{U}_j &= \mathcal{A}_{w_j}(T_1, T_2, Z_0, U_j), \\ U_{j+1} &= \text{IPOTProcessor}_j(\bar{U}_j),\end{aligned}$$

processor 内部的两条加法残差也仍然存在。

原版 AttnRes 则让子层输入本身通过历史选择形成。**SA 和 FFN 可以分别使用自己的聚合参数，选择不同的历史来源。**

当前“每个 processor 前融合一次”的版本，SA 得到融合结果，FFN 得到 SA 普通残差更新后的结果；FFN 并没有再独立选择历史。

需要补充：Block AttnRes 内部仍保留分支求和，不能将原版简单理解成“取消所有加法”。它改写的是整体的历史汇聚规则。

第三，当前逐块复用版依然不是一个不断扩展的历史库。

假设后段已经执行了三层，产生：

$$U_1, \quad U_2, \quad U_3.$$

按目前的逐块版本，下一层读取的是：

$$\{T_1, T_2, Z_0, U_3\}.$$

它不能单独选择“第一层后段 SA 的输出”，也不能直接回到 U_1 ；这些后段历史都被递归地包含在 U_3 中。

原版 AttnRes 会将新的分支输出，或者新的块级分支和，加入可以访问的历史来源。

因此，当前逐块复用版具有：

固定的前段压缩历史 + 不断更新的当前潜状态。

原版本则具有：

随着深度不断产生、可继续被单独选择的历史分支／块表示。

所以，目前设计主要解决的是**前段压缩结果如何被后段反复利用**，并没有完整解决原版 AttnRes 关注的“后段自身所有历史更新如何被选择性访问”。

此外，你的迁移还多了一个 latent 对应条件。

在 Kimi 中，不同层的第 m 个 token 都对应同一个输入 token 身份。

在你的模型中， $T_1[m]$ 、 $T_2[m]$ 、 $Z_0[m]$ 来自不同的可学习压缩过程，没有天然一致的 slot（槽位）含义。

直接融合在计算上合法，也可以通过端到端训练协调表示；但它增加了一个隐含要求：

不同来源的相同 latent 编号，应当适合进行组合。

RMSNorm 不能完成这种对应，也不能自动统一不同通道的含义。这是迁移到低秩神经算子后需要检查的问题，不是原版同一 token 跨层连接所具有的同等问题。

从成本看，当前版本也比完整维护后段历史更简单。

假设有 r 个后段块，只比较历史聚合的成本：

方式	聚合成本
三来源入口融合一次	$O(3Md)$
固定历史 + 当前状态，逐块融合	$O(4rMd)$
Full AttnRes，保存各子层历史	随子层数呈二次增长
Block AttnRes	取决于子层数和历史块数

这些版本都可以完全在 latent 中执行；区别不在是否重新 slice/deslice，而在**如何组织和更新 latent 历史**。

因此，当前方案最准确的描述是：

采用 AttnRes 的聚合算子，对不同压缩阶段的 latent 表示进行融合或持续复用，同时保留原 IPOT processor 的残差结构。

如果进一步让后段 SA、FFN 的分支输出进入历史库，并由 AttnRes 重新组织它们的输入汇聚，才会更接近在后段完整采用原版 AttnRes 残差架构。这是另一项明确的结构变化，不是当前融合公式已经自动实现的功能。

我主要考虑两个问题：1.

| 历史是否持续增加 |
| —— |

| 随计算产生新的块级历史 |
| —— |

| 不增加，对于这点你觉得除了在后续的persistent latent部分中的而首次压缩与前面的过程连接外 |
| ————— |

,可不可以后续的persistent latent空间中也在每层与前面的LRSA风格的block和persistent latent空间中它前面的latent block也进行类似attenres的操作（即除了前面两次的LRSA风格的block外后续的每一个block都对前面的block（无论是LRSA中反复压缩和重建中的block还是在后续persistent latent中前面的latent block中）；2.对于“

| token 对应关系 |
| —— |

| 同一个 token 身份贯穿不同深度 |
| —— |

| 独立压缩产生的 latent，需要学出兼容性”我想到了这样的改进：意思上还采用attenres的跨深度范式，但具体实现上我有两种想法（计算过程还不完全，你可以在此基础上补充）：第一种方式是类似于LRSA/IPOT本身的压缩过程，只不过目的不再是压缩而是通过类似于LRSA/IPOT的压缩一样通过注意力计算将前面每一个block(需要通过attenres连接到当前block的block)通过注意力计算汇聚到当前的M个latent token中（压缩是把N个token汇聚到M个latent token中，现在我像采用的跨深度汇聚是把前面每层的M个latent token的信息再汇聚到当前block的M个latent token中，由于被前面有多个block所以在对不同深度层的同时汇聚上可以借鉴attenres的处理方式），第二种方式是像transolver一样的软分配将前面block中的M个latent block中通过软分片（或者叫软分配）到当前block中的M个latent block中，在对多个深度同时汇聚到当前block的处理方式上仍然借鉴kimi k3的处理方式。以后我不再管这个叫一种attenres而是称为物理跨深度注意力（或者叫物理纵向注意力），总之为了与transolver中的physics attention相对应也称之为一种physics attention，只不过是不同深度间的，你可以根据我的意思给这个操作起一个名字 |

| _____
| _____
| _____
| _____ |

我会按“每个后段块都能访问前段两份 latent，以及此前后段块的表示”来完善连接方式，再分别补全你提出的 cross-attention 对齐和软分配对齐。重点是怎样完成跨来源融合、是否保持 token 置换性质，以及历史增长后的计算成本。

两种方式都能扩展到不断增长的 latent 历史，而且不需要再读取物理点。它们的关键区别是：cross-attention 让每个当前 token 选择历史 token；软分配则先让每个历史 token 在当前目标 token 之间分配，再计算加权均值。还需注意，原版 Transolver 的分配目标是可学习的固定槽位；如果要直接匹配当前 latent 的内容，需要让分配权重也依赖当前表示。

可以。你提出的两个改动能够组成一个完整的新模块：**历史随着后段计算持续增加；读取历史时，先把不同来源的 latent 对齐到当前 token，再决定各个深度贡献多少信息。**

我建议将它命名为：

Cross-Depth Physics Attention (CDPA, 跨深度物理注意力)

两种实现分别称为 **CDPA-Cross (交叉注意力对齐)** 和 **CDPA-Slice (软分配对齐)**。“跨深度”比“纵向”更明确地指出这里讨论的是网络深度。下面给出的具体组合是对你方案的补全，并非原论文已经展示的架构。

首先，你希望的“不断增加历史”可以直接实现。

前两个完整 LRSA 块仍分别产生：

$$T_1, T_2 \in \mathbb{R}^{M \times d},$$

衔接 encoder 产生：

$$U_0 = Z_0 \in \mathbb{R}^{M \times d}.$$

将第 j 个后段块的完整输出记为 U_j 。那么各块可以访问：

后段块	当前输入	可以使用的全部来源
第 1 块	$U_0 = Z_0$	T_1, T_2, Z_0
第 2 块	U_1	T_1, T_2, Z_0, U_1
第 3 块	U_2	T_1, T_2, Z_0, U_1, U_2
第 j 块	U_{j-1}	$T_1, T_2, Z_0, U_1, \dots, U_{j-1}$

每完成一个后段块，就将新的 U_j 加入历史。

这样，后续块既可以访问两次 LRSA 中间的压缩表示，也可以重新使用任何此前后段块的处理结果。**历史不再仅通过一个当前状态递归传递。**

这里有两个实现细节：

- U_0 就是 Z_0 ，只作为一个来源，不能重复加入。
- 当前输入 U_{j-1} 已经处于当前 token 系统中，可以直接作为恒等候选；其余历史来源再执行 $M \rightarrow M$ 对齐，省去不必要的自对齐。

以下默认缓存完整 block 输出，符合你“访问前面每个 block”的要求。

两种方式可以共享同一个外层结构：先对齐来源，再选择深度。

设历史来源为 B_s ，当前输入为 U_{j-1} 。先计算：

$$R_{j,s} = \text{Align}_j(U_{j-1}, B_s) \in \mathbb{R}^{M \times d}.$$

两种方案的差别，就在这个 Align 如何实现。

然后将当前状态作为额外候选：

$$R_{j,0} = U_{j-1}.$$

对每个当前 token m ，做跨深度选择：

$$\begin{aligned} e_{j,s,m} &= w_j^\top \text{RMSNorm}(R_{j,s,m}), \\ \alpha_{j,s,m} &= \text{softmax}_s(e_{j,s,m}), \\ \bar{U}_{j,m} &= \sum_s \alpha_{j,s,m} R_{j,s,m}. \end{aligned}$$

再执行原来的 latent processor：

$$\begin{aligned}V_j &= \bar{U}_j + \text{SA}_j(\text{Norm}(\bar{U}_j)), \\U_j &= V_j + \text{FFN}_j(\text{Norm}(V_j)).\end{aligned}$$

最后将 U_j 保存到历史库。

这就形成了一个完整 block：

读取并对齐历史 latent → 选择历史深度 → latent self-attention → FFN → 保存新历史。

当前状态已经参与上面的加权融合，因此这里不再额外无条件加一次 U_{j-1} 。其后的 SA、FFN 残差仍然保留。

跨深度选择借鉴了 Kimi 的伪查询、RMSNorm 打分和来源维 softmax；但你新增的 token 对齐，使它成为了新的两级机制。[Kimi K3, §2.2](#)

第一种方式：让当前 token 主动查询每个历史来源。

这最接近你描述的 LRSA/IPOT 风格。为清楚起见，以下使用单头形式；实现时可以采用多头。

由当前状态生成 query：

$$Q_j = \text{Norm}(U_{j-1})W_{Q,j}.$$

由历史来源生成 key、value：

$$K_s = \text{Norm}(B_s)W_K, \quad V_s = \text{Norm}(B_s)W_V.$$

计算相似度：

$$L_{j,s} = \frac{Q_j K_s^\top}{\sqrt{d}} \in \mathbb{R}^{M \times M}.$$

这里 $L_{j,s}[m, i]$ 表示：当前第 m 个 token 对历史第 i 个 token 的匹配程度。

沿历史 token 维度归一化：

$$A_{j,s}[m, i] = \frac{\exp L_{j,s}[m, i]}{\sum_{n=1}^M \exp L_{j,s}[m, n]}.$$

然后：

$$R_{j,s} = A_{j,s} V_s W_{O,j}.$$

它与 LRSA/IPOT 压缩操作的主要区别是：

- 原压缩通常使用可学习 latent query，将 N 个点汇聚成 M 个 token。
- 这里使用**当前 latent 内容产生的 query**，将历史的 M 个 token 转换为当前 M 个 token 所需的表示。[LRSA](#)、[IPOT](#)

我更推荐这里采用当前内容 query，而不是重新放一组独立的可学习 query。因为你的目标是“对齐到当前 token”，当前内容正好可以定义要对齐的目标。

它解决 token 对应问题的依据很清楚。假设某份历史的 token 顺序被任意置换，置换矩阵为 P ：

$$K'_s = P K_s, \quad V'_s = P V_s.$$

则：

$$\text{softmax}(Q_j K'^\top_s) V'_s = \text{softmax}(Q_j K_s^\top) V_s.$$

因此，**历史来源可以各自采用不同的 token 编号，读取结果仍然由当前 query 的编号决定。**

这消除了“历史第 m 个 token 必须与当前第 m 个 token 对应”的要求。但它保证的是编号置换性质，实际学习到的对应是否符合有用的物理结构，仍由训练决定。

第二种方式：让历史 token 向当前目标 token 进行竞争性软分配。

这里必须区分两种落地方式。

最忠于原 Transolver 的实现是：

$$S_s[i, m] = \text{softmax}_m \left[(\phi_R(B_s) W_{\text{slot}})_{i,m} / \tau \right],$$

其中：

$$W_{\text{slot}} \in \mathbb{R}^{d_r \times M}.$$

然后按照目标槽位接收到的总权重计算均值：

$$R_s[m] = \frac{\sum_i S_s[i, m] \phi_V(B_s)[i]}{\sum_i S_s[i, m] + \varepsilon}.$$

这对应 Transolver 的“目标槽位维 softmax+按接收质量归一化”的 slice 原语。[Transolver 源码](#)

不过，这里的目标由 W_{slot} 定义，是一组**可学习的固定槽位**，并不会自动对应任意当前 $U_{j-1}[m]$ 。

如果你希望严格按照“当前 latent 的内容”进行分配，我建议补全为下面这个版本：

继续使用第一种方式的相似度：

$$L_{j,s}[m, i] = \frac{Q_j[m]^\top K_s[i]}{\sqrt{d}}.$$

但首先沿**当前目标 token**维度归一化：

$$S_{j,s}[i, m] = \frac{\exp L_{j,s}[m, i]}{\sum_{q=1}^M \exp L_{j,s}[q, i]}.$$

这意味着，对每个历史 token i ：

$$\sum_m S_{j,s}[i, m] = 1.$$

然后计算每个当前目标接收的总分配量：

$$\mu_{j,s,m} = \sum_i S_{j,s}[i, m],$$

并形成目标特征：

$$R_{j,s}[m] = \left[\frac{\sum_i S_{j,s}[i, m] V_s[i]}{\mu_{j,s,m} + \varepsilon} \right] W_{O,j}.$$

这样，目标槽位由当前 $Q_j[m]$ 定义，同时保留 Transolver 式的竞争分配与加权均值。

这是对原分配方式的明确修改：**用当前 query 定义目标，替代固定的 Linear(d, M) 目标槽位。**

两种方式的实质区别，是谁在归一化时彼此竞争。

比较项	CDPA-Cross	当前内容条件化的 CDPA-Slice
第一次 softmax 的维度	历史来源 token	当前目标 token
基本解释	每个当前 token 独立选择历史内容	每个历史 token 在当前目标间分配
是否需要接收质量归一化	不需要额外做	需要
当前目标之间是否在分配时竞争	没有显式竞争	有
对齐依据	当前 query	当前 query
主要矩阵计算量	$O(M^2 d)$	$O(M^2 d)$

第一种方式允许多个当前 token 同时高度关注同一个历史 token。

第二种方式先规定一个历史 token 的分配份额总和为 1，因此某些目标获得更多份额时，其他目标获得的份额会减少。它可能鼓励不同目标分工，但**不能保证不发生 token 塌缩**；完全对称的目标仍可能得到相似结果。

两者不是同一个 softmax 简单转置，也不能仅凭都叫 attention 就认为等价。

这类“沿目标槽位归一化，再计算加权均值”的机制也出现在 **Slot Attention** 中。该论文可以为你的第二种方式提供直接的数学与实现参考；你的使用场景则是跨深度 latent 对齐。[Slot Attention, §2.1、算法 1](#)

多份历史应当先分别对齐，再进行跨深度融合。

无论选择哪种方式，都建议先得到：

$$R_{j,1}, R_{j,2}, \dots \in \mathbb{R}^{M \times d},$$

再计算来源权重 $\alpha_{j,s,m}$ 。

这样，两级权重分别承担：

$$\underbrace{\beta_{j,s,m,i}}_{\text{某一历史内部，读取哪些 token}} \quad \underbrace{\alpha_{j,s,m}}_{\text{不同历史之间，采用多少信息}} \cdot$$

忽略投影，整体可以写成：

$$\bar{U}_{j,m} = \alpha_{j,0,m} U_{j-1,m} + \sum_s \alpha_{j,s,m} \sum_i \beta_{j,s,m,i} V_{s,i}.$$

这给新模块提供了清楚的功能分工：

token 对齐决定“从这一层读什么”；**深度选择**决定“这一层值得用多少”。

它通常不同于把所有历史 token 拼接起来做一次 attention。普通拼接 attention 会让各个深度在同一个 token softmax 中竞争；你的两级结构单独学习深度权重，更容易检查不同历史是否被使用。

对于 CDPA-Slice，各历史来源的 $\mu_{j,s,m}$ 也应分别计算。否则某一深度接收到的分配量会混入另一深度的归一化，两个机制的含义就不再清楚。

另外，虽然深度选择仍可以使用 Kimi 风格的固定可学习 w_j ，但现在：

$$R_{j,s} = R_{j,s}(U_{j-1}, B_s),$$

已经依赖当前状态。因此，来源之间的选择也会间接受到当前计算需求影响。

这两个改动比先前的直接 AttnRes 融合更完整，但成本也发生了实质变化。

原来的逐 token 深度融合，对一份历史主要只需：

$$O(Md).$$

现在先做 $M \rightarrow M$ 对齐，每份历史主要需要：

$$O(M^2d),$$

此外还有投影成本。

对 r 个后段块，当前状态直接作为候选，其余历史进行对齐。第 j 块需要对齐 $j + 1$ 份历史，总次数为：

$$\sum_{j=1}^r (j + 1) = \frac{r(r + 3)}{2}.$$

因此，新增对齐的主要成本随深度增长为：

$$O(r^2 M^2 d).$$

历史表示的存储量则主要为：

$$O(rMd).$$

这仍然没有新增物理点数 N 相关的读取，也不会重新运行前段原网格卷积，但不能再沿用先前“AttnRes 几乎只有加权求和成本”的估计。

例如：

$$r = 6, \quad M = 64, \quad d = 128,$$

需要对齐历史共 27 次。仅计算 cross-attention 的 $QK^\top$ 与 AV ，合计约：

$$0.0283 \text{ GMAC}.$$

投影、归一化和 softmax 需要另计。作为量级比较，前面假设 $N = 7225, D = 128$ 时，一条全通道 3×3 原网格卷积约为 1.065 GMAC。因此，在这个小 M 配置下，新增跨深度对齐仍有合理的计算预算。

但如果 M 从 64 增到 256，注意力乘积成本会增加到 16 倍；后段加深也会增加历史对数。实际延迟还可能受大量小矩阵运算和 kernel 启动影响。

为了控制成本，我建议：

- 历史 K/V 投影跨目的层共享，使每份历史产生后可以投影一次、后续复用。
- 每个目的层保留自己的 Q 和输出投影，以适应不同深度的读取需求。
- 历史缓存保持梯度连接。
- 不为每一个“来源层-目的层”组合都单独创建一套完整 Q/K/V 模块。

如果采用固定目标槽位的 Transolver 式版本，而且目标槽位与投影跨目的层共享，还可以将每份历史预先映射一次、随后复用。它的成本可能更低，但目标不再随当前 latent 内容变化，应该作为独立版本比较。

对于两种方案，我更建议先以 CDPA-Cross 建立基线，再用 CDPA-Slice 检验竞争分配是否更合适。

原因是 CDPA-Cross 的当前 query 对齐含义直接，也容易使用标准 SDPA 实现。第二种方式具有不同的归纳偏置，但不是天然更便宜或更准确。

最有解释力的比较是让两者共享：

- 相同历史来源；
- 相同 Q/K/V 投影形式；
- 相同跨深度选择；
- 相同 latent 数、宽度和后段块数；

只改变 token 对齐时的归一化方式。

同时保留“不断增长的历史”和“只使用固定 T_1, T_2, Z_0 ”的对照，才能判断新增后段历史是否真的有用。若只与无跨深度模块的模型比较，还无法分清收益来自历史复用、额外 attention 计算，还是竞争分配。

这类同时沿 token 与网络深度访问表示的总体思想，也可参考 OmniNet；因此，新工作的重点应落在你的低秩算子结构、两种对齐方式及其精度-效率结果上。OmniNet, ICML 2021

CDPA 目前能够明确解决的是两个结构问题：历史结果可以持续被访问；历史 token 不再必须按编号直接对应。后段新产生的表示仍来自已有输入信息，它们增加的是加工结果和可访问路径，不能恢复所有初始压缩表示都遗漏的内容。

名称中的 Physics 可以沿用 Transolver 的表达习惯，说明模块用于物理场潜表示交互；当前设计本身没有引入 PDE 方程残差或守恒约束。你的研究目标可以明确写成：

通过内容对齐和深度选择，持续复用不同阶段的物理 latent 表示，以较少的物理空间压缩重建支撑更充分的后段计算。

2026-09-12 21:23:05 You 

我整个模型的架构最终实现的目的应该是计算量/推理延迟还是要小于LRSA的，如果每层都加了CDPA-Cross 后还可不可以保持这个优势，如果还有优势优势还明显吗，如果还是只有 persistent latent中的第一次压缩时与前面的block计算一次会不会效率上的优势更明显

 ChatGPT gpt-6-astra-wm

我会按相同的前段层数、后段深度和 latent 大小，比较“每个后段块都加 CDPA-Cross”和“仅后段入口融合一次”的成本，并把规则网格卷积、投影和 attention 都计入。推理延迟会单独讨论，避免把计算量比例直接当成加速比。

估算显示：当点数远大于 latent 数时，每层加入 CDPA-Cross 仍能保留较大的计算量优势；改成只在入口融合一次，虽然显著减少了 CDPA 自身的开销，但整网计算量可能只再降约 1%。延迟上的差别更值得关注：逐个历史层调用 attention，和把同一目标层的历史批量计算，可能有很不同的表现。

可以。在点数 N 远大于 latent token 数 M 、后段深度适中的情况下，每层加入 CDPA-Cross，仍然有条件保留相对 LRSA 明显的计算量优势。但推理延迟需要单独验证。

只在 **persistent latent** 阶段入口融合一次，计算量和实现复杂度都会更低；不过，整网计算量的进一步下降可能没有想象中大，延迟上的改善反而可能更值得关注。

下面按照你当前的架构计算，不改变两个阶段的设计。

先明确比较对象。假设 LRSA baseline 有 8 个完整 block，你的模型是：

- 前段：2 个完整 LRSA block，保留其中的 latent 表示 T_1, T_2 。
- 衔接：将第二个完整 block 输出的点特征再次编码，得到 Z_0 。
- 后段：6 个 persistent latent block，每个仍保留 self-attention 和 FFN。
- 输出：按照 IPOT 的方式，用输出位置构造 query，最后解码一次。

这里必须计入衔接编码器。因此，你的模型总共有 **3 次点到 latent 的压缩、3 次 latent 到点的重建**；baseline 则各有 8 次。这与 LRSA 的逐块压缩—处理—重建，以及 IPOT 的编码—潜空间处理—解码结构相对应。[LRSA 原文](#)，[IPOT 原文](#)

CDPA-Cross 增加的是 $M \rightarrow M$ 的**历史读取**，没有重新执行 $N \rightarrow M$ 的点特征压缩。这是效率优势能够保留的基础。

每层加入 CDPA-Cross，究竟增加多少计算？

按照我们讨论的“历史持续增加”版本，第 j 个后段 block：

- 对前段的 T_1, T_2 进行对齐；
- 对此前已经产生的、更早的 persistent latent 表示进行对齐；
- 当前输入 U_{j-1} 作为恒等候选直接参与深度融合，不需要再做一次对自己的 cross-attention。

于是，6 个后段 block 分别需要对齐：

$$2, 3, 4, 5, 6, 7$$

份历史，总计：

$$A = \sum_{j=1}^6 (j+1) = 27.$$

只在入口融合一次，则只需要把 T_1, T_2 对齐到 Z_0 ，共 **2 次历史对齐**。

但不能据此认为逐层方案让整网开销增加了 $27/2 = 13.5$ 倍。这个比例只涉及历史对齐的数量，而且还没有考虑投影复用。

设特征宽度为 d 。一次 $M \rightarrow M$ cross-attention 中，两个主要注意力矩阵乘法的成本是：

$$QK^\top : M^2d, \quad AV : M^2d.$$

合计为：

$$2M^2d.$$

除此之外，还要计算 $Q/K/V/O$ 投影，不能只统计 attention 矩阵乘法。

如果沿用我们之前的设置——**历史 K/V 投影跨目标层共享并缓存，同一目标层复用 Q**，输出投影在各来源进入深度门控之前执行——那么，后段有 r 层时：

$$A = \frac{r(r+3)}{2},$$

$$C_{\text{CDPA,all}} \approx 2AM^2d + (A + 3r + 2)Md^2$$

而入口仅融合一次：

$$C_{\text{CDPA,once}} \approx 4M^2d + 7Md^2.$$

这里使用 MAC，即乘加运算次数，省略归一化、softmax 和深度门控等较小项。

因此，逐层版本的新增开销随深度近似为：

$$O(r^2M^2d + r^2Md^2).$$

它确实比一次融合贵，但主要仍发生在 M 个 token 上。LRSA 被省去的点特征投影、点 FFN、压缩和重建，则涉及 N 个点。

真正的判断条件是：

CDPA 新增成本 < 减少点空间往返及点空间处理所节省的成本.

在一个具体配置下，这个优势有多大？

下面给出统一架构预算，不是论文报告的 FLOPs，也不是实测速度。假设：

$$N = 85^2 = 7225, \quad M = 64, \quad d = 128,$$

并采用：

- 8 个 LRSA block，对比“2 个 LRSA+6 个 latent block”；
- 普通 FFN，隐藏层扩张率为 4；
- 点特征与 latent 特征使用相同宽度；
- attention 按标准投影计数；
- 统计主干主要矩阵运算，端点只计 cross-attention，未纳入任务相关输入输出头和端点 FFN。

在这个口径下，一个完整 LRSA block 的成本约为：

$$C_{\text{LRSA}} = 12Nd^2 + 24Md^2 + 4NMd + 2M^2d.$$

一个后段 latent self-attention+FFN block 的成本约为：

$$C_{\text{latent}} = 12Md^2 + 2M^2d.$$

代入后，CDPA 本身的成本为：

CDPA 使用方式	新增成本
只在入口融合一次	0.0094 GMAC
后段每层融合全部既有历史	0.0776 GMAC

再计入整个主干：

点空间卷积配置	8 层 LRSA	你的模型：每层 CDPA	你的模型：入口一次 CDPA
不额外加入卷积	13.47 GMAC	4.24 GMAC	4.17 GMAC
每次压缩前额外加入一条 3×3 全通道卷积	21.99 GMAC	7.44 GMAC	7.37 GMAC
用两条 3×3 全通道卷积替换对应的两个逐点投影	28.62 GMAC	9.92 GMAC	9.85 GMAC

后两行分别代表两种卷积接入方式的预算，不能混用：**额外添加卷积按完整卷积成本计，替换已有线性投影则需要扣掉被替换的线性成本**。表中 LRSA 执行 8 组点空间卷积，你的模型按前段两次加衔接编码一次，执行 3 组。

这些数字说明：

1. **在这个配置下，逐层 CDPA 仍保留了明显的计算优势**。相对采用同样卷积处理的 LRSA，主干主要计算量下降约 65%–66%。
2. **入口一次融合更便宜，但整网差距较小**。在有卷积的两种配置下，相对逐层 CDPA，整网计算量只进一步下降约 0.7%–0.9%。
3. 原因是此时 CDPA 的 $M = 64$ 很小，主要成本仍然在前段点空间处理。大部分节省已经由“后段不再反复回到点空间”获得。

这里还需要注意比较基准：如果只给你的模型加卷积，而 baseline 不加，就不能引用上述 65%–66%。例如，在同一预算中，对比不加额外卷积的 13.47 GMAC baseline，两个卷积版本的逐层 CDPA 模型分别减少约 **45% 和 26%**。

所以，卷积可以增加减少点空间往返所节省的**绝对计算量**，但不意味着相对降幅必然更大。

推理延迟为什么可能呈现不同的结果？

计算量低，不一定按相同比例转化为低延迟。这里尤其要关注 CDPA 的实现方式。

在上述 8 层对比中，主干 attention 计算次数可以列成：

架构与实现	Attention 调用次数示意
8 个完整 LRSA block	24
两阶段模型，不加 CDPA	14
入口一次 CDPA，两个来源批量处理	15
每层 CDPA，各层历史来源批量处理	20
每层 CDPA，对每份历史分别调用	41

这里统计的是 attention 调用，不是 GPU kernel 的精确数量，也不能直接用它预测速度。

关键区别在于：同一个目标 block 的多份历史，可以沿“历史来源”这个维度批量处理。因此，27 次来源-目标对齐，可以分布在 6 次批量 attention 调用中完成。

要保留这个实现的数学含义，需要：

- 每份历史内部独立计算 token softmax；
- 再进行跨深度的门控融合。

不能直接把所有历史沿 token 维拼接，改成一次统一 softmax，然后认为与原来的 CDPA 等价。

逐层版本更容易受到以下因素影响：

- 多次小矩阵运算的调度开销；
- 历史特征和 K/V 的读取；
- 深度门控、归一化等算子的额外开销；
- 是否重复计算了本来可以缓存的历史 K/V。

因此：

逐层 CDPA 在理论计算量上有较大的获胜空间，但需要实现得当，才能把这个空间转化为延迟优势。入口一次融合更容易获得稳定的延迟收益。

特别是在小 batch 推理中，入口一次方案减少的小算子和依赖步骤，可能比它减少的那不到 1% 的整网 MAC 更有意义。具体幅度必须实测。

针对你的研究目标，我更建议怎样安排？

我建议把“入口一次 CDPA + 后续持续 latent 计算”作为优先验证的效率版本，把“每个后段 block 都读取不断增长的历史”作为表达能力增强版本，与它进行直接比较。

这个建议有两个原因：

- 入口一次版本已经能够检验你的核心假设：复用前面不同压缩过程产生的 latent，能否让后段减少点空间往返，同时维持精度。
- 逐层版本是否值得保留，取决于它带来的精度提升，是否足以抵偿额外的延迟、历史存储和实现复杂度。

一次融合也有明确限制：历史被融合进当前 M 个 token 后，后续 block 不能再独立查询原始的 T_1, T_2 ；逐层版本则允许不同深度重新选择历史。因此，不能预先断言两者精度相同。

对你而言，最终最有价值的结果是：在达到 LRSA 相当或更低误差的前提下，哪个版本实际推理更快。目前的计算预算支持两者都有机会，其中入口一次方案更适合作为首先实现和验证的版本。

2026-09-12 21:41:59 You 

好的，我准备实现的是CDPA-Cross，CDPA-Slice 以后再考虑（作为后续补充），现在你仔细阅读slotattention,omninet, kimi k3的论文，确定一个最终计算的方式（在处理单历史block融合时已经准备采用CDPA-Cross 的方式，关键在于如何融合历史的多个block）

我会重点核对三篇论文对“多个历史 block 如何汇聚”的处理，确定 CDPA-Cross 中跨历史的归一化、权重计算、当前状态的保留方式，以及融合在 block 中的位置；也会明确哪些来自原论文，哪些是针对你模型的设计。

我会使用 PDF 阅读技能核对 Kimi 报告中的公式。当前要区分两个操作：从某一历史 block 中读取哪些 token，以及从多个历史 block 中选择哪些结果。Slot Attention 和 OmniNet 对这两步的组织方式不同，不能只因它们都用了 attention 就直接互换。

三篇论文的方法核对后，我建议采用“每份历史独立对齐，再对每个当前 latent token 做跨历史 softmax”。深度评分使用 Kimi 的可学习向量即可，因为对齐结果已经依赖当前 query，融合权重仍会随样本变化。另外，Kimi 保存的残差分支输出或块内累加量，与我们保存的 latent 表示不同，这部分需要明确写成针对本模型的改造。

我建议把多历史 block 的融合方式最终确定为：

各历史 block 独立做 CDPA-Cross 对齐 → 逐 latent token 的跨深度 softmax → 加权融合后进入 latent processor

跨深度评分采用 Kimi K3 的 AttnRes 式可学习向量；当前 latent 表示也作为一个候选参与融合。不需要为“多个历史 block 的融合”再增加一套完整的 Transformer。

这套融合规则同时适用于入口一次融合和后续逐层融合。按照上一轮的效率优先建议，**首个实现版本仍建议在 persistent latent 阶段入口调用一次**；下面先以两个历史表示 T_1, T_2 和当前编码结果 Z_0 给出完整定义。

以下完整组合是针对你模型的设计，三篇论文提供的是不同部分的依据，不能把它描述为其中某篇已经提出并验证的架构。

先把三篇论文中与这个问题直接相关的计算区分清楚。

论文	原文怎样汇聚信息	对当前方案的作用
Slot Attention	输入与 slots 计算匹配，softmax 沿目标 slot 轴进行；再沿输入轴归一化为加权平均，通过 GRU 更新 slots，可附加残差 MLP，并重复迭代	提醒我们区分归一化轴，以及不要假设不同 latent 集合具有相同编号对应关系
OmniNet	组织不同深度的全部 token，进行联合 attention；随后对同一 token 位置跨深度做逐通道 max pooling，并与最深层表示建立残差连接	支持跨深度复用表示，但其同位置池化不能直接套到独立压缩产生的 latent 上
Kimi K3 / AttnRes	对同一 token 位置的不同历史来源，用可学习向量与归一化后的特征计算分数；沿历史来源维度 softmax，再融合未经此次归一化的值	最适合作为“完成对齐以后，多个历史结果如何融合”的直接参考

对应原文：[Slot Attention §2.1、Algorithm 1](#)，[OmniNet §3.2、公式 4–6](#)，[Kimi K3 §2.2、公式 8–10](#)。

其中有两个细节必须保留：

- OmniNet 的 max pooling 是**每个通道分别取最大值**，不是为一个 token 选择某一份完整历史向量。
- Kimi 的 Full AttnRes 使用嵌入和残差分支输出；Block AttnRes 使用块内残差分支输出的累加量。**这些来源并不等同于你的 LRSA 中间 latent 表示。**

因此，我们借鉴的是 AttnRes 的深度评分和融合方式；历史来源的定义、跨 latent 对齐和接入位置，都按你的模型重新定义。

下面给出最终计算。

设：

$$T_1, T_2, Z_0 \in \mathbb{R}^{B \times M \times d},$$

其中：

- T_1, T_2 ：两个前段 LRSA block 内，完成 latent 处理、尚未重建回点空间的表示；
- Z_0 ：第二个完整 LRSA block 输出点特征后，经衔接编码器压缩得到的表示；
- B, M, d ：batch size、latent token 数、通道宽度。

这里保留完整的第二个 LRSA block，不能直接用 T_2 代替 Z_0 ，否则会绕过该 block 后面的重建和点空间更新。

首先，以同一个当前表示查询各份历史。

下面省略 batch 维度。对第 h 个 attention head：

$$Q^{(h)} = \text{LN}_q(Z_0)W_Q^{(h)},$$
$$K_s^{(h)} = \text{LN}_{kv}(T_s)W_K^{(h)}, \quad V_s^{(h)} = \text{LN}_{kv}(T_s)W_V^{(h)}, \quad s = 1, 2.$$

计算：

$$\beta_{s,m,n}^{(h)} = \text{softmax}_n \left(\frac{Q_m^{(h)} (K_{s,n}^{(h)})^\top}{\sqrt{d_h}} \right),$$
$$R_s = \text{ConcatHeads} \left[\sum_{n=1}^M \beta_{s,m,n}^{(h)} V_{s,n}^{(h)} \right] W_O.$$

于是：

$$R_1, R_2 \in \mathbb{R}^{M \times d}.$$

$R_1[m]$ 与 $R_2[m]$ 都是“针对当前第 m 个 latent query，从不同历史中读取的结果”。它们现在拥有共同的目标索引，才适合逐 token 比较和融合。

具体实现固定为：

- 同一个融合模块的 W_Q, W_K, W_V, W_O 在历史来源之间共享。
- Q 只计算一次。
- 每份历史内部独立执行 softmax。
- 每份 R_s 在进入深度评分前完成 W_O 投影。
- R_s 是读取结果，此处不分别加入 Z_0 ，也不为每份历史再添加 FFN。

这一步解决的是 token 对齐。它不要求 $T_1[m]$ 与 $T_2[m]$ 原本就代表相同区域，也不保证一定学到正确的物理区域对应关系。

然后，把当前表示加入候选集合。

定义：

$$R_0 = Z_0.$$

需要融合的候选变成：

$$\mathcal{R} = \{R_0, R_1, R_2\}.$$

当前表示走恒等路径，不需要对自己再执行一次 CDPA-Cross。

这样，模型可以学习在不同 token 上保留不同程度的当前信息，而不是被迫只从历史中取值。

接着，对每个当前 token，计算不同来源的分数。

采用一个可学习向量：

$$w \in \mathbb{R}^d.$$

对每个来源 s 和当前 token m ：

$$e_{s,m} = w^\top \text{RMSNorm}_{\text{depth}}(R_s[m]).$$

这里的 RMSNorm 沿通道维度进行，并在所有来源之间共享参数。

再沿来源维度执行：

$$\alpha_{s,m} = \frac{\exp(e_{s,m})}{\sum_{t=0}^2 \exp(e_{t,m})}.$$

因此：

$$\alpha_{s,m} \geq 0, \quad \sum_{s=0}^2 \alpha_{s,m} = 1.$$

最终融合为：

$$Z_{\text{fused}}[m] = \alpha_{0,m} Z_0[m] + \alpha_{1,m} R_1[m] + \alpha_{2,m} R_2[m].$$

RMSNorm 只用于这一步的评分；加权融合使用 R_s 本身。

例如，某个当前 token 可以得到：

$$(\alpha_0, \alpha_1, \alpha_2) = (0.2, 0.7, 0.1),$$

另一个 token 则可以得到：

$$(\alpha_0, \alpha_1, \alpha_2) = (0.6, 0.1, 0.3).$$

这表示不同 token 可以选择不同的历史组合。它不是为整个样本学习三个统一权重。

张量层面的定义如下：

张量	形状	运算轴
对齐结果与当前候选	$[B, 3, M, d]$	来源维度包含当前、历史 1、历史 2
RMSNorm 后的评分特征	$[B, 3, M, d]$	沿最后的通道维度归一化
深度分数 e	$[B, 3, M]$	与 w 做通道内积
深度权重 α	$[B, 3, M]$	沿大小为 3 的来源维度 softmax
融合结果	$[B, M, d]$	沿来源维度加权求和

跨深度权重在多头合并后计算，**每个 token、每个来源产生一个标量权重，作用于整个 d 维向量**。这是首版采用的粒度。

融合结果随后进入正常的 latent processor。

入口一次融合版本：

$$\begin{aligned} U_0 &= Z_{\text{fused}}, \\ \widetilde{U}_\ell &= U_{\ell-1} + \text{SA}_\ell(\text{LN}(U_{\ell-1})), \\ U_\ell &= \widetilde{U}_\ell + \text{FFN}_\ell\left(\text{LN}(\widetilde{U}_\ell)\right). \end{aligned}$$

最后按照 IPOT 风格，用输出坐标构造 query，对最终 latent 表示解码。

这里要明确两种残差的关系：

- CDPA 融合时，当前 Z_0 已作为 R_0 参与来源选择，因此采用的输出就是 Z_{fused} 。
- 后续 SA 和 FFN 保留各自正常的残差连接。

当前方案不再把 CDPA 输出定义成 $Z_0 + Z_{\text{fused}}$ 。后者会变成另一种“当前表示始终以单位系数保留，再叠加历史”的架构，和我们现在的来源选择机制不同。

mermaid

```
graph TD
    T1["前段历史 T1"] --> C1["CDPA-Cross : 得到 R1"]
    T2["前段历史 T2"] --> C2["CDPA-Cross : 得到 R2"]
    Z["当前编码 Z0"] --> C1
    Z --> C2
    Z --> F["逐 token 的跨深度 softmax 融合"]
    C1 --> F
    C2 --> F
    F --> P["持续潜空间 SA 与 FFN"]
    P --> D["输出坐标 query 解码"]
```

为什么我选择这个多历史融合方式，可以从三个数学关系理解。

第一个关系：它把“历史内部读什么”和“不同历史如何取舍”分开了。

为了简洁，考虑单 head，并把输出投影吸收到值向量中。某份历史的读取结果为：

$$R_s[m] = \sum_n \beta_{s,m,n} \widetilde{V}_s[n].$$

融合后：

$$Z_{\text{fused}}[m] = \alpha_{0,m} Z_0[m] + \sum_{s=1}^2 \sum_n \underbrace{\alpha_{s,m} \beta_{s,m,n}}_{\omega_{s,m,n}} \widetilde{V}_s[n].$$

其中：

$$\omega_{s,m,n} = \underbrace{\alpha_{s,m}}_{\text{这份历史应占多少}} \underbrace{\beta_{s,m,n}}_{\text{在这份历史内部读哪里}}.$$

这就是采用两级归一化的意义：

- 第一级 softmax 沿历史内部的 token 维度；
- 第二级 softmax 沿不同历史来源维度。

它与 Slot Attention 的两步归一化不同。Slot Attention 的第一步让目标 slots 竞争解释输入；这里第一步已经固定为普通 cross-attention，并不引入这种 slots 竞争。

第二个关系：它与“拼接全部历史再做一次 cross-attention”不等价。

假设直接拼接历史，内部匹配分数为 $a_{s,m,n}$ ，则统一 softmax 为：

$$p_{s,m,n} = \frac{\exp(a_{s,m,n})}{\sum_t \sum_j \exp(a_{t,m,j})}.$$

它隐含的历史来源权重只能是：

$$\alpha_{s,m}^{\text{concat}} = \text{softmax}_s (\text{LogSumExp}_n a_{s,m,n}).$$

也就是说，拼接方案的“这份历史占多少”，由匹配 logits 的总量决定。

我们的方案则对**已经读取出的内容** $R_s[m]$ 进行评分。因此，来源选择不再被上述 LogSumExp 关系唯一确定。

还有一个直接区别：如果给某份历史的所有内部 logits 加上同一个常数，其内部 softmax、读取结果和我们的深度权重都不变；拼接方案中，该来源的总权重却会变化。

这说明两者具有不同的归纳偏置，**不能据此断言我们的方案一定更准确**，但它确实更直接地实现了你提出的“先读取每次压缩的结果，再决定如何利用不同压缩历史”。

第三个关系：可学习向量 w 固定，并不意味着融合权重是静态的。

虽然推理时 w 是固定参数，但：

$$R_s = \text{CrossAttention}(Z_0, T_s)$$

已经依赖当前状态和对应历史。于是：

$$\alpha_s = \text{softmax}_s [w^\top \text{RMSNorm}(\text{CrossAttention}(Z_0, T_s))]$$

仍然随样本、当前 token 和历史内容变化。

因此，首版无需再为深度融合添加一套“由当前 token 生成 Q、由各历史结果生成 K/V”的完整注意力投影。后者可以增加显式的当前-历史匹配能力，但其必要性需要实验验证。

还有几个实现细节，我建议现在就固定下来。

设计项	首版选择	原因
多历史融合	逐 token 的深度 softmax 加权和	保留多来源贡献，同时控制来源总权重
深度评分	$w^\top \text{RMSNorm}(R_s)$	对齐已包含当前 query 条件，评分保持轻量
当前状态	作为恒等候选 R_0	可以学习保留当前编码

设计项	首版选择	原因
来源独立参数	共享 Cross 投影及归一化参数	控制参数量，避免为每份历史建立一套大投影
深度或来源编号偏置	首版不引入	先验证基于对齐内容的来源选择是否有效
深度权重 dropout	设为 0	保持明确的归一化加权融合
额外迭代	每次调用完成一轮对齐和融合	保持已讨论的计算预算
数值计算	深度评分、softmax 和加权归约用 FP32	这部分张量小，便于控制混合精度误差
历史梯度	保留计算图	让后段监督通过历史读取路径训练前段表示

初始化采用：

$$w = 0,$$

RMSNorm 的缩放参数初始化为 1，Cross 的投影使用常规非零初始化。此时：

$$\alpha_{0,m} = \alpha_{1,m} = \alpha_{2,m} = \frac{1}{3}.$$

这是均匀融合初始化，不是恒等初始化。值路径仍有梯度，深度评分向量也能够学习；当前状态的初始系数就是 1/3，需要如实记录。

首版选择它，是为了少引入一个人为的来源偏好。若后续实验发现当前编码在入口融合时被过度削弱，再单独测试当前来源偏置；不把这个尚未验证的稳定化措施混入主结果。

深度 RMSNorm 的作用也应准确表述：它减小特征整体幅度对评分的直接影响，但**不负责保证不同来源的物理单位或语义一致**。兼容性仍由投影、前段网络和端到端训练共同学习。

在融合这一步，非负且归一化的权重满足：

$$\|Z_{\text{fused}}[m]\| \leq \sum_s \alpha_{s,m} \|R_s[m]\| \leq \max_s \|R_s[m]\|.$$

因此，它不会仅因为来源数量增加，就像直接求和那样自然放大输出幅度。这个界只针对当前融合步骤，不是整个模型的稳定性证明。

效率方面，多历史门控本身很轻。有 S 份历史时：

$$C_{\text{对齐}} = O(SM^2d + SMd^2),$$

$$C_{\text{深度评分与融合}} = O((S + 1)Md).$$

所以，新增成本主要来自已经确定的 CDPA-Cross，跨深度评分并没有再增加一个 $M \times M$ 或 $(SM) \times (SM)$ 的 attention 矩阵。不同历史的 Cross 可以合并到 batch 维度批量处理，同时保留各来源独立的 softmax。

最后，研究结论应落在一个可验证的表述上：

通过当前 latent 对历史压缩表示进行查询对齐，再学习逐 token 的跨深度选择，检验能否减少点空间压缩-重建次数，并保持预测精度。

它可以增加对历史信息的访问路径，但融合后仍只有 $M \times d$ 的状态，不能预先宣称无损，也不能宣称严格等价于多次新的压缩-重建。

首轮最直接的三个对照是：**不使用历史**、**Cross 对齐后固定均匀融合**、**Cross 对齐后采用上述可学习深度融合**。这能分别回答“历史是否有用”和“多个历史之间的自适应选择是否有用”，且三者的比较不会被其他结构改动混淆。

好的没有问题就按你提供的建议来，现在我准备通过codex修改代码实现与你讨论的模型了，请给一个推荐的仓库（这个仓库的作用是作为初始仓库，通过修改它实现我跟你讨论的新模型）

我会检查候选仓库的实际代码、数据入口和训练脚本，重点比较 LRSA 与 Transolver，判断哪个更适合作为起点实现我们已经确定的两阶段模型和 CDPA-Cross。

我倾向于以 LRSA 仓库为起点：它已经把压缩、latent 处理和重建拆成了独立模块，便于保留前段完整 block，并在同一训练流程下比较原始 LRSA 与新模型。需要新增的主要是衔接编码、CDPA 融合、持续潜空间处理和输出坐标解码接口；后者在现有顶层接口中还没有。

推荐使用 [Adversarr/LRSA-Operator](#) 作为初始仓库，在其中新增我们讨论的两阶段 CDPA 模型，并保留原始 LRSA 作为 baseline。

我检查了实际代码。对你当前的目标，它比从 Transolver 或 IPOT 开始更合适，最主要的原因是：**前段可以直接保留完整 LRSA block，训练和评价流程也可以沿用，便于判断新架构是否真正优于 LRSA。**

具体对应关系如下：

你的模型部分	仓库已有基础	修改方式
前段两个完整 LRSA block	已有 down-attention、latent 处理、up-attention，以及外层点残差和 FFN	保留完整计算，额外返回 up-attention 之前的 T_1, T_2
衔接编码器	已有独立的 PerceiverDownProject	新建编码器，将第二个完整 block 的点特征压缩成 Z_0
CDPA-Cross	已有标准多头注意力和原生 SDPA 路径	新增独立历史对齐、逐 token 深度评分和融合
后段 persistent latent blocks	已有 latent self-attention、FFN 等基础模块	新建持续潜空间处理模块
输出位置解码器	已有 up-attention 和位置编码基础模块	新增由独立输出坐标产生 query 的解码器
规则网格卷积	已有规则网格模型分支	按约定新增 Transolver/LinearNO 风格的压缩前卷积

压缩、latent attention 和重建的实现集中在 `attn.py`，完整 block 的外层残差与点 FFN 位于 `perceiver.py`。这种拆分使我们可以复用前段结构，并单独编排后段。

另外，仓库已有 Darcy、Elasticity、Airfoil/NACA、Pipe、Plasticity、Navier–Stokes 的预处理、训练配置和评价入口，也有 AirFRANS、ShapeNet-Car 的专用代码。标准任务需要先转为其 HDF5 数据格式。[仓库说明](#)

我建议三个仓库承担不同作用：

仓库	在本项目中的作用
LRSA-Operator	唯一主仓库： 模型实现、数据处理、训练、评价及 LRSA 对照
IPOT	参考持续潜空间处理和输出坐标解码方式
Transolver	参考规则网格的压缩前卷积处理

选择 Transolver 为主仓库，需要先重新实现你要求保留的完整 LRSA block；选择 IPOT，则需要补建整个前段。以 LRSA 为基础，可以更直接地保留你的主要比较对象。

交给 Codex 时，有四个实际问题需要明确。

- 保留原始 LRSA，另加新模型类和配置。**
让两个模型使用相同的数据划分、归一化、损失和评价代码。这样，精度和延迟差异才更容易归因于架构。
- 必须新增独立的输出坐标接口。**
现有 `PerceiverModel.forward(x, fx, mass)` 默认按输入点形状输出，没有我们需要的独立 `output_coords`。新模型需要同时修改解码 query 和输出形状逻辑；同分辨率训练时，可以默认输出坐标等于输入坐标。
- 不能把已有卷积开关当成我们要求的卷积。**
仓库的规则网格卷积主要位于重建后的点 FFN 中。你要求的是 Transolver/LinearNO 风格的压缩前特征处理，位置和功能不同，需要明确实现。[规则网格模型代码](#)
- 环境与实际配置需要单独固定。**
当前依赖声明使用 PyTorch 2.9 系列，并配置 CUDA 12.8 安装源，还包含 xFormers、Liger、Lightning、Hydra 等依赖。实现时应统一 attention 后端，并按实际配置统计效率；之前用于估算的 $M = 64, d = 128$ 不能直接当成仓库各任务的默认值。[依赖配置](#)

我检查的版本是提交：

```
text

47b03f8c8c8da30bbcc0737b008dc4548f9cb98e
```

可以从这个明确版本创建开发分支：

```
bash

git clone https://github.com/Adversarr/LRSA-Operator.git
cd LRSA-Operator
git switch -c cdpa-cross 47b03f8c8c8da30bbcc0737b008dc4548f9cb98e
```

这次完成的是仓库和代码结构检查，尚未运行真实训练。后续 Codex 的实现应首先保证：**原始 LRSA 仍可运行，新模型能够完成前向、反向和独立输出坐标解码，再进入真实数据上的精度与效率对比。**

2026-09-12 22:09:22 You 

由于数据集比较方便我还是准备使用transolver作为基本仓库进行修改，但你也要仔细浏览LRSA和IPOT原仓库，因为会用到它们的block，同时注意只是修改block要保证依旧可以进行正常的的数据读取，我的思考是只要保持transolver原来对下载好的数据的处理方式（也就是不修改处理数据的部分或其他不影响模型架构只影响数据读取的部分），只是调整模型架构就可以在transolver仓库上直接修改，我这样思考有问题吗，注意先不要生成修改计划，我不让你生成修改计划就不要生成。另外补充一个问题：对于规则网格的处理，LRSA和transolver的处理方式一样吗（我指的是卷积部分及卷积后续的接入），这部分我需要调整吗

 ChatGPT gpt-6-astra-wm

你的思路总体可行。我会核对 Transolver 的数据到模型接口，以及 LRSA、IPOT 的 block 输入输出，确认哪些部分可以直接替换；再单独比较 LRSA 和 Transolver 的卷积位置与后续接入方式。本轮只回答这些问题，不生成修改计划。

你的思路是成立的。可以确定以 Transolver 为基础仓库，保留它对已下载数据的读取、采样、归一化和标签处理，在模型内部实现我们讨论的架构。使用 LRSA、IPOT 的 block，不要求同时采用它们的数据处理代码。

需要准确区分两件事：可以把改动限制在模型架构侧，但不能把“只修改 block”理解为只替换原来的 attention 函数、其余模型组织完全不变。

我核对了三个仓库的相关实现，下面只解释兼容性和卷积差异，不展开修改计划。

保留 Transolver 数据处理，为什么可行？

数据读取程序负责提供坐标、输入场、时间条件和标签；模型负责把这些输入变成对应的预测。只要新模型满足原来的输入输出约定，内部使用 slice、LRSA cross-attention，还是持续潜空间计算，都不会要求改变原始数据文件格式。

Transolver 不同任务的接口略有区别：

任务	原模型接口的关键约定
Darcy	输入坐标与系数场，输出对应节点上的标量场
Elasticity、Airfoil、Pipe	原脚本可以用 <code>model(x, None)</code> ，输出对应节点上的标量预测
Navier–Stokes	输入历史场窗口，每次调用输出下一时刻；外部脚本组织时间推进
Plasticity	除坐标和输入特征外，还传入时间条件 <code>T</code>
ShapeNet-Car、AirfRANS	使用各自的数据对象，输出必须保持原节点顺序、通道含义及表面掩码对应关系

这些差异来自实际的[标准任务模型接口](#)、[ShapeNet-Car 模型](#)和 [AirfRANS 模型](#)。

因此，你的想法可以更准确地表述为：

保持 Transolver 原有的数据到模型、模型到损失函数的接口，在接口内部替换网络架构。

这里不仅包括输入输出的形状，还包括坐标与节点顺序、时间条件、预测通道以及输出是否仍处于原来的归一化空间。无需为使用 LRSA 或 IPOT 而重新转换成它们的数据格式。

但两阶段模型确实涉及网络整体的计算组织。原 Transolver 的每个 block 基本上都是：

$$N \text{ 个点特征} \rightarrow N \text{ 个点特征}.$$

你的模型后段则是：

$$M \text{ 个 latent} \rightarrow M \text{ 个 latent},$$

最后才解码到输出点。因此，模型内部的 block 串联方式和最后输出头需要符合这个结构。**这仍然属于模型架构变化，不属于数据读取变化。**

另外，“持续潜空间”目前指的是一次 forward 内跨网络深度持续计算，并不意味着自动在 Navier–Stokes 的多个物理时间步之间复用 latent。保持这个区别，原来的时间推进脚本就可以继续承担原来的职责。

输出位置解码器也可以使用原任务已有的坐标进行同节点预测，因此普通训练不需要额外的数据输入。不过，真正评价不同输入、输出分辨率时，仍要提供相应查询坐标与真值；坐标解码器本身不会自动补齐超分辨率评价流程。

原 LRSA、IPOT 的模块需要接口适配，但这些适配可以留在模型内部。

例如：

- LRSA 的完整规则网格模型从 $[B, H, W, p]$ 的坐标张量获取网格形状，而 Transolver 通常已经将坐标展平为 $[B, N, p]$ ，再通过模型的 H, W 参数恢复网格。
- IPOT 原解码器的位置编码路径在直接接收 $[B, N, p]$ 坐标时，会把批次和点维度一起展平；不能未经适配直接用于 Transolver 的逐样本坐标输入。[IPOT 解码器代码](#)

这些是“模块怎样接收现有张量”的问题，不构成更换数据读取流程的理由。

LRSA 与 Transolver 的规则网格卷积不一样，而且最重要的区别是卷积的位置和用途。

这里的 LRSA 指其仓库提供的 **StructuredPerceiver / ConvFFN 分支**。并不是所有规则网格任务都自动启用这个分支。

对比项	Transolver 的结构化网格 attention	LRSA 的 StructuredPerceiver
卷积位置	压缩之前，位于 attention 内部	压缩、latent 处理、重建之后，位于点 FFN 分支
卷积数量	两条并行卷积分支	一条空间卷积
两条分支的作用	一条生成分片依据，一条提供被汇聚的特征	对重建后的点特征进行局部处理
2D 卷积设置	默认 3×3 , stride 1, padding 1	3×3 , stride 1, padding 1
是否是 depthwise convolution（逐通道卷积）	不是	也不是 ；虽然变量名叫 dwconv，实际未设置逐通道分组
卷积后紧接的计算	分头，然后分别进入分片权重生成和加权汇聚	LayerNorm $\rightarrow$ Linear $\rightarrow$ GELU/SiLU $\rightarrow$ Linear
是否改变网格分辨率	不改变	不改变
外层点 FFN	普通逐点 MLP	含卷积的 FFN

来源：[Transolver Physics Attention](#)、[LRSA 结构化网格模型](#)。

具体来看，Transolver 先对当前点特征 H 归一化：

$$F = \text{LN}(H).$$

然后执行两条卷积：

$$X_c = \text{Conv}_x(F), \quad V_c = \text{Conv}_v(F).$$

第一条分支生成分片权重：

$$A_{nm} = \text{softmax}_m \left(\frac{L_{\text{slice}}(X_{c,n})}{\tau} \right).$$

第二条分支提供被汇聚的值：

$$T_m = \frac{\sum_n A_{nm} V_{c,n}}{\sum_n A_{nm} + \varepsilon}.$$

随后在 latent 中计算 attention，并使用同一份 A 完成 deslice，最后才接外层残差和逐点 MLP。

因此，Transolver 的卷积同时影响：

1. 当前点应该如何分片；
2. 每个分片汇聚到什么内容。

由于 deslice 复用分片权重，卷积还通过 A 影响当层重建。

LRSA 的卷积顺序则是：

$$U = H + \text{LRSA}(N_{\text{att}}(H)),$$

这里的 LRSA 已经完成：

$$\text{down-attention} \rightarrow \text{latent FFN/attention} \rightarrow \text{up-attention}.$$

之后才执行：

$$H_{\text{next}} = U + L_2 [\sigma (L_1 [\text{LN} (\text{Conv}_{3 \times 3} (N_{\text{ffn}}(U))))].$$

也就是说，LRSA 的这个卷积分支先接收已经融合全局信息的点特征，再进行局部处理。卷积前有 FFN 分支的归一化，卷积后还另有 LayerNorm。

它当然会影响下一层的压缩，但不会改变已经完成的当层压缩和重建。

这个区别对你的 CDPA 历史尤其重要：

- T_i 缓存在第 i 个前段 block 的重建之前。
- 如果使用 LRSA 原来的后置 ConvFFN， T_i 尚未包含本层后置卷积的结果，但可以包含此前各层卷积的影响。
- 如果采用 Transolver 风格的前置卷积，局部处理在本次压缩之前就已经生效，因此会进入本次缓存的 latent。

所以，你需要调整的是卷积与新压缩模块的接入关系，数据读取不需要因此改变。

Transolver 的卷积分支原本接的是软分片：一路接 Linear $\rightarrow$ slice softmax，一路参与分片加权平均。LRSA 的压缩则使用可学习 query 查询点特征的 K/V。两者后续算子不同，不能把原连接原样保留，就认为已经变成 LRSA。

按照你此前明确的意图，采用的是 Transolver 风格的压缩前局部处理，结合 LRSA 风格的压缩与重建。这是新模型中的组合方式，而不是原 LRSA 规则网格 block 的逐行复制。

如果把 LRSA 原来的整个 StructuredPerceiver block 搬过来，又保留 Transolver 的前置卷积，就会同时存在：

$$\text{压缩前卷积} + \text{重建后 ConvFFN}.$$

这将增加另一组局部处理及其计算成本，不能把它当成两种卷积实现的等价替换。

还有一个与数据顺序有关的细节：Transolver 的 Airfoil、Pipe 等任务使用的是具有二维索引邻接的**结构化网格**，物理坐标未必均匀。卷积依赖原来的 $H \times W$ 排列。因此，保留你已有的数据处理和展平顺序，恰好有利于保留正确的卷积邻接关系。

2026-09-12 23:16:15 You 

好的，那就先采用LRSA的卷积方式吧，IPOT的论文中是不是没有卷积操作，那你觉得对于我这个模型在后续的persistant latent层中有没有必要加入卷积操作

我会按你现在确定的 LRSA 卷积方式来判断，核对 IPOT 的论文与实现，并区分“有空间邻接关系的网格”与“当前的 latent token 集合”，看后段卷积是否有合理作用。

可以，规则网格部分就采用 LRSA 的卷积方式：在前段每个完整 block 重建回点空间后，通过含卷积的 FFN 进行局部处理。对于后续 persistent latent 层，我建议首版保留 self-attention+FFN，不额外加入跨 latent token 的卷积。

你对 IPOT 的理解是正确的：IPOT 原论文的模型没有这里所说的 Conv1d、Conv2d、Conv3d 空间卷积模块。其主体由 cross-attention 编码器、self-attention 潜空间处理器和 cross-attention 解码器组成，配合归一化、FFN 和位置编码；原仓库相应模型代码也没有空间卷积层。
[IPOT 原文](#)，[模型代码](#)

是否需要给你的后段加入卷积，关键在于：当前 latent token 之间有没有适合卷积利用的邻接关系。

目前，latent 的编号并不代表物理空间邻接。

前段卷积作用于：

$$H \in \mathbb{R}^{B \times H_g \times W_g \times d},$$

相邻数组位置具有明确的网格邻接关系，因此 3×3 卷积可以处理局部结构。

压缩后的表示则是：

$$Z \in \mathbb{R}^{B \times M \times d}.$$

这 M 个 token 由可学习 query 对点特征进行汇聚得到。它们可以具有稳定的槽位身份，但：

latent 编号接近 $\nrightarrow$ 所代表的物理区域相邻。

例如， $M = 64$ 时，虽然可以把它重排成 8×8 ，但这个重排本身没有建立一个真实的二维空间。此时加入 3×3 卷积，实际上是在规定：人工排列中相邻的 latent 应当具有局部交互关系。

CDPA-Cross 也没有解决这个问题。它将不同历史表示对齐到当前 query，使历史结果可以逐 token 融合；它没有为当前这些 query 建立二维网格邻接。

数学上，可以用排列性质说明这种差异。对于不额外加入编号位置偏置的 latent self-attention 和逐 token FFN：

$$F(PZ) = PF(Z),$$

其中 P 是 latent token 的排列矩阵。模型的计算关系不依赖某种特定的 token 排序。

固定索引上的局部卷积 C 则通常满足：

$$C(PZ) \neq PC(Z).$$

因为打乱编号就改变了卷积邻居。

这不表示 latent 卷积一定无效。可学习 query 与卷积可能共同学出有用的组织方式。但那是一个额外的学习假设，目前我们的架构并没有给出它的物理邻接依据。

采用 LRSA 卷积以后，后段也并不是从“没有局部信息”的表示开始。

按照你现在确定的位置，信息经过：

$$\begin{aligned} H_0 &\rightarrow T_1 \rightarrow \text{重建}_1 \rightarrow \text{ConvFFN}_1 \rightarrow H_1, \\ H_1 &\rightarrow T_2 \rightarrow \text{重建}_2 \rightarrow \text{ConvFFN}_2 \rightarrow H_2 \rightarrow Z_0. \end{aligned}$$

因此，进入 CDPA 的三个来源具有不同的处理经历：

表示	是否包含前段卷积的影响
T_1	尚未经过第一个 block 的后置卷积
T_2	可以包含第一次卷积的影响，尚未经过第二次后置卷积
Z_0	由 H_2 压缩得到，可以包含两次卷积的影响

这里缓存位置仍然是 latent 处理完成、up-attention 之前，与 LRSA 原 block 的计算顺序一致。

后段通过 CDPA 融合这些来源，再使用 self-attention 和 FFN 加工它们。因此，你已经拥有：

- 前段真实网格上的局部特征处理；
- 潜空间中的全局交互；
- 跨深度历史信息的选择与复用。

这为首版后段不额外加卷积提供了合理依据，虽然它不能保证压缩已经保留了全部局部细节。

还要区分两类“latent 卷积”。

操作	实际增加的能力
沿 latent 编号做 kernel size > 1 的 Conv1d，或重排成网格后做 Conv2d	引入基于该排列的邻域混合，但邻接是否合理尚未确定
每个 latent 上做 1 × 1 卷积	只是通道线性变换，与逐 token 的 Linear 等价，FFN 已包含这类运算

所以，加入 1 × 1 卷积不会额外获得物理空间的局部建模能力；加入跨 token 卷积则需要解释邻接关系从哪里来。

从效率上看，直接在小规模 latent 上增加卷积，开销可能并不大。例如，全通道 $k \times k$ 卷积的主要成本为：

$$O(k^2 M d^2).$$

因此，我并不认为它必然会破坏相对 LRSA 的效率优势。**目前不加入它的主要理由，是其空间假设和额外收益尚不明确，而现有后段已经具备 token 混合与通道非线性。**

将来如果 latent 明确对应某个规则粗网格，或者拥有经过验证的稳定邻接结构，卷积会有更直接的作用依据。在当前表示方式下，不能仅因为输入数据是规则网格，就认为压缩后的 latent 也适合使用同样的空间卷积。

2026-09-12 23:22:35 You 

再考虑一个问题：就是latent token数量，你首先下仔细了解transolver,linearno,LRSA,IPOT,LinnearNO的分片数量（根据论文，再哪些数据集上都采用了哪些分片/latent token数，要仔细阅读论文，论文中所有的实验设置都要覆盖到）

 ChatGPT gpt-6-astra-wm

我会以论文及附录为主，逐项整理 Transolver、LinearNO、LRSA 和 IPOT 在各数据集、消融及泛化实验中使用的分片／latent token 数，并区分默认配置、变化范围和论文未明确说明的设置。你提到的“LinnearNO”我先按重复的 LinearNO 处理。

已经发现几个容易混淆的设置：LRSA 的 Navier–Stokes 主实验用 32 个 latent，但效率实验用 64 个；Transolver 附录还测试了前四层 64、后四层 32 的多尺度配置。IPOT 多个任务则使用 256 或 512 个 latent，明显高于另外几篇的常见配置。我会把这些例外单独列出。

核对时还发现，LRSA 的配置表与部分结果表不能完全对应：Navier–Stokes 配置表写的是 32 个 latent，但主表误差与消融表中 64 个 latent 的结果相同。我会保留论文明确写出的配置，并标注这些疑点，避免把推测当成确定设置。

我把这四篇论文的正文、附录配置表、分片数消融、效率测试及泛化实验都核对了一遍。**不能简单概括成“Transolver、LinearNO、LRSA 用 32/64，IPOT 用 256/512”**：这只覆盖了部分默认配置，遗漏了若干重要实验，而且 LRSA 的配置表与部分结果表存在对应不清的问题。

你最后写的“LinnearNO”我按重复的 **LinearNO** 处理。下面的页码均指你上传的 PDF 页码。

为避免混淆，以下统一用 M 表示分片数或 latent token 数，并采用三个标记：

- **直接给出的数值**：论文配置表或对应实验明确报告。
- $M^\dagger$ ：对应实验没有重新报告 M ，按该数据集的总体配置理解；不能视为该实验独立确认了这一数值。
- **未说明**：论文不足以确定，不能用仓库默认值代填。

一、所有数据集的总体配置

这张表汇总的是各论文**架构配置表明确报告的 M** ，不是在消融实验中取得最低误差的 M 。

数据集／任务	Transolver	LinearNO	LRSA	IPOT
Elasticity	64	64	64	512
Plasticity	64	64	64	256
Airfoil	64	64	64	128
Pipe	64	64	32	—
Navier–Stokes	32	32	32，见后文疑点	512
Darcy	64	64	64，见后文疑点	256
ShapeNet-Car	32	32	64	—
AirfRANS	32	32	64	—
Irregular Darcy	—	—	32	—
Pipe Turbulence	—	—	32	—
Heat Transfer	—	—	32	—
Composite	—	—	32	—
Burgers	—	256，Cross-LinearNO	—	256
Spherical shallow water	—	—	—	256
ERA5	—	—	—	512

“—”表示该论文没有报告这一任务的对应配置，不表示模型不能处理该任务。

原文依据分别是：

- **Transolver**：附录 B.3、Table 8，第 16–17 页。[原文](#)
- **LinearNO**：附录 B、Table 8，第 12 页。[原文](#)
- **LRSA**：附录 D.1、Table 9，第 18 页。[原文](#)
- **IPOT**：Table 5，第 13 页。[原文](#)

这里先补充三个容易忽视的细节。

第一，IPOT 的 Burgers 出现在附录的数据、架构和训练配置表中，但没有出现在正文 Table 1 的主结果表里。因此，可以确定它报告了 $M = 256$ 的设置，不能说主结果表完整展示了这个任务的性能。

第二，IPOT 对 Navier–Stokes 只给出一套总体架构配置 $M = 512$ ，附录另外比较了三种黏度；没有分别为三种黏度重列 latent 数量。

第三，这里的 M 都不能再乘上头数或层数，当成某一层的 latent 序列长度：

模型	M 的具体含义
Transolver	每个 attention head 内生成的 slice token 数；不同 head 的分片可以不同
LinearNO	每个 head 内的分片／低秩维度，中间聚合结果具有 $M \times d_h$ 的形状
LRSA	每个 block 压缩后 latent 序列的长度；多头在通道维组合
IPOT	持续计算的 latent 序列长度，即可学习查询矩阵 $M \times d_z$ 的第一维

例如，8 个头、 $M = 64$ ，不能表述为一个由 512 个全宽 token 构成的 latent 序列。同样，8 层各有 64 个 latent，也不意味着某一层有 512 个 latent。

二、四篇论文实际扫描过哪些 M

这里的“六个标准任务”统一指：Elasticity、Plasticity、Airfoil、Pipe、Navier–Stokes、Darcy。

论文	分片数／latent 数消融覆盖的数据集	实际测试的全部 M	完整结果出处
Transolver	六个标准任务，全部覆盖	1、8、16、32、64、96、128、256、512、1024	Table 9，第 18 页

论文	分片数／latent 数消融覆盖的数据集	实际测试的全部 M	完整结果出处
LinearNO	六个标准任务，全部覆盖	16、32、48、64、96、128、256	Table 11，第 14 页
LRSA	六个标准任务，全部覆盖	8、16、32、64、128、256	Table 12，第 20 页
IPOT	ERA5	64、128、256、512	Table 3，第 6 页

这张表有几个重要区别：

- Transolver 正文 Table 4 只展示 Elasticity、Darcy，但附录扩展到了全部六个标准任务。
- LinearNO 正文 Table 5 只展示部分任务，附录 Table 11 才是完整六任务结果；其中 **48、96** 都不能遗漏。
- LRSA 正文 Figure 5 展示四个任务，附录 Table 12 另外覆盖了 Pipe、Plasticity。
- **IPOT 没有报告在所有任务上系统扫描 M** ；其明确的 latent 数消融集中在 ERA5。

这些是原论文的扫描范围，**不能把扫描中最好的 M 直接替换成上一张表的默认配置**。[Transolver 消融](#)、[LinearNO 消融](#)、[LRSA 消融](#)、[IPOT 消融](#)

三、Transolver：除主配置以外，所有相关实验设置

Transolver 的主模型均为 **8 层、8 个头**：

- Elasticity、Plasticity、Airfoil、Pipe、Darcy：通道宽度 128， $M = 64$ 。
- Navier–Stokes、ShapeNet-Car、AirFRANS：通道宽度 256， $M = 32$ 。

作者将这一配置解释为性能与效率之间的平衡。因此，**它没有按输入点数 N 设置固定压缩比例 N/M** 。[Transolver](#)，附录 B.3、Table 8

除了前述完整的 M 扫描，其他实验如下。

实验	数据集和变化设置	使用的 M	位置
分片数变化的时间／显存测试	$N = 1024$ ，batch size 为 1	1、8、16、32、64、96、128、256、512、1024	Tables 4、9，第 7、18 页
固定方块代替可学习分片	Plasticity、Airfoil、Pipe、NS、Darcy	使用 4×4 大小的规则方块；总 token 数未明确报告	Tables 4、9
跨层多尺度	六个标准任务，全部覆盖	全 8 层 32；全 8 层 64；前 4 层 64、后 4 层 32	Table 11、Figure 20，第 22 页
多尺度方案的效率测试	$N = 1024$ ，batch size 为 1	同上三种设置	Table 11
Darcy 分辨率扩展	85^2 、 103^2 、 141^2 、 211^2 、 411^2	$64^\dagger$	Figure 8(a)，第 9 页
与普通 Transformer 的多分辨率比较	Darcy： 22^2 、 41^2 、 58^2 、 85^2 、 103^2 、 141^2 、 211^2 、 411^2	Transolver 为 $64^\dagger$ ；普通 Transformer 无分片 M	Table 10，第 18 页
训练数据量扩展	Darcy：1000、2000、3000、4000、5000 个训练样本	$64^\dagger$	Figure 8(b)
模型深度扩展	六个标准任务；8、16、24、32、40 层	NS 为 $32^\dagger$ ，其余为 $64^\dagger$ ；未报告随深度改变 M	Figure 8(c)、Figure 19，第 9、22 页
注意力分析和 Elasticity 重采样	Elasticity 保留 50% 点；Darcy 注意力分布统计	均为 $64^\dagger$	Figure 5、Table 5，第 8 页
分片与重采样可视化	ShapeNet-Car、Airfoil、Pipe、NS、Darcy	图注明确分别为 32、64、64、32、64 ；重采样不改变 M	Figures 9–17，第 19–21 页
OOD Reynolds 数与攻角	AirFRANS，两种 OOD 设置	均为 $32^\dagger$	Table 6；附录 E. 3、Tables 12–13

实验	数据集和变化设置	使用的 M	位置
WaterDrop 拉格朗日任务	2000 粒子；GNN 消息传递旁并联额外 Transolver 层	未说明	附录 E.4、Table 14，第 23 页
三次重复与标准差	八个主任务	对应主配置，未报告新的 M 设置	Table 15，第 24 页
实际任务效率比较	Elasticity、ShapeNet-Car	分别为 $64^\dagger$ 、 $32^\dagger$ ；图未独立重列计时模型的 M	Figure 6，第 8 页
输入点数扩展的独立效率测试	$N = 1024、2048、4096、8192、16384、32768$	统一计时模型的 M 未单独说明	附录 F、Figure 21、Table 16
其他预测误差图	八个主任务的相关展示	对应主配置，未另报 M	Figures 1、7、18

以上设置均来自上传版的正文和附录。[Transolver 原文](#)

这里特别强调两点：

4×4 规则方块不等于论文明确设置了 $M = 16$ 。原文描述的是方块大小／形状，没有充分说明总分块数和边缘处理。

Transolver 已经做过 $64 \rightarrow 32$ 的跨层 latent 数变化。但这个多尺度版本仍然在每一层执行 slice/deslice，并没有变成后半段持续在 latent 空间计算。

四、LinearNO：主配置、结构消融与 Cross-LinearNO

LinearNO 在前八个共有任务上的总体 M 、宽度、层数和头数与 Transolver 对应配置相同；额外的 Burgers/Cross-LinearNO 使用：

$$M = 256, \quad d = 96, \quad h = 8, \quad L = 8.$$

这里的 $M = 256$ 不是普通六任务实验的统一设置。[LinearNO，Table 8](#)

实验	数据集和变化设置	使用的 M	位置
完整分片数消融	六个标准任务	16、32、48、64、96、128、256	Table 11，第 14 页
与 Transolver 的分片数曲线比较	Airfoil，两个模型	双方均为 16、32、48、64、96、128、256	Figure 4(e)，第 7 页
逐层注意力有效秩	Airfoil，两个模型，8 层，按 head 平均	双方均明确为 64	Figure 4(d)
Gen/Sim 解耦与去除中间 SA 的组合消融	六个标准任务，四种组合	按总体配置：NS 为 $32^\dagger$ ，其余为 $64^\dagger$ ；各组合未重新列 M	Tables 4、10，第 6、14 页
softmax 归一化轴消融	Airfoil、Darcy、Elasticity	均为 $64^\dagger$	Table 6，第 7 页
去除 slice-attention 的动机实验	六个标准任务，Transolver 及其变体	图中未独立报告 M ；只能参考总体实验设置	Figure 1，第 1 页
参数量和 FLOPs 比较	六个标准任务，batch size 为 1	LinearNO 按任务默认配置；表中未重新列 M	Tables 3、9，附录 C
OOD Reynolds 数	AirfRANS	$32^\dagger$	附录 D、Tables 12–13
OOD 攻角	AirfRANS	$32^\dagger$	同上
稀疏输入补全／超分辨率	Burgers；采样率 20%、10%、5%、1%、0.5%	全部使用 256 ；未给各采样率不同的 M	附录 E、Table 14，第 15 页
Airfoil 权重、误差可视化	Airfoil	$64^\dagger$ ；不能把 Figure 4(d) 的明确说明扩写成所有 panel 均单独报告	Figure 4(a–c)
补充权重可视化	Darcy、Elasticity、Pipe	均为 $64^\dagger$	Figures 6–8，第 17–19 页

实验	数据集和变化设置	使用的 M	位置
补充预测误差图	AirfRANS、ShapeNet-Car、NS 及其余标准任务	前三者 32 [†] ，其余 64 [†]	Figures 9–10，第 20 页

附录 F 是注意力形式之间的推导，没有增加新的数值实验或新的 M 配置。[LinearNO 原文](#)

还需要区分两个对象：**LinearNO 自己的配置**，以及 **LinearNO 论文中复现的 Transolver 配置**。论文没有为每张对比表逐模型重列 M 。可以明确确认双方相同 M 的，是 Figure 4(d) 的 64 和 Figure 4(e) 的七档扫描；其他表不能凭总体配置表就声称每个 baseline run 都被独立说明。

五、LRSA：尤其要区分精度配置和效率配置

LRSA 的总体配置没有随层变化的 M_ℓ ：各层使用该任务配置的同一个 M 。

- 六个标准任务和两个工业任务：8 层。
- Irregular Darcy、Pipe Turbulence、Heat Transfer、Composite：4 层，均为 $M = 32$ 。

它与 Transolver/LinearNO 的主要默认差异是：

$$\text{Pipe} : 64 \rightarrow 32, \quad \text{ShapeNet-Car、AirfRANS} : 32 \rightarrow 64.$$

这些数值来自 LRSA 的 Table 9。[LRSA 原文](#)

实验	数据集和变化设置	使用的 M	位置
完整 latent rank 消融	六个标准任务	8、16、32、64、128、256	Table 12，第 20 页
LRSA 与 Transolver 的 rank 曲线比较	Airfoil、Darcy、Elasticity、NS	双方均扫描 8、16、32、64、128、256	Figure 5 上部，第 8 页
零样本分辨率泛化	Airfoil；训练 221×51 ，测试 221×51 、 111×51 、 221×26 、 111×26	64 [†] ，该实验未重新列 M	Table 4，第 7 页
小样本训练	Darcy、NS；200、600、1000 个训练样本	未单独披露；总体配置分别为 64、32，但 NS 存在下述对应疑点	Table 5，第 7 页
去除 latent 内部 attention	六个标准任务	未逐项重列 M ，只能参考总体配置	Table 12，Components
强制压缩／重建对称	六个标准任务	未逐项重列 M ，只能参考总体配置	同上
FP32、FP16、BF16 的训练／评估精度消融	六个标准任务	各行未重列 M ，不能凭误差相同反推配置	Table 12
混合精度准确性展示	Airfoil、NS、ShapeNet-Car surface	图中未独立报告 M	Figure 4 左侧
前向／反向延迟及显存测试	NS、Airfoil、ShapeNet-Car	LRSA 和 Transolver 全部使用 64	Table 11，第 19 页
五个随机种子重复	六个标准任务，LRSA 与 Transolver	没有重列 M ；LRSA 可参考总体配置，但保留下述疑点	Table 10，第 19 页
预测可视化	NS、Elasticity、Airfoil、Plasticity	对应总体配置，图未重列 M	Figure 3，第 5 页

LRSA Figure 1 的一维 Poisson/Green 函数低秩示意，是数值低秩性质的展示，不能当成一个额外 LRSA 训练任务的 latent 配置。[LRSA，Figures 1、4–5，Tables 4–5、9–12](#)

效率表的三个设置非常明确：

效率测试数据集	输入点数 N	latent 数 M	Batch size
Navier–Stokes	4096	64	20
Airfoil	11271	64	20
ShapeNet-Car	32186	64	8

因此，**LRSA 的 NS 效率测试不是按照 Table 9 的 $M = 32$ 进行的**。此外，该效率表中的 Transolver 在 ShapeNet-Car 上也使用 64，区别于 Transolver 原论文的主配置 32。[LRSA, Table 11](#)

还有三处原文问题，需要保留下来：

1. **NS 配置与结果不能一一对应。**

Table 9 写 $M = 32$ ，主结果为 0.0484；但 Table 12 中：

$$M = 32 : 0.0691, \quad M = 64 : 0.0484.$$

这说明不能仅凭配置表，完全确认主结果对应哪次运行；也不能擅自把作者写的 32 改成 64。

2. **Darcy 也有跨表数值差异。**

配置表写 $M = 64$ ，主结果为 0.0043；rank 表中 $M = 32$ 为 0.00431， $M = 64$ 为 0.00452。可能涉及运行差异或配置记录问题，原文没有解释。

3. **NS 的 $M = 8$ 误差疑似有指数印刷错误。**

Table 12 印为 1.03×10^{-2} ，Figure 5 对应曲线却约为 1.03×10^{-1} 。这不影响确认“实验测试过 $M = 8$ ”，但不能据此声称 $M = 8$ 的准确率最好。[LRSA, Tables 1、9、12 与 Figure 5](#)

六、IPOT：各任务 latent 数与其余实验

IPOT 的 Table 5 把可学习查询写成“token 数 × 通道宽度”。完整配置如下，避免把后面的 64/128 误读成 token 数。

任务	可学习查询形状 $M \times d_z$	Processor 的头数	Processor block 数
Burgers	256 × 64	8	1
Darcy	256 × 64	8	4
Navier–Stokes	512 × 128	4	2
Airfoil	128 × 64	4	2
Elasticity	512 × 64	4	4
Plasticity	256 × 64	2	2
Spherical shallow water	256 × 128	8	2
ERA5	512 × 128	8	4

这里列的是 **processor** 的头数；IPOT 的 encoder、processor、decoder 的头数并不总相同。无论头数如何设置，都不改变第一列定义的 M 。[IPOT, Table 5, 第 13 页](#)

实验	数据集和变化设置	使用的 M	位置
主准确率、参数、训练时间、显存比较	Darcy、NS、Airfoil、Elasticity、Plasticity、ERA5	按任务配置依次为 256、512、128、512、256、512 ；结果表未另列 M	Tables 1、5
latent 数消融	ERA5, $N = 16200$	64、128、256、512	Table 3, 第 6 页
去掉 inducing-point 压缩	ERA5, IPOT w.o ip	不再保留小规模瓶颈；该实验表列 $M = N = 16200$	Table 3
输入规模与推理时间／显存扩展	ERA5 不同规模，最大到 $N = 10^6$	分别固定为 256、512 ；另比较无 inducing-point 版本	Figure 5, 第 7 页
分辨率泛化	ERA5；训练 2° ，测试 4° 、 1° 、 0.25°	$512^\dagger$ ；同一预训练模型，未报告随测试分辨率改变 M	Table 2；数据说明
陆地区域遮挡	ERA5, Masked land	$512^\dagger$	Table 2、Figures 4、6
海域遮挡设置	ERA5, Masked sea	$512^\dagger$ ；该设置列在附录数据表，正文 Table 2 没有另列对应定量结果	Table 4, 第 12 页
稀疏观测可视化	Darcy；输入保留 100%、50%、25%	均为 $256^\dagger$	Figure 3, 第 7 页

实验	数据集和变化设置	使用的 M	位置
不规则任务预测可视化	Airfoil、Elasticity、Plasticity	分别为 $128^\dagger$ 、 $512^\dagger$ 、 $256^\dagger$	Figure 3
球面浅水长期预测	时间段 0-10、10-20、20-30、30-40	$256^\dagger$	Figure 4、Table 7, 第 7、14 页
与 Perceiver IO 比较	NS, $\nu = 10^{-3}$ 、 10^{-4} 、 10^{-5}	IPOT 统一参考 $512^\dagger$; 未逐黏度重列	Table 8, 第 14 页
Burgers 配置	附录列出数据、架构和训练设置	256 ; 正文 Table 1 未展示该任务结果	Tables 4-6

以上覆盖了 IPOT 主文及附录报告的相关实验。[IPOT 原文](#)

这里还有两个计数细节：

- ERA5 的 16200 是输入空间点数，不是标准 IPOT 的 latent 数；标准配置是 **512**。
- Figure 5 的无 inducing-point 版本，其序列长度随 N 增长，不能把 Table 3 中的 16200 当成整条效率曲线都固定使用的 token 数。

七、这些配置对我们当前讨论最有用的信息

首先，persistent latent 架构在原论文中确实经常配合更大的 M 。在共同任务上：

数据集	LRSA 配置表 M	IPOT 配置表 M	IPOT / LRSA
Elasticity	64	512	8
Plasticity	64	256	4
Airfoil	64	128	2
Darcy	64	256	4
Navier-Stokes	32	512	16

但这不能证明“一次压缩必须使用更多 latent”：IPOT 同时使用了不同的宽度、深度、训练设置，部分任务设置也不同。它只能说明，不能直接把原版 LRSA 的 32/64 和原版 IPOT 的效率结论拼接起来。

其次， M 不应该被当成固定压缩率。例如 Transolver 中，Elasticity 的 972 个点压到 64，而 ShapeNet-Car 的 32186 个点压到 32；原论文的选择显然不只是根据 N 决定。

最后，后续讨论我们模型的 token 数时，需要分清三种量：前段每个 LRSA block 的 M 、后段 persistent latent 的 M ，以及 CDPA 访问的历史 token 总数。它们不是同一个计数。前两者决定各阶段的表示规模，第三者还取决于缓存了多少历史 block。原论文的 M 表只能作为参照，不能直接把这些量合并成一个“latent 数”。

我要对新模型进行的实验包括:六个标准pdebench\ :Darcy ,Elasticity、Airfoil、Pipe ,Navier-Stokes （包含长时rollout,可以参考这几篇论文中已有的实验）,Plasticity ,两个非结构化不规则网格数据集: ShapeNet-Car、AirFRANS ,在darcy中还打算进行IPOT中进行过的“Darcy：稀疏系数观测 \(\rightarrow\) 完整解场”的实验，但是在接下来制定修改计划时暂时不啷哪个考虑“Darcy：稀疏系数观测 \(\rightarrow\) 完整解场”的实验”这个实验，只需考虑其他的实验即可（其他的实验应该是transolver现有的仓库就都包含在内的（了解下transolver论文中有没有做ns方程长时rollout的实验（比如有已有的10帧预测接下来的10帧/20帧/40帧等情况））。另外需要考虑“根据我前面让你分析过的各模型包含的实验配置，你应当如何设置新模型在我打算使用的数据集上的超参数（尤其各层latent token数，在第一版中我打算保持前面的LRSA风格的block中的latent token的数量和后面persistant latent中的latent token数量一致，后续可能补充在persistant latent中latent token数逐渐增加的实验，我前面忽略了一个问题，现在你在补充了解一下各个模型中的block总数，以便确定新模型的block总数并相应的调整LRSA风格的block数以及后续persistant latent中的block数）。最后还需要考虑一个问题，就是这里面对大部分问题（除了darcy稀疏重建外，参考IPOT原实验）输入点数和输出点数其实都是一致的，所以可不可以迁移IPOT的persistant latent部分时可不可以做一定调整，具体为：在处理darcy稀疏重建问题时保持IPOT的原始设定，而处理其他问题时虽然使用IPOT的persistant

latent但在重建时调整为LRSA风格的，也就是让重建于输入位置和特征有关（但仍要保持输入和输出解耦，也就是保持LRSA的原始设定），这样可不可以增强处理一般正向求解的能力（也就是说我现在希望改进后的模型有两种形式，对于darcy稀疏重建问题采用IPOT原始设置，对于其他问题按我上述内容修改，你判断下合不合理）

我会先核对 Transolver 的 Navier–Stokes 预测时间窗和仓库中的 rollout 实现，再结合四篇论文的层数、宽度和 latent 数，给出新模型首版的配置建议，并分析两种 decoder 的适用性。Darcy 稀疏观测任务暂不纳入首版修改范围。

Transolver 的 NS 设置已核实：论文和仓库默认都是前 10 帧预测后 10 帧，测试时逐帧自回归；训练时则用真实下一帧更新输入窗口。默认数据只有 20 帧，不能仅把预测长度改成 20 或 40 就完成有真值的评估。层数方面，你选八个任务在 Transolver、LinearNO、LRSA 的论文中都用 8 个 block；LRSA 的 4 层设置对应另外四个数据集。

两种 decoder 的区分是合理的。普通正向任务可以用前两层输出的点特征 H_2 构造最终重建 query；若同时保留 LRSA 的 H_2 残差连接，还能让点特征直接绕过最终 latent 瓶颈。它仍只需一次最终重建。我会把“输入相关 query”和“点特征残差通道”的作用分开说明，因为两者带来的表达能力不同。

你的两个 decoder 设定是合理的，也兼容目前的两阶段架构。**我的首版建议是：2 个完整 LRSA block + 一次过渡编码和 CDPA-Cross 融合 + 6 个 persistent latent block，最后采用 LRSA 风格的输入特征相关重建。**各阶段的 latent 数保持一致；Darcy 稀疏系数观测任务暂不纳入首版实现。

但需要先明确两个事实：**Transolver 原有 NS 实验是 10 → 10，没有直接提供 10 → 20/40 的完整评估条件；最终采用输入相关 decoder 后，模型还会增加一条绕过最终 latent 瓶颈的点特征通道。**后者对解释模型的表达能力很重要。

以下是实验范围、配置和架构判断，不是代码修改步骤。

1. 你的八个主要任务，Transolver 仓库已经覆盖

严格说，这六个任务应称为“六个标准 PDE benchmark（基准任务）”，避免与专名 **PDEBench** 数据库混淆。它们主要沿用 FNO/Geo-FNO 等工作的数据。

任务	当前仓库一次模型前向中的空间点数	原有预测方式	首版纳入
Darcy	$85 \times 85 = 7225$	完整系数场 → 完整解场	是
Elasticity	972	材料点几何 → 每点应力	是
Airfoil	$221 \times 51 = 11271$	几何及已有条件 → 每点流场量	是
Pipe	$129 \times 129 = 16641$	管道几何 → 每点速度量	是
Navier–Stokes	$64 \times 64 = 4096$	10 帧历史 → 下一帧，再自回归	是
Plasticity	$101 \times 31 = 3131$	输入条件 + 查询时间 → 该时刻的输出	是
ShapeNet-Car	论文配置约 32186	对已有体积 / 表面节点预测物理量	是
AirfRANS	论文配置约 32000	对已有流场节点预测物理量	是
Darcy 稀疏观测	输入点少于输出点	稀疏系数观测 → 完整解场	后续补充

工业任务的实际点数以当前样本和采样设置为准。仓库包含标准任务、ShapeNet-Car、AirfRANS 三套入口，模型接口不同；保留各自的数据读取、节点顺序、表面掩码和训练接口，可以继续使用这些数据。[Transolver 官方仓库](#)

特别纠正一个可能影响效率估计的细节：**当前 Transolver 的 Plasticity 单次前向不是处理 $3131 \times 20 = 62620$ 个时空 token。**它每次处理 3131 个空间点，通过时间条件分别预测 20 个时刻；这些调用也不是 NS 式的预测反馈。因此，迁移模型时要保留时间条件，不能直接照搬 IPOT 对 Plasticity 的时空展平方式。[Plasticity 实验代码](#)

2. Transolver 的 NS 到底做了什么 rollout

原论文附录 B.1 和仓库一致：

$$\nu = 10^{-5}, \quad N = 64^2, \quad T_{\text{in}} = 10, \quad T_{\text{pred}} = 10.$$

数据文件为：

```
text

NavierStokes_V1e-5_N1200_T20.mat
```

它包含 20 帧；前 10 帧作输入，后 10 帧作标签。模型每次前向输出一帧，并非一次输出全部未来 10 帧。[Transolver 原论文](#)，第 15 页、[NS 数据与模型设置](#)

训练和测试还有一个关键区别：

阶段	下一次输入窗口如何更新
训练	加入 真实下一帧 ，即 teacher forcing（真实历史回填）
测试	加入 模型预测帧 ，即 autoregressive rollout（自回归滚动预测）

因此，现有仓库已经包含 **10 步自回归测试**，但不能将它描述为“训练时也展开预测反馈的 10 步 rollout”。

对于更长预测：

目标	至少需要的完整轨迹长度	当前 T20 数据是否足够
前 10 帧 → 后 10 帧	20 帧	是
前 10 帧 → 后 20 帧	30 帧	否
前 10 帧 → 后 40 帧	50 帧	否

模型可以继续生成，但没有后续真值就不能评价长期误差。**只把代码里的 T=10 改成 20 或 40，不足以构成有效实验。**

IPOT 可以提供参考，不过其三种预测长度对应不同黏度：

IPOT 的 NS 设置	预测任务
$\nu = 10^{-3}$	10 → 40
$\nu = 10^{-4}$	10 → 20
$\nu = 10^{-5}$	10 → 10

这不是同一动力学条件下的预测长度消融。IPOT 明确将测试预测范围进一步延长的实验还包括球面浅水方程，不能把它算作 NS 的同类证据。[IPOT, Tables 4、7–8](#)

所以，你的实验范围可以这样理解：

- **首版直接覆盖现有 NS 10 → 10 自回归实验。**
- 10 → 20/40 保留为 NS 长时实验目标，但需要合适的长轨迹数据。
- 如果要研究“超出训练范围的长期外推”，应在同一黏度、同一时间采样间隔下，区分“训练到 10 步、测试到 40 步”和“训练与测试都覆盖 40 步”。

同时，我们目前的 persistent latent 指的是一次**模型前向内部，跨网络深度保持 latent**。沿用 Transolver 外部 rollout 时，每预测下一帧，仍然重新执行整个模型；它没有自动变成 IPOT 那种跨时间持续推进 latent 的时间架构。

3. 四个模型的 block 数，应该怎样对齐

针对你准备使用的八个任务，论文报告如下：

数据集	Transolver	LinearNO	LRSA	IPOT processor
Darcy	8	8	8	4
Elasticity	8	8	8	4
Airfoil	8	8	8	2
Pipe	8	8	8	未报告
Navier–Stokes	8	8	8	2

数据集	Transolver	LinearNO	LRSA	IPOT processor
Plasticity	8	8	8	2
ShapeNet-Car	8	8	8	未报告
AirfRANS	8	8	8	未报告

来源分别为各论文的架构配置表：Transolver Table 8、LinearNO Table 8、LRSA Table 9、IPOT Table 5。[Transolver](#)、[LinearNO](#)、[LRSA](#)、[IPOT](#)

LRSA 的 4 层配置对应另外四个不规则任务，不包括 ShapeNet-Car、AirfRANS。不能因为这两个工业数据集也是不规则网格，就套用 4 层。

另外，这里的 block 并不是完全相同的计算单位：

模型	一个主要 block 包含什么
Transolver	slice → latent attention → deslice，再接点域 FFN 和残差
LinearNO	解耦的分片聚合／重建及线性运算，再接点域 FFN 和残差
LRSA	down-attention → latent FFN → latent self-attention → latent FFN → up-attention，再接点域 FFN／ConvFFN 和残差
IPOT processor	latent self-attention＋latent FFN，编码器和解码器另外计算

因此，“都是 8 层”只能说明某一种深度计数相近，**不能自动说明参数量、FFN 数量和计算量相同。**

4．新模型首版：建议保持 2+6，而不是先缩成 2+2 或 2+4

我的建议是：

2 个完整 LRSA block + 6 个 persistent latent block

另外显式包含：

- 一次从前段点特征进入后段 latent 的过渡编码；
- 一次后段入口的 CDPA-Cross 历史融合；
- 一次最终重建。

用统一符号表示：

$$\begin{aligned}
 H_0 &\xrightarrow[\text{保存 } T_1]{\text{LRSA}_1} H_1 \xrightarrow[\text{保存 } T_2]{\text{LRSA}_2} H_2, \\
 Z_0 &= \text{Encoder}(H_2), \\
 \tilde{Z}_0 &= \text{CDPACross}(Z_0; T_1, T_2), \\
 Z_6 &= \text{LatentBlock}_6 \circ \cdots \circ \text{LatentBlock}_1(\tilde{Z}_0).
 \end{aligned}$$

这里继续采用已经讨论的**入口融合一次**；后续不在每层重新访问全部历史。

选择 2+6 的理由是：

- 保留 8 次 latent 内部 self-attention 混合。**
前两层各一次，后段六次。这样主要考察“把反复点域处理改成持续 latent 处理”的影响，减少一开始就大幅削减表达深度的干扰。
- 最大的节省来自减少点域计算。**
从 2+6 改成 2+4，只会再减少两个相对便宜的 latent block；点域压缩、重建次数并不继续减少。
- 前两层可以形成两份不同处理深度的历史。**
它们也提供两次点域更新和规则网格局部处理，为 CDPA 提供不同来源。

不过，首版的“总深度 8”应准确记录为 **8 个含 latent self-attention 的处理 block**，不能遗漏额外 encoder、decoder、CDPA 后，宣称整个网络只有 8 个模块。

我建议后段六个 block **参数独立**。IPOT 官方 processor 实现中存在重复应用同一 attention/FFN 模块的参数共享写法；如果直接照搬，会把“持续 latent”和“跨层参数共享”两个因素同时引入。首版先把持续计算结构作为主要变化，也不能直接继承 IPOT 原模型的参数量数字。[IPOT processor 实现](#)

5. 各任务首版超参数建议

下面是我为新模型建议的起始配置，不是论文已经验证过的新模型最优值。

所有任务都满足：

$$M_{\text{LRSA}_1} = M_{\text{LRSA}_2} = M_{\text{bridge}} = M_{\text{latent},1} = \cdots = M_{\text{latent},6}.$$

数据集	通道宽度 d	头数 h	各阶段统一的 M	前段 LRSA blocks	后段 latent blocks
Darcy	128	8	64	2	6
Elasticity	128	8	64	2	6
Airfoil	128	4	64	2	6
Pipe	128	4	32	2	6
Navier–Stokes	256	8	64	2	6
Plasticity	128	8	64	2	6
ShapeNet-Car	256	8	64	2	6
AirfRANS	256	8	64	2	6

这组设置主要沿用 LRSA 的宽度、头数和 latent 规模，其中 NS 需要单独解释。

为什么 NS 首版建议 64，而不是配置表中的 32？

- 后段不再反复回到点域，持续表示的容量值得适当保留。
- LRSA 的 NS 效率测试明确使用 64。
- LRSA 的 rank 消融表显示，NS 对 M 较敏感；其主结果与配置表还有此前指出的对应疑点。

因此，64 是兼顾效率和容量的起点。**这不代表已经证明 64 足够支撑更长 rollout**。如果之后发现主要瓶颈集中在 NS，128 是有依据的容量对照，而不必首先增加全分辨率 block。

为什么不直接采用 IPOT 的 256 或 512？

因为我们的前段已有两次点域更新、历史 latent 融合，而且新 decoder 还保留输入点特征；它并不是原始 IPOT 的同一个信息瓶颈。另一方面，大幅增大 M 会增加：

$O(NMd)$ 压缩／重建成本， $O(M^2d)$ 后段和 CDPA 成本。

因此，没有充分理由一开始就照搬 256/512。

其他起始设置建议：

- 前段保留完整 LRSA 的 FFN、残差和规则网格 ConvFFN 结构。
- 后段保留 self-attention+FFN；若沿用 IPOT 的 GEGLU FFN，隐藏宽度先取 $2d$ 。
- 后段不额外加入沿 latent 编号的卷积。
- dropout 先取 0，与基仓库常用设置接近。
- attention 使用标准 SDPA 实现；精度比较时，新模型和重跑基线采用一致设置。

其中 FFN 宽度 $2d$ 是起始建议，不是四篇论文统一规定的值。

训练预算方面，首版架构比较建议先沿用 Transolver 的任务预算：

任务	Epochs	Batch size	优化器	学习率配置值
Darcy	500	4	AdamW	10^{-3}
Elasticity	500	1	AdamW	10^{-3}
Airfoil	500	4	AdamW	10^{-3}
Pipe	500	4	AdamW	10^{-3}

任务	Epochs	Batch size	优化器	学习率配置值
NS	500	2	AdamW	10^{-3}
Plasticity	500	8	AdamW	10^{-3}
ShapeNet-Car	200	1	Adam	10^{-3}
AirfRANS	400	1	Adam	10^{-3}

标准任务的 AdamW 权重衰减可沿用 10^{-5} ；调度器、损失、归一化和训练划分继续采用对应 Transolver 入口。OneCycle 中的 10^{-3} 是传入的最大学习率参数，不应误写为整个训练过程恒定的学习率。

这里有两个需要准确记录的差异：

- LRSA 论文的 ShapeNet-Car 使用 **400 epochs**，Transolver 使用 **200 epochs**。若直接比较论文数字，训练预算不同。
- 当前 Transolver AirfRANS 配置文件写的是 **398 epochs**，论文表格写 400。首版如果保留原脚本，就让新模型与重跑基线都用 398，并如实记录；若采用 400，也应对双方统一。[Transolver 配置表](#)、[AirfRANS 配置文件](#)

6. 普通正向任务采用 LRSA 风格最终重建，是合理的

你的判断成立，但判断条件应从“输入输出点数相同”进一步收紧为：

输出节点与输入节点有明确的一一对应关系，并且每个输出节点都有可用的输入侧特征。

仅有 $N_{\text{in}} = N_{\text{out}}$ 不够。例如两套大小相同但位置不同的点集，不能直接按索引复用输入特征。

当前八个任务在 Transolver 的处理方式下，基本满足所需的节点对应关系。Plasticity 对应的是材料参考点身份，并不要求预测的变形后坐标与输入坐标相同。

对于新模型，最终 decoder 的输入应是：

$$H_2 \in \mathbb{R}^{B \times N \times d}, \quad Z_6 \in \mathbb{R}^{B \times M \times d}.$$

其中 H_2 是两个前段 block 完成后的点特征，已经包含输入位置、任务条件和前段处理结果。

采用 LRSA 风格重建时：

$$\begin{aligned} Q &= \text{Norm}(H_2)W_Q^\uparrow, \\ K &= \text{Norm}_z(Z_6)W_K^\uparrow, \quad V = \text{Norm}_z(Z_6)W_V^\uparrow, \\ \Delta H &= \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)VW_O^\uparrow. \end{aligned}$$

如果保持 LRSA 的完整读出后半段，还包括：

$$\begin{aligned} H_D &= H_2 + \Delta H, \\ H_{\text{out}} &= H_D + \text{PointFFN/ConvFFN}(\text{Norm}(H_D)), \\ \hat{u} &= \text{OutputProjection}(H_{\text{out}}). \end{aligned}$$

这是一次 **up-attention+点特征残差+点域 FFN**，没有因此再增加一次 down-attention 或 latent self-attention。原始 LRSA 的 up-query 也是来自该 block 的输入点特征，而不是直接使用未知输出场。[LRSA，式 \(10\) 及实现](#)、[LRSA attention 代码](#)

这里的压缩与重建仍然解耦：

- 编码端的可学习 latent queries 与解码端的输入特征 queries 不同；
- 两端投影参数独立；
- 重建权重不被强制等于压缩权重的转置。

输入输出节点相同，不等于压缩与重建必须共享权重。

7. 为什么它可能提高普通正向任务的表达能力

需要把两种作用分开。

第一，输入相关 query 提供更灵活的读取方式。

同一位置，在不同 Darcy 系数场、NS 历史状态或工业流动条件下，可以形成不同的 Q_i ，进而从 latent 中读取不同内容：

$$\beta_{ij} = \text{softmax}_j \left(q(H_{2,i})^\top k(Z_{6,j}) \right).$$

坐标 query 的 decoder 也会通过输入相关的 latent keys 改变注意力权重；因此不能说它“完全不能适应输入”。准确的区别是：**LRSA 风格 decoder 还允许查询端直接由该点的条件特征变化。**

第二，保留 H_2 残差，提供真正的点特征旁路。

只改 query，主要改变 latent 值的读取权重；加入

$$H_D = H_2 + \Delta H$$

才让点特征可以直接绕过最终压缩瓶颈。

模型因此具有如下形式：

$$\hat{u} = D\left(H_2, F\left(\text{CDPA}(E(H_2), T_1, T_2)\right)\right).$$

即便两份输入在最终 latent 路径中形成相同或接近的表示，decoder 仍可能利用不同的 H_2 区分它们。这个旁路对局部条件、细节和高频信息有明确的潜在价值。

但这不是“恢复全部丢失信息”的保证：前段本身已经丢掉的信息不会自动回来，最终能否利用旁路也取决于学习结果。

对研究归因尤其要注意：**CDPA 的历史信息收益，与最终 H_2 旁路的收益，是两个不同来源。**后面解释实验结果时，应避免把 decoder 改动带来的提升全部归功于 CDPA。

另外，最终相加发生在相同宽度的隐藏特征空间，不是直接把 Darcy 渗透率数值加到压力解上。

8. 两种 decoder 模式的适用范围与效率

这两种形式可以共用整体研究框架：

模式	最终 query 来源	点特征残差	适用任务
输入特征条件解码	对应节点的 H_2 ，其中包含位置和任务条件	保留 H_2	首版八个主要任务
输出坐标解码	输出位置编码 $\phi(y)$	不添加没有定义的 $H_2(y)$	后续 Darcy 稀疏观测 → 完整解场

未来的稀疏模式使用：

$$Q_{\text{out}}(y) = \phi(y)W_Q,$$

因此可以查询未被观测的输出位置，而不要求这些位置上存在输入系数特征。不能使用未观测位置的真实 $a(y)$ 构造 query，否则改变了任务条件。[IPOT decoder 设计](#)

普通模式仍可处理“输入和输出共同变化到另一分辨率、且节点对应”的情况；但它**不自动具有固定稀疏输入、任意密集输出的能力**。

还需预留一个认识：未来稀疏 Darcy 不一定只切换 decoder 就完成适配。当前前段的规则网格 ConvFFN 依赖完整网格；稀疏点输入如何进入该前段，也要另行处理。这个问题本版暂不展开。

采用 LRSA 风格最终读出后，效率优势依然有结构依据：

操作	8 层 LRSA	建议的新模型
点域 → latent	8 次	3 次：前段 2+过渡编码 1
latent → 点域	8 次	3 次：前段 2+最终重建 1
latent self-attention	8 次	8 次：前段 2+后段 6
规则网格 ConvFFN	8 次	3 次：前段 2+最终读出 1
历史 latent 的 CDPA-Cross	0	入口一次，访问两份历史

在相同 N, M, d 下，只比较 attention 的主要矩阵乘法，忽略共同常数、投影和 FFN：

$$C_{\text{LRSA}} \propto d(16NM + 8M^2),$$
$$C_{\text{new}} \propto d(6NM + 10M^2).$$

新模型的 $10M^2$ 包括 8 次 latent self-attention，以及对两份历史进行的 CDPA-Cross。当 $N \gg M$ ：

$$\frac{C_{\text{new}}}{C_{\text{LRSA}}} \approx \frac{6}{16} = 0.375.$$

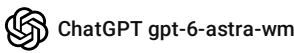
这个 37.5% 仅是上述 attention 矩阵乘法项的近似比例，不是端到端延迟比例。完整耗时还包含点域投影、卷积、FFN、归一化、内存访问和 kernel 调度。

你选择的 LRSA 规则网格卷积尤其值得正确计入：其实现中名为 dwconv 的层实际是没有设置 groups=d 的普通 3×3 卷积，不能按深度卷积估算成本。把这类处理从 8 次减少到 3 次，可能进一步扩大规则网格任务的效率收益。[LRSA ConvFFN 实现](#)

代价则是最后需要保留一份 H_2 ，增加约 $O(BNd)$ 的点特征驻留空间，并多执行一次最终点域 FFN/ConvFFN。后段六层仍然只更新小规模 latent，效率目标没有被这个 decoder 改动破坏。

后续逐渐增加 persistent latent 数量可以保留为补充实验。但增加 token 数本身不会创造新观测信息；它增加的是表示和计算容量。较大的后段 M 还会进入最终重建的 NM 成本，不能只按“发生在 latent 空间，所以很便宜”来判断。

“还需预留一个认识：未来稀疏 Darcy 不一定只切换 decoder 就完成适配。当前前段的规则网格 ConvFFN 依赖完整网格；稀疏点输入如何进入该前段，也要另行处理。这个问题本版暂不展开。”对这个问题的考虑是完全采用IPOT的设计方式，即新模型的前面的部分（LRSA block）不再使用卷积，后面的persistant latent部分则完全采用IPOT原来的设计（当然会前面说明的CDPA-cross仍要保留）。注：在以后的说明中就称CDPA-cross为CDPA，视为我确定的新机制（跨深度物理注意力），至于前面的CDPA-slice版本如果我后面再提起它再补充它的设计和实验。



明确。**稀疏 Darcy 版本采用无卷积的前段 LRSA block、IPOT 风格的持续 latent 处理和输出坐标解码，并保留 CDPA。**这样就解决了前段依赖完整规则网格的问题。

两种任务模式可以明确为：

部分	八个主要任务	后续稀疏 Darcy
前段 LRSA block	规则网格使用 ConvFFN；非结构化点使用逐点 FFN	不使用空间卷积，采用非结构化点版本的逐点 FFN
前段压缩与重建	保留完整 LRSA 逻辑	同样保留，只在已观测点集合上压缩、重建
进入后段	过渡编码 + CDPA	过渡编码 + CDPA
后段 persistent latent	在 latent 中持续计算	采用 IPOT 的 latent 处理方式
最终 decoder	使用前段点特征构造 query，并保留点特征残差	采用 IPOT 输出位置 query，不要求输出位置具有输入特征

稀疏 Darcy 的计算关系因此是：

$$\{(x_i, a(x_i))\}_{i=1}^{N_{\text{obs}}} \longrightarrow \text{无卷积 LRSA 前段} \longrightarrow \text{过渡编码 + CDPA} \longrightarrow \text{IPOT latent 处理} \longrightarrow \{\hat{u}(y_j)\}_{j=1}^{N_{\text{out}}}.$$

其中：

- 前段每次重建都返回 N_{obs} 个**已观测输入点**，不需要补齐完整系数网格。
- 最终 decoder 才根据完整输出坐标 y_j ，生成 N_{out} 个**解场值**。
- 不需要使用未观测位置的**真实系数 $a(y_j)$** 。

因此，这个版本在架构上可以自然处理 $N_{\text{obs}} \neq N_{\text{out}}$ 。整体仍是我们设计的混合模型，其中后段处理和最终读出采用 IPOT 方式。

命名也按你的决定固定：以后“CDPA”专指已经确定的跨深度物理注意力机制，即此前的 CDPA-Cross。它继续包含历史 latent 的跨注意力对齐和多历史块融合。CDPA-Slice 不纳入当前设计与实验范围，等你再次提出时再展开。

稀疏 Darcy 的这一架构设定现在已经明确，但仍按你的要求，暂不纳入首版代码修改范围。

2026-09-13 08:20:15 You 

准备好以transolver仓库为baseline,制定修改计划，充分考虑我和你以前的对话记录，以我和你讨论的结果为准进行设计，我说几个关键设计（我说的不全或可能有误，应以我和你的对话记录为准）：1.数据的问题，本质只改模型架构，不要影响数据读取，要保证改完后任然可以训练（具体参考我和你的对话内容，同时注意我在修改时并没有实际下载这个仓库所用的数据集，所以核查需要完全基于文件判断，没有数据，我觉得一方面你按我和你讨论过的形式设计可以直接避免数据方面的错误，另一方面你可以直接了解这些数据的格式和其他信息辅助进行核查而不是真的使用下载好的数据，用真的数据运行程序来检查）2.分阶段的设计，前面的阶段使用LRSA风格的block，后面的persistant latent部分采用IPOT风格的block(并非完全照搬，具体修改参照你我聊天记录，但你应当把这两个模型仓库当作参考仓库（这两个仓库本身的代码可能也有误，所以要检查好后再做参考）以便不要凭空捏造这两种block而是有依据的设计)。同时注意初始设置为两个LRSA风格的block，6个IPOT风格的Latent block，但这应当作为可以在训练中进行调整的内容（可以硬编码，但要，可以做不同比例的实验，从0到6都要兼容，根据你的经验选择一个合适的修改方式）；3.新增的主要机制CDPA（即按你我商定的cross attention风格，以kimi k3 attenres的方式将不同深度连接在当前block的方式），在初始设定中是在后面的persistant latent中第一次压缩时计算一次，但后续每个persistant latent中的block都要可以做CDPA（这个将作为补充的实验），另外block总数也应当可以扩展。4.darcy稀疏数据的模型架构部分暂时不纳入主计划中，可以作为记录，记录如何实现，但不要影响主计划的进行，即除非我特别提出，否则不要执行这部分的修改，计划中一定要说明这一点；5.切片数对各个数据集暂时按照我刚才与你讨论过的直接参考其他模型架构的结果，初始训练轮次和训练，评估的逻辑与transolver保持一致。上面可能不全，还是仔细阅读我和你的聊天记录，来制定完整的计划

 ChatGPT gpt-6-astra-wm

我会把已确认的架构整理成可交给 Codex 执行的修改计划，并逐项核对三个仓库的模型接口、数据格式和训练逻辑。重点会明确阶段比例与CDPA 模式如何配置、没有真实数据时如何验收，以及哪些内容只作后续记录、不得执行。

核对源码后，有几处会写进计划，避免执行时误改：Pipe 原训练脚本的 batch size 是 8；AirFRANS 实际配置是 398 个 epoch；Plasticity 的 20 个时间查询各自更新参数。CDPA 的逐层补充模式也会明确历史入库时机，避免同一个当前状态被重复加权。完整计划会整理成可直接使用的 Markdown 文件。

已把主版和补充实验的边界定清：默认是“总深度 8、前段 2、后段 6、入口 CDPA 一次”，阶段比例和总深度都通过参数调整。逐层 CDPA 的历史会增长，但不会重复存入当前状态；Darcy 稀疏模式和递增 latent 数只写入附录，不进入本次执行清单。参考代码中未使用的参数、意外共享的层，以及 CUDA 写死的位置编码，也会逐项说明如何处理。

完整草稿已写好，正在做最后的一致性审阅。文档包含逐文件修改范围、CDPA 公式与历史时序、八任务的合成输入验收，以及检查点加载和原训练逻辑的保护措施。此前未逐项约定的 FFN、归一化和初始化参数，也已明确标为本版的实现默认，避免与论文配置混淆。

最后审阅补上了三个容易导致执行失败的细节：评估时必须先读取已有架构配置，不能覆盖后再校验；只注册实际会使用的 placeholder 和时间模块；原仓库固定的 PyTorch 1.10.1 不支持 SDPA，因此需要单独的 CDPA 环境检查，避免安装旧 requirements 时降级。以上都已写入计划，数据和训练协议仍保持原样。

完整修改计划已完成，并核对了三个参考仓库的源码与此前确认的设计：

[下载完整修改计划：CDPA_Transolver_Implementation_Plan_v1.md](#)

文件包含精确计算公式、逐文件修改范围、执行顺序、八任务接口、无数据验收标准，以及可直接发给 Codex 的启动指令。**目前只完成计划，未修改模型代码、未下载数据。**

计划固定了以下内容：

项目	确定设置
默认结构	2 个完整 LRSA block+过渡编码+入口 CDPA+6 个持续 latent block+LRSA 风格最终读出
阶段比例	参数配置；总深度默认 8，前段支持 0-6，后段数量自动由两者相减得到
深度扩展	总深度可增加，不把后段固定为 6
CDPA	实现 off / entry / every_block ，默认 entry
latent 数	全模型各阶段一致；Pipe 为 32，其余七个任务为 64
规则网格	前段及最终读出使用 LRSA 风格 ConvFFN；后段 latent 不加卷积
最终重建	query 来自前段点特征，并保留点特征残差
后续内容	稀疏 Darcy、递增 latent 数、CDPA-Slice 不进入本次执行范围

逐层 CDPA 的历史规则也已消除歧义。例如第三个后段 block 的当前状态是 Z_2 ，它访问前段历史和 Z_0, Z_1 ； Z_2 只作为当前恒等候选，不再重复作为历史来源。默认 2+6 时，入口模式共进行 2 次历史对齐，逐层模式共 27 次，效率必须分别测量。

数据部分明确保护原来的读取、采样、归一化、损失和训练循环，特别包括：

- NS：训练真实帧回填，测试预测帧回填，保留现有 $10 \rightarrow 10$ 。
- Plasticity：保留时间条件和每个时间查询的参数更新方式。
- 两个工业任务：保留节点顺序、表面掩码、原模型接口及完整评估流程。
- 当前脚本口径：**Pipe batch size 为 8，AirfRANS 为 398 epochs。**

无数据验收包括数学参考对照、八任务合成输入前反向、原损失连接、阶段配置矩阵、CDPA 梯度与历史检查、检查点往返及冻结代码审查。另明确了 SDPA 需要支持它的 PyTorch 环境，不能沿用原文件中的 PyTorch 1.10.1 固定版本。

此前没有逐项确定的归一化、FFN 和初始化细节，已在文档中标为“实现级默认”，不会冒充论文原配置。

当前无法完成持久保存，请下载文件留存。

关于环境我在远端使用的时cuda12.8，以及2.11版本的torch，下载依赖时还有"# 1. 装 pyg-lib (匹配 torch 2.11.0+cu128)

```
pip install -U torch_geometric
```

pip install pyg-lib -f https://data.pyg.org/whl/torch-2.11.0+cu128.html"这个命令（这个依赖我已经在transolver下car-design-shapenetcar数据集上要整过了可以完成训练，但不知道与其他文件夹下数据训练是否兼容），你可以使用这个依赖也可以使用其他的依赖（但必须基于 cuda12.8，总之可以给一个具体的依赖）。还有关于计算量和推理延迟的设计你和我也有过讨论，你也给出了相应的设计（比如前面对如果后面persistant ;atent中的而每个block都使用 CDPA的考虑，你还给出了对应的设计），这些设计被考虑到当前的方案中了吗，除此之外，再完整的检查一下当前计划以及你和我的聊天记录，确保完整详细

我会按你已验证的 CUDA 12.8+PyTorch 2.11 环境，核对八个任务实际需要的依赖，再复查计划里的效率设计，尤其是逐层 CDPA 的历史增长、投影共享、缓存和计时口径。核查结果会补入更新版计划，并明确区分已验证的兼容性和仍需远端检查的部分。

你的安装方式有官方依据：PyG 2.8 已把 radius_graph 等操作接到 pyg-lib>=0.6，所以不能再一概要求额外安装 torch-cluster。不过，ShapeNet-Car 训练成功还不能覆盖 AirfRANS 的 PyVista 处理和反复构图路径。我也确认了计划的效率主设计已纳入；这次会补强来源批处理的 SDPA 实现、逐层 CDPA 的投影成本，以及与 LRSA 的实际计时对照。

这次复核有两点需要写得更明确：多份历史可以折叠到 batch 中计算，减少 SDPA 调用次数，但不会减少数学计算量；不同 CDPA 层的投影参数独立，因此不能直接复用上一层算好的 K/V。我会把这两个边界写入计划，并附上不需要数据集的环境检查脚本，方便你在远端验证八个任务涉及的依赖。

完整复核后，主结构仍保持已商定的“2 个 LRSA block+入口 CDPA+6 个持续 latent block”。规则网格卷积、最终特征条件读出、可调阶段比例，以及稀疏 Darcy 暂不实施的边界都已保留。这次主要补齐环境和性能验收：候选依赖的版本与直接依赖关系已核查，但远端 GPU 兼容性仍需运行预检脚本确认，不能用 ShapeNet-Car 的成功替代另外七个任务的检查。

已完成复核，并更新为 **v1.1 完整计划**。原来的效率主设计已经纳入；这次进一步补齐了依赖版本、来源批处理、逐层 CDPA 的投影成本，以及可实际计时的 LRSA 对照。

- [完整修改计划 v1.1](#)
- [完整交付包 ZIP：计划、依赖清单、环境预检脚本](#)

关于你的 CUDA 12.8+torch 2.11 环境

优先保留你已经完成 ShapeNet-Car 训练的环境，先检查实际版本和缺失依赖，不要求重新安装。

如果另外创建环境，计划中提供了以下具体组合，以及完整的 Python 依赖清单：

组件	新环境候选版本
Python	3.11
PyTorch	2.11.0+cu128
torchvision	0.26.0+cu128
torch-geometric	2.8.0.post1
pyg-lib	0.6.0+pt211cu128
PyVista / VTK	0.46.4 / 9.3.1

PyTorch 与 torchvision 的上述 cu128 配对有官方安装支持；这里安装 torchvision 是因为 timm 的导入链需要它。[PyTorch 官方说明](#)

你的 pyg-lib 安装方向也成立：**PyG 2.8 的 radius_graph 已通过 pyg-lib 实现，因此不能笼统要求必须再安装 torch-cluster**。对应 torch 2.11/cu128 的二进制包有官方索引。[PyG 2.8 源码](#)、[对应 wheel](#)

但 ShapeNet-Car 训练成功尚不能覆盖 AirFRANS 的依赖路径：AirFRANS 还有反复调用的半径图构造，以及 PyVista/VTK 场处理。交付包中的脚本可在**不下载数据**的情况下检查这些依赖：

```
bash

python check_cdpa_environment.py \
  --device cuda \
  --repo /你的绝对路径/Transolver \
  --json-path environment-report.json
```

候选版本的发布信息和直接依赖关系已经核查，远端 GPU 联合兼容性仍需运行上述检查确认。

关于计算量和推理延迟设计

默认结构保持：

2 个完整 LRSA block → 一次 bridge → 入口 CDPA → 6 个 persistent latent block → LRSA 特征条件读出。

以相同宽度、latent 数和卷积设置的 **8 层 LRSA 结构对照**计算：

操作	LRSA 对照	新模型 entry	新模型 every_block
点域→latent	8 次	3 次	3 次
latent→点域	8 次	3 次	3 次
规则网格 ConvFFN	8 次	3 次	3 次
CDPA 历史对齐	0 份	2 份	27 份

这次明确补入了以下执行要求：

- 同次融合共享投影，当前 Q 只计算一次。

- **支持来源批处理及分块。** 全来源批处理下，entry 的历史 SDPA 调用可合并为 1 次，every_block 合并为 6 次；对应的历史计算量仍然是 2 份和 27 份。
- **禁止错误缓存。** 不同 CDPA 层参数独立，不能直接复用其他层投影后的 K/V；原始历史保留梯度。
- **完整统计投影成本。** 不只计算 M^2 attention，还计入 Q/K/V/O、归一化、深度融合及临时内存。
- **加入实际 LRSA 结构计时对照。** 复用已移植 block 做合成性能比较，无需引入 LRSA 的数据读取或训练器。
- **统一性能测量条件。** 精度、batch、TF32、SDPA backend、编译设置一致，分别报告参数、计算量、延迟和显存。

因此，**entry 仍适合作为效率优先的主设置；every_block 保留为表达能力与额外成本的补充实验。** 来源批处理有望改善调用开销，但不能据此提前保证逐层 CDPA 一定更快。

完整性复核也已写入计划

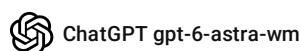
新增逐项核对表，覆盖：八任务数据合同、完整 LRSA 前段、后段独立参数、历史提取位置、双层 softmax、零初始化、最终 decoder、规则网格卷积、F/L 配置、各任务 M、原训练预算、NS 时间循环、checkpoint，以及稀疏 Darcy 等暂不实施的边界。

本次只更新计划并提供环境工具，三个参考仓库均未修改。脚本语法、帮助入口和缺依赖时的失败报告已检查；尚未执行远端 GPU 验证、新模型测试或真实训练。

当前无法完成文件的持久保存，请下载交付包留存。

2026-09-13 09:06:39 You 

完成了上述检查后再给上面的计划补充一些数学定理的支持（你的参考依据有以下几级：第一级：神经算子，PDE，线性代数，低秩空间等本身的一些相关理。第二级：论前面展示的几篇论文：LRSA，IPOT，LNO，LinearNO，transolver及它们相关的数学原理，第三级：与这个主题类似的其他论文的数学原理（低秩空间神经算子，PDE的相关理论（谱，低秩等）），第四级：适当的选取与pde有关的其他的且发表在ICLR上的论文使用的数学原理）。要利用设计的新模型的架构给出有新意且合理的数学原理支撑。我有几个思考：为什么在LRSA这样压缩重建解耦的block中两个相邻block可以有不同的表达能力；为什么PDE可以在低秩空间，甚至潜在低秩空间中学习；潜在低秩空间（也就是后面的persistant latent空间）中加入CDPA的表达能力要强于不加入（即不增加CDPA这个模块时）。你可以根据已有ICLR论文的经验和本身需要直接复用参考论文呢的一部分定理（但也许需要适当的更换表达方式？你的输出可能被直接用于组织ICLR或其他顶会（有可能时ICML或NeurPIS）的论文理论部分，要注意在形式上严格要求你的输出）



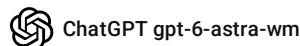
我会先核对当前计划和已完成检查的边界，再补充可用于论文理论部分的数学框架。重点回答你提出的三个问题，并明确区分已有定理、针对新架构可证明的命题，以及仍需实验验证的假设；尤其会严格检查“加入 CDPA 后表达能力更强”在什么条件下成立。

这项工作我会按深度研究技能整理成带假设、证明和来源说明的理论补充稿。先说明一个关键点：最终 decoder 还能读取 H_F 点特征，因此比较 CDPA 的表达能力时，必须把这条旁路纳入分析，不能只比较“单份 latent”和“多份 latent”。另外，更换已有定理的符号或表述仍需明确引用，不能将其写成新定理。

目前核对出的理论主线更清楚了：椭圆与扩散算子的谱衰减，为“一次编码后在潜空间处理”提供了条件明确的依据；相邻 LRSA 层可以更新不同方向，但不代表后一层天然具有更大的函数类。CDPA 则更适合用“历史是否保留了当前表示未保留、且对目标有用的信息”来分析，而不能直接断言增加模块必然提高精度。这些结论会写成条件明确、附完整证明的命题。

又确认了一个更贴近实际结构的关键点：虽然最终 decoder 保留 H_F ，但每个输出点直接读取的范围有限——不规则任务是该点特征，规则网格任务是最终卷积的 3×3 邻域；远距离信息仍需经过 latent。因此，即使完整 H_F 没有丢失输入，CDPA 也可能把 bridge 未保留的远处信息提供给当前输出点。这将作为理论稿的核心条件命题，比笼统声称“恢复全部信息”更准确。

理论稿和证明已完成独立复核，并附了一个保留完整 H_F 、采用非零均匀池化 bridge 的有限样本例子：同一冻结前端下，局部读出无法通过 bridge 区分的两个输入，可以经历史 CDPA 区分。另有 8 项 NumPy 代数与导数核查通过，英文理论稿已编译为 9 页 PDF。稿中也明确保留限制：这不等于重新训练后的完整 CDPA 模型必然优于 off；LayerNorm、对齐和固定大小融合仍可能丢弃或抵消信息。



我继续完成交付整理：将理论补充与修改计划合并为 v1.2，并附上中文完整证明、英文 LaTeX/PDF 和数值核查结果。模型架构、数据读取及已确定的实验配置保持不变。

已经完成，数学补充已纳入 **v1.2 计划**，并整理成中文完整证明和英文论文理论稿。**模型架构、数据读取和既定实验配置没有改变。**

- [完整计划 v1.2](#)
- [中文数学理论与完整证明](#)
- [英文理论稿 PDF, 9 页](#)
- [完整交付包：以上文件、LaTeX、核查脚本及依赖](#)

最值得作为论文核心的结论是：

CDPA 可以复用早期压缩产生的全局统计量，补充当前 bridge 和局部 decoder 无法访问的目标相关方向，而不必再次遍历全部点特征。

这里有明确的成立条件。不能把它扩大成“增加 CDPA 后，所有 PDE 上的表达能力和准确率必然提高”。

下面直接解释你提出的三个问题，以及这次形成的主要定理。

第一个问题：为什么相邻两个 LRSA block 可以发挥不同作用？

首先要区分：**相邻 block 的结构容量相同，但学到的函数、观察到的特征和选择的交互方向可以不同。**

对于你的完整 LRSA 前段，每层具有独立参数，并处理上一层更新后的点特征。down 决定提取哪些全局统计量，up 决定把这些统计量写回哪些点；点域残差则把当前层没有充分处理的特征继续传下去。

为了严格解释这一点，可以先分析一个**冻结注意力路由的线性代理**：

$$h_j = (I + R_j)h_{j-1}, \quad R_j = B_j K_j C_j.$$

其中 C_j 表示压缩， K_j 表示潜空间处理， B_j 表示重建。若每个 R_j 的秩至多为 r_j ，两层复合满足

$$\text{rank}[(I + R_2)(I + R_1) - I] \leq r_1 + r_2.$$

这个上界可以达到。例如：

$$R_1 = \begin{bmatrix} a & 0 \\ 0 & 0 \end{bmatrix}, \quad R_2 = \begin{bmatrix} 0 & 0 \\ 0 & b \end{bmatrix}.$$

两层各自只有一个更新方向，但合起来可以修正两个不同方向。

反过来，去掉残差，仅反复相乘，则

$$\text{rank}(R_2 R_1) \leq \min\{\text{rank } R_1, \text{rank } R_2\}.$$

这说明：**重复压缩重建的潜在价值，与点域残差保留特征、后续层重新选择方向密切相关。**

但真实 LRSA 还包含输入条件的注意力、FFN、归一化和卷积，不能把完整 block 当作固定低秩矩阵。特别是 up-query 依赖点特征，其导数包含路由变化项，因此不能直接声称“完整 block 的 Jacobian 秩不超过 M ”。

解耦也不自动意味着正交或双正交。只有另外满足相应代数条件，才能把分析/重建解释成严格的投影；当前方案没有这些约束，理论稿没有假装它们已经存在。

第二个问题：为什么 PDE 可以在低秩空间，甚至非线性潜空间中学习？

这部分最稳妥的依据是**谱衰减、解流形可压缩性和非线性重建**，而不是笼统地说“PDE 天然低秩”。

设 A 是正自伴算子，且 A^{-1} 紧，具有特征对

$$Ae_j = \lambda_j e_j, \quad 0 < \lambda_1 \leq \lambda_2 \leq \cdots.$$

对于解算子 $S = A^{-1}$ ，保留前 m 个模式：

$$S_m f = \sum_{j=1}^m \lambda_j^{-1} \langle f, e_j \rangle e_j.$$

经典谱理论给出

$$\|S - S_m\| = \inf_{\substack{B \text{ 为有界线性算子} \\ \text{rank } B \leq m}} \|S - B\| = \lambda_{m+1}^{-1}.$$

因此，这类问题确实可以采用：

$$\text{一次编码} \longrightarrow \text{潜空间对角映射} \longrightarrow \text{一次解码}.$$

对正时间的热方程，相应误差为

$$\|e^{-tA} - S_{t,m}\| = e^{-t\lambda_{m+1}}, \quad t > 0.$$

这为你的 persistent latent 设计提供了明确的存在性依据：**对于某些 PDE 类，反复返回点域不是数学上的必要条件。**

不过，Darcy 的系数场到解场映射 $a \mapsto u(a)$ 是非线性的，不能直接由某一个固定 A_a^{-1} 的谱衰减推出整个数据集共享同一个快速衰减空间。因此文档还加入了参数化椭圆 PDE 的解流形宽度条件，以及 POD 的平均重建误差公式。

非线性潜空间又是另一回事。考虑移动前沿

$$u_t(x) = \mathbf{1}_{[0,t]}(x).$$

它的最佳公共 n 维线性近似误差仅按

$$d_n \asymp n^{-1/2}$$

衰减，但一个标量 t 就能精确描述整个函数族。这里低维的是**非线性坐标**，不是公共线性输出空间；相应 decoder 在 L^2 中也只有 Hölder 连续性，不能忽略稳定性。

ICLR 2023 的非线性重建论文正是讨论这类区别：固定线性重建与输入条件非线性重建，可以具有不同的逼近效率。[论文](#)

另外，已有理论证明：一次全局平均，配合足够宽的非线性编码和重建，也可以获得算子普适性。因此，LinearNO 关于反复 slice/deslice 的实验观察，不能被理解为“反复压缩是所有神经算子表达能力的唯一来源”。你的研究应聚焦于**有限宽度、有限 latent 数和计算预算下，怎样更有效地使用压缩结果。**[相关理论](#)

第三个问题：加入 CDPA 后，到底能严格证明什么？

这次最重要的补充，是把最终 decoder 的真实访问范围纳入证明。

虽然最终 decoder 保留了 H_F ，但对某个输出点 i ：

- 不规则网格版本直接读取 $H_{F,i}$ ；
- 规则网格版本经过最终一次 3×3 ConvFFN，直接读取对应邻域；
- 其他远距离信息仍需通过 latent 提供。

记这个邻域为 $\mathcal{N}_i$ ，定义

$$V_i(x) = (H_F(x)|_{\mathcal{N}_i}, Z_0(x)).$$

这里 x 表示整个输入样本。**不加 CDPA 时，无论后段如何计算，该点的预测都只能是 $V_i(x)$ 的函数：**

$$\hat{u}_i^{\text{off}} = f_i(V_i(x)).$$

由此得到一个严格下界。

如果两个输入满足

$$V_i(x_+) = V_i(x_-),$$

但真实目标不同，那么任何使用同一冻结前端和 bridge 的 off 后段都必须对它们输出相同预测。因此

$$\max_{\pm} \|\hat{u}_i(x_{\pm}) - u_i(x_{\pm})\| \geq \frac{1}{2} \|u_i(x_+) - u_i(x_-)\|.$$

这个条件不要求整个 H_F 相同。 即使完整 H_F 保留了输入差异，当前输出点也可能无法通过“局部上下文+bridge”访问它。

接下来，才是 CDPA 真正对应的条件定理。

设早期历史为 $T_s(x)$ ，对齐结果为 $R_s(Z_0, T_s)$ 。对于一个局部读出和 bridge 都看不到的扰动方向 v ：

$$DV_i(x_*)v = 0,$$

在既定的零 scorer 初始化 $w = 0$ 下，CDPA 的导数满足

$$DC(x_*)v = \frac{1}{S+1} \sum_{s=1}^S D_{T_s} R_s DT_s(x_*)v.$$

这条式子把三个必要环节写得很清楚：

- 1. 早期历史 T_s 必须仍对该扰动敏感；
- 2. Cross 对齐不能消掉这个方向；
- 3. 各历史融合后不能相互抵消，后续 processor 和 decoder 也必须能利用它。

这些条件成立时，**CDPA 输出可以对一个 off 后段必然不敏感的方向产生响应**。这就是针对当前架构可严格陈述的表达能力支持。

文档还给出了实际 LRSA+CDPA 模块可实现的有限样本构造：完整 H_F 保留两输入，非零均匀池化 bridge 将远端相反特征抵消；早期历史采用不同压缩权重保留差异，CDPA 将其送到当前输出点。同一冻结前端下，off 的该点最优均方误差为 1，CDPA 可以为 0。

这个构造证明了历史机制**可能严格有用**；它没有证明重新训练 bridge 后的 off 模型仍然无法解决该任务。

为什么我没有把它写成“CDPA 模型类必然严格包含 off”？

因为你当前确定的融合机制不支持最常见的直接证明：

把新增模块参数置零，就恢复 baseline。

当前候选包含 $R_0 = Z$ ，但没有额外的外部残差。若历史输出全部置零，同时 $w = 0$ ，结果是

$$C = \frac{Z}{S+1},$$

并非 $C = Z$ 。

此外，历史 LN 会精确消掉 token 的通道均值变化，Cross 可能丢弃方向，多历史融合可能相消。历史增加后，输出仍是 $M \times d$ ，不能说它无损保存了所有历史。

所以，更适合论文的主张是：

在固定前端和计算预算下，CDPA 可以通过历史对齐，补充局部读出与当前 bridge 未提供的全局可访问方向；实际精度和效率收益由消融实验验证。

关于你要求的四级参考依据

理论稿已经逐项标明来源和适用范围：

层级	主要依据
第一层	谱截断、POD、Kolmogorov 宽度、残差秩累积、条件期望、局部 Jacobian
第二层	LRSA、IPOT、LNO、LinearNO、Transolver 的真实公式和已出现的理论
第三层	连续 attention、参数化椭圆解流形、单次全局平均的算子逼近理论
第四层	FNO (ICLR 2021)、非线性重建 (ICLR 2023)、抛物 PDE 定量逼近 (ICLR 2025)

其中 ICLR 2025 的理论用于参考“核近似+非线性处理+复合误差”的组织方式，没有把当前 block 冒充为满足其假设的 Picard 迭代。正式论文

已有定理即使更换符号或表述，也必须继续引用原作者。 本稿将经典结果、针对本架构的条件推导和待实验验证的假设分开；建议论文正文集中展示“局部访问限制–历史对齐–条件方向恢复”这条理论链，其余证明放附录。

本次已完成独立数学复核、**8 项 NumPy 代数与导数核查**以及英文 PDF 编译和排版检查。远端 GPU 验证、新模型实现和真实 PDE 训练仍未执行，理论与数值核查不能替代它们。

当前无法完成文件的持久保存，请下载交付包留存。

给这个新模型确定一个名字，我将用它来命名对应的项目名和github仓库名

 ChatGPT gpt-6-astra-wm

我会根据模型已确定的两阶段结构和 CDPA 机制选一个正式名称，并检索是否有明显重名，兼顾论文标题、项目名和 GitHub 仓库名。
我建议正式确定为 **CDLNO**。

英文全称：Cross-Depth Latent Neural Operator
中文名称：跨深度潜空间神经算子

这个名字对应模型的两个核心特点：

- **Cross-Depth（跨深度）**：通过 CDPA 对齐并融合不同深度的 latent 表示，复用前段 LRSA 压缩得到的信息。这是新模型最值得突出的方法贡献。
- **Latent（潜空间）**：后段持续在潜空间中计算，通过减少点空间与潜空间之间的往返来追求效率。

建议统一采用以下命名：

用途	名称
模型名、项目展示名	CDLNO
GitHub 仓库名	cdlno
Python 包名	cdlno
模型类名	CDLNO
核心机制名	CDPA — Cross-Depth Physics Attention

论文中可以这样首次介绍：

We propose **CDLNO (Cross-Depth Latent Neural Operator)**, which reuses intermediate latent representations through cross-depth physics attention to support persistent latent computation.

CDLNO 是完整模型，CDPA 是其中的核心机制。 名称准确表达架构特色，也为后续调整阶段比例、融合频率和 latent 数量保留了空间。